

磐维数据库（PanWeiDB）

V2.0-S3.0.0_Beta

应用程序开发指南

中移（动）信息技术有限公司

2024 年 4 月

【版权声明】

© 中移（动）信息技术有限公司 版权所有

本文档著作权归 **中移（动）信息技术有限公司**（简称“中移信息”）所有，未经中移信息事先书面许可，任何主体不得以任何形式复制、 修改、抄袭、传播全部或部分本文档内容。

中移（动）信息技术有限公司保留所有的权利。

【服务声明】

本文档意在向客户介绍中移信息全部或部分产品、服务的当时的整体概况，部分产品、服务的内容可能有所调整。您所购买的产品、服务的种类、服务标准等应由您与中移信息之间的商业合同约定，除非双方另有约定，否则，中移信息对本文档内容不做任何明示或模式的承诺或保证。

修订历史记录

序号	修订描述	修订日期	修订者
1	创建文档	2024-04-15	
2			
3			
4			
5			

目 录

1. .NET 接口开发指南 20
1.1. 介绍 20
1.1.1. PostgreSQL .NET 驱动版本 20
1.1.2. Npgsql 与.NET 的兼容性 20
1.2. Npgsql 主要类与接口 20
1.3. 设置.NET 驱动 21
1.4. 使用.NET 连接数据库 22
1.4.1. 导入 22
1.4.2. 连接数据库 22
1.4.2.1. 连接参数 22
1.4.2.2. 获取连接 23
1.4.2.3. 关闭连接 23
1.5. 执行 SQL 语句并处理结果 24
1.5.1. 执行查询 24
1.5.2. 读取结果 25
1.5.2.1. 获取单一结果 25
1.5.2.2. 使用 DataReader 获取结果 25
1.5.3. 使用 NpgsqlCommand 对象 27
1.5.4. 使用 NpgsqlDataReader 对象 29
1.5.5. 执行更新 29
1.5.5.1. 删除 30
1.5.5.2. 更新 30
1.5.5.3. 插入 30
1.5.6. 创建或修改数据库对象 31
1.5.7. 小结 31
1.6. 调用存储过程 32
1.7. 处理大数据类型 34
1.7.1. 二进制类型 34
1.7.1.1. BYTEA 34
1.7.2. 插入记录 34
1.7.3. 读取记录 36
1.7.3.1. 大对象功能 37
1.7.4. 插入记录 37
1.7.5. 读取记录 38
1.7.6. 字符串类型 40
1.7.6.1. 插入记录 41
1.7.6.2. 读取记录 41
1.8. 总结 42
2. Django 框架开发指南 44
2.1. 介绍 44
2.2. 安装 Django 44

2.2.1. 安装 psycopg2 44
2.2.1.1. 安装 PanWeiDB V2.0-S3.0.0_Beta Libpq 44
2.2.1.2. 安装 python3 和 gcc 45
2.2.1.3. 安装 psycopg2 模块 45
2.2.2. 安装 Django 46
2.2.3. Django 与 python 版本对应关系 46
2.3. 使用 Django 连接数据库 47
2.3.1. 创建 Django 项目 47
2.3.2. 创建 app 48
2.3.3. 修改 DATABASES 配置 48
2.3.4. 启动项目 49
2.4. 执行 SQL 语句并处理结果 50
2.4.1. 创建或修改数据库对象 50
2.4.2. 执行更新 51
2.4.2.1. 插入 51
2.4.2.2. 更新 52
2.4.2.3. 删除 52
2.4.3. 执行查询 52
2.4.3.1. 基础查询 52
2.4.3.2. 过滤查询 53
2.4.4. 获取结果 54
2.4.5. 执行存储过程 54
2.4.6. 处理数据类型 55
2.4.6.1. 二进制类型 55
2.4.6.2. 字符类型 56
2.4.6.3. 数字类型 57
2.4.6.4. 时间/日期类型 58
2.5. 连接 panweidb 时需注意的问题 60
2.5.1. 连接 60
2.5.1.1. 问题描述 60
2.5.1.2. 原因 60
2.5.1.3. 解决办法 60
2.5.2. 冲突时插入 64
2.5.2.1. 问题描述 64
2.5.2.2. 原因 64
2.5.2.3. 解决办法 64
2.6. 附录 1: 主要接口 65
2.7. 附录 2: 调用示例 68
3. GoLang 接口开发指南 73
3.1. 介绍 73
3.2. 设置 pq 驱动 74
3.2.1. 安装 GoLang 74
3.2.2. 安装数据库对应的 GoLang 语言驱动 75

3.3. 使用连接数据库 75
3.3.1. 3.1 导入 75
3.3.2. 3.2 连接数据库 76
3.3.2.1. 3.2.1.连接参数 76
3.3.2.2. 3.2.2 获取连接 77
3.3.2.3. 3.2.3 关闭连接 77
3.4. 执行 SQL 语句并处理结果 78
3.4.1. 执行查询 78
3.4.2. 读取结果 78
3.4.2.1. 获取单一结果 78
3.4.2.2. 使用数组获取结果 79
3.4.2.3. 支持 PBE 80
3.4.3. 执行更新 81
3.4.3.1. 删除 82
3.4.3.2. 更新 83
3.4.3.3. 插入 84
3.4.4. 创建或修改数据库对象 85
3.5. 调用存储过程与函数 85
3.6. 处理数据类型 87
3.6.1.1. 二进制类型 87
3.6.1.2. 字符类型 89
3.6.1.3. 数字类型 90
3.6.1.4. 时间/日期类型 93
3.7. 附录 1: 主要接口 95
3.8. 附录 2: 调用示例 96
4. JDBC 接口开发指南 98
4.1. 介绍 98
4.1.1. PanWeiDB V2.0-S3.0.0_Beta JDBC 版本 98
4.1.2. PanWeiDB V2.0-S3.0.0_Beta JDBC 与 JDK 的兼容性 98
4.2. PanWeiDB V2.0-S3.0.0_Beta JDBC 主要类与接口 99
4.3. 设置 JDBC 驱动 100
4.3.1. 设置类路径 100
4.3.2. 为 JDBC 准备数据库 101
4.4. 使用 JDBC 连接数据库 101
4.4.1. 导入 101
4.4.2. 说明 101
4.4.3. 加载驱动 102
4.4.4. 连接数据库 102
4.4.4.1. URL 格式 102
4.4.4.2. 获取 JDBC 连接 103
4.4.4.3. 关闭 JDBC 连接 103
4.4.5. 连接参数 103
4.5. 执行 SQL 语句并处理结果 107

4.5.1. 执行查询 107
4.5.1.1. 使用 Statement 执行查询 107
4.5.1.2. 使用 PreparedStatement 执行查询 108
4.5.2. 使用游标获取结果 108
4.5.3. 使用 Statement 和 PreparedStatement 接口 110
4.5.4. 使用 ResultSet 接口 111
4.5.5. 执行更新 111
4.5.5.1. 删除 111
4.5.5.2. 更新 112
4.5.5.3. 插入 112
4.5.6. 创建或修改数据库对象 113
4.6. 调用存储过程 114
4.6.1. 获取结果集 114
4.7. 处理大数据类型 116
4.7.1. 二进制类型 116
4.7.1.1. BYTEA 116
4.7.2. 插入记录 117
4.7.3. 读取记录 117
4.7.3.1. BLOB 118
4.7.4. 插入记录 118
4.7.5. 读取记录 120
4.7.5.1. CLOB 120
4.7.6. 插入记录 121
4.7.7. 读取记录 121
4.7.7.1. 大对象功能 122
4.7.8. 插入记录 123
4.7.9. 读取记录 124
4.7.10. 字符类型 126
4.7.10.1. 插入记录 126
4.7.10.2. 读取记录 127
4.8. 调用 BulkloadAPI 128
4.8.1. 加载配置文件 128
4.8.1.1. 连接串参数 128
4.8.1.2. setBulkloadInfo() 128
4.8.2. 获取 bulkload 管理器 129
4.8.3. 使用 bulkload 管理器 129
4.8.3.1. bulkloadIn() 129
4.8.3.2. bulkloadIn(InputStream from) 130
4.8.3.3. bulkloadIn(String folderPath, Pattern fileNamePattern) 132
4.8.3.4. bulkloadIn(File[] files) 133
4.8.4. 使用批量接口 134
4.9. 调用 CopyManager 135

4.9.1. 获取 Copy 管理器 135
4.9.2. 使用 Copy 管理器 135
4.9.2.1. copyIn(String sql, InputStream from, int bufferSize) 135
4.9.2.2. copyIn(String sql, Reader from, int bufferSize) 136
4.9.2.3. copyOut(String sql, OutputStream to) 136
4.9.2.4. copyOut(String sql, Writer to) 137
4.9.3. 从 MY 向 PanWeiDB 进行数据迁移 137
4.10. 逻辑复制 142
4.10.1. 配置选项 142
4.10.1.1. 解码线程并行度 142
4.10.1.2. 解码格式 142
4.10.1.3. 限制仅备机解码 143
4.10.1.4. 批量发送 143
4.10.1.5. 并行解码队列长度 143
4.10.1.6. 逻辑解码内存阈值 143
4.10.1.7. 逻辑解码发送超时阈值 144
4.10.2. 代码示例 145
4.11. 使用连接池 150
4.11.1. 无框架配置 150
4.11.2. Tomcat 配置 155
4.11.3. Tomcat 依赖修改 155
4.11.4. web.xml 配置 155
4.11.5. context.xml 配置 156
4.11.6. catalina.properties 文件 157
4.11.6.1. 项目中配置 158
4.11.7. 引入 pom 依赖 158
4.11.8. context.xml 与 web.xml 配置 158
4.11.9. filter 相关配置 159
4.11.10. Spring 配置 160
4.11.10.1. xml 配置 161
4.11.10.2. 结合 mybatis 的 xml 配置 163
4.11.10.3. yml 配置 165
4.12. 其他功能与优化 168
4.12.1. 读写分离&负载均衡 168
4.12.1.1. 参数说明 168
4.12.1.2. 样例 168
4.12.2. 写操作 169
4.12.3. 读操作 169
4.12.4. Socket 相关 169
4.12.4.1. 参数说明 169
4.12.4.2. 样例 169
5. libpq 接口开发指南 171

5.1. 介绍 171
5.1.1. libpq 库与 PanWeiDB V2.0-S3.0.0_Beta 171
5.2. libpq 库主要接口类型 171
5.3. 编译 libpq 程序 174
5.4. 使用 libpq 连接数据库 176
5.4.1. 连接数据库 176
5.4.2. 非阻塞连接数据库 178
5.4.3. 连接字符串 180
5.4.3.1. URL 格式 180
5.4.3.2. 参数关键字 181
5.4.4. 关闭到数据库的连接 182
5.5. 同步命令处理 183
5.5.1. 执行查询 183
5.5.2. 使用 PQprepare 接口 188
5.6. 异步命令处理 193
5.6.1. 使用预备语句 194
5.6.2. 提交命令 195
5.6.3. 获取结果 196
5.6.4. 示例 197
5.7. 逐行获取查询结果 204
5.7.1. libpq 设置逐行返回结果 204
5.7.2. 示例 204
5.8. 事件系统 208
5.8.1. 概览 208
5.8.2. 事件类型和触发函数 209
5.8.3. 示例 210
5.9. 处理大对象 218
5.9.1. 创建一个大对象 218
5.9.2. 导入一个大对象 219
5.9.3. 导出一个大对象 220
5.9.4. 打开一个现有的大对象 221
5.9.5. 向一个大对象写入数据 222
5.9.6. 从一个大对象读取数据 222
5.9.7. 在一个大对象中查找 223
5.9.8. 获取一个大对象的查找位置 223
5.9.9. 截断一个大对象 224
5.9.10. 关闭一个大对象描述符 224
5.9.11. 移除一个大对象 225
5.9.12. 示例 225
5.10. 其他功能 238
5.10.1. 杂项函数 238
5.10.2. 控制函数 239
5.10.3. 批处理函数 240

6. Lua 接口开发指南 246
6.1. 介绍 246
6.2. 安装 LuaSQL 246
6.2.1. 安装 Lua 246
6.2.2. 安装 LuaRocks 246
6.3. 安装 PanWeiDB V2.0-S3.0.0_Beta 247
6.4. 使用 luasql.postgres 连接数据库 247
6.4.1. 导入 247
6.4.2. 连接数据库 248
6.4.2.1. 获取数据库连接 248
6.4.2.2. 关闭数据库连接 248
6.5. 执行 SQL 语句并处理结果 248
6.5.1. 执行查询 248
6.5.1.1. 直接执行 248
6.5.1.2. 预编译方式执行 248
6.5.2. 使用游标获取结果 249
6.5.3. 执行更新 249
6.5.3.1. 插入 249
6.5.3.2. 更新 249
6.5.3.3. 删除 250
6.5.4. 创建或修改数据库对象 250
6.5.5. 执行存储过程 250
6.5.6. 处理大数据类型 251
6.5.6.1. 二进制类型 251
6.5.6.2. 字符类型 252
6.6. 附录 1: 主要接口 253
6.7. 附录 2: 调用示例 255
7. OCI 接口开发指南 259
7.1. 介绍 259
7.2. panweidb OCI 主要接口 259
7.3. 设置 OCI 驱动 263
7.3.1. 安装 libpq 263
7.3.2. 安装 unixODBC 264
7.3.3. 安装 ODBC 264
7.3.3.1. 安装包安装 openGauss-2.0.0-ODBC 264
7.3.3.2. 安装包安装 PanWeiDB V2.0-S3.0.0_Beta ODBC 265
7.3.3.3. 手动编译安装 265
7.3.3.4. 配置数据源 266
7.3.3.5. 配置环境变量 267
7.3.3.6. 测试数据源配置 267
7.3.4. 安装 OCI 268
7.3.4.1. 安装包安装 PanWeiDB V2.0-S3.0.0_Beta OCI 268
7.4. 使用 OCI 连接数据库 269

7.4.1. 连接数据库 269
7.4.1.1. 获取 OCI 连接 269
7.4.1.2. 关闭 OCI 连接 270
7.5. 执行 SQL 语句并处理结果 271
7.5.1. 执行查询、获取结果 271
7.5.2. 执行插入 273
7.5.3. 获取结果 275
7.6. 处理数据类型 275
7.6.1. 大对象类型 276
7.6.1.1. 插入记录 276
7.6.1.2. 读取 LOB 大对象定位器 277
7.6.1.3. 操作大对象类型 282
7.7. 句柄和描述符属性管理 287
7.7.1. 句柄属性管理 287
7.7.2. 描述符属性管理 289
7.8. 服务与会话 293
7.9. 事务管理 295
7.10. 杂项接口 296
7.11. 日志系统 297
7.11.1. 日志的级别 297
7.11.2. 日志的文件格式 298
7.11.3. 日志内容的格式 298
7.12. 分段操作 299
7.12.1. 执行插入或者更新流程 299
7.12.2. 执行查询操作 303
7.13. 游标支持 308
7.14. 内存池 310
7.15. 连接池 310
8. ODBC 接口开发指南 316
8.1. 介绍 316
8.2. openGauss ODBC 主要接口 316
8.3. 设置 ODBC 驱动 321
8.3.1. 安装 PanWeiDB V2.0-S3.0.0_Beta ODBC 321
8.3.1.1. 通过 PanWeiDB V2.0-S3.0.0_Beta 安装包安装 321
8.3.2. 安装 unixODBC 322
8.3.3. 安装 ODBC 322
8.3.3.1. 安装包安装 openGauss-2.0.0-ODBC 322
8.3.3.2. 安装包安装 PanWeiDB V2.0-S3.0.0_Beta ODBC 323
8.3.3.3. 手动编译安装 323
8.3.3.4. 配置数据源 324
8.3.3.5. 配置环境变量 325
8.3.3.6. 测试数据源配置 326
8.4. 使用 ODBC 连接数据库 327

8.4.1. 连接数据库 327
8.4.1.1. 获取 ODBC 连接 327
8.4.1.2. 关闭 ODBC 连接 331
8.4.2. 连接参数 332
8.5. 执行 SQL 语句并处理结果 333
8.5.1. 执行查询 333
8.5.1.1. 直接执行 333
8.5.1.2. 预编译方式执行 334
8.5.2. 使用游标获取结果 336
8.5.3. 执行更新 339
8.5.3.1. 删除 339
8.5.3.2. 批处理删除 340
8.5.3.3. 更新 341
8.5.3.4. 批处理更新 342
8.5.3.5. 插入 343
8.5.3.6. 批处理插入 343
8.5.4. 创建或修改数据库对象 345
8.6. 调用存储过程 345
8.7. 处理数据类型 347
8.7.1. 二进制类型 347
8.7.1.1. BYTEA 347
8.7.2. 插入记录 348
8.7.3. 读取记录 349
8.7.3.1. 大对象 350
8.7.4. 插入记录 350
8.7.5. 读取记录 350
8.7.6. 字符类型 351
8.7.6.1. 插入记录 351
8.7.6.2. 读取记录 353
8.7.7. 数字类型 353
8.7.7.1. 插入数据 353
8.7.7.2. 读取数据 360
8.7.8. 时间/日期类型 364
8.7.9. 插入数据 364
8.7.10. 读取数据 366
9. peewee 框架开发指南 369
9.1. 介绍 369
9.2. 安装 peewee 369
9.2.1. 安装 psycopg2 369
9.2.1.1. 安装 PanWeiDB V2.0-S3.0.0_Beta Libpq 369
9.2.1.2. 安装 python3 和 gcc 370
9.2.1.3. 安装 psycopg2 模块 370
9.2.2. 安装 peewee 371

9.3. 使用 peewee 连接数据库 371
9.3.1. 导入 371
9.3.2. 连接数据库 372
9.3.2.1. 定义表结构体 373
9.4. 执行 SQL 语句并处理结果 373
9.4.1. 创建或修改数据库对象 373
9.4.2. 删除数据库对象 374
9.4.3. 导出数据库对象 374
9.4.4. 执行更新 376
9.4.4.1. 插入 376
9.4.4.2. 更新 376
9.4.4.3. 删除 377
9.4.5. 执行查询 377
9.4.5.1. 查询单条数据 377
9.4.5.2. 查询多条数据 378
9.4.6. 获取结果 378
9.4.6.1. 遍历数组 378
9.4.7. 执行存储过程 378
9.4.8. 处理数据类型 379
9.4.8.1. 二进制类型 379
9.4.8.2. 字符类型 380
9.4.8.3. 数字类型 381
9.4.8.4. 时间/日期类型 383
9.5. 附录 1: 主要接口 384
9.6. 附录 2: 调用示例 388
10. Perl 接口开发指南 392
10.1. 介绍 392
10.2. 相关依赖版本 392
10.3. DBI 主要类方法 392
10.4. 使用 Perl 连接数据库 393
10.4.1. 导入 393
10.4.2. 连接数据库 394
10.4.2.1. URL 格式 394
10.4.2.2. 获取 Perl 连接 394
10.4.2.3. 关闭 Perl 连接 395
10.5. 执行 SQL 语句并处理结果 395
10.5.1. 执行查询 395
10.5.1.1. 使用 prepare 查询 395
10.5.2. 执行更新 396
10.5.2.1. 删除 396
10.5.2.2. 插入 397
10.5.3. Insert 插入数据 397
10.5.4. copy 插入数据 399

10.5.4.1. 更新 399
10.5.5. LOB 类型读写 400
10.5.5.1. BYTEA 类型 400
10.5.5.2. BLOB 类型 402
10.5.5.3. CLOB 类型、TEXT 类型 404
10.5.6. 创建或修改数据库对象 406
10.6. 存储过程或函数调用 406
10.6.1. 无参数 406
10.6.2. 带参数 408
11. PHP 接口开发指南 410
11.1. 介绍 410
11.1.1. PDO 类主要方法 410
11.2. 启用动态链接库 411
11.2.1. Windows 平台 411
11.2.2. Linux 平台 413
11.2.2.1. 安装 PanWeiDB V2.0-S3.0.0_Beta 413
11.2.2.2. 安装 postgresql-devel 413
11.2.2.3. 安装驱动拓展 413
11.3. 使用 PDO 连接数据库 416
11.3.1. 连接数据库 416
11.3.1.1. 获取数据库连接 416
11.3.1.2. 关闭数据库连接 417
11.3.1.3. 获取持久化连接 417
11.4. 执行 SQL 语句并处理结果 417
11.4.1. 直接执行 417
11.4.2. 预编译方式执行 419
11.5. 存储过程 420
11.5.1. 创建存储过程 420
11.5.2. 调用存储过程 421
11.6. 函数 422
11.6.1. 创建函数 422
11.6.2. 调用函数 423
11.7. 处理大数据类型 424
11.7.1. 二进制类型 424
11.7.1.1. BYTEA 插入记录 424
11.7.1.2. BYTEA 读取记录 425
11.7.2. 字符类型 426
12. Python 接口开发指南 427
12.1. 介绍 427
12.2. 安装 psycopg2 模块 427
12.2.1. 安装 PanWeiDB V2.0-S3.0.0_Beta Libpq 427
12.2.1.1. 通过安装 PanWeiDB V2.0-S3.0.0_Beta 安装包安装 427
12.2.2. 安装 python3 和 gcc 428

12.2.3. 安装 psycopg2 模块 428
12.2.4. psycopg2 对 python 的版本要求 429
12.3. 使用 psycopg2 连接数据库 429
12.3.1. 导入 429
12.3.2. 连接数据库 429
12.3.2.1. 获取数据库连接 429
12.3.2.2. 关闭数据库连接 429
12.4. 执行 SQL 语句并处理结果 430
12.4.1. 执行插入 430
12.4.1.1. 直接执行 430
12.4.1.2. 预编译方式执行 430
12.4.2. 使用游标获取结果 430
12.4.2.1. 遍历游标 431
12.4.2.2. cur.fetchone() 431
12.4.2.3. cur.fetchall() 431
12.4.2.4. cur.fetchmany([size=fetchsize]) 431
12.4.3. 执行更新 432
12.4.3.1. 插入 432
12.4.3.2. 更新 432
12.4.3.3. 删除 432
12.4.4. 创建或修改数据库对象 433
12.4.5. 执行存储过程 433
12.4.6. 处理数据类型 434
12.4.6.1. 二进制类型 434
12.4.6.2. 字符类型 435
12.4.6.3. 数字类型 436
12.4.6.4. 时间/日期类型 438
12.5. 附录 1: 主要接口 439
12.6. 附录 2: 调用示例 442
13. soci 接口开发指南 446
13.1. 介绍 446
13.2. Windows 搭建 soci 开发环境 446
13.2.1. 安装 Visual Studio 446
13.2.2. 安装 cmake 446
13.2.3. 安装 PostgreSQL 447
13.2.4. 下载 soci 源码 447
13.2.5. 编译 soci 447
13.2.5.1. cmake 生成 visual studio 工程 447
13.2.5.2. msbuild 编译 448
13.2.6. Demo 工程引入 soci 448
13.2.6.1. 创建 Demo 工程 448
13.2.6.2. 引入头文件 448
13.2.6.3. 引入 lib 文件 449

13.2.6.4. 修改包含目录 449
13.2.6.5. 修改附加库目录 449
13.2.6.6. 修改附加依赖项 450
13.2.6.7. 添加 dll 文件 450
13.3. 安装 PanWeiDB V2.0-S3.0.0_Beta 451
13.4. 使用 soci::session 连接数据库 451
13.4.1. 导入 451
13.4.2. 连接数据库 452
13.4.2.1. 获取数据库连接 452
13.4.2.2. 关闭数据库连接 452
13.5. 执行 SQL 语句并处理结果 452
13.5.1. 执行查询 452
13.5.1.1. 直接执行 452
13.5.1.2. 预编译方式执行 453
13.5.2. 使用 sql.prepare 获取多行结果 453
13.5.3. 执行更新 454
13.5.3.1. 插入 454
13.5.3.2. 更新 454
13.5.3.3. 删除 454
13.5.4. 创建或修改数据库对象 454
13.5.5. 执行存储过程 455
13.5.6. 处理大数据类型 455
13.5.6.1. 二进制类型 455
13.5.6.2. 字符类型 457
13.6. 附录 1: 主要接口 458
13.7. 附录 2: 调用示例 460
14. SQLAlchemy 框架开发指南 465
14.1. 介绍 465
14.2. 安装 SQLAlchemy 465
14.2.1. 安装 psycopg2 465
14.2.1.1. 安装 PanWeiDB V2.0-S3.0.0_Beta Libpq 465
14.2.1.2. 安装 python3 和 gcc 466
14.2.1.3. 安装 psycopg2 模块 466
14.2.2. 安装 SQLAlchemy 467
14.3. 使用 SQLAlchemy 连接数据库 467
14.3.1. 导入 467
14.3.2. 连接数据库 467
14.4. 执行 SQL 语句并处理结果 469
14.4.1. 创建或修改数据库对象 469
14.4.2. 删除数据库对象 470
14.4.3. 执行更新 470
14.4.3.1. 插入 470
14.4.3.2. 更新 471

14.4.3.3. 删除 471
14.4.4. 执行查询 471
14.4.4.1. 简单查询 471
14.4.4.2. 复杂条件查询 472
14.4.5. 获取结果 473
14.4.6. 执行存储过程 473
14.4.7. 处理数据类型 474
14.4.7.1. 二进制类型 474
14.4.7.2. 字符类型 475
14.4.7.3. 数字类型 477
14.4.7.4. 时间/日期类型 478
14.5. 连接 panweidb 时需注意的问题 480
14.5.1. 连接 480
14.5.2. 冲突时插入 480
14.5.2.1. 问题描述 480
14.5.2.2. 原因 481
14.5.2.3. 解决办法 481
14.6. 附录 1: 主要接口 482
14.7. 附录 2: 调用示例 485
15. SQLAPI++开发指南 489
15.1. 介绍 489
15.2. sqlapi++适配 489
15.2.1. gcc 安装 489
15.2.2. sqlapi++包下载 489
15.3. 使用 sqlapi++连接数据库 491
15.3.1. 包含头文件 491
15.3.2. 连接与断开 491
15.3.2.1. 连接数据库 491
15.3.2.2. 断开连接 491
15.4. 执行 SQL 语句并处理结果 492
15.4.1. 执行查询 492
15.4.1.1. 绑定参数 492
15.4.1.2. 输入参数 492
15.4.1.3. 执行查询 492
15.4.2. 获取结果 493
15.4.3. 执行更新 493
15.4.3.1. 插入 493
15.4.3.2. 更新 494
15.4.3.3. 删除 495
15.4.4. 创建或修改数据库对象 496
15.4.5. 执行存储过程 496
15.4.6. 处理数据类型 497
15.4.6.1. 二进制类型 497

15.4.6.2. 字符类型 498
15.4.6.3. 时间/日期类型 499
16. SQLObject 框架开发指南 502
16.1. 介绍 502
16.2. 安装 SQLObject 502
16.2.1. 安装 psycopg2 502
16.2.1.1. 安装 PanWeiDB V2.0-S3.0.0_Beta Libpq 502
16.2.1.2. 安装 python3 和 gcc 503
16.2.1.3. 安装 psycopg2 模块 503
16.2.2. 安装 SQLObject 504
16.3. 使用 SQLObject 连接数据库 504
16.3.1. 导入 504
16.3.2. 连接数据库 504
16.4. 执行 SQL 语句并处理结果 506
16.4.1. 创建或修改数据库对象 506
16.4.2. 删除数据库对象 506
16.4.3. 执行更新 507
16.4.3.1. 插入 507
16.4.3.2. 更新 507
16.4.3.3. 删除 507
16.4.4. 执行查询 507
16.4.4.1. 根据 ID 号查询 507
16.4.4.2. 根据条件查询 508
16.4.5. 获取结果 508
17. xorm 框架开发指南 509
17.1. 介绍 509
17.2. 安装 xorm 509
17.2.1. 安装 GoLang 509
17.2.2. 安装 xorm 510
17.2.3. 安装数据库对应的 GoLang 语言驱动 510
17.3. 使用 xorm 连接数据库 511
17.3.1. 导入 511
17.3.2. 连接数据库 511
17.3.2.1. 创建 Engine 引擎 511
17.3.2.2. 定义表结构体 512
17.4. 执行 SQL 语句并处理结果 512
17.4.1. 执行查询 512
17.4.1.1. 直接执行 512
17.4.1.2. 使用 xorm 中的方法查询 512
17.4.2. 获取结果 513
17.4.2.1. 遍历数组 513
17.4.3. 执行更新 513
17.4.3.1. 插入 514

17.4.3.2. 更新 514
17.4.3.3. 删除 514
17.4.4. 创建或修改数据库对象 514
17.4.5. 执行存储过程 515
17.4.6. 处理数据类型 516
17.4.6.1. 二进制类型 516
17.4.6.2. 字符类型 517
17.4.6.3. 数字类型 518
17.4.6.4. 时间/日期类型 521
17.5. 修改 xorm 框架代码以正常获取 DBVersion 524
17.5.1. 问题 524
17.5.2. 影响范围 524
17.5.3. 原因 524
17.5.4. 解决办法 525
17.6. 附录 1: 主要接口 526
17.7. 附录 2: 调用示例 530
17.7.1. 处理数据类型 533
17.7.1.1. 二进制类型 533
17.7.1.2. 字符类型 534
17.7.1.3. 数字类型 535
17.7.1.4. 时间/日期类型 536
17.8. 附录 1: 主要接口 537
17.9. 附录 2: 调用示例 540

1. .NET 接口开发指南

1.1. 介绍

PanWeiDB V2.0-S3.0.0_Beta 数据库通过 PostgreSQL 的.NET 驱动为使用.NET 开发的应用程序提供访问 PanWeiDB V2.0-S3.0.0_Beta 的接口。

1.1.1. PostgreSQL .NET 驱动版本

使用 PostgreSQL .NET 驱动访问 PanWeiDB V2.0-S3.0.0_Beta，建议使用 Npgsql 5.0.10 版本。官方下载路径为：

https://www.nuget.org/packages/Npgsql/5.0.10

1.1.2. Npgsql 与.NET 的兼容性

Npgsql 5.0.10 版本兼容.NET Core3.1 版本。

1.2. Npgsql 主要类与接口

.NET 驱动由具有建立和管理与 PanWeiDB V2.0-S3.0.0_Beta 数据库连接、执行 SQL 语句和对结果集进行储存管理的若干功能类组成。

- NpgsqlConnection

  与特定数据库的连接（会话）。在连接上下文中执行 SQL 语句并返回结果。

  示例：

```
var connString = "Host=myserver;Port=5432;Username=mylogin;Password=
```

```
mypass;Database=mydatabase";

var conn = new NpgsqlConnection(connString);
```

- NpgsqlCommand

  用于执行 SQL 语句并返回结果。

```
using (var cmd = new NpgsqlCommand("INSERT INTO data (some_field) V

ALUES (@p)", conn))

{

cmd.Parameters.AddWithValue("p", "Hello world");

cmd.ExecuteNonQueryAsync();

}
```

- NpgsqlDataReader

  存储执行 SQL 语句产生的结果集。

  示例：

```
using (var cmd = new NpgsqlCommand("SELECT some_field FROM data",

conn))

using (var reader = await cmd.ExecuteReaderAsync())

    while (await reader.ReadAsync())

        Console.WriteLine(reader.GetString(0));
```

1.3. 设置.NET 驱动

此处只介绍离线设置，在线设置方法请自行查询。

点击 visual studio 的“工具” -> “选项” -> “NuGet 包设置管理器” -> “程序包源” 设置 Npgsql 驱动所在的路径。然后右击项目，选择“管理 NuGet 程序包”，选择设置的程序包源名称，在“浏览”页面找到 Npgsql 驱动包点击安装，安装完成即设置成功。

1.4. 使用.NET 连接数据库

1.4.1. 导入

任何使用 PanWeiDB V2.0-S3.0.0_Beta 的.NET 程序都需要导入命名空间 Npgsql：

```
using Npgsql;
```

1.4.2. 连接数据库

1.4.2.1. 连接参数

使用 Npgsql 驱动访问 PanWeiDB V2.0-S3.0.0_Beta 的 URL 格式如下：

```
Host=myserver;Username=mylogin;Password=mypass;Database=mydatabase;Port=5432;
```

URL 中各参数的含义如下：

- Host

服务端的主机名，默认为 localhost。

- Port

  服务端监听的端口，默认为 5432。

- Database

  数据库名称，默认连接的数据库是与用户同名的数据库。比如连接用户为 panweidb,如果不指定 database 参数，则默认连接到 panweidb 这个数据库。

- Username

  数据库用户名

- Password

  数据库用户的密码

1.4.2.2. 获取连接

使用 new NpgsqlConnection(connString)获取连接：

```
var connString = "Host=myserver;Username=mylogin;Password=mypass;Dat
abase=mydatabase";
var conn = new NpgsqlConnection(connString);
conn.Open();
```

1.4.2.3. 关闭连接

关闭连接时调用 NpgsqlConnection 的 Close()方法即可：

```
......
conn.Close();
```

1.5. 执行 SQL 语句并处理结果

向数据库发出执行 SQL 的请求时，需要使用 NpgsqlCommand 实例发出查询，服务端执行完成后将返回一个 NpgsqlDataReader 实例，该实例包括查询的结果。

1.5.1. 执行查询

使用 NpgsqlCommand 执行查询：

```
/将返回两个结果集
using (var cmd = new NpgsqlCommand("SELECT ...; SELECT ...",conn))
using (var reader = cmd.ExecuteReader())
{
while (reader.Read())
{
// Read first resultset
        }
        reader.NextResult();
        while (reader.Read())
{
// Read second resultset
}
}
```

1.5.2. 读取结果

1.5.2.1. 获取单一结果

执行查询，并返回查询所返回的结果集中第一行的第一列。 所有其他的列和行将被忽略。此方法适合使用 ExecuteScalar 方法从数据库中检索单个值（例如一个聚合值），比如统计数量。

- 对于查询单个字段，不包括函数的 select 语句，例如 select a 字段 from 表 A ,如果不存在值， DbCommand.ExecuteScalar 方法的返回值是 null，调用者需要使用 result == null 来判断。
- 对于使用函数的 select 语句，例如 select sum(a 字段) from 表 A,如果不存在值，DbCommand.ExceuteScalar 方法的返回值是 DBNull.Value，调用者需要用 result is DBNull 或者 result == DBNull.Value 来判断。

```
    string sql = "select count(*) from test";
using (var cmd = new NpgsqlCommand(sql,conn))
{
int count = Convert.ToInt32(cmd.ExecuteScalar());     //将结果转换成整数
}
```

1.5.2.2. 使用 DataReader 获取结果

驱动程序获取某个查询的结果集，并以流的方式返回给客户端。示例：

```
//将返回两个结果集
using (var cmd = new NpgsqlCommand("SELECT ...; SELECT ...",conn))
using (NpgsqlDataReader reader = cmd.ExecuteReader())
```

```
{
    while (reader.Read())
{
        // Read first resultset
        for (i = 0; i < reader.FieldCount; i++)
{
            Console.Write("{0} \t", reader[i]); //获得字段名
string  fieldType =  reader.GetFieldType(i).ToString();  //获得指定字段的类型
        }
        var fieldValue = reader["fieldName"]; // 获得指定字段的值（fieldNa
me 是表的某个字段）
}

    reader.NextResult();
    while (reader.Read()) {
        // Read second resultset
       ......
    }
    reader.Close();
}
```

1.5.3. 使用 NpgsqlCommand 对象

在使用 NpgsqlCommand，需要注意以下问题：

NpgsqlCommand 传入 SQL 命令是在构造函数中进行的，因此最好一次执行对应一个 NpgsqlCommand 实例。为了避免.NET 垃圾回收不及时，可以指定空间回收时机使得命令执行完成且结果获取之后释放 NpgsqlCommand 空间，可以通过 using(……)方式或者 try-finally 来指定：

```
using (var cmd = new NpgsqlCommand( “……", conn))

using (NpgsqlDataReader reader = cmd.ExecuteReader()){

……

}
```

或者

```
var cmd = new NpgsqlCommand( “……");

Try

{

NpgsqlDataReader reader = cmd.ExecuteReader();

Try

{

……

}

Finally

{
```

```
                if (reader != null)
((IDisposable)reader).Dispose();
}
}
Finally
{
if (cmd != null)
((IDisposable)cmd).Dispose();
}
```

通过上述两种方式都可以进行及时的空间回收，不会造成空间浪费，并且可以保证使用安全。另外，也可以进行 NpgsqlCommand 的复用，即可以多次使用一个 NpgsqlCommand 实例，可以在打开连接时创建 NpgsqlCommand 然后在连接的整个生命周期内使用它，每次在执行之前可以修改 NpgsqlCommand 的参数：

```
cmd.Connection = conn;    //设置连接
    cmd.CommandType = CommandType.Text;     //指明命令类型
    cmd.CommandText = "selelct * from data";    //SQL 语句
NpgsqlDataReader reader = cmd.ExecuteReader();
```

但是这用方式要注意使用新连接时，要手动断掉旧链接。

- 如果在处理 NpgsqlDataReader 时需要执行查询，可以创建另一个 NpgsqlCommand 来执行。
- 如果使用多线程 ，并且有多个线程正在使用数据库，则每个线程必须使用

单独的 NpgsqlCommand.，建议每个命令使用单独的 NpgsqlCommand，保证安全。

- 使用完 NpgsqlCommand 应该及时释放空间，以及及时断开连接。

1.5.4. 使用 NpgsqlDataReader 对象

使用 NpgsqlDataReader 接口时必须考虑下面的问题：

- NpgsqlDataReader 对象是 NpgsqlCommand 对象在执行 SQL 语句后返回的一个结果对象，当结果集有多个时，默认指向的是第一个结果集，如果要访问下一个结果集时，需要使用 NpgsqlDataReader 的 NextResult 方法；在某一个结果集中，通过不断调用 NpgsqlDataReader 的 Read 方法，依次获取该结果集的一条记录，构建 NpgsqlDataReader 对象的结构，可以通过 NpgsqlDataReader 下标数字的方法获取字段信息，通过 NpgsqlDataReader 下标字段名的方法获取值。
- NpgsqlDataReader 的 Read 方法，保证了对一个结果集的一条记录只访问一次。
- 在结束对一个 NpgsqlDataReader 的处理后，必须调用 close()来关闭它，并及时释放空间。
- 为了保证使用安全和线程安全，最好是一个 NpgsqlCommand 对应一个 NpgsqlDataReader 对象，其实一个 NpgsqlDataReader 对象就是由 NpgsqlCommand 执行产生的。当使用那个创建 NpgsqlDataReader 的 NpgsqlCommand 做另一个查询请求， 当前打开的 NpgsqlDataReader 实例不会自动关闭，此处小心使用。

1.5.5. 执行更新

要更改数据(执行一个 insert，update 或者 delete)，可以使用 NpgsqlCommand 对象的 ExecuteNonQuery() 方法． ExecuteNonQuery()

方法类似于发出 select 语句的 ExecuteReader()方法，不过，它不返回 NpgsqlDataReader ，它返回的是：

- 对于 Update、insert、Delete 语句执行成功是返回值为该命令所影响的行数，如果影响的行数是 0，则返回值就是 0；
- 对于所有其他类型的语句，返回值为-1；
- 如果发生回滚，返回值也为-1；

1.5.5.1. 删除

```
string sql = "delete from test where id=1";
NpgsqlCommand cmd = new NpgsqlCommand(sql, conn);
cmd .ExecuteNonQuery();
```

1.5.5.2. 更新

```
string sql = "update test set price=300 where id=1";
NpgsqlCommand cmd = new NpgsqlCommand(sql, conn);
cmd .ExecuteNonQuery();
```

1.5.5.3. 插入

插入单行：

```
string sql = "insert into test values (1,200)";
NpgsqlCommand cmd = new NpgsqlCommand(sql, conn);
cmd .ExecuteNonQuery();
```

其中，SQL 语句的参数，可以通过格式化字符串的方式去设置与替换，这是

最简单的调用方式。当然也可以使用指定参数的方式去调用，请参见 1.6 调用存储过程。

1.5.6. 创建或修改数据库对象

要创建、更改或者删除一个类似表或者视图这样的数据库对象， 仍然可以使用 ExecuteNonQuery() 方法，但是返回值只返回-1。

例如 DROP 表：

```
string sql = "DROP TABLE mytable";
NpgsqlCommand cmd = new NpgsqlCommand(sql, conn);
cmd .ExecuteNonQuery();
```

DROP 数据库：

```
string sql = "DROP DATABASE mydatabase";
NpgsqlCommand cmd = new NpgsqlCommand(sql, conn);
cmd .ExecuteNonQuery(); s
```

1.5.7. 小结

总结来说，SQL 语句的主要执行对象就是 NpgsqlCommand 对象，只不过具体的执行过程不同，针对查询功能，调用的是 NpgsqlCommand.ExecuteScalar 功能或者 NpgsqlCommand.ExecuteReader 功能，查询返回的结果会返回一个 NpgsqlDataReader 对象，可以从这个对象里去获取数据；而

其它类型的 SQL，则使用 NpgsqlCommand.ExecuteNonQuery 功能，在针对数据记录的增删改操作，会返回受影响的记录数量，其它操作比如针对数据库对象的操作，返回-1。

1.6. 调用存储过程

在调用存储过程时，也是基于 NpgsqlCommand 来进行，只不过是要指定过程名以及参数，调用方法与跟 SQL 中的存储过程调用类似。

假设有存储过程：

```
CREATE OR REPLACE FUNCTION "public"."userreg" (p_passport varchar, p_n
ickname varchar, out p_password varchar, out p_userid integer) RETURNS
 record AS
.....
```

此存储过程名称为"userreg" ，四个参数，类型分别为 varchar，varchar，varchar 以及 integer，其中前两个参数是 in 参数，后两个参数是 out 参数。

调用存储过程示例：

```
NpgsqlConnection conn = new NpgsqlConnection();
    conn.ConnectionString = ".....";  //连接参数
    NpgsqlCommand cmd = new NpgsqlCommand();
  /*设置参数*/
    cmd.Connection = conn;    //设置连接
    cmd.CommandType = CommandType.StoredProcedure;     //指明为存储
过程
```

```
    cmd.CommandText = "userreg";    //存储过程名称
    NpgsqlParameter p1 = new NpgsqlParameter("p_passport", DbType.Str
ing, 50);    //参数名，参数类型，长度，下同
    NpgsqlParameter p2 = new NpgsqlParameter("p_nickname", DbType.St
ring, 50);
    NpgsqlParameter p3 = new NpgsqlParameter("p_nickname", DbType.St
ring, 50);
    NpgsqlParameter p4 = new NpgsqlParameter("p_userid", DbType.Int3
2);
    p1.Value = ......;     //对于 in 参数，设置参数值
    p2.Value = ......;
    p3.Direction = ParameterDirection.Output;     //对于 out 参数，指明参数方
向
p4.Direction = ParameterDirection.Output;
cmd.Parameters.Add(p1);    //给 cmd 添加参数
cmd.Parameters.Add(p2);
cmd.Parameters.Add(p3);
cmd.Parameters.Add(p4);
conn.Open();    //连接数据库
NpgsqlDataReader dr =  cmd.ExecuteReader();    //执行并返回结果
.....  //结果处理
```

1.7. 处理大数据类型

1.7.1. 二进制类型

PanWeiDB V2.0-S3.0.0_Beta 提供两种不同的方法存储二进制数据。 二进制数据可以使用二进制数据类型 BYTEA 存储在表中，或者使用大对象特性以一种特殊的格式将二进制数据存储在一个独立的表中， 然后通过在表中保存一个类型为 OID 的值来引用该表。以下为这两种方法的使用示例。

1.7.1.1. BYTEA

使用 BYTEA 类型存储二进制数据，可以用.NET 自身提供的 byte[]类型的操作方式进行读写。

例如：

```
CREATE TABLE images (imgname text, img bytea);
```

1.7.2. 插入记录

可以通过直接将参数赋值，直接将数据传入 PanWeiDB V2.0-S3.0.0_Beta。

```
FileStream fs = new FileStream(fileUrl, FileMode.Open, FileAccess.Read);
    try
    {
        byte[] buffur = new byte[fs.Length];
        fs.Read(buffur, 0, (int)fs.Length);    //读取二进制文件
```

```
        Int Len = (int)fs.Length;
        //关闭资源
        fs.Close();

        ..... //建立连接

NpgsqlCommand cmd = new NpgsqlCommand();
        cmd.CommandType = CommandType.Text;     //指明为文本
        cmd.CommandText = "insert into image values ('I1', @content)";
//SQL 语句

        NpgsqlParameter p = new NpgsqlParameter("@content", DbType.I
mage, Len);
        p.Value = buffur ;
        cmd.Parameters.Add(p);    //给 cmd 添加参数

        cmd.ExecuteNonQuery();      //执行
        ......
    }
```

还有一种方式，就是上述提到的通过 DataSet 与数据库连接的方式，通过 DataSet，可以直接将数据传入数据库：

```
DataSet tempDataSet=new DataSet();
string sql = "SELECT * FROM images  WHERE 1=0";    //不会有内容，只有表
```

```
头
NpgsqlDataAdapter objAdapter = new NpgsqlDataAdapter(sql, conn);
objAdapter .Fill(tempDataSet,” temp” ); //与数据库建立了连接，并在 tempDat
aSet 创建一个空表

//在 DataSet 的表中插入一条记录
DataRow tempDataRow = tempDataSet.Tables[“temp”].NewRow(); //创建表
的新行
tempDataRow["imgname "] =” Image1” ; //给新行的 imgname 赋值
tempDataRow["img "] =buffur ; //给新行的 img 赋值，为二进制数据

//更新至数据库
tempDataSet.Tables[ “temp” ].Rows.Add(tempDataRow);
stempAdapter.Update(tempDataSet);
```

1.7.3. 读取记录

```
using(NpgsqlCommand cmd = new NpgsqlCommand("select * from image
s '", conn));
using(NpgsqlDataReader dr = cmd.ExecuteReader();
{
While(dr.Read())
{
byte[] imageData = (byte[])dr[ “img ” ];
```

```
                ......                    //其它处理
}
}
```

在这里，二进制数据被读取为 byte[]。

1.7.3.1. 大对象功能

使用大对象功能存储二进制数据，需要开启事务来进行读写，使用到 LargeObjectManager 类对象与 LargeObject 类对象 。

例如：

```
CREATE TABLE imageslo (imgname text, imgoid oid);
```

1.7.4. 插入记录

```
.....     //建立 conn 连接
int noid;
using (NpgsqlTransaction trans = conn.BeginTransaction())
{     //开启事务
try
{
LargeObjectManager lom = new LargeObjectManager(conn);    //创建大对
象管理器
byte[] buf = new byte[8192];
int readIn = 0;
```

```
int noid = lom.Create(LargeObjectManager.WRITE);    //创建大对象 oid
LargeObject lo = lom.Open(noid, LargeObjectManager.WRITE);   //创建大对
象
using (FileStream fs = File.OpenRead(fileName)){    //读取文件二进制流
while ((readIn = fs.Read(buf, 0, buf.Length)) > 0)
{    //写入大对象
lo.Write(buf, 0, readIn);
}
}
Lo.Close();    //写完大对象，关闭

string sql = string.Format( "insert into test values ({0},{1})", “Image1”,noid);
  //格式化字符串构建语句
NpgsqlCommand cmd = new NpgsqlCommand(sql, conn);
cmd .ExecuteNonQuery();     //执行插入记录

trans.Commit();    //事务提交
}
```

1.7.5. 读取记录

```
.....     //建立 conn 连接

using (NpgsqlTransaction trans = conn.BeginTransaction())
```

```
{    //开启事务
/*获取大对象 oid*/
int  loOid = 0;
string sql = "select  imgoid from imageslo where ......";
using (var cmd = new NpgsqlCommand(sql,conn))
{
loOid  = Convert.ToInt32(objCommand.ExecuteScalar());      //将结果转换成
整数
}

LargeObjectManager lom = new LargeObjectManager(conn);      //创建大
对象管理器
LargeObject lo = lom.Open(loOid , LargeObjectManager.READ);    //打开大
对象文件
using (FileStream fs = File.OpenWrite(@"MyFile"))
{    //创建目的文件，将数据写入
int tempmax = lo.Size();
int tempread = 0;
int temppos = 0;

do
{
byte[] buf = lo.Read(8192);      //一次性从大对象读取 8192 个字节
```

```
tempread = buf.Length;
if (tempread > 0)
{
fs.Write(buf, 0, tempread);
temppos += tempread;
lo.Seek(temppos);      //当前读取位置更新
}
} while (tempread > 0);
fs.Flush();
}

lo.Close(); //读取完毕，关闭大对象
trans.Commit();  //提交事务
}
```

1.7.6. 字符串类型

PanWeiDB V2.0-S3.0.0_Beta 中 TEXT 类型与 VARCHAR 类型都是可变长的字符串类型，区别在于 VARCHAR 类型通过 VARCHAR(n)中的 n 来限制最大长度，而 TEXT 类型没有。TEXT 类型与 VARCHAR 类型几乎没有性能差别，TEXT 类型最多可存储 1G 数据 。

使用 TEXT 类型存储数据时，可用.NET 中的 Convert 功能将二进制数据与文本数据之间来进行转换。例如：

```
CREATE TABLE images (id int, msg text);
```

1.7.6.1. 插入记录

```
Stream s = File.Open(MapPath("MyImage.jpg"), FileMode.Open);
int leng = (int)s.Length;
byte[] by = new byte[leng];
s.Read(by, 0, leng);//把图片读到字节数组中
s.Close();
string str = Convert.ToBase64String(by);//把字节数组转换成字符串

string sql = string.Format( "insert into test values ({0},{1})", “Image1”,noid);
  //格式化字符串构建语句
NpgsqlCommand cmd = new NpgsqlCommand(sql, conn);
cmd .ExecuteNonQuery();     //执行插入记录
```

1.7.6.2. 读取记录

在读取记录时，按照获取字符串的方式，就可以获取到在数据库中的 TEXT 数据。

```
using (var cmd = new NpgsqlCommand("SELECT * FROM images WHER
E id = 1;",conn))
using (NpgsqlDataReader reader = cmd.ExecuteReader())
{
while (reader.Read())
```

```
{
// Read
            string   fieldValue = reader["msg "]; // 获得指定字段的值
            ......   //后期处理
}
    }
reader.Close();
```

在这里，TEXT 数据被读取为 string。

1.8. 总结

使用 C#接口与 PanWeiDB V2.0-S3.0.0_Beta 进行交互时，主要有三种方式：

- 通过命令直接交互
- 通过设置参数进行交互
- 通过 DataSet 绑定进行交互

通过命令直接交互是最简单最常用的方式，在普通的增删改查应用场景中最为适用，主要流程就是直接定义 SQL 语句，让 NpgsqlCommand 直接执行，在定义 SQL 语句时，可能会用到格式化字符串的方式，去更改参数。

通过设置参数进行交互，主要用于调用存储过程，在存储过程中需要指定参数，本质上也是给 NpgsqlCommand 指定了一些参数，再去执行。在二进制 byte 方式存储时，由于无法格式化二进制数据，因此也采用这种方式。当然一般情况下也可以采用指定参数方式。

通过 DataSet 绑定进行交互，是.NET 提供了一个类对象 DataSet，以及 Npgsql 驱动工具提供了 DataSet 与数据库的适配器，使得 DataSet 与数据库可以直接进行数据交互，省去了细节的操作。这一方法可以应用于.NET 的 DataGridView 控件的显示，使得在客户端显示数据的数据或者查询结果非常方便。当前，其它的数据操作也可以用这种方式，比如介绍的二进制 byte 方式存储数据。

综上所述，以上三种方式都可以应用于具体场景，只是方便程度与侧重不同。

2. Django 框架开发指南

2.1. 介绍

Django 是一个由 Python 编写的一个开放源代码的 Web 应用框架。

使用 Django，只要很少的代码，Python 的程序开发人员就可以轻松地完成一个正式网站所需要的大部分内容，并进一步开发出全功能的 Web 服务 Django 本身基于 MVC 模型，即 Model（模型）+ View（视图）+ Controller（控制器）设计模式，MVC 模式使后续对程序的修改和扩展简化，并且使程序某一部分的重复利用成为可能。

2.2. 安装 Django

安装 Django 需要安装 Django 包和数据库对应的 python 语言驱动 psycopg2。

2.2.1. 安装 psycopg2

安装 psycopg2 模块前需要先装好 PanWeiDB V2.0-S3.0.0_Beta Libpq，并配置环境变量。

2.2.1.1. 安装 PanWeiDB V2.0-S3.0.0_Beta Libpq

安装 Libpq 有两种方式。

1.通过提供的 Libpq 安装包安装

解压 PanWeiDB V2.0-S3.0.0_Beta 提供的 Libpq，新建/usr/local/libpq 目录，将解压后的 lib 目录拷贝到/usr/local/libpq 目录下，配置环境变量如下：

```
export LD_LIBRARY_PATH=/usr/local/libpq/lib:$LD_LIBRARY_PATH
```

2.通过安装 PanWeiDB V2.0-S3.0.0_Beta 安装包安装

按照安装说明安装 PanWeiDB V2.0-S3.0.0_Beta，并配置环境变量。如果是使用安装包自带安装程序进行安装，则环境变量会自动配置无需手动配置；如果是手动安装数据库，则需配置以下环境变量（只作为参数，应按实际的环境进行配置）：

```
export GAUSSHOME=/usr/local/panweidb

export PATH=$GAUSSHOME/bin:$PATH

export LD_LIBRARY_PATH=$GAUSSHOME/lib:$LD_LIBRARY_PATH
```

2.2.1.2. 安装 python3 和 gcc

推荐使用 python3，安装命令如下：

```
yum install -y python3 python3-devel gcc
```

2.2.1.3. 安装 psycopg2 模块

推荐使用源码包安装，获取 psycopg2 源码包：

```
# 1.卸载已安装的 psycopg2（如果没有安装则跳过，此步骤可能需要 root 用户执行）

pip3 uninstall psycopg2 -y
```

```
# 2.上传并解压源码包
unzip Psycopg-panwei.zip
# 3.安装
cd Psycopg-panwei
python3 setup.py build
# 此步骤可能需要 root 用户执行
python3 setup.py install

# 4.检查安装结果
进入 python3 命令行，执行 import psycopg2 可以正常导入即表示安装成功。
```

2.2.2. 安装 Django

推荐使用源码包安装，安装至本地可以通过命令：

```
python -m pip install -e Django/# Django/为源码包存放的路径
```

2.2.3. Django 与 python 版本对应关系

Django 版本	Python 版本
2.2	3.5，3.6，3.7，3.8（2.2.8 添加），3.9（2.2.17 添加）
3.1	3.6，3.7，3.8，3.9（3.1.3 添加）

3.2	3.6, 3.7, 3.8, 3.9, 3.10 (在 3.2.9 中就已经加入了)
4.0	3.8，3.9，3.10
4.1	3.8, 3.9, 3.10, 3.11 (4.1.3 添加)

2.3. 使用 Django 连接数据库

2.3.1. 创建 Django 项目

打开命令行，cd 到一个你想放置你代码的目录，然后运行以下命令：

```
python -m django startproject myproject
```

stratproject 会创建如下文件：

```
myproject/
    manage.py
    myproject/
        __init__.py
        settings.py
        urls.py
        asgi.py
        wsgi.py
```

这些目录和文件的用处是：

- 最外层的 myproject/ 根目录只是你项目的容器， 根目录名称对 Django

没有影响，你可以将它重命名为任何你喜欢的名称。

- manage.py：一个让你用各种方式管理 Django 项目的命令行工具。
- 里面一层的 myproject/ 目录包含你的项目，它是一个纯 Python 包。它的名字就是当你引用它内部任何东西时需要用到的 Python 包名。(比如 myproject.urls).
- myproject/__init__.py：一个空文件，告诉 Python 这个目录应该被认为是一个 Python 包。
- myproject/settings.py：Django 项目的配置文件。
- myproject/urls.py：Django 项目的 URL 声明，就像你网站的“目录”
- myproject/asgi.py:作为你的项目的运行在 ASGI 兼容的 Web 服务器上的入口。
- myproject/wsgi.py:作为你的项目的运行在 WSGI 兼容的 Web 服务器上的入口。

2.3.2. 创建 app

进入项目根目录，创建一个名称为 myapp 的 app。

```
python manage.py startapp myapp
```

2.3.3. 修改 DATABASES 配置

打开 myproject 目录下的 settings.py 配置文件，把默认的 sqlite3 数据库换成 panweidb 数据库，使用 Django 连接 panweidb 时需要将连接引擎设置为 panweidb：

```
DATABASES = {
    'default': {
```

```
        'ENGINE': 'django.db.backends.panweidb',# 数据库引擎
        'NAME': 'panweidb',# 数据库名字
        'USER': 'pwadmin',# 数据库用户名
        'PASSWORD': 'password',# 密码
        'HOST': '127.0.0.1',# 本地主机或数据库服务器的 ip
        'PORT': '5432',# 连接端口
    }
}
```

并把 app 加入到 INSTALLED_APPS 列表里：

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'myapp' # 加入 app
]
```

2.3.4. 启动项目

```
python manage.py runserver
```

```
# 指定端口

python manage.py runserver ip:端口
```

2.4. 执行 SQL 语句并处理结果

2.4.1. 创建或修改数据库对象

Django 采用模型映射数据库表，模型准确且唯一的描述了数据。它包含了储存的数据的字段和行为。一般来说，每一个模型都映射一张数据库表。

- 每个模型都是一个 Python 的类，这些类继承 django.db.models.Model
- 模型类的每个属性都相当于一个数据库的字段。

创建模型类，生成对应的数据库表格。

```
from django.db import models


class Person(models.Model):
    id = models.AutoField(primary_key=True)
    name = models.CharField(max_length=32)
    age = models.IntegerField()
    birthday = models.DateTimeField()  # 日期


    class Meta():
        db_table = 'person'  # 修改表格名称为 person
```

然后在命令行下执行：

```
python manage.py makemigrations # 同步数据库

python manage.py migrate # 写入数据库
```

2.4.2. 执行更新

2.4.2.1. 插入

```
# save 方法

u = Person(name='user1',age='22',birthday=datetime(2000,1,1))

u.save()


# create 方法

u = Person.objects.create(name = 'user2',age='21',birthday=datetime(2001,1,

1))
```

在 Django 中向数据库中插入多条数据时，每使用 save 或 create 方法保存一条就会执行一次 SQL。而 Django 提供的 bulk_create 方法可以一次 SQL 添加多条数据，效率要高很多，如下所示：

```
# 内存生成多个对象实例

user = [Person(name='user3',age='1',birthday=datetime(2021,1,1)),

Person(name='user4',age='2',birthday=datetime(2020,3,28)),

Person(name='user5',age='3',birthday=datetime(2019,2,26))]


# 执行一次 SQL 插入数据
```

```
Person.objects.bulk_create(user)
```

为了避免重复创建数据表中已存在的条目，Django 还提供了 get_or_create 方法。它会返回查询到的或新建的模型对象实例，还会返回这个对象实例是否是刚刚创建的。

```
obj, created = Person.objects.get_or_create(name = '用户 6',
age='32',birthday=datetime(1990,1,1))
```

2.4.2.2. 更新

```
# 调用 save()函数更新
obj = Person.objects.first()
obj.name = 'user7'
obj.save()

# 使用模型类.objects.filter().update(),会返回受影响的行数
num = Person.objects.filter(birthday=date(2019,2,26).update(age='4')
print(num)
```

2.4.2.3. 删除

```
obj = Person.objects.filter(id=1)
print(obj)
obj.delete()
```

2.4.3. 执行查询

2.4.3.1. 基础查询

get 查询单一结果，如果不存在会抛出模型类.DoesNotExist 异常。all 查询多个结果。count 查询结果数量。

```
obj = Person.objects.get(id=2)
print(obj)
obj = Person.objects.get(pk=3)
print(obj)
obj = Person.objects.all()
print(obj)
num = Person.objects.count()
print(num)
```

2.4.3.2. 过滤查询

实现 SQL 中的 where 功能，包括

- filter 过滤出多个结果
- exclude 排除掉符合条件剩下的结果
- get 过滤单一结果

对于过滤条件的使用，上述三个方法相同，故仅以 filter 进行示例。
过滤条件的语法：属性名称__比较运算符=值

```
obj = Person.objects.filter(id__exact=2) # exact:表示判等
print(obj)
obj = Person.objects.filter(name__contains='user') # contains:是否包含
print(obj)
obj = Person.objects.filter(id__gt=1) # 查询 id 大于 3 的用户
```

```
print(obj)

obj = Person.objects.filter(birthday__year=2001) # 查询 2001 年出生的用户

print(obj)
```

2.4.4. 获取结果

查询集获取的结果需要遍历取值输出。

```
for i in obj:

print(i.id,i.name,i.age,i.birthday)
```

2.4.5. 执行存储过程

下面的例子展示了如何调用存储过程。

```
from django.db import connection

cursor = connection.cursor()

sql_procedure = " CREATE OR REPLACE PROCEDURE pro_test(a integer,inou

t b integer) as \

                    begin \

                        b := a+b; \

                    end"

cursor.execute(sql_procedure)

cursor.execute("call pro_test(1,5)")

result = cursor.fetchone()
```

下面的例子展示了如何调用函数。

```
from django.db import connection
```

```
cursor = connection.cursor()
sql_function = "CREATE OR REPLACE FUNCTION func_test (a int,out b int)
RETURNS int AS \
                $$ \
                begin \
                    b := a+2; \
                return ;  \
                end; \
                $$ LANGUAGE 'plpgsql';"
cursor.execute(sql_function)
cursor.execute("select func_test(1)")
result = cursor.fetchone()
```

2.4.6. 处理数据类型

2.4.6.1. 二进制类型

PanWeiDB V2.0-S3.0.0_Beta 提供两种不同的方法存储二进制数据。二进制数据可以使用二进制数据类型 BYTEA 存储在表中，或者使用大对象特性以一种特殊的格式将二进制数据存储在一个独立的表中， 然后通过在表中保存一个类型为 OID 的值来引用该表。以下为 BYTEA 类型使用示例。

❖ 创建表

```
class ByteaTest(models.Model):
id = models.AutoField(primary_key=True)
```

```
col1 = models.BinaryField()


class Meta():
db_table = 'byteatest'
```

❖ 插入数据

```
data = bytes(range(256))
u = ByteaTest(col1=data)
u.save()
```

❖ 读取数据

```
b1 = ByteaTest.objects.get(id=1)
print(b1.id,b1.col1.tobytes())
assert b1.col1.tobytes() == data
```

2.4.6.2. 字符类型

PanWeiDB V2.0-S3.0.0_Beta 中 TEXT 类型与 VARCHAR 类型都是可变长的字符类型，区别在于 VARCHAR 类型通过 VARCHAR(n)中的 n 来限制最大长度，而 TEXT 类型没有。TEXT 类型与 VARCHAR 类型几乎没有性能差别，TEXT 类型最多可存储 1G 数据 。

1）创建表

```
class StringTest(models.Model):
id = models.AutoField(primary_key=True)
col1 = models.CharField(max_length=256)# varchar 类型
col2 = models.TextField()# text 类型
```

```
class Meta():
db_table = 'stringtest'
```

2）插入数据

```
a='test1'
b='test2'
u = StringTest(col1=a,col2=b)
u.save()
```

3）读取数据

```
s1 = StringTest.objects.get(id=1)
print(s1.id,s1.col1,s1.col2)
assert s1.col1==a
assert s1.col2==b
```

2.4.6.3. 数字类型

数字类型有整数类型、任意精度数字类型、浮点类型和序数类型。下面用整数类型 integer、任意精度数字类型 numeric 和 double precision 举例说明。

1）创建表

```
class NumberTest(models.Model):
id = models.AutoField(primary_key=True)
```

```
col1 = models.IntegerField()# integer 类型
col2 = models.DecimalField(max_digits=4,decimal_places=2)# numeric 类型
col3 = model.FloatField()# double precision 类型


class Meta():
db_table = 'numbertest'
```

2) 插入数据

```
a = 1
b= decimal.Decimal(2.141)
c = 3.1415926
u = NumberTest(col1=a,col2=b,col3=c)
u.save()
```

3) 读取数据

```
n1 = NumberTest.objects.get(id=1)
print(n1.col1)
print(n1.col2)
print(n1.col3)
```

2.4.6.4. 时间/日期类型

日期/时间类型有 timestamp 、date、time 、interval ； 下面以 timestamp 、date、time 举例说明。

1）创建表

```
class DatetimeTest(models.Model):

id = models.AutoField(primary_key=True)

col1 = models.DateTimeField()# timestamp 类型

col2 = models.DateField()# date 类型

col3 = models.TimeField()# time 类型



class Meta():

db_table = 'datetimetest'
```

2）插入数据

```
_now = datetime.now().replace(microsecond = 0)

u = DatetimeTest(col1=_now,col2=_now,col3=_now.time())
```

3）读取数据

```
d1 = DatetimeTest.objects.get(id=1)

print(d1.col1,d1.col2,d1.col3)

assert isinstance(d1.col1,datetime)

assert isinstance(d1.col2,date)

assert isinstance(dt.col3,time)
```

2.5. 连接 panweidb 时需注意的问题

2.5.1. 连接

2.5.1.1. 问题描述

在将 settings.py 文件中的数据库引擎设置为 postgresql，使用 Django 连接 panweidb 数据库时，会报如下错误：

```
Traceback (most recent call last):
  File "/workspace/python/Django/Django-4.0.1/django/db/backends/utils.py", line 83, in _execute
    return self.cursor.execute(sql)
psycopg2.errors.UndefinedColumn: column c.relispartition does not exist
LINE 3:             CASE WHEN c.relispartition THEN 'p' WHEN c.relki...
                             ^
CONTEXT:  referenced column: case
```

2.5.1.2. 原因

Django 在连接数据库时会检索扫描数据库中存在的表，inspectdb 的时候扫描了 relispartition 这个参数的值，但是 relispartition 是在 postgresql10 引入的，panweidb 是基于 postgresql9.2.4，尚未这个参数，所以报错。此外，panweidb 没有兼容 with ordinality 语法，需将其改为相应的窗口函数 row_number()。

2.5.1.3. 解决办法

如果在连接数据库时不想使用 panweidb 提供的 Django 源码包，也可根据以下步骤自行修改源码，以适配 panweidb。

1.复制 django.db.backends 下的 postgresql 文件夹并重命名为 panweidb，这是为了同时兼容 postgresql 和 panweidb。

2.将 Postgresql 中查表时检测分区表 relispartition 属性改为 panweidb 判断分区表的属性 parttype，打开 django.db.backends.panweidb.introspection.py 文件：

（1）找到 get_table_list(self, cursor)函数，修改其执行的 SQL 语句，改动如下：

```
47      def get_table_list(self, cursor):
48          """Return a list of table and view names in the current database."""
49          cursor.execute("""
50              SELECT c.relname,
51              CASE WHEN c.parttype <> 'n' THEN 'p' WHEN c.relkind IN ('m', 'v') THEN 'v' ELSE 't' END
52              FROM pg_catalog.pg_class c
53              LEFT JOIN pg_catalog.pg_namespace n ON n.oid = c.relnamespace
54              WHERE (c.relkind IN ('f', 'm', 'r', 'v') OR c.parttype <> 'n')
55                  AND n.nspname NOT IN ('pg_catalog', 'pg_toast')
56                  AND pg_catalog.pg_table_is_visible(c.oid)
57          """)
58          return [TableInfo(*row) for row in cursor.fetchall() if row[0] not in self.ignored_tables]
59
```

（2）找到 get_table_description(self, cursor,table_name)函数，修改其执行的 SQL 语句，改动如下：

```
60      def get_table_description(self, cursor, table_name):
61          """
62          Return a description of the table with the DB-API cursor.description
63          interface.
64          """
65          # Query the pg_catalog tables as cursor.description does not reliably
66          # return the nullable property and information_schema.columns does not
67          # contain details of materialized views.
68          cursor.execute("""
69              SELECT
70                  a.attname AS column_name,
71                  NOT (a.attnotnull OR (t.typtype = 'd' AND t.typnotnull)) AS is_nullable,
72                  pg_get_expr(ad.adbin, ad.adrelid) AS column_default,
73                  CASE WHEN collname = 'default' THEN NULL ELSE collname END AS collation
74              FROM pg_attribute a
75              LEFT JOIN pg_attrdef ad ON a.attrelid = ad.adrelid AND a.attnum = ad.adnum
76              LEFT JOIN pg_collation co ON a.attcollation = co.oid
77              JOIN pg_type t ON a.atttypid = t.oid
78              JOIN pg_class c ON a.attrelid = c.oid
79              JOIN pg_namespace n ON c.relnamespace = n.oid
80              WHERE (c.relkind IN ('f', 'm', 'r', 'v') OR c.parttype <> 'n')
81                  AND c.relname = %s
82                  AND n.nspname NOT IN ('pg_catalog', 'pg_toast')
83                  AND pg_catalog.pg_table_is_visible(c.oid)
84          """, [table_name])
85          field_map = {line[0]: line[1:] for line in cursor.fetchall()}
86          cursor.execute("SELECT * FROM %s LIMIT 1" % self.connection.ops.quote_name(table_name))
87          return [
88              FieldInfo(
89                  line.name,
90                  line.type_code,
91                  line.display_size,
92                  line.internal_size,
93                  line.precision,
94                  line.scale,
95                  *field_map[line.name],
96              )
97              for line in cursor.description
98          ]
```

3.panweidb 没有兼容 with ordinality 语法，将其改为相应的窗口函数 row_number()

（1）找到 get_constraints(self, cursor, table_name)函数，修改其执行的 SQL 语句，改动如下：

```
142     def get_constraints(self, cursor, table_name):
143         """
144         Retrieve any constraints or keys (unique, pk, fk, check, index) across
145         one or more columns. Also retrieve the definition of expression-based
146         indexes.
147         """
148         constraints = {}
149         # Loop over the key table, collecting things as constraints. The column
150         # array must return column names in the same order in which they were
151         # created.
152         cursor.execute("""
153             SELECT
154                 c.conname,
155                 array(
156                     SELECT attname
157                     FROM (
158                         SELECT unnest(c.conkey) AS colid,
159                                generate_series(1, array_length(c.conkey, 1)) AS arridx
160                     ) AS cols
161                     JOIN pg_attribute AS ca ON cols.colid = ca.attnum
162                     WHERE ca.attrelid = c.conrelid
163                     ORDER BY cols.arridx
164                 ),
165                 c.contype,
166                 (SELECT fkc.relname || '.' || fka.attname
167                 FROM pg_attribute AS fka
168                 JOIN pg_class AS fkc ON fka.attrelid = fkc.oid
169                 WHERE fka.attrelid = c.confrelid AND fka.attnum = c.confkey[1]),
170                 cl.reloptions
171             FROM pg_constraint AS c
172             JOIN pg_class AS cl ON c.conrelid = cl.oid
173             WHERE cl.relname = %s AND pg_catalog.pg_table_is_visible(cl.oid)
174         """, [table_name])
175         for constraint, columns, kind, used_cols, options in cursor.fetchall():
176             constraints[constraint] = {
177                 "columns": columns,
178                 "primary_key": kind == "p",
179                 "unique": kind in ["p", "u"],
180                 "foreign_key": tuple(used_cols.split(".", 1)) if kind == "f" else None,
181                 "check": kind == "c",
182                 "index": False,
183                 "definition": None,
184                 "options": options,
185             }
```

```
        # Now get indexes
        cursor.execute("""
            SELECT
                indexname, array_agg(attname ORDER BY arridx), indisunique, indisprimary,
                array_agg(ordering ORDER BY arridx), amname, exprdef, s2.attoptions
            FROM (
                SELECT
                    row_number() OVER () as arridx, c2.relname as indexname,
                    idx.*, attr.attname, am.amname,
                    CASE
                        WHEN idx.indexprs IS NOT NULL THEN
                            pg_get_indexdef(idx.indexrelid)
                    END AS exprdef,
                    CASE am.amname
                        WHEN %s THEN
                            CASE (option & 1)
                                WHEN 1 THEN 'DESC' ELSE 'ASC'
                            END
                    END as ordering,
                    c2.reloptions as attoptions
                FROM (
                    SELECT
                        *, unnest(i.indkey) as key, unnest(i.indoption) as option
                    FROM pg_index i
                ) idx
                LEFT JOIN pg_class c ON idx.indrelid = c.oid
                LEFT JOIN pg_class c2 ON idx.indexrelid = c2.oid
                LEFT JOIN pg_am am ON c2.relam = am.oid
                LEFT JOIN pg_attribute attr ON attr.attrelid = c.oid AND attr.attnum = idx.key
                WHERE c.relname = %s AND pg_catalog.pg_table_is_visible(c.oid)
            ) s2
            GROUP BY indexname, indisunique, indisprimary, amname, exprdef, attoptions;
        """, [self.index_default_access_method, table_name])
```

2.5.2. 冲突时插入

2.5.2.1. 问题描述

在使用 bulk_create()函数批量插入值时，若将 ignore_conflicts 参数设置为 True 代表 告诉数据库忽略插入任何不合格的约束条件的行，如重复的唯一值。此时会报如下错误：

```
Traceback (most recent call last):
  File "/workspace/python/Django/Django-4.0.1/django/db/backends/utils.py", line 85, in _execute
    return self.cursor.execute(sql, params)
psycopg2.errors.FeatureNotSupported: Conflict without target is not yet supported within INSERT ON CONFLICT KEY DO UPDATE statement.
```

2.5.2.2. 原因

PostgreSQL 的 ON CONFLICT DO NOTHING 语法在 panweidb 未支持。

2.5.2.3. 解决办法

将 ON CONFLICT DO NOTHING 改为 panweidb 相应的语法 ON DUPLICATE KEY UPDATE NOTHING。

打开 django.db.backends.panweidb.operations.py 文件，找到 ignore_conflicts_suffix_sql(self,ignore_conflicts=None) 函数，改动如下：

```
274
275     def ignore_conflicts_suffix_sql(self, ignore_conflicts=None):
276         return 'ON DUPLICATE KEY UPDATE NOTHING' if ignore_conflicts else super().ignore_conflicts_suffix_sql(ignore_conflicts)
277
```

2.6. 附录 1：主要接口

主要接口说明如下：

- create(**kwargs)

  一种方便的方法，用于创建一个对象并一步到位地保存。因此：

  ```
  p = Person.objects.create(first_name="Bruce", last_name="Springsteen")
  ```

  和：

  ```
  p = Person(first_name="Bruce", last_name="Springsteen")
  p.save(force_insert=True)
  ```

  是等效的。

  调用示例见附录 2。

- bulk_create(objs, batch_size=None, ignore_conflicts=False)

  这个方法以有效的方式将提供的对象列表插入数据库（一般来说，不管有多少个对象，只需进行一次查询），并以列表形式返回创建的对象，顺序与提供的相同：

关键字	参数说明
obj	需要保存的模型实例

batch_size	控制在一次查询中创建多少对象
ignore_conflicts	告诉数据库忽略插入任何不合格的约束条件的行

调用示例见附录 2。

- get_or_create(defaults=None, **kwargs)

此方法根据类创建表。参数说明如下:

一个方便的方法，用于查找具有给定 kwargs 的对象（如果你的模型对所有字段都有默认值，则可能为空），必要时创建一个对象。

返回值说明如下:

关键字	参数说明
(object, created) 的元组	object 是检索或创建的对象，created 是指定是否创建新对象的布尔值

调用示例见附录 2。

- update(**kwargs)

对指定的字段执行 SQL 更新查询，并返回匹配的行数（如果有些行已经有了新的值，则可能不等于更新的行数）。参数说明如下:

关键字	参数说明
**kwargs	模型需更新的字段

调用示例见附录 2。

- delete()

对 QuerySet 中的所有行执行 SQL 删除查询，并返回删除的对象数量和每个对象类型的删除数量的字典。

调用示例见附录 2。

- filter(*args, **kwargs)

返回一个新的 QuerySet，其中包含与给定查找参数相匹配的对象。

参数说明如下：

关键字	参数说明
**kwargs	查询参数

调用示例见附录 2。

- exclude(*args, **kwargs)

返回一个新的 QuerySet，其中包含与给定查找参数不匹配的对象。

参数：

关键字	参数说明
**kwargs	查询参数

调用示例见附录 2。

- get(*args, **kwargs)

返回与给定的查找参数相匹配的对象。使用时保证唯一的查询，比如主键或唯一约束中的字段。

参数：

关键字	参数说明
**kwargs	查询条件

调用示例见附录 2。

2.7. 附录 2：调用示例

Django 的 ORM，它一般都是在 web 项目上使用的，很少有单独使用的，这里就提炼出一个小的 Demo 展示独立使用 Django ORM， 即非 Web 项目上使用 ORM， 代码目录等如下：

1. 目录结构

```
|   main.py
|
├──myproject
|   |   settings.py
|   |   __init__.py
|   |
|   ├──myapp
|   |   |   models.py
|   |   |   __init__.py
```

2. 数据库迁移设置

在创建的项目__init__.py 文件中把所有的 app 加载到 Django 里，并提供 migrate 和 make_migrations 方法用于数据库迁移。

```
from django.apps import apps
from django.conf import settings
```

```
from django.core.management import call_command


apps.populate(settings.INSTALLED_APPS)

print("----------apps.populate----------")



def migrate(app_name: str):

    call_command("migrate", app_name)



def make_migrations(app_name: str):

    call_command("makemigrations", app_name)
```

3. 数据库对象

```
from django.db import models

from datetime import datetime,date,time


class Person(models .Model):

id = models.AutoField(primary key=True)

name = models.CharField(max length=32)

age = models .IntegerField()

birthday = models .DateField()# 日期

class Meta():
```

```
db_table = 'person'# 修改数据库中表格的名称为 person
```

4. 代码 main.py

为了独立测试 Django 的 orm 框架，在入口处配置 Django 的设置文件系统环境变量 DJANGO_SETTINGS_MODULE 值为设置文件的目录 myproject.settings，这取值跟目录结构有关。

```
import os,django

# project_name 指项目名

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "myproject.settings")


from myproject import migrate, make_migrations

from django.db import models

from datetime import datetime,date,time

from myapp.models import Person


def addperson():

u = Person(name='user1',age='22',birthday=date(2000,1,1))

u.save()

u = Person.objects.create(name = 'user2',age='21',birthday=date(2001,1,1))

# 内存生成多个对象实例

user = [Person(name='user3', age='1', birthday=date(2021,1,1)),

Person(name='user4', age='2', birthday=date(2020,3,28)),

Person(name='user5', age='3', birthday=date(2019,2,26))]

# 执行一次 SQL 插入数据
```

```
Person.objects.bulk_create(user)

obj, created = Person.objects.get_or_create(name='user5',

age='3',birthday=date(2019,2,26))


def updatePerson():

obj = Person.objects.first()

obi.name = 'user7'

obj.save()

obj =Person.objects.filter(birthday=date(2019,2,26)).update(age='4')

print(obj)


def deletePerson():

Person.objects.filter(id=1)

print(obj)

obj.delete()


def seletePerson():

obj = Person.objects.get(id=2)

print(obj.name,obj.age,obj.birthday)

obj = Person.objects.get(pk=3)

print(obj.name,obj.age,obj.birthday)

obj = Person.objects.all()

print(obj)

num = Person.objects.count()
```

```
print(num)


obj = Person.objects.filter(id__exact=2)# exact:表示判等

print(obj)

obj = Person.objects.filter(name__contains='user')# contains:是否包含

print(obj)

obj = Person.objects.filter(id__gt=3)# 查询 id 大于 3 的用户

print(obj)

for i in obj:

print(i.id,i.name,i.age,i.birthday)

obj = Person.objects.filter(birthday__year=2001)# 查询 2001 年出生的用户

print(obj)


make_migrations("myapp")

migrate("myapp")

addPerson()

updatePeerson()

selectPerson()

deletePerson()
```

3. GoLang 接口开发指南

3.1. 介绍

PanWeiDB V2.0-S3.0.0_Beta 数据库支持通过 pq 驱动来基于 GoLang 进行数据库访问。pq 是用于用于 Go 数据库/sql 包的 GoLang panweidb 驱动程序。

具有的特性如下：

- 适配 openGauss SHA256/SM3 密码认证
- 支持 SSL
- 处理 database/sql 坏连接
- 正确扫描 time.Time（即 timestamp[tz], time[tz], date）
- 正确扫描二进制 Blob（即 bytea）
- 支持 hstore 软件包
- 支持 COPY FROM
- pq.ParseURL 用于将 URL 转换为 sql.Open 的连接字符串。
- libpq 兼容的环境变量
- 支持 Unix 套接字
- 通知：LISTEN/NOTIFY
- 支持 pgpass
- GSS（Kerberos）验证

3.2. 设置 pq 驱动

3.2.1. 安装 GoLang

若机器已经安装 GoLang 编译环境并可以正确编译 GoLang 程序，此处可以跳过。

1、获取 go1.16.6.linux-amd64.tar.gz 安装包，若可以联网的话，可以使用 wget 命令直接获取：

```
wget https://dl.google.com/go/go1.16.6.linux-amd64.tar.gz
```

2、进行解压，目录自行确定，建议解压到 /usr/local 目录下：

```
tar -C /usr/local -zxvf go1.16.6.linux-amd64.tar.gz
```

3、添加 /usr/local/go/bin 到PATH变量中，建议添加至 /etc/profile 中；以及添加 GOROOT,GOPATH 等变量：

```
export GOROOT=/usr/local/go
export GOPATH=$HOME/go
export GO111MODULE=on
export GOPROXY=https://goproxy.cn,direct
```

添加完成后 source /etc/profile

3.2.2. 安装数据库对应的 GoLang 语言驱动

获取 Go 语言驱动的 pg 的代码，将其放在前面设置的 GOPATH/pkg/mod 中。在运行的项目代码中,如果没有 go.mod 文件,则运行如下命令生成 go.mod 文件。

```
go mod init golangproject // golangproject 根据实际项目名修改
```

在 go.mod 文件中，添加如下代码：

```
require pq v0.0.0

replace pq => /root/go/pkg/pq // pq 目录根据实际情况修改
```

导入 pq 通过命令：

```
go get pq
```

如果报错请检查是否存在重复的 pq 驱动，数据库的 Go 驱动无法与 PostgreSQL 的 Go 驱动并存。

3.3. 使用连接数据库

3.3.1. 3.1 导入

pq 包是一个纯 Go Postgres 驱动的数据库/sql 包。在大多数情况下，客户端会使用 database/sql 包，而不是直接使用这个包。引入 pq 包时使用了_进行匿名加载，而不是直接使用驱动包。在对数据库的操作仍然是使用自带的 sql 包。例如：

```
import (
```

```
...
"database/sql"
_ "pq"
)
```

3.3.2. 3.2 连接数据库

3.3.2.1. 3.2.1.连接参数

使用 pq 驱动访问 PanWeiDB V2.0-S3.0.0_Beta 的连接串格式如下:

```
host=myip port=myport user=myuser password=mypasswd dbname=myna
me sslmode=enable/disable
```

URL 中各参数的含义如下:

- host

  服务端的主机名，默认为 localhost。

- port

  服务端监听的端口，默认为 5432。

- dbname

  数据库名称，默认连接的数据库是与用户同名的数据库。比如连接用户为 a tlasuser,如果不指定 database 参数,则默认连接到 atlasuser 这个数据库。

- user

  数据库用户名。

- password

  数据库用户的密码。

❖ sslmode

是否需要 ssl 加密。

3.3.2.2. 3.2.2 获取连接

使用 sql 库中 Open 方法获取连接：

```
connStr := "host=127.0.0.1 port=5432 user=pwadmin password=Aa123456
dbname=panweidb sslmode=disable"
db, err := sql.Open("panweidb", connStr)
if err != nil {
log.Fatal(err)
}
```

3.3.2.3. 3.2.3 关闭连接

关闭连接时调用 sql 库的 Close()方法即可：

```
connStr := "host=127.0.0.1 port=5432 user=pwadmin password=Aa123456
dbname=panweidb sslmode=disable"
db, err := sql.Open("panweidb", connStr)
db.Close();
```

3.4. 执行 SQL 语句并处理结果

向数据库发出执行 SQL 的请求时，需要使用 Query 实例发出查询，服务端执行完成后将返回一个 rows 实例，该实例包括查询的结果。

3.4.1. 执行查询

使用 Query 执行查询：

```
rows, err := db.Query("select * from products")
if err != nil {
fmt.Println("Some amazing wrong happens in the process of query", err)
return err
}
Products := make([]product, 0)
defer rows.Close()
```

3.4.2. 读取结果

3.4.2.1. 获取单一结果

若希望获取单一结果,则进行查询的接口有所不同,并通过连接对象的 Scan 方法获取结果值，Scan 是通过参数来返回结果值的，参数是指针类型。

示例：

```
var date string
err = db.QueryRow("select current_date").Scan(&date)
```

```
if err != nil {
    Log.Fatal(err)
}
```

3.4.2.2. 使用数组获取结果

驱动程序获取某个查询的结果集，客户端可通过数组来存储数据，同样是通过 Scan 来获取，一次获取一条数据。

示例：

用于一条记录结果的结构：

```
type product struct {
   ProductNo string
   Name      string
   Price     float64
}
```

逐条获取数据并存入数组：

```
rows, err := db.Query(`Select * from products`)
 if err != nil {
    fmt.Println("Some amazing wrong happens in the process of Query.",
err)
   return err, []product{}
 }
 products := make([]product, 0)
```

```
  defer rows.Close() //关闭连接
  index := 0
  var p product
  for rows.Next() {
  err := rows.Scan(&p.ProductNo, &p.Name, &p.Price)
    products = append(products, p)
  if err != nil { // 获得的都是字符串
      fmt.Println("Some amazing wrong happens in the process of query
All.", err)
      return err, products
    }
    index++
  }
```

使用注意事项：

在获取到 rows 结果集后，建立及时声明 defer rows.Close() ，避免忘记释放空间导致后续空间紧张。

3.4.2.3. 支持 PBE

本驱动支持 panweidb 的缓存计划以及参数绑定，其中缓存计划以及后期的绑定参数执行接口分别为：prepare，Query。

缓存计划示例：

```
sqlStr := "select * from products where name=$1"
```

```
stmt, err := db.Prepare(sqlStr)
if err != nil {
    fmt.Printf("prepare failed, err:%v\n", err)
  return err, []product{}
}
defer stmt.Close()
```

此时返回的是一个 stmt 对象，并非结果，该对象必须绑定参数才能执行出结果。

传入参数执行计划示例：

```
rows, err := stmt.Query(pname)
if err != nil {
    fmt.Printf("query failed, err:%v\n", err)
    return err, []product{}
}
defer rows.Close()
```

调用 stmt 对象的 Query 方法并传入参数，得到最终结果，此执行计划可以多次复用，传入不同参数即可。

3.4.3. 执行更新

要更改数据(执行一个 insert，update 或者 delete)，可以使用 Prepare 加 Exec 的方法，最终返回的是：

- 对于 update、insert、delete 语句执行成功是返回值为该命令所影响的行

数，如果影响的行数是 0，则返回值就是 0；

- 对于所有其他类型的语句，返回值为-1；
- 如果发生回滚，返回值也为-1；

3.4.3.1. 删除

```
stmt, err := db.Prepare(`Delete FROM products  where name = $1`)
  if err != nil {
    fmt.Println("Some amazing wrong happens in preparation for the del
etion.")
    return err
  }
  res, err := stmt.Exec(pname)
  if err != nil {
    fmt.Println("Some amazing wrong happens in execution for the deleti
on.")
    return err
  }
  num, err := res.RowsAffected()
  if err != nil {
    fmt.Println("Some amazing wrong happens in the affected fo deletion.
")
    return err
  }
  fmt.Println("Successfully delete", num)
```

3.4.3.2. 更新

```
stmt, err := db.Prepare("Update products set price = $2 where  product_n
o = $1")
  if err != nil {
    fmt.Println("Some amazing wrong happens in preparation for the upd
ate.")
    return err
  }
  defer stmt.Close()
  res, err := stmt.Exec(pno, newValue)
  if err != nil {
    fmt.Println("Some amazing wrong happens in execution for the updat
e.")
    return err
  }
  num, err := res.RowsAffected()
  if err != nil {
    fmt.Println("Some amazing wrong happens in the affected fo update.
")
    return err
  }
  fmt.Println("Successfully update ", num)
```

3.4.3.3. 插入

```
stmt, err := db.Prepare("Insert into products(Product_No,Name,Price) value
s($1,$2,$3)")
  if err != nil {
    fmt.Println("Some amazing wrong happens in preparation for the ins
ert.")
    return err
  }
  defer stmt.Close()
  res, err := stmt.Exec(project.ProductNo, project.Name, project.Price)
  if err != nil {
    fmt.Println("Some amazing wrong happens in the process of doInsert
One.")
    return err
  }
  num, err := res.RowsAffected()
  if err != nil {
    fmt.Println("Some amazing wrong happens in the affected fo deletion.
")
    return err
  }
  fmt.Println("Successfully add ", num)
```

3.4.4. 创建或修改数据库对象

创建表可以使用 Prepare 和 Exec 函数。

例如:

```
stmt1,err := db.Prepare("drop table if exists student")

res1,err := stmt1.Exec()

fmt.Println(res1,err)

stmt2,err := db.Prepare("create table student(id int)")

res2,err := stmt2.Exec()

fmt.Println(res2,err)
```

要更改或者删除一个类似表或者视图这样的数据库对象， 仍然可以使用 Query() 方法，但是没有结果集返回.

例如 DROP 表:

```
 _, err := db.Query("DROP TABLE mytable;")
```

DROP 数据库:

```
_, err := db.Query("DROP DATABASE mydatabase;")
```

3.5. 调用存储过程与函数

在调用存储过程与函数时，也是基于 sql.QueryRow()来进行，方法就是构建相应的 sql 语句来进行数据库访问，并通过 Scan()方法获取结果。

```
sql_procedure := ` CREATE OR REPLACE PROCEDURE pro_test(a integer,ino
ut b integer) as
                    begin
                        b := a+b;
                    end;`
stmt,err :=db.prepare(sql_procedure)
res,err := stmt.Exec()
var num string
err1 := db.QueryRow("call pro_test(1,5)").Scan($num)
fmt.Println(num)
```

下面的例子展示了如何调用函数。

```
sql_function := "CREATE OR REPLACE FUNCTION func_test (a int,out b int)
RETURNS int AS
                $$
                begin
                    b := a+2;
                return ;
                end;
                $$ LANGUAGE 'plpgsql';"
stmt,err :=db.prepare(sql_function)
res,err := stmt.Exec()
var num string
err1 := db.QueryRow("select func_test(1)").Scan($num)
```

```
fmt.Println(num)
```

3.6. 处理数据类型

3.6.1.1. 二进制类型

PanWeiDB V2.0-S3.0.0_Beta 提供两种不同的方法存储二进制数据。二进制数据可以使用二进制数据类型 BYTEA 存储在表中，或者使用大对象特性以一种特殊的格式将二进制数据存储在一个独立的表中， 然后通过在表中保存一个类型为 OID 的值来引用该表。以下为这两种方法的使用示例。

- 使用 BYTEA 类型存储

1） 创建表

```
stmt,err := db.Prepare("create table lo_test_tab (id int, img bytea)")
res,err := stmt.Exec()
```

2） 插入数据

```
file,_ := os.Open("image.png")
defer file.Close()
bytes,_ := ioutil.ReadAll(file)
stmt,err := db.Prepare("insert into lo_test_tab values(1,$1)")
res,err := stmt.Exec(bytes)
```

3） 读取数据

```
var bytedata []byte
err := db.QueryRow("select img from lo_test_tab where id=1").Scan(&byted
ata)
_ = IfNoFileToCreate("newimage.png")
if err := ioutil.WriteFile("newimage.png",bytedata,0666);err !=nil{
    fmt.Println("写入文件失败")
}
```

- 使用 OID 存储

1） 创建表

```
stmt,err := db.Prepare("create table lo_test_tab (name text, img oid)")
res,err := stmt.Exec()
```

2） 导入数据

该步骤会将操作系统文件 “image” 导入成一个大对象并生成一个 oid，查询 lo_test_tab 表时则显示 oid。

```
stmt,err := db.Prepare("insert into  lo_test_tab values('beautiful image', lo
_import('/tmp/image'))")
res,err := stmt.Exec()
```

3） 导出数据

该操作将数据库中存储的大对象导出到一个操作系统文件。

```
stmt,err := db.Prepare("select lo_export(lo_test_tab.img, '/tmp/image-bak')
```

```
 from lo_test_tab where name = 'beautiful image'")

res,err := stmt.Exec()
```

3.6.1.2. 字符类型

PanWeiDB V2.0-S3.0.0_Beta 中 TEXT 类型与 VARCHAR 类型都是可变长的字符类型，区别在于 VARCHAR 类型通过 VARCHAR(n)中的 n 来限制最大长度，而 TEXT 类型没有。TEXT 类型与 VARCHAR 类型几乎没有性能差别，TEXT 类型最多可存储 1G 数据 。

使用 TEXT 类型存储数据时，可用以下方式来进行读写。

1) 创建表

```
stmt,err := db.Prepare("create table images (id int, msg text)")

res,err := stmt.Exec()
```

2) 插入数据

```
stmt,err := db.Prepare("insert into images values(1, $1)")

res,err := stmt.Exec("the text data")
```

3) 读取数据

```
row,err := db.Query("select msg from images where id = 1")

for row.Next(){

var t string

err := row.Scan(&t)

fmt.Println(t)
```

```
fmt.Println(err)
}
```

3.6.1.3. 数字类型

数字类型有整数类型、任意精度数字类型、浮点类型和序数类型。下面用整数类型 integer、任意精度数字类型 numeric、 浮点类型 real 和 double precision 举例说明。

1)创建表

- 整数类型 integer 的建表

```
stmt,err := db.Prepare("create table test_int(Num int)")
res,err := stmt.Exec()
```

- 任意精度数字类型 numeric 的建表

```
stmt,err := db.Prepare("create table test_numeric(num numeric)")
res,err := stmt.Exec()
```

- 浮点类型 real 和 double precision 的建表

```
// real 类型
stmt,err := db.Prepare("create table test_real(num real)")
res,err := stmt.Exec()

// double precision 类型
stmt,err := db.Prepare("create table test_doubleprecision(num double prec
ision)")
res,err := stmt.Exec()
```

2)插入数据

- 整数类型 integer 的插入

```
stmt,err := db.Prepare("insert into  test_int values($1)")
res,err := stmt.Exec(3)
```

- 任意精度数字类型 numeric 的插入

```
stmt,err := db.Prepare("insert into  test_numeric values($1)")
res,err := stmt.Exec(123.456)
```

- 浮点类型 real 和 double precision 的插入

```
// real 类型
stmt,err := db.Prepare("insert into  test_real values($1)")
res,err := stmt.Exec(3.1234)
// double precision 类型
stmt,err := db.Prepare("insert into  test_doubleprecision values($1)")
res,err := stmt.Exec(4.1234)
```

3)读取数据

- 整数类型 integer 的读取

```
type test_int struct{
    Num int
}
row,err := db.Query("select Num from test_int")
var i test_int
for row.Next(){
err := row.Scan(&i.Num)
```

```
fmt.Println(i.Num)

}
```

- 任意精度数字类型 numeric 的读取

```
type test_numeric struct{

    Num string

}

row,err := db.Query("select Num from test_numeric")

var i test_numeric

for row.Next(){

err := row.Scan(&i.Num)

fmt.Println(i.Num)

}
```

- 浮点类型 real 和 double precision 的读取

```
// real 类型

type test_real struct{

    Num string

}

row,err := db.Query("select Num from test_real ")

var i test_real

for row.Next(){

err := row.Scan(&i.Num)

fmt.Println(i.Num)

}
```

```
// double precision 类型
type test_doubleprecision struct{
    Num string
}
row,err := db.Query("select Num from test_doubleprecision ")
var i test_doubleprecision
for row.Next(){
err := row.Scan(&i.Num)
fmt.Println(i.Num)
}
```

3.6.1.4. 时间/日期类型

日期/时间类型有 timestamp 、date、time 、interval ； 下面以 timestamp 、time 举例说明。

1)创建表

```
// timestamp 类型
type test_timestamp struct{
    Dt time.Time
}
stmt,err := db.Prepare("create table test_timestamp(dt timestamp)")
res,err := stmt.Exec()
// time 类型
```

```
type test_time struct{
    Dt time.Time
}
stmt,err := db.Prepare("create table test_time(dt time)")
res,err := stmt.Exec()
```

2)插入数据

```
// timestamp 类型
stmt,err := db.Prepare("insert into test_timestamp values($1)")
res,err := stmt.Exec(time.Date(2018,10,1,10,30,30,0,time.Local))
// time 类型
stmt,err := db.Prepare("insert into test_time values($1)")
res,err := stmt.Exec(time.Date(1,1,1,10,10,10,time.Local))
```

3)读取数据

```
// timestamp 类型
row,err := db.Query("select dt from test_timestamp")
var i test_timestamp
for row.Next(){
err := row.Scan(&i.Dt)
fmt.Println(i.Dt)
}

// time 类型
```

```
row,err := db.Query("select dt from test_time")
var i test_time
for row.Next(){
err := row.Scan(&i.Dt)
fmt.Println(i.Dt.Hour(),i.Dt.Minute(),i.Dt.Second())
}
```

3.7. 附录 1：主要接口

pq 驱动由具有建立和管理与 PanWeiDB V2.0-S3.0.0_Beta 数据库连接、执行 SQL 语句和对结果集进行储存管理的若干功能类组成。

- sql.Open

  与特定数据库的连接（会话）。在连接上下文中执行 SQL 语句并返回结果。

  示例：

```
connStr := "host=127.0.0.1 port=5433 user=gaussdb password=Aa123456 d
bname=panweidb sslmode=disable"
db, err := sql.Open("panweidb", connStr)
```

- QueryRow

  用于执行 SQL 语句并通过 Scan 返回结果。

```
err = db.QueryRow("select current_data").Scan(&date)
```

- Query

存储执行 SQL 语句产生的结果集。

示例：

```
rows,err := db.Query("select * from products where name=$1", pname)
```

3.8. 附录 2：调用示例

```
import (
  "fmt"
  "os"
  _ "pq"
  "database/sql"
)

func main(){
  var err error
connStr := "host=127.0.0.1 port=5432 user=pwadmin password=Aa123456
dbname=panweidb sslmode=disable"
db, err := sql.Open("panweidb", connStr)
if err != nil {
log.Fatal(err)
}
 //创建表
    stmt1,err := db.Prepare("create table student(id int)")
```

```
res1,err := stmt1.Exec()



//插入数据

stmt2,err := db.Prepare("insert into test_int values($1)")

res2,err := stmt2.Exec(3)

//更新数据

stmt3,err := db.Prepare("update test_int set Num= $2 where Num = $1")

res3,err := stmt3.Exec(3,1)

//查询数据 Get 方法

var n int

err = db.QueryRow("select Num from test_int").Scan(&n)

fmt.Println(n)

}
```

4. JDBC 接口开发指南

4.1. 介绍

PanWeiDB V2.0-S3.0.0_Beta 数据库通过提供 JDBC 驱动为使用 JAVA 语言的应用程序提供访问 PanWeiDB V2.0-S3.0.0_Beta 的接口。

4.1.1. PanWeiDB V2.0-S3.0.0_Beta JDBC 版本

如果要使用 panweidb JDBC 驱动访问 panweidb，建议使用 2.0 及以上版本的驱动。

PanWeiDB V2.0-S3.0.0_Beta 的 JDBC 驱动基于开源 openGauss-connector-jdbc 开发，PanWeiDB V2.0-S3.0.0_Beta JDBC 驱动包名称为 panweidb_jdbc_$version<v|p>_$buildtime.jar。

PanWeiDB V2.0-S3.0.0_Beta 的 JDBC 驱动主要增加了以下功能：

- 增加对 nvarchar2 类型数据的处理。
- 增加 db_compatibility 连接参数。
- 增加 bulkload 高速导入功能。

4.1.2. PanWeiDB V2.0-S3.0.0_Beta JDBC 与 JDK 的兼容性

2.0 版本及以上的 JDBC 驱动兼容 JDK1.8 及以上版本。

4.2. PanWeiDB V2.0-S3.0.0_Beta JDBC 主要类与接口

PanWeiDB V2.0-S3.0.0_Beta JDBC 驱动由具有建立和管理与 PanWeiDB V2.0-S3.0.0_Beta 数据库连接、执行 SQL 语句和对结果集进行储存管理的若干功能类组成。

- Driver 类(org.postgresql.Driver)

  当注册驱动的时候或配置软件以使用 PanWeiDB V2.0-S3.0.0_Beta JDBC 驱动的时候，应该使用这个类名。示例：

```
Class.forName("org.postgresql.Driver");
```

- DriverManager 类(java.sql.DriverManager)

  跟踪可用的驱动程序，在数据库与相应的驱动程序之间建立连接。应用服务器使用 JDBC 时，DriverManager 类管理连接的建立。首先需要告诉 DriverManager 应该与哪个 JDBC 驱动进行连接，最简单的方法就是使用实现了接口 java.sql.Driver 的类的 Class.forName()方法。在 PanWeiDB V2.0-S3.0.0_Beta JDBC 驱动中，这个类的名字叫做 org.postgresql.Driver。用这种方法，就可以在连接一个数据库时使用一个外部配置文件来给驱动提供类名和驱动参数。

- Connection 接口(org.postgresql.PGConnection)

  与特定数据库的连接（会话）。在连接上下文中执行 SQL 语句并返回结果。示例：

```
Connection con=DriverManager.getConnection("jdbc:postgresql://host:port/
dbname ", "user","password");
```

- Statement 接口(org.postgresql.PGStatement)

  用于执行 SQL 语句并返回结果。

- ResultSet 接口(org.postgresql.PGResultSetMetaData)

  存储执行 SQL 语句产生的结果集。示例：

```
Statement st = con.createStatement();

ResultSet rs = st.executeQuery("select c_custkey from customer");
```

4.3. 设置 JDBC 驱动

4.3.1. 设置类路径

要使用驱动， 必须将驱动 jar 包含在类路径里，可以将 jar 包路径添加到 CLASSPATH 环境变量中,或者使用 java 命令行标记的方式将驱动 jar 包引入.

比如，有一个使用 PanWeiDB V2.0-S3.0.0_Beta JDBC 驱动的应用安装在 /usr/local/lib/myapp.jar，而 PanWeiDB V2.0-S3.0.0_Beta JDBC 驱动安装在/usr/local/panweidb/java/panweidb-2.0_2021072214.jar．可以用以下方式运行应用：

```
export CLASSPATH=/usr/local/lib/MyApp.jar:/usr/local/panweidb/java/pan
weidb-2.0_2021072214.jar:.
java MyApp
```

4.3.2. 为 JDBC 准备数据库

因为 java 只支持使用 TCP/IP 连接，所以 PanWeiDB V2.0-S3.0.0_Beta 必须配置为可接受 TCP/IP 连接。可以通过修改 listen addresses 来进行配置。同样，也需要配置主机认证文件，保证客户端程序可以访问数据库。

4.4. 使用 JDBC 连接数据库

4.4.1. 导入

任何使用 JDBC 的程序都需要导入 java.sql 包：

```
import java.sql.*
```

4.4.2. 说明

驱动默认提供 PostgreSQL 版本的驱动包（postgresql.jar），在需要同时使用 PostgreSQL 及 panweidb 驱动时，为避免包名与连接串名冲突，我们提供了 PanWeiDB 版本的驱动包（panweidb-jdbc-<$version>v.jar），请联系相关人员获取。

v 版本驱动包使用说明：

1）使用 “jdbc:panweidb” 作为 URL 前缀替换 “jdbc:postgresql”；

2）使用 “cn.com.panweidb.Driver” 作为驱动名称替换 “org.postgresql.Driver”。

4.4.3. 加载驱动

在试图与数据库建立连接之前，首先需要加载驱动，加载驱动有两种方法：

- 在代码中，用 Class.forName(“org.postgresql.Driver”)方法显示加载驱动；
- 在 JVM 启动时作为参数传递，比如：

```
java  -Djdbc.drivers=org.postgresql.Driver
```

4.4.4. 连接数据库

4.4.4.1. URL 格式

使用 PanWeiDB V2.0-S3.0.0_Beta JDBC 驱动访问 PanWeiDB V2.0-S3.0.0_Beta 的 URL 格式如下：

- jdbc:postgresql://host:port/database
- jdbc:postgresql://host:port/
- jdbc:postgresql://host/database
- jdbc:postgresql://host/
- jdbc:postgresql:/

URL 中各参数的含义如下：

- host

  服务端的主机名，默认为 localhost,如果要指定 IPV6 的地址 ，必须用方括号括起主机参数，例如：jdbc:postgresql://[::1]:5432/atlas.

- port

服务端监听的端口，默认为 5432.

- database

数据库名称，默认连接的数据库是与用户同名的数据库。比如连接用户为 pwuser，如果不指定 database 参数，则默认连接到 pwuser 这个数据库。

4.4.4.2. 获取 JDBC 连接

使用 DriverManager.getConnection()获取连接：

```
Connection connection = DriverManager.getConnection(url, username,pass
word);
```

4.4.4.3. 关闭 JDBC 连接

关闭连接时调用 Connection 的 close()方法即可：

```
Connection connection = DriverManager.getConnection(url, username,pass
word);
connection.close();
```

4.4.5. 连接参数

名称	类型	定义
user	String	连接数据库的用户
password	String	数据库用户的密码
sendBufferSize	int	在连接流上设置 SO_SNDBUF
recpwufferSize	int	在连接流上设置 SO_RCpwUF

名称	类型	定义
tcpKeepAlive	boolean	启用或禁用 TCP keep-alive
socketTimeout	int	socket 读取操作的超时时间
ApplicationName	String	指定正在使用连接的应用程序名称
currentSchema	String	指定设置到 search-path 中的 schema
readOnly	boolean	将连接设置为只读模式
characterEncoding	String	设置 client_encoding，支持 gbk、utf8，设置为 gbk 时，需要将 allowEncodingChanges 设置为 true
allowEncodingChanges	boolean	设置该参数值为“true”进行字符集类型更改，配合 characterEncoding 设置字符集
db_compatibility	String	兼容模式，默认值为 postgresql，可设置有 postgresql、oracle、mysql。设置为 postgresql 时，返回对象名的字母大小写与数据库中一致，设置为 oracle 时，使用 ResultSetMetaData 获取对象名时，对象名的字母将以大写返回，并且 disableColumnSanitiser=fasle 设置无效。设置为 mysql 时，对二进制数据的处理会有

名称	类型	定义
		对应不同。注：当对象名是使用双引号引住的小写名称(没有大写字母)，也会转换成全大写字母返回，因此使用此参数时需要注意对象名的大小写。
prepareThreshold	int	指定使用 PreparedStatement 构造的 sql 语句在重复执行多少次后会缓存在数据库服务端，默认值 5，即执行 5 次后会把查询计划等信息缓存到数据库端，随后的请求指挥发送该语句缓存后的句柄，建议默认值
preparedStatementCacheQueries	int	每个连接中可缓存的 PreparedStatement，默认值 256，建议默认值
preparedStatementCacheSizeMiB	int	每个连接中语句缓存的最大值，默认 5MB,单位 5MB，建议默认值
defaultRowFetchSize	int	指定在一个查询中，每次抓取的行数，主要是防止一次性从数据库抓取的数据量太大造成应用端大量的内存消耗，进而导致内存溢出。默认为 0，即一次性抓取全部符合条件的行，

名称	类型	定义
		一般应用端在进行查询时都指定了分页条件，建议使用默认值
reWriteBatchedInserts	boolean	折叠批量 insert 的语句(使用 executeBatch 方法)，建议开启(true)。样例：insert into foo (col1, col2, col3) values (1,2,3);insert into foo (col1, col2, col3) values (4,5,6)；会合并为 insert into foo (col1, col2, col3) values (1,2,3),(4,5,6);
socketFactory	String	指定 socket 工厂类
socketFactoryArg	String	指定 socket 工厂类参数
targetServerType	String	any：连接到任意一个可用的数据库节点 master：连接到任意一个可写的数据库节点 slave：连接到任意一个只读的数据库节点 secondary：连接到任意一个只读的数据库节点 preferSlave：连接到任意一个只读的数据库节点，如果没有可用的只读节点，则连接到可读可写节点

名称	类型	定义
		preferSecondary ：连接到任意一个只读的数据库节点，如果没有可用的只读节点，则连接到可读可写节点
loadBalanceHosts	boolean	配合 targetServerType 使用，若为 true，则随机分配节点，若为 false，则按顺序分配节点

4.5. 执行 SQL 语句并处理结果

向数据库发出执行 SQL 的请求时，需要使用 Statement 或 PreparedStatement，使用 Statement 或 PrepareStatement 实例发出查询，服务端执行完成后将返回一个 ResultSet 实例，该实例包括查询的结果。

4.5.1. 执行查询

4.5.1.1. 使用 Statement 执行查询

```
Statement st = conn.createStatement();
ResultSet rs = st.executeQuery("SELECT * FROM mytable WHERE columnfoo
 = 500");
while (rs.next())
{
    System.out.print("Column 1 returned ");
```

```
    System.out.println(rs.getString(1));
}
rs.close();
```

4.5.1.2. 使用 PreparedStatement 执行查询

```
int foovalue = 500;
PreparedStatement st = conn.prepareStatement("SELECT * FROM mytable
WHERE columnfoo = ?");
st.setInt(1, foovalue);
ResultSet rs = st.executeQuery();
while (rs.next())
{
    System.out.print("Column 1 returned ");
    System.out.println(rs.getString(1));
}
rs.close();
st.close();
```

4.5.2. 使用游标获取结果

默认情况下，驱动程序将一次性获取查询的所有结果，当某个查询的结果集很庞大时，一次获取所有结果很不合理，因此 JDBC 驱动程序提供了一种基于数

据库游标的 ResultSet 方法，只获取少量的行缓存在客户端。通过重新定位游标来获取下一批数据。

基于游标的 ResultSet 不能在所有情况下使用。有许多限制会使驱动程序一次性获取整个结果集：

- 连接不能使用自动提交模式 ，因为后端会在事务结束时关闭游标，因此在自动提交模式下，事务结束后，在从游标获取数据之前，游标已经被关闭。
- Statement 必须使用 ResultSet.TYPE_FORWARD_ONLY 类型创建，这意味着不能在结果集中进行跳转或向后滚动。这也是默认值，因此不需要在代码中额外指定。
- 给定的查询语句必须是单条语句，不能是用分号连接在一起的多个语句。
- 必须设置 fetch size，且 fetch size 不能为 0.

示例：

```
// make sure autocommit is off
conn.setAutoCommit(false);
Statement st = conn.createStatement();


// Turn use of the cursor on.
st.setFetchSize(50);
ResultSet rs = st.executeQuery("SELECT * FROM mytable");
while (rs.next())
{
    System.out.print("a row was returned.");
}
```

```
rs.close();


// Turn the cursor off.
st.setFetchSize(0);
rs = st.executeQuery("SELECT * FROM mytable");
while (rs.next())
{
    System.out.print("many rows were returned.");
}
rs.close();


// Close the statement.
st.close();
```

4.5.3. 使用 Statement 和 PreparedStatement 接口

在使用 Statement 和 PreparedStatement 时，需要注意以下问题：

- 可以多次使用一个 Statement 实例，可以在打开连接时创建 Statement 然后在连接的整个生命周期内使用它。但是必须注意在给定时间内每个 Statement 或 PreparedStatement 只能存在一个 ResultSet.
- 如果在处理 ResultSet 时需要执行查询，可以创建另一个 Statement 来执行。
- 如果使用多线程 ，并且有多个线程正在使用数据库，则每个线程必须使用单独的 Statement.

- 使用完 Statement 或 PreparedStatement 应该及时关闭。
- 在 JDBC 中，问号(?)是 PreparedStatement 的占位符，但是有一些操作符包含一个问号，为避免 SQL 语句中的问号被当成占位符，可以使用两个问号(??)进行转义。也可以在 Statement 中使用上述转义，但是不是必须的，因为在 Statement 中单个问号也可以用作操作符。

4.5.4. 使用 ResultSet 接口

使用 ResultSet 接口时必须考虑下面的问题：

- 在读取任何数值的时候，必须调用 next()． 如果还有结果则返回真（true）。
- 在 JDBC 规范里，对一个字段应该只访问一次． 遵循这个规则是最安全的，不过目前 PanWeiDB V2.0-S3.0.0_Beta JDBC 驱动允许对一个字段访问任意次．
- 在结束对一个 ResultSet 的处理后，必须调用 close()来关闭它。
- 使用那个创建 ResultSet 的 Statement 做另一个查询请求， 当前打开的 ResultSet 实例将自动关闭．

4.5.5. 执行更新

要更改数据(执行一个 insert，update 或者 delete)，可以使用 executeUpdate() 方法． executeUpdate() 方法类似于发出 select 语句的 executeQuery()方法，不过，它不返回 ResultSet，它返回的是 insert,update 或者 delete 语句影响的行数．

4.5.5.1. 删除

```
int foovalue = 500;
```

```
PreparedStatement st = conn.prepareStatement("DELETE FROM mytable W
HERE columnfoo = ?");
st.setInt(1, foovalue);
int rowsDeleted = st.executeUpdate();
System.out.println(rowsDeleted + " rows deleted");
st.close();
```

4.5.5.2. 更新

```
int foovalue = 500;
PreparedStatement st = conn.prepareStatement("UPDATE mytable  SET col
umninfo='updated' WHERE columnfoo = ?");
st.setInt(1, foovalue);
int rowsUpdated = st.executeUpdate();
System.out.println(rowsDeleted + " rows updated");
st.close();
```

4.5.5.3. 插入

插入单行：

```
PreparedStatement st = conn.prepareStatement("INSERT INTO mytable(col_
1,col_2) values(?,?)");
st.setInt(1, 0);
st.setString(2, "row1");
int rowsInserted = st.executeUpdate();
```

```
System.out.println(rowsDeleted + " rows inserted");

st.close();
```

插入多行：

```
PreparedStatement st = conn.prepareStatement("INSERT INTO mytable(col_

1,col_2) values(?,?)");

st.setInt(1, 0);

st.setString(2, "row1");

st.addBatch();

st.setInt(1,1);

st.setString(2, "row2");

st.addBatch();

st.executeBatch();

st.close();
```

4.5.6. 创建或修改数据库对象

要创建、更改或者删除一个类似表或者视图这样的数据库对象， 可以使用 execute() 方法， execute()方法类于发出 select 语句的 executeQuery()方法 ，不过它不返回结果.

例如 DROP 表：

```
Statement st = conn.createStatement();

st.execute("DROP TABLE mytable");
```

```
st.close();
```

4.6. 调用存储过程

调用存储过程，可以使用 prepareCall 方法，问号(?)是 prepareCall 的占位符，对于存储过程的输出，需要使用 CallableStatement 的 registerOutparameter 方法先注册，执行存储过程使用 execute()方法。

下面的例子展示了如何调用存储过程：

```
CallableStatement upperProc = conn.prepareCall("{? = call upper( ? ) }");
upperProc.registerOutParameter(1, Types.VARCHAR);
upperProc.setString(2, "lowercase to uppercase");
upperProc.execute();
String upperCased = upperProc.getString(1);
upperProc.close();
```

4.6.1. 获取结果集

函数可以通过两种方式返回结果集，根据函数被调用的方式决定结果集的返回方式。

- 方式一

```
Statement stmt = conn.createStatement();
stmt.execute("CREATE OR REPLACE FUNCTION setoffunc() RETURNS SETOF i
nt AS "
    + "' SELECT 1 UNION SELECT 2;' LANGUAGE sql");
```

```
ResultSet rs = stmt.executeQuery("SELECT * FROM setoffunc()");
while (rs.next())
{
    // do something
}
rs.close();
stmt.close();
```

❖ 方式二

```
// Setup function to call.
Statement stmt = conn.createStatement();
stmt.execute("CREATE OR REPLACE FUNCTION refcursorfunc() RETURNS refc
ursor AS "
    + " DECLARE "
    + "     mycurs refcursor; "
    + " BEGIN "
    + "     OPEN mycurs FOR SELECT 1 UNION SELECT 2; "
    + "     RETURN mycurs; "
    + " END; ");
stmt.close();

// We must be inside a transaction for cursors to work.
conn.setAutoCommit(false);
```

```
// Procedure call.
CallableStatement proc = conn.prepareCall("{? = call refcursorfunc() }");
proc.registerOutParameter(1, Types.OTHER);
proc.execute();
ResultSet results = (ResultSet) proc.getObject(1);
while (results.next())
{
    // do something with the results.
}
results.close();
proc.close();
```

4.7. 处理大数据类型

4.7.1. 二进制类型

PanWeiDB V2.0-S3.0.0_Beta 提供两种不同的方法存储二进制数据。二进制数据可以使用二进制数据类型 BYTEA 存储在表中，或者使用大对象特性以一种特殊的格式将二进制数据存储在一个独立的表中， 然后通过在表中保存一个类型为 OID 的值来引用该表。

4.7.1.1. BYTEA

使用 BYTEA 类型存储二进制数据，可以用 getBytes()，setBytes()进行读写。

例如：

```
CREATE TABLE images (imgname text, img bytea);
```

4.7.2. 插入记录

```
File file = new File("myimage.gif");
FileInputStream fis = new FileInputStream(file);
byte[] b = new byte[(int)file.length()];
fis.read(b);
PreparedStatement ps = conn.prepareStatement("INSERT INTO images VAL
UES (?, ?)");
ps.setString(1, file.getName());
ps.setBytes(2, b);
ps.executeUpdate();
ps.close();
fis.close();
```

在这里，数据存储在 byte[]中，使用 setBytes()方法来实现数据写入。

4.7.3. 读取记录

```
PreparedStatement ps = conn.prepareStatement("SELECT img FROM image
s WHERE imgname = ?");
```

```
ps.setString(1, "myimage.gif");
ResultSet rs = ps.executeQuery();
while (rs.next())
{
    byte[] imgBytes = rs.getBytes(1);
    // use the data in some way here
}
rs.close();
ps.close();
```

在这里，二进制数据被读取为 byte[] 。

4.7.3.1. BLOB

使用 BLOB 类型存储二进制数据，可以用 getBlob()，setBlob()，getBinaryStream()，setBinaryStream() 进行读写。

例如：

```
CREATE TABLE test_blob(uid integer, blob_text blob);
```

4.7.4. 插入记录

```
File file = new File("myimage.gif");
FileInputStream fis = new FileInputStream(file);
PreparedStatement ps = conn.prepareStatement("INSERT INTO test_blob V
```

```
ALUES (?,?)");
ps.setInt(1, 1);
ps.setBinaryStream(2, fis, (int)file.length());
ps.executeUpdate();
ps.close();
fis.close();
```

在这里，setBinaryStream()将一组字节从流传输到 Blob 类型的列中。setBinaryStream 的长度参数必须正确。

也可以使用 setBlob()方法来实现数据写入。

```
File file = new File("myimage.gif");
FileInputStream fis = new FileInputStream(file);
Blob blob = new PGBlob();
byte[] tmp = new byte[(int)file.length()];
fis.read(tmp);
blob.setBytes(1L, tmp);
PreparedStatement ps = conn.prepareStatement("INSERT INTO test_blob V
ALUES (?,?)");
ps.setInt(1, 2);
ps.setBlob(2, blob);
ps.executeUpdate();
ps.close();
fis.close();
```

4.7.5. 读取记录

```
PreparedStatement ps = conn.prepareStatement("SELECT blob_text FROM t
est_blob WHERE uid= ?");
ps.setInt(1, 1);
ResultSet rs = ps.executeQuery();
while (rs.next())
{
    Blob blob = rs.getBlob("blob_text");
InputStream in = blob.getBinaryStream();
byte[] buff = new byte[1024];
in.read(buff);
// use the data in some way here
In.close();
}
rs.close();
ps.close();
```

在这里，二进制数据被读取为 byte[]，也可以使用 getBinaryStream()来实现。

4.7.5.1. CLOB

使用 CLOB 类型存储二进制数据，可以用 getCharacterStream()，setCharatcerStream() 进行读写。

例如：

```
CREATE TABLE test_clob(uid integer, clob_text clob);
```

4.7.6. 插入记录

```
String data = "123abc一二三";
PreparedStatement ps = conn.prepareStatement("INSERT INTO test_clob VA
LUES (?, ?)");
ps.setInt(1,1);
StringReader reader = new StringReader(data);
ps.setCharacterStream(2,reader,data.length())
ps.executeUpdate();
reader.close();
ps.close();
```

在这里，setCharacterStream()获取到字符流写入器。writer 将数据写入到字符流中进行输出。

4.7.7. 读取记录

```
PreparedStatement ps = conn.prepareStatement("SELECT clob_text FROM t
est_clob WHERE uid= ?");
ps.setString(1, 1);
ResultSet rs = ps.executeQuery();
while (rs.next())
```

```
{
    Clob clob = rs.getClob("clob_text");
Reader rd = clob.getCharacterStream();
char[] str = new char[1024];
while (true) {
int i = rd.read(str);
if (-1 == i) {
break;
}
    // use the data in some way here
}
}
rs.close();
ps.close();
```

在这里，二进制数据被读取为 char[]。

4.7.7.1. 大对象功能

使用大对象功能存储二进制数据，可以用 LargeObject API 进行读写。

例如：

```
CREATE TABLE imageslo (imgname text, imgoid oid);
```

4.7.8. 插入记录

```
// All LargeObject API calls must be within a transaction block
conn.setAutoCommit(false);

// Get the Large Object Manager to perform operations with
LargeObjectManager lobj = conn.unwrap(org.postgresql.PGConnection.clas
s).getLargeObjectAPI();

// Create a new large object
long oid = lobj.createLO(LargeObjectManager.READ | LargeObjectManager.
WRITE);

// Open the large object for writing
LargeObject obj = lobj.open(oid, LargeObjectManager.WRITE);

// Now open the file
File file = new File("myimage.gif");
FileInputStream fis = new FileInputStream(file);

// Copy the data from the file to the large object
byte buf[] = new byte[2048];
int s, tl = 0;
```

```
while ((s = fis.read(buf, 0, 2048)) > 0)
{
    obj.write(buf, 0, s);
    tl += s;
}

// Close the large object
obj.close();

// Now insert the row into imageslo
PreparedStatement ps = conn.prepareStatement("INSERT INTO imageslo V
ALUES (?, ?)");
ps.setString(1, file.getName());
ps.setLong(2, oid);
ps.executeUpdate();
ps.close();
fis.close();

// Finally, commit the transaction.
conn.commit();
```

4.7.9. 读取记录

```
// All LargeObject API calls must be within a transaction block
```

```
conn.setAutoCommit(false);

// Get the Large Object Manager to perform operations with
LargeObjectManager lobj = conn.unwrap(org.postgresql.PGConnection.clas
s).getLargeObjectAPI();

PreparedStatement ps = conn.prepareStatement("SELECT imgoid FROM ima
geslo WHERE imgname = ?");
ps.setString(1, "myimage.gif");
ResultSet rs = ps.executeQuery();
while (rs.next())
{
    // Open the large object for reading
    long oid = rs.getLong(1);
    LargeObject obj = lobj.open(oid, LargeObjectManager.READ);

    // Read the data
    byte buf[] = new byte[obj.size()];
    obj.read(buf, 0, obj.size());
    // Do something with the data read here

    // Close the object
    obj.close();
```

```
}
rs.close();
ps.close();

// Finally, commit the transaction.
conn.commit();
```

4.7.10. 字符类型

PanWeiDB V2.0-S3.0.0_Beta 中 TEXT 类型与 VARCHAR 类型都是可变长的字符类型，区别在于 VARCHAR 类型通过 VARCHAR(n)中的 n 来限制最大长度，而 TEXT 类型没有。TEXT 类型与 VARCHAR 类型几乎没有性能差别，TEXT 类型最多可存储 1G 数据 。

使用 TEXT 类型存储数据时，可用 getString()，getCharacterStream(), setString(), setCharacterStream()来进行读写。

例如：

```
CREATE TABLE images (id int, msg text);
```

4.7.10.1. 插入记录

```
File file = new File("myimage.gif");
Reader reader = new InputStreamReader(new FileInputStream(file));
PreparedStatement ps = conn.prepareStatement("INSERT INTO images VAL
```

```
UES (?, ?)");
ps.setString(1, 12);
ps.setCharacterStream (2, reader);
ps.executeUpdate();
ps.close();
fis.close();
```

如果数据内容已经在 String 中，那么也可以使用 setString()方法来实现数据写入。

4.7.10.2. 读取记录

```
PreparedStatement ps = conn.prepareStatement("SELECT msg FROM image
s WHERE id= ?");
ps.setString(1, 12);
ResultSet rs = ps.executeQuery();
while (rs.next())
{
    String imgBytes = rs.getString(1);
    // use the data in some way here
}
rs.close();
ps.close();
```

在这里，TEXT 数据被读取为 String，也可以调用 getCharacterStream() 处理 。

4.8. 调用 BulkloadAPI

4.8.1. 加载配置文件

驱动提供两种不同的方法加载 bulkload 的配置文件参数。

4.8.1.1. 连接串参数

在连接串中通过 bulkloadCtlFile 参数配置 bulkload 的配置文件的路径。

例如：

```
String url = "jdbc:postgres://[ip]:[port]/[dbname]?user=xxx&password=xxxx&
bulkloadCtlFile=E:\\bulkload.ctl"
```

4.8.1.2. setBulkloadInfo()

通过调用 setBulkloadInfo 方法设置 bulkload 参数。

例如：

```
String url = "jdbc:postgres://[ip]:[port]/[dbname]?user=xxx&password=xxxx"
Connection conn = DriverManager.getConnection(url);
BulkloadManager bulkloadAPI = ((PGconnection) conn).getBulkloadAPI();
BulkloadProperties bulkloadInfo = new BulkloadProperties();
bulkloadInfo.put("INPUT","stdin");
```

```
bulkloadInfo.put("PARSE_ADFILE","/home/panweidb/tbl_bulkload.txt");
bulkloadInfo.put("LIMIT","INFINITE");
bulkloadInfo.put("PARSE_ERRORS","0");
bulkloadInfo.put("TYPE","CSV");
bulkloadInfo.put("SKIP","0");
bulkloadInfo.put("OUTPUT","tbl_buklload");

BulkloadAPI.setBulkloadInfo(bulkloadInfo);
```

4.8.2. 获取 bulkload 管理器

```
BulkloadManager bulkloadAPI = ((PGconnection) conn).getBulkloadAPI();
```

4.8.3. 使用 bulkload 管理器

通过调用 bulkloadIn()方法将数据导入数据库

4.8.3.1. bulkloadIn()

当 bulkloadInfo 中的 INPUT 参数为 path/function 时，可直接调用 bulkloadIn()方法调用数据库的 bulkload 接口

例如：

```
public void testBulkloadWithServerLocalFileInput() throws SQLException, IO
Execption {
bulkloadAPI.getBulkloadInfo().put("INPUT","/home/panweidb/tbl_bulkload_o
utput.txt");
try {
      ResultSet result = bulkloadAPI.bulkloadIn();
      // ResultSet result = bulkloadAPI.bulkloadIn((InputStream)null);
      While(result.next()) {
          for(int i = 1; i< result.getMetaData().getColumnCount() + 1; i ++)
{
            System.out.println(result.getMetaData().getColumnLabel(i) + resu
lt.getObject(result.getMetaData().getColumnLabel(i)));
         }
}
}
}
```

4.8.3.2. bulkloadIn(InputStream from)

当 bulkloadInfo 中的 INPUT 参数为 path/function 时，用此方法会直接调研数据库的 bulkload 接口，忽略入参 from 的输入， 当 INPUT 参数为 stdin 时，JDBC 驱动会读取 form 的输入，然后发送至数据库。

例如：

```
Public void testBulkloadWithJDBCRemoteFileInput() throws SQLException, IO
Execption {
bulkloadAPI.getBulkloadInfo().put("INPUT","stdin");
InputStream is = null;
try {
   Is = readfile()
      ResultSet result = bulkloadAPI.bulkloadIn(is);
      // ResultSet result = bulkloadAPI.bulkloadIn(is, 655360);
      While(result.next()) {
          for(int i = 1; i< result.getMetaData().getColumnCount() + 1; i ++)
{
             System.out.println(result.getMetaData().getColumnLabel(i) + resu
lt.getObject(result.getMetaData().getColumnLabel(i)));
         }
}
} finally {
   if (is != null) {
        is.close();
}
}
}
```

4.8.3.3. bulkloadIn(String folderPath, Pattern fileNamePattern)

当 bulkloadInfo 中的 INPUT 参数为 path/function 时，用此方法会直接调研数据库的 bulkload 接口，忽略入参 folerPath 和 fileNamePattern，当 INPUT 参数为 stdin 时，JDBC 驱动会根据 folderPath 和 fileNamePattern 参数读取指定文件内容，发送至数据库。

例如：

```
Public void testBulkloadWithFolderPathAndPattern() throws SQLException, I
OExecption {
bulkloadAPI.getBulkloadInfo().put("INPUT","stdin");
File file = new File("src/test/resources");
String folderPath = file.getAbsolutePath();
Pattern pattern = Pattern.compile("tbl_bulkload_small*", Pattern.CASE_INSE
NSITIVE);
try {
   Is = readfile()
     ResultSet result = bulkloadAPI.bulkloadIn(folderPath, pattern);
     While(result.next()) {
          for(int i = 1; i< result.getMetaData().getColumnCount() + 1; i ++)
{
            System.out.println(result.getMetaData().getColumnLabel(i) + resu
lt.getObject(result.getMetaData().getColumnLabel(i)));
        }
```

```
}
}
}
```

4.8.3.4. bulkloadIn(File[] files)

当 bulkloadInfo 中的 INPUT 参数为 path/function 时，用此方法会直接调研数据库的 bulkload 接口，忽略入参 files， 当 INPUT 参数为 stdin 时，JDBC 驱动会根据 files 读取指定文件内容，发送至数据库。

例如：

```
Public void testBulkloadWithFolderPathAndPattern() throws SQLException, I
OExecption {
bulkloadAPI.getBulkloadInfo().put("INPUT","stdin");
File[] file = new File[] {
    new File("src/test/resources/tbl_bulkload_small_1.txt"),
new File("src/test/resources/tbl_bulkload_small_3.txt"),
}
try {
    Is = readfile()
      ResultSet result = bulkloadAPI.bulkloadIn(files);
      While(result.next()) {
          for(int i = 1; i< result.getMetaData().getColumnCount() + 1; i ++)
{
```

```
            System.out.println(result.getMetaData().getColumnLabel(i) + resu
lt.getObject(result.getMetaData().getColumnLabel(i)));
        }
}
}
}
```

4.8.4. 使用批量接口

是用 bulkloadAPI 的接口，可以将内存数据批量加载至数据库中

例如：

```
Connection conn = DriverManager.getConnection(ulr);
BulkloadManager bulkloadAPI = ((PGConnection) conn).getBulkloadAPI();
int count = 0;
Record record;
While((record = S1Loader.parseFile(file))!= null) {
   count++;
     bulkloadAPI.setString(1,(String)record.get(0));
     bulkloadAPI.setNumber(2,(Number)record.get(1));
bulkloadAPI.addBatch();
if(count % 10000 == 0) {
   ResultSet rs = bulkloadAPI.executeBatch();
   Count = 0;
```

```
}
}
ResultSet res = bulkloadAPI.executeBatch();
```

4.9. 调用 CopyManager

CopyManager 是 openGauss JDBC 驱动中提供的一个 API 接口类，用于批量向 openGauss 中导入数据。

4.9.1. 获取 Copy 管理器

```
Connection connection = DriverManager.getConnection(URL,properties)
PGConnection pgConnection = connection.unwrap(PGConnection.class);
CopyManager copyManager = pgConnection.getCopyAPI();
```

4.9.2. 使用 Copy 管理器

4.9.2.1. copyIn(String sql, InputStream from, int bufferSize)

使用 COPY FROM STDIN 从 InputStream 中快速向数据库中的表加载数据。

例如：

```
Connection connection = DriverManager.getConnection(URL,properties)
PGConnection pgConnection = connection.unwrap(PGConnection.class);
```

```
CopyManager copyManager = pgConnection.getCopyAPI();
String sql = "COPY copytest FROM STDIN";
copyManager.copyIn(sql,
new ByteArrayInputStream(testInfo.getBytes()));
```

存在变形，即 bufferSize 为 DEFAULT_BUFFER_SIZE

copyIn(String sql, InputStream from)

4.9.2.2. copyIn(String sql, Reader from, int bufferSize)

使用 COPY FROM STDIN 从 Reader 中快速向数据库中的表加载数据。

例如

```
Connection connection = DriverManager.getConnection(URL,properties);
PGConnection pgConnection = connection.unwrap(PGConnection.class);
CopyManager copyManager = pgConnection.getCopyAPI()
String sql = "COPY copytest FROM STDIN";
copyManager.copyIn(sql,
        new StringReader(testInfo));
```

存在变形，即 bufferSize 为 DEFAULT_BUFFER_SIZE

4.9.2.3. copyOut(String sql, OutputStream to)

将一个 COPY TO STDOUT 的结果集从数据库发送到 OutputStream 类中。

```
Connection connection = DriverManager.getConnection(URL,properties);
```

```
PGConnection pgConnection = connection.unwrap(PGConnection.class);
CopyManager copyManager = pgConnection.getCopyAPI();
String sql = "COPY copytest to STDOUT";
ByteArrayOutputStream byteArrayOutputStream = new ByteArrayOutputStre
am();
copyManager.copyOut(sql,byteArrayOutputStream);
```

4.9.2.4. copyOut(String sql, Writer to)

将一个 COPY TO STDOUT 的结果集从数据库发送到 Writer 类中。

```
Connection connection = DriverManager.getConnection(URL,properties);
PGConnection pgConnection = connection.unwrap(PGConnection.class);
CopyManager copyManager = pgConnection.getCopyAPI();
String sql = "COPY copytest to STDOUT";
StringWriter stringWriter = new StringWriter();
copyManager.copyOut(sql,stringWriter);
```

4.9.3. 从 MY 向 PanWeiDB 进行数据迁移

下面示例演示如何通过 CopyManager 从 MY 向 PanWeiDB 进行数据迁移。

```
import java.io.StringReader;
import java.sql.Connection;
```

```
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

import org.postgresql.copy.CopyManager;
import org.postgresql.core.BaseConnection;

public class Migration{

    public static void main(String[] args) {
        String url = new String("jdbc:postgresql://1.2.4.5:8000/postgres");
//数据库 URL
        String user = new String("jack");              //用户名
        String pass = new String("Password@123");              //密码
        String tablename = new String("migration_table"); //定义表信息
        String delimiter = new String("|");                //定义分隔符
        String encoding = new String("UTF8");              //定义字符集
        String driver = "org.postgresql.Driver";
        StringBuffer buffer = new StringBuffer();        //定义存放格式化数
据的缓存

        try {
```

```
            //获取源数据库查询结果集
            ResultSet rs = getDataSet();


            //遍历结果集，逐行获取记录
            //将每条记录中各字段值，按指定分隔符分割，由换行符结束，拼成一
个字符串
            //把拼成的字符串，添加到缓存 buffer
            while (rs.next()) {
                buffer.append(rs.getString(1) + delimiter
                        + rs.getString(2) + delimiter
                        + rs.getString(3) + delimiter
                        + rs.getString(4)
                        + "\n");
            }
            rs.close();


            try {
                //建立目标数据库连接
                Class.forName(driver);
                Connection conn = DriverManager.getConnection(url, user,
 pass);
                BaseConnection baseConn = (BaseConnection) conn;
                baseConn.setAutoCommit(false);
```

```
                //初始化表信息
                String sql = "Copy " + tablename + " from STDIN DELIMIT
ER " + "'" + delimiter + "'" + " ENCODING " + "'" + encoding + "'";

                //提交缓存 buffer 中的数据
                CopyManager cp = new CopyManager(baseConn);
                StringReader reader = new StringReader(buffer.toString());
                cp.copyIn(sql, reader);
                baseConn.commit();
                reader.close();
                baseConn.close();
            } catch (ClassNotFoundException e) {
                e.printStackTrace(System.out);
            } catch (SQLException e) {
                e.printStackTrace(System.out);
            }

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
```

```
    //********************************
    // 从源数据库返回查询结果集
    //********************************
    private static ResultSet getDataSet() {
        ResultSet rs = null;
        try {
            Class.forName("com.MY.jdbc.Driver").newInstance();
            Connection conn = DriverManager.getConnection("jdbc:MY://1.
2.3.4:3306/jack?useSSL=false&allowPublicKeyRetrieval=true", "jack", "Passwo
rd@123");
            Statement stmt = conn.createStatement();
            rs = stmt.executeQuery("select * from migration_table");
        } catch (SQLException e) {
            e.printStackTrace();
        } catch (Exception e) {
            e.printStackTrace();
        }
        return rs;
    }
}
```

4.10. 逻辑复制

4.10.1. 配置选项

针对逻辑复制的配置选项，除了在逻辑解码中的配置选项外，还有专门给 JDBC 等流式解码工具增加的配置项，如下所示：

4.10.1.1. 解码线程并行度

通过配置选项 parallel-decode-num，指定并行解码的 Decoder 线程数量。其取值范围为 1~20 的 int 型，取 1 表示按照原有的串行逻辑进行解码，取其余值即为开启并行解码。默认值为 1。当该选项配置为 1 时，禁止配置以下选项：解码格式选项 decode-style、批量发送选项 batch-sending 和并行解码队列长度选项 parallel-queue-size。

4.10.1.2. 解码格式

通过配置选项 decode-style，指定解码格式。其取值为 char 型的字符'j'、't'或'b'，分别代表 json 格式、text 格式及二进制格式。默认值为'b'即二进制格式解码。该选项仅允许并行解码时设置，且二进制格式解码仅在并行解码场景下支持。与二进制格式对应的 json 和 text 格式，在批量发送的解码结果中，每条解码语句的前 4 字节组成的 uint32 代表该条语句总字节数（不包含该 uint32 类型占用的 4 字节，0 代表本批次解码结束），8 字节 uint64 代表相应 lsn（begin 对应 first_lsn，commit 对应 end_lsn，其他场景对应该条语句的 lsn）。

4.10.1.3. 限制仅备机解码

通过配置选项 standby-connection，指定是否限制仅备机解码。其取值为 bool 型（可用 0 或 1 表示），取 true（或 1）代表限制仅允许连接备机解码，连接主机解码时会报错退出；取 false（或 0）时不做限制。默认值为 false（0）。

4.10.1.4. 批量发送

通过配置选项 batch-sending，指定是否批量发送。其取值范围为 0 或 1 的 int 型，取 0 表示逐条发送解码结果，取 1 表示解码结果累积到达 1MB 则批量发送解码结果。默认值为 0。该选项仅允许并行解码时设置。开启批量发送的场景中，当解码格式为'j'或't'时，在原来的每条解码语句之前会附加一个 uint32 类型,表示本条解码结果长度(长度不包含当前的 uint32 类型),以及一个 uint64 类型，表示当前解码结果对应的 lsn。

4.10.1.5. 并行解码队列长度

通过配置选项 parallel-queue-size，指定并行逻辑解码线程间进行交互的队列长度。取值范围【2，1024】，且必须为 2 的幂数，默认值为 128。队列长度和解码过程的内存使用量正相关。

4.10.1.6. 逻辑解码内存阈值

通过配置选项 max-txn-in-memory 指定单个事务解码中间结果缓存的内存阈值；单位为 MB，范围【0，100】，默认值为 0，表示不管控内存使用。通过配置选项 max-reorderbuffer-in-memory 指定所有事务解码中间结果缓存的内存阈值；单位为 GB，范围【0，100】，默认值为 0，表示不管控内存使用。当超过内存阈值时，解码过程将出现解码中间结果写临时文件的现象，影响逻辑解码的性能。

4.10.1.7. 逻辑解码发送超时阈值

通过配置选项 sender-timeout 指定内核与客户端的心跳超时阈值。当该时间段内没有收到客户端任何消息，逻辑解码将主动停止，并断开和客户端的连接。单位为毫秒（ms），范围【0, 2147483647】，默认值取决于 GUC 参数 logical_sender_timeout 配置。

在并行解码的标准场景下（16 核 CPU、内存 128G、网络带宽 ＞200MBps、表的列数为 10~100、单行数据量 0.1KB~1KB、DML 操作以 insert 为主、不涉及落盘事务即单个事务中语句数量小于 4096、parallel-decode-num 为 8、解码格式为'b'且开启批量发送功能），解码性能（这里以 xlog 消耗量为标准）不低于 100MBps。为保证解码性能达标以及尽量降低对业务的影响，一台备机上应尽量仅建立一个并行解码连接，保证 CPU、内存、带宽资源充足。

4.10.2. 代码示例

注意：逻辑复制类 PGReplicationStream 为非线程安全类，并发调用可能导致数据异常。

```
//逻辑复制功能示例：文件名，LogicalReplicationDemo.java
//前提条件：添加 JDBC 用户机器 IP 到数据库白名单里，在 pg_hba.conf 添加以下内容即可：
//假设 JDBC 用户 IP 为 10.10.10.10
//host     all             all             10.10.10.10/32          sha256
//host     replication     all             10.10.10.10/32          sha256

import org.postgresql.PGProperty;
import org.postgresql.jdbc.PgConnection;
import org.postgresql.replication.LogSequenceNumber;
import org.postgresql.replication.PGReplicationStream;

import java.nio.ByteBuffer;
import java.sql.DriverManager;
import java.util.Properties;
import java.util.concurrent.TimeUnit;

public class LogicalReplicationDemo {
```

```
    private static PgConnection conn = null;
    public static void main(String[] args) {
        String driver = "org.postgresql.Driver";
//此处配置数据库 IP 以及端口，这里的端口为 haPort，通常默认是所连接 DN 的 p
ort+1 端口
        String sourceURL = "jdbc:postgresql://$ip:$port/postgres";
//默认逻辑复制槽的名称是：replication_slot
//测试模式：创建逻辑复制槽
        int TEST_MODE_CREATE_SLOT = 1;
//测试模式：开启逻辑复制（前提条件是逻辑复制槽已经存在）
        int TEST_MODE_START_REPL = 2;
//测试模式：删除逻辑复制槽
        int TEST_MODE_DROP_SLOT = 3;
        //开启不同的测试模式
        int testMode = TEST_MODE_START_REPL;


        try {
            Class.forName(driver);
        } catch (Exception e) {
            e.printStackTrace();
            return;
        }
```

```
        try {
            Properties properties = new Properties();
            PGProperty.USER.set(properties, "user");
            PGProperty.PASSWORD.set(properties, "passwd");
    //对于逻辑复制，以下三个属性是必须配置项
            PGProperty.ASSUME_MIN_SERVER_VERSION.set(properties, "9.4");
            PGProperty.REPLICATION.set(properties, "database");
            PGProperty.PREFER_QUERY_MODE.set(properties, "simple");
            conn = (PgConnection) DriverManager.getConnection(sourceUR
L, properties);
            System.out.println("connection success!");

            if(testMode == TEST_MODE_CREATE_SLOT){
                conn.getReplicationAPI()
                        .createReplicationSlot()
                        .logical()
                        .withSlotName("replication_slot") //这里字符串如包
含大写字母则会自动转化为小写字母
                        .withOutputPlugin("mppdb_decoding")
                        .make();
            }else if(testMode == TEST_MODE_START_REPL) {
                //开启此模式前需要创建复制槽
                LogSequenceNumber waitLSN = LogSequenceNumber.valu
```

```
eOf("6F/E3C53568");
                PGReplicationStream stream = conn
                        .getReplicationAPI()
                        .replicationStream()
                        .logical()
                        .withSlotName("replication_slot")
                        .withSlotOption("include-xids", false)
                        .withSlotOption("skip-empty-xacts", true)
                        .withStartPosition(waitLSN)
                        .withSlotOption("parallel-decode-num", 10) //解码
线程并行度
                        .withSlotOption("white-table-list", "public.t1,public.t
2") //白名单列表
                        .withSlotOption("standby-connection", true) //强制
备机解码
                        .withSlotOption("decode-style", "t") //解码格式
                        .withSlotOption("sending-batch", 1) //批量发送解码
结果
                        .withSlotOption("max-txn-in-memory", 100) //单个
解码事务落盘内存阈值为 100MB
                        .withSlotOption("max-reorderbuffer-in-memory", 50)
 //正在处理的解码事务落盘内存阈值为 50GB
                        .start();
```

```
                while (true) {
                    ByteBuffer byteBuffer = stream.readPending();

                    if (byteBuffer == null) {
                        TimeUnit.MILLISECONDS.sleep(10L);
                        continue;
                    }

                    int offset = byteBuffer.arrayOffset();
                    byte[] source = byteBuffer.array();
                    int length = source.length - offset;
                    System.out.println(new String(source, offset, length));

                    //如果需要 flush lsn，根据业务实际情况调用以下接口
                    //LogSequenceNumber lastRecv = stream.getLastRecei
veLSN();
                    //stream.setFlushedLSN(lastRecv);
                    //stream.forceUpdateStatus();

                }
            }else if(testMode == TEST_MODE_DROP_SLOT){
                conn.getReplicationAPI()
                        .dropReplicationSlot("replication_slot");
```

```
            }
        } catch (Exception e) {
            e.printStackTrace();
            return;
        }
    }
}
```

4.11. 使用连接池

PanWeiDB V2.0-S3.0.0_Beta jdbc 连接可使用连接池，比如阿里巴巴开源连接池 Druid。

4.11.1. 无框架配置

下载 Druid 的 jar 包，除 PanWeiDB V2.0-S3.0.0_Beta JDBC 驱动外，引入 pom 依赖如下：

```
<dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>druid</artifactId>
    <version>1.2.6</version>
</dependency>
<dependency>
    <groupId>log4j</groupId>
```

```
    <artifactId>log4j</artifactId>
<version>1.2.16</version>
</dependency>
```

在 resource 目录下配置 db_server.properties 文件

```
driverClassName=org.postgresql.Driver
url=jdbc:postgresql://127.0.0.1:5432/test
username=root
password=123456
filters=stat
initialSize=2
maxActive=300
maxWait=60000
timeBetweenEvictionRunsMillis=60000
minEvictableIdleTimeMillis=300000
validationQuery=SELECT 1
testWhileIdle=true
testOnBorrow=false
testOnReturn=false
poolPreparedStatements=false
maxPoolPreparedStatementPerConnectionSize=200;
```

Java 代码

```
// DBPoolConnection 类
```

```
import com.alibaba.druid.pool.*;
import org.apache.log4j.*;
import java.io.FileInputStream;
import java.sql.SQLException;
import java.util.Properties;

public class DBPoolConnection {
  static Logger log = LogManager.getLogger(DBPoolConnection.class);
  private static DBPoolConnection dbPoolConnection = null;
  private static DruidDataSource druidDataSource = null;

  static {
    Properties properties = new Properties();
    try {
      properties.load(new FileInputStream(ClassLoader.getSystemResource
("db_server.properties").getPath()));
      druidDataSource = (DruidDataSource)DruidDataSourceFactory.createD
ataSource(properties); //DruidDataSrouce 工厂模式
    } catch (Exception e) {
      log.error("获取配置失败");
    }
  }
```

```
    /**
    * 数据库连接池单例
    * @return
    */
    public static synchronized DBPoolConnection getInstance(){
        if (null == dbPoolConnection){
            dbPoolConnection = new DBPoolConnection();
        }
        return dbPoolConnection;
    }

    /**
    * 返回 druid 数据库连接
    * @return
    * @throws SQLException
    */
    public DruidPooledConnection getConnection() throws SQLException{
        return druidDataSource.getConnection();
    }
}
```

在 PanWeiDB V2.0-S3.0.0_Beta 创建表 student：

```
create table student(id int,name varchar(20),num int,score numeric(4,2));
```

运行以下代码连接 PanWeiDB V2.0-S3.0.0_Beta 并写入一条数据到表 student

```
import com.alibaba.druid.pool.DruidPooledConnection;
import java.sql.*;


public class TestDruid{
    public static void main(String[] args) throws SQLException {
        DruidPooledConnection dsConn = DBPoolConnection.getInstance().g
etConnection();
        Connection conn = dsConn.getConnection();


        //execute SQL
        String sql = "INSERT INTO student VALUES (NULL, ?, ?, ?);";
        PreparedStatement pstmt = conn.prepareStatement(sql);
        pstmt.setString(1, "sam");
        pstmt.setInt(2, 35);
        pstmt.setDouble(3, 88.5);
        int i = pstmt.executeUpdate();
        System.out.println("insert rows:  " + i);


        pstmt.close();
        //release connection and give back to pool
        conn.close();
```

```
    }
}
```

4.11.2. Tomcat 配置

Tomcat 中植入。

4.11.3. Tomcat 依赖修改

下载 druid 的依赖 jar，放入到 tomcat/lib 目录下。

4.11.4. web.xml 配置

修改 tomcat 下的 web.xml，里面增加 druid 配置：增加在 web.xml 中 servlet 的 default 配置后面

```
<!-- druid view -->
<servlet>
    <servlet-name>DruidStatView</servlet-name>
    <servlet-class>com.alibaba.druid.support.http.StatViewServlet</servlet-cl
ass>
    <init-param>
        <!-- 允许清空统计数据 -->
        <param-name>resetEnable</param-name>
        <param-value>true</param-value>
```

```
    </init-param>
    <init-param>
        <!-- 用户名 -->
        <param-name>loginUsername</param-name>
        <param-value>admin</param-value>
    </init-param>
    <init-param>
        <!-- 密码 -->
        <param-name>loginPassword</param-name>
        <param-value>admin</param-value>
    </init-param>
</servlet>

<servlet-mapping>
    <servlet-name>DruidStatView</servlet-name>
    <url-pattern>/druid/*</url-pattern>
</servlet-mapping>
<!-- druid view -->
```

4.11.5. context.xml 配置

将 source 中的 factory 修改为：

com.alibaba.druid.pool.DruidDataSourceFactory

```
<Resource  name="prpDataSource"

           auth="Container"

           type="javax.sql.DataSource"

           factory="com.alibaba.druid.pool.DruidDataSourceFactory"

           defaultTransactionIsolation="READ_COMMITTED"

           username="用户名"

           password="密码"

           filters="stat,wall"

           driverClassName="org.postgresql.Driver"

           url="jdbc:postgresql://127.0.0.1:5432/dbname"

           maxActive="96"

           minIdle="8"

           removeabandoned="true"

           removeabandonedtimeout="60"

           logabandoned="true">

</Resource>
```

4.11.6. catalina.properties 文件

如果 druid 没有加载到，再修改 catalina.properties 文件

```
115 tomcat-api.jar,tomcat-util.jar,tomcat-util-scan.jar,tomcat-coyote.jar,\
116 druid-1.1.9.jar,tomcat-dbcp.jar,tomcat-jni.jar,tomcat-websocket.jar,\
117 tomcat-i18n-en.jar,tomcat-i18n-es.jar,tomcat-i18n-fr.jar,tomcat-i18n-ja.jar,\
118 tomcat-juli-adapters.jar,catalina-jmx-remote.jar,catalina-ws.jar,\
119 tomcat-jdbc.jar,\
120 tools.jar,\
121 commons-beanutils*.jar,commons-codec*.jar,commons-collections*.jar,\
122 druid-1.1.9.jar,commons-dbcp*.jar,commons-digester*.jar,commons-fileupload*.jar,\
123 commons-httpclient*.jar,commons-io*.jar,commons-lang*.jar,commons-logging*.jar,\
```

4.11.6.1. 项目中配置

4.11.7. 引入 pom 依赖

```
<dependency>
    <groupId>com.alibaba.druid</groupId>
    <artifactId>druid-wrapper</artifactId>
    <version>0.2.9</version>
</dependency>
<dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>druid</artifactId>
    <version>1.1.9</version>
</dependency>
<dependency>
    <groupId>log4j</groupId>
    <artifactId>log4j</artifactId>
    <version>1.2.8</version>
</dependency>
```

4.11.8. context.xml 与 web.xml 配置

参考 4.11.4 中的相关配置。

4.11.9. filter 相关配置

如果项目中有使用 Authorization 权限拦截的，需要在白名单中配置/druid/路径

```
<filter>
    <filter-name>Authorization</filter-name>
    <filter-class>
        com.sinosoft.common.AuthorizationFilter
    </filter-class>
    <init-param>
        <param-name>loginPage</param-name>
        <param-value>/index.html</param-value>
    </init-param>
    <init-param>
        <param-name>valid</param-name>
        <param-value>true</param-value>
    </init-param>
    <init-param>
        <param-name>ignore</param-name>
        <param-value>
            /login.do;
            /claim/swfFlowBeforeQuery.do;
            /interFaces/;
```

```
            /prpall/;
            /www/;
            /druid/;
        </param-value>
    </init-param>
</filter>
```

4.11.10. Spring 配置

引入相关依赖

```
<!-- SPRINGBOOT WEB -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <!-- SPRINGBOOT DRUID -->
        <dependency>
            <groupId>com.alibaba</groupId>
            <artifactId>druid-spring-boot-starter</artifactId>
            <version>1.1.10</version>
        </dependency>

        <!-- SPRINGBOOT JDBC -->
```

```
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-jdbc</artifactId>
        </dependency>

        <!-- PanWeiDB V2.0-S3.0.0_Beta -->
        <dependency>
            <groupId>cn.com.panweidb</groupId>
            <artifactId>panweidb</artifactId>
            <version>${panweidb-jdbc.version}</version>
        </dependency>

        <!-- LOG4J -->
        <dependency>
            <groupId>log4j</groupId>
            <artifactId>log4j</artifactId>
            <version>1.2.16</version>
            <scope>compile</scope>
        </dependency>
```

4.11.10.1. xml 配置

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:aop="http://www.springframework.org/schema/aop" xmlns:xsi="
http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd
        http://www.springframework.org/schema/aop
        http://www.springframework.org/schema/aop/spring-aop.xsd">
    <!-- 配置_Druid 和 Spring 关联监控配置 -->
    <bean id="druid-stat-interceptor"
        class="com.alibaba.druid.support.spring.stat.DruidStatInterceptor"><
/bean>
    <!-- 方法名正则匹配拦截配置 -->
    <bean id="druid-stat-pointcut" class="org.springframework.aop.support.
JdkRegexpMethodPointcut"
        scope="prototype">
        <property name="patterns">
            <list>
              <!--此处根据实际项目的包名更改-->
                <value>com.beini.*</value>
            </list>
        </property>
    </bean>
```

```
    <aop:config proxy-target-class="true">
        <aop:advisor advice-ref="druid-stat-interceptor"
  pointcut-ref="druid-stat-pointcut" />
    </aop:config>
</beans>
```

4.11.10.2. 结合 mybatis 的 xml 配置

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:context="http://www.springframework.org/schema/context" xmlns:
p="http://www.springframework.org/schema/p"
xmlns:aop="http://www.springframework.org/schema/aop" xmlns:tx="http:
//www.springframework.org/schema/tx"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:util="http://www.springframework.org/schema/util"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
 http://www.springframework.org/schema/beans/spring-beans-4.0.xsd
  http://www.springframework.org/schema/context http://www.springfra
mework.org/schema/context/spring-context-4.0.xsd
  http://www.springframework.org/schema/aop http://www.springframew
ork.org/schema/aop/spring-aop-4.0.xsd http://www.springframework.org/
schema/tx http://www.springframework.org/schema/tx/spring-tx-4.0.xsd
```

```
  http://www.springframework.org/schema/util http://www.springframew
ork.org/schema/util/spring-util-4.0.xsd">
<!-- 配置 dao 层扫描 -->
  <context:property-placeholder location="classpath:conf/database.properti
es" />
     <!-- 通过 JDBC 模版获取数据库连接 -->
    <bean scope="singleton" id="jdbcTemplate"
        class="org.springframework.jdbc.core.JdbcTemplate">
        <property name="dataSource" ref="dataSource"></property>
    </bean>
  <!-- 数据库连接池 -->
  <bean id="dataSource" class="com.alibaba.druid.pool.DruidDataSource"
    destroy-method="close">
    <property name="url" value="${jdbc.url}" />
    <property name="username" value="${jdbc.username}" />
    <property name="password" value="${jdbc.password}" />
    <property name="driverClassName" value="${jdbc.driverClassName}" />
  </bean>

  <!-- 让 spring 管理 sqlsessionfactory 使用 mybatis 和 spring 整合包中的 -->
  <bean id="sqlSessionFactory" class="org.mybatis.spring.SqlSessionFactory
Bean">
    <!-- 数据库连接池 -->
```

```
    <property name="dataSource" ref="dataSource" />
    <!-- 加载 mybatis 的全局配置文件 -->
    <property name="mapperLocations" value="classpath:com/mapper/*.x
ml" />
  </bean>

  <!-- 配置 mapper 扫描包 -->
  <bean class="org.mybatis.spring.mapper.MapperScannerConfigurer">
    <property name="basePackage" value="com.mapper" />
  </bean>
</beans>
```

4.11.10.3. yml 配置

```
spring:
  # 数据源配置
  datasource:
    type: com.alibaba.druid.pool.DruidDataSource
    driver-class-name: org.postgresql.Driver
    url: jdbc:postgresql://127.0.0.1:5432/test
    username: root
    password: 123456
    # 连接池配置
```

```
    druid:
      # 初始化大小，最小，最大
      initial-size: 5
      min-idle: 5
      max-active: 20
      # 配置获取连接等待超时的时间
      max-wait: 60000
      # 配置间隔多久才进行一次检测，检测需要关闭的空闲连接，单位毫秒
      time-between-eviction-runs-millis: 60000
      # 配置一个连接在池中最小生存时间
      min-evictable-idle-time-millis: 300000
      validation-query: SELECT 1 FROM sys_user
      test-while-idle: true
      test-on-borrow: false
      test-on-return: false
      # 打开 PSCache，并且指定每个连接上 PSCache 的大小
      pool-prepared-statements: true
      max-pool-prepared-statement-per-connection-size: 20
      # 配置监控统计拦截的 Filter，去掉后监控界面 SQL 无法统计，wall 用于防
火墙
      filters: stat,wall,log4j
      # 通过 connection-properties 属性打开 mergeSql 功能；慢 SQL 记录
      connection-properties: druid.stat.mergeSql\=true;druid.stat.slowSqlMi
```

```
llis\=5000
        # 配置 DruidStatFilter
        web-stat-filter:
          enabled: true
          url-pattern: /*
          exclusions: .js,*.gif,*.jpg,*.bmp,*.png,*.css,*.ico,/druid/*
        # 配置 DruidStatViewServlet
        stat-view-servlet:
          url-pattern: /druid/*
          # IP 白名单，没有配置或者为空，则允许所有访问
          allow: 127.0.0.1
          # IP 黑名单，若白名单也存在，则优先使用
          deny: 192.168.31.253
          # 禁用 HTML 中 Reset All 按钮
          reset-enable: false
          # 登录用户名/密码
          login-username: root
          login-password: 123
```

4.12. 其他功能与优化

4.12.1. 读写分离&负载均衡

PanWeiDB V2.0-S3.0.0_Beta 数据库使用流复制可搭建主从架构，即主机可读可写，从机只读。

假设应用端可支持读写分离，即为读写操作分别构建不同的连接或连接池。假设使用连接池，则可以创建一个读写连接池，一个只读连接池，当写操作时使用读写连接池，读操作时使用只读连接池。

PanWeiDB V2.0-S3.0.0_Beta 数据库 JDBC 驱动内置了一个简单的负载均衡功能，JDBC 驱动在连接到数据库服务端后，会对测试数据库是否符合 targetServerType 要求，如符合要求则会进行 sql 请求。

当正在执行 sql 语句的数据库节点发生当宕机，使用 JDBC 驱动的客户端会接收到错误 “FATAL: terminating connection due to administrator command”，JDBC 驱动不会对 sql 命令进行二次分发。

4.12.1.1. 参数说明

参考 4.4 章节中 targetServerType 和 loadBalanceHosts 的参数设置说明。

4.12.1.2. 样例

4.12.2. 写操作

写操作连接或连接池的 JDBC 连接字符串

jdbc:postgresql://node1,node2,node3/accounting?targetServerTyp
e=master

4.12.3. 读操作

读操连接或连接池的 JDBC 连接字符串

```
jdbc:postgresql://node1,node2,node3/accounting?targetServerType=preferSl
ave&loadBalanceHosts=true
```

4.12.4. Socket 相关

Unix Domain Socket 是进程间通讯的一种方式，适用于同一台服务器内不同进程的通讯。PanWeiDB V2.0-S3.0.0_Beta 支持使用 Unix Domain Socket 访问数据库，Java 应用希望以 Unix Domain Socket 访问 PanWeiDB V2.0-S3.0.0_Beta 数据库，需要添加新的类库支持。

4.12.4.1. 参数说明

参考 4.4 章节中 socketFactory 和 socketFactoryArg 的参数设置说明。

4.12.4.2. 样例

使用 maven 添加相关类库支持

```
<dependency>
  <groupId>no.fiken.oss.junixsocket</groupId>
  <artifactId>junixsocket-common</artifactId>
  <version>1.0.2</version>
</dependency>
<dependency>
  <groupId>no.fiken.oss.junixsocket</groupId>
  <artifactId>junixsocket-native-common</artifactId>
  <version>1.0.2</version>
</dependency>
```

JDBC 连接字符串添加参数：

```
jdbc:postgresql://host:port/database?socketFactory=org.newsclub.net.unix.socketfactory.PostgresqlAFUNIXSocketFactory&socketFactoryArg=[path-to-the-unix-socket]
```

5. libpq 接口开发指南

5.1. 介绍

PanWeiDB V2.0-S3.0.0_Beta 数据库通过提供 libpq 库为使用 C 语言的应用程序提供访问 PanWeiDB V2.0-S3.0.0_Beta 的接口，它们允许客户端程序传递查询给 PanWeiDB V2.0-S3.0.0_Beta 后端服务器并且接收这些查询的结果。

5.1.1. libpq 库与 PanWeiDB V2.0-S3.0.0_Beta

libpq 库与相关头文件是 PanWeiDB V2.0-S3.0.0_Beta 产品的一部分，随 PanWeiDB V2.0-S3.0.0_Beta 产品一起发布。使用时应注意 PanWeiDB V2.0-S3.0.0_Beta 和 libpq 库与头文件的匹配性，即 PanWeiDB V2.0-S3.0.0_Beta 主产品与 libpq 相关开发库及头文件必须来自于同一版本。

5.2. libpq 库主要接口类型

PanWeiDB V2.0-S3.0.0_Beta libpq 库由具有建立与 PanWeiDB V2.0-S3.0.0_Beta 数据库连接、执行 SQL 语句和对结果集进行获取的若干类型接口组成。

- 数据库连接控制函数

  这类函数会建立一个 PanWeiDB V2.0-S3.0.0_Beta 后端服务器的连接。一个应用程序可以在一个时刻打开多个后端连接。每个连接用一个 PGconn

对象表示，它从函数 PQconnectdb、PQconnectdbParams 或 PQsetdbLogin 得到。简单列举相关函数：

```
PQconnectdbParams

PQconnectdb

PQsetdbLogin

PQsetdb

PQconnectStartParams(非阻塞)

PQconnectStart（非阻塞）

PQconnectPoll（非阻塞）
```

❖ 连接状态函数

当需要获取一个已有数据库连接对象的状态时，如获取连接的数据库名、用户名、主机名等信息，可以调用连接状态函数来获取，简单列举相关函数：

```
PQdb

PQuser

PQpass

PQhost

PQport
```

❖ 同步命令执行函数

与特定数据库的连接，在连接上下文中执行 SQL 语句并返回结果。这些函数均为同步函数，需要等待命令完成才会返回。简单列举相关函数：

```
PQexec
```

```
PQexecParams

PQprepare

PQexecPrepared
```

❖ 异步命令执行函数

与特定数据库的连接，在连接上下文中执行 SQL 语句并返回结果。这些函数均为异步函数，不等待结果立即返回。示例：

```
PQsendQuery

PQsendQueryParams

PQsendPrepare

PQsendQueryPrepared

PQsendDescribePrepared
```

❖ 获取查询结果函数

获取 SQL 语句产生的结果集。简单列举相关函数：

```
PQntuples

PQnfields

PQfname

PQfnumber
```

❖ 逐行获取查询结果函数

通常，libpq 会收集一个 SQL 命令的整个结果并且把它作为单个 PGresult 返回给应用。这对于返回大量行的命令是行不通的。对于这类情况，应用可以使用 PQsendQuery 和 PQgetResult 的单行模式。在这种模式中，结果行以一次一行的方式被返回给应用。以此来获取 SQL 语句产生的结果集。要进入到单行模式，在一次成功的 PQsendQuery（或者其他兄弟函数）调用后立即调用 PQsetSingleRowMode。这种模式选择只对当前正在执行的

查询有效。然后反复调用 PQgetResult，直到它返回空。

❖ 取消查询函数

当需要请求取消一个仍在被服务器处理的命令时，可以采用此类函数。

```
PQgetCancel

PQfreeCancel

PQrequestCancel
```

❖ 控制函数

当需要对 libpq 行为中的各项细节进行控制时，可以调用此类函数。如获取客户端编码、设置客户端编码、设置错误消息的详细程度、跟踪客户端和服务端的通讯等。

```
PQclientEncoding

PQsetClientEncoding

PQsetErrorVerbosity

PQtrace
```

5.3. 编译 libpq 程序

要编译（即编译并且链接）一个使用 libpq 的程序，你需要做下列所有的事情：

包括 libpq-fe.h 头文件：

```
#include <libpq-fe.h>
```

如果你无法这样做，那么你通常会从你的编译器得到像这样的错误消息：

```
foo.c: In function `main':

foo.c:34: `PGconn' undeclared (first use in this function)
```

```
foo.c:35: `PGresult' undeclared (first use in this function)
foo.c:54: `CONNECTION_BAD' undeclared (first use in this function)
foo.c:68: `PGRES_COMMAND_OK' undeclared (first use in this function)
foo.c:95: `PGRES_TUPLES_OK' undeclared (first use in this function)
```

通过为你的编译器提供 -Idirectory 选项，向你的编译器指出 PanWeiDB V2.0-S3.0.0_Beta 头文件安装在哪里（在某些情况下编译器默认将查看该目录，因此你可以忽略这个选项）。例如你的编译命令行可能看起来像：

```
cc -c -I/usr/local/panweidb/include testprog.c
```

如果你在使用 makefile，那么把该选项加到 CPPFLAGS 变量中：

```
CPPFLAGS += -I/usr/local/panweidb/include
```

如果你的程序可能由其他用户编译，那么你不应该像那样硬编码目录位置。你可以运行工具 pg_config 在本地系统上找出头文件在哪里：

```
$ pg_config --includedir
/usr/local/include
```

无法为编译器指定正确的选项将导致一个错误消息，例如：

```
testlibpq.c:8:22: libpq-fe.h: No such file or directory
```

当链接最终的程序时，指定选项-lpq，这样 libpq 库会被编译进去，也可以用选项-Ldirectory 向编译器指出 libpq 库所在的位置（再次，编译器将默认搜索某些目录）。为了最大的可移植性，将-L 选项放在-lpq 选项前面。例如：

```
cc -o testprog testprog1.o testprog2.o -L/usr/local/pgsql/lib -lpq
```

你也可以使用 pg_config 找出库目录：

```
$ pg_config --libdir

/usr/local/pgsql/lib
```

指出这一部分问题的错误消息可能看起来像：

```
testlibpq.o: In function `main':

testlibpq.o(.text+0x60): undefined reference to `PQsetdbLogin'

testlibpq.o(.text+0x71): undefined reference to `PQstatus'

testlibpq.o(.text+0xa4): undefined reference to `PQerrorMessage'
```

这意味着你忘记了 -lpq.

```
/usr/bin/ld: cannot find -lpq
```

这意味着你忘记了-L 选项或者没有指定正确的目录。

5.4. 使用 libpq 连接数据库

5.4.1. 连接数据库

一个应用程序可以在一个时刻打开多个后端连接（原因之一就是为了访问多个数据库）。每个连接用一个 PGconn 对象表示，它从函数 PQconnectdb、PQconnectdbParams 或 PQsetdbLogin 得到。在通过该连接对象发送查询之前，应该调用 PQstatus 函数来检查返回值以确定是否得到了一个成功的连接。

```
#include <stdio.h>

#include <stdlib.h>

#include "libpq-fe.h"
```

```
int
main(int argc, char **argv)
{
    const char *conninfo;
PGconn      *conn;
    conninfo = "user = pwadmin password = 1234Abcde dbname = panw
eidb";

    /* 建立到数据库的一个连接 */
    conn = PQconnectdb(conninfo);

    /* 检查看后端连接是否成功建立 */
    if (PQstatus(conn) != CONNECTION_OK)
    {
        fprintf(stderr, "Connection to database failed: %s",
                PQerrorMessage(conn));
        PQfinish(conn);
    }
    /* 关闭到数据库的连接并且清理 */
PQfinish(conn);

return 0;
```

```
}
```

5.4.2. 非阻塞连接数据库

以非阻塞的方式建立一个到数据库服务器的连接：

PQconnectStartParams、PQconnectStart、PQconnectPoll 三个函数被用来开启一个到数据库服务器的连接，这样你的应用的执行线程不会因为远程的 I/O 而被阻塞。这种方法的要点在于等待 I/O 完成可能在应用的主循环中发生，而不是在 PQconnectdbParams 或 PQconnectdb 中，因此应用能够把这种操作和其他动作并行处理。

```
#include <stdio.h>
#include <stdlib.h>
#include "libpq-fe.h"

int
main(int argc, char **argv)
{
    const char *conninfo, feedback;
    PGconn      *conn;
    int          sock;

    conninfo = "postgresql://pwadmin:1234Abcde@localhost:5432/panwei
db"
```

```
  /* 建立到数据库的一个连接 */
  conn = PQconnectStart(conninfo);

 switch(PQstatus(conn))
 {
  case CONNECTION_STARTED:
    feedback = "Connecting...";
    break;

  case CONNECTION_MADE:
    feedback = "Connected to server...";
    break;
  default:
    feedback = "Connecting...";
}
fprintf(stderr, "Connection to database failed: %s\n", feedback);

sock = PQsocket(conn);
  if (sock < 0)
{
  /* 不应该发生 */
  PQfinish(conn);
```

```
    return 0;
  }
    /* 关闭到数据库的连接并且清理 */
PQfinish(conn);

return 0;
}
```

5.4.3. 连接字符串

几个 libpq 函数会解析一个用户指定的字符串来获得连接参数。这些字符串有两种被接受的格式：纯关键词 = 值字符串以及 URI。URI 通常遵循 RFC 3986，除非像下文进一步描述的那样允许多主机连接字符串。

5.4.3.1. URL 格式

一个连接 URI 的一般形式是：

```
postgresql://[user[:password]@][netloc][:port][,...][/dbname][?param1=value1&...]
```

URI 模式标志符可以是 postgresql://或 postgres://。每一个 URI 部分都是可选的。如：

```
postgresql://pwadmin:1234Abcde@localhost:5432/panweidb?host=/tmp&hostaddr=127.0.0.1
```

5.4.3.2. 参数关键字

主要的参数关键词如下

- host

要连接的主机名，如果一个主机名以斜线开始，则表示一个 Unix 域通信而不是 TCP/IP 通信，其值是存储套接字文件的目录名。当 host 没有指定或者为空时的默认行为是连接到一个/tmp 中的 Unix 域套接字。在没有 Unix 域套接字的机器上，默认是连接到 localhost。

- hostaddr

要连接的主机的数字 IP 地址。它应该是标准的 IPv4 地址格式，例如 172.28.40.9。如果你的机器支持 IPv6，也可以使用 IPv6 地址。

- port

服务端监听的端口，默认为 5432.

- dbname

数据库名。

- user

连接用户。

- password

服务器要求口令认证时要使用的口令。

- passfile

指定用于存放口令的文件名）。默认是~/.pgpass，或者是 Microsoft Windows 上的%APPDATA%\postgresql\pgpass.conf（如果该文件不存在也不会报错）。

- connect_timeout

  连接的最大等待时间，以秒为单位（写作一个十进制整数，例如 10）。零、负值或者不指定表示无限等待。允许的最小超时是 2 秒，因此值 1 会被解释为 2。这个超时单独应用于每个主机名或者 IP 地址。例如，如果指定两个主机并且 connect_timeout 为 5，每个主机都会在 5 秒内没有建立起连接的情况下超时，因此花费在等待连接上的总时间可能高达 10 秒。

- client_encoding

  为连接设置 client_encoding 配置参数。除了被相应服务器选项所接受的值，你还能使用 auto 从客户端的当前区域（Unix 系统上的 LC_CTYPE 环境变量）决定正确的编码。

- options

  指定在连接开始时发送给服务器的命令行选项。例如，设置这个参数为-c geqo=off 会把会话的 geqo 参数值设置为 off。这个字符串中的空格被认为是命令行参数的分隔符，除非用一个反斜线（\）对它转义，用\\可以表示一个字面意义上的反斜线。可用选项的详细讨论请参考《用户手册》。

- application_name

  为 application_name 配置参数指定一个值。

5.4.4. 关闭到数据库的连接

调用 PQfinish 关闭与服务器的连接。同时释放 PGconn 对象使用的内存。

注意，即使与服务器的连接尝试失败（由 PQstatus 指示），应用也应当调用 PQfinish 来释放 PGconn 对象使用的内存。不能在调用 PQfinish 之后再使用 PGconn 指针。

```
PQfinish(conn);
```

```
connection.close();
```

5.5. 同步命令处理

数据库服务器的连接被成功建立后，这里描述的函数可以被用来执行 SQL 查询和命令。这里的函数均为同步函数，在返回结果之前会阻塞。

5.5.1. 执行查询

使用 PQnfields、PQfname 等函数，程序将一次性获取查询的所有结果。开发者可以通过使用游标，配合 fetch 命令逐条数据取出结果。也可通过 libpq 提供的接口来逐行获取数据。本例中采用 pqexec 的方式简单的使用游标，并采用 Fetch all 命令一次性获取所有的结果。并调用 PQfname 函数获取所有字段的名称，调用 PQntuples 函数获取元组的数量，并最终循环调用 PQgetvalue 函数得到每个元组的数据。

```
#include <stdio.h>
#include <stdlib.h>
#include "libpq-fe.h"

static void
exit_nicely(PGconn *conn)
{
    PQfinish(conn);
    exit(1);
```

```
}


int
main(int argc, char **argv)
{
    const char *conninfo;
    PGconn     *conn;
    PGresult   *res;
    int         nFields;
    int         i,j;


    /*
     * 如果用户在命令行上提供了一个参数，将它用作连接信息串。
     * 否则默认用设置 dbname=panweidb 并且为所有其他链接参数使用环境变
量或默认值。
     */
    if (argc > 1)
        conninfo = argv[1];
    else
        conninfo = "dbname = panweidb";


    /* 建立到数据库的一个连接 */
    conn = PQconnectdb(conninfo);
```

```
    /* 检查看后端连接是否成功建立 */
    if (PQstatus(conn) != CONNECTION_OK)
    {
        fprintf(stderr, "Connection to database failed: %s",
                PQerrorMessage(conn));
        exit_nicely(conn);
    }

    /*
     * 我们的测试案例这里涉及使用一个游标，对它我们必须用在一个事务块内。
     * 我们可以在一个单一的 "select * from pg_database" 的 PQexec() 中做
整个事情，
     */

    /* 开始一个事务块 */
    res = PQexec(conn, "BEGIN");
    if (PQresultStatus(res) != PGRES_COMMAND_OK)
    {
        fprintf(stderr, "BEGIN command failed: %s", PQerrorMessage(con
n));
        PQclear(res);
        exit_nicely(conn);
```

```
    }

    PQclear(res);

    /*
     * 从 pg_database 取得行，它是数据库的系统目录
     */
    res = PQexec(conn, "DECLARE myportal CURSOR FOR select * from pg
_database");
    if (PQresultStatus(res) != PGRES_COMMAND_OK)
    {
        fprintf(stderr, "DECLARE CURSOR failed: %s", PQerrorMessage(con
n));
        PQclear(res);
        exit_nicely(conn);
    }
    PQclear(res);

    res = PQexec(conn, "FETCH ALL in myportal");
    if (PQresultStatus(res) != PGRES_TUPLES_OK)
    {
        fprintf(stderr, "FETCH ALL failed: %s", PQerrorMessage(conn));
        PQclear(res);
```

```
        exit_nicely(conn);
    }

    /* 首先，打印出属性名 */
    nFields = PQnfields(res);
    for (i = 0; i < nFields; i++)
        printf("%-15s", PQfname(res, i));
    printf("\n\n");

    /* 接下来，打印出行 */
    for (i = 0; i < PQntuples(res); i++)
    {
        for (j = 0; j < nFields; j++)
            printf("%-15s", PQgetvalue(res, i, j));
        printf("\n");
    }

    PQclear(res);

    /* 关闭入口，我们不需要考虑检查错误 */
    res = PQexec(conn, "CLOSE myportal");
    PQclear(res);
```

```
    /* 结束事务 */
    res = PQexec(conn, "END");
    PQclear(res);

    /* 关闭到数据库的连接并且清理 */
    PQfinish(conn);

    return 0;
}
```

5.5.2. 使用 PQprepare 接口

PQprepare 创建一个后面会由 PQexecPrepared 执行的预备语句，这个特性允许命令被反复执行而无需每次都进行解析和规划，各函数的使用方式及必要的参数使用举例说明请参考代码注释。

```
#include <stdio.h>
#include <stdlib.h>
#include "libpq-fe.h"

static void
exit_nicely(PGconn *conn)
{
 PQfinish(conn);
```

```
 exit(EXIT_SUCCESS);
}


int
main(int argc, char **argv)
{
 int     nFields;
 int     i,j;
 const char *conninfo;
 PGconn      *conn;
 PGresult    *res;


 if (argc > 1)
         conninfo = argv[1];
    else
         conninfo = "dbname = panweidb";


    /* 建立到数据库的一个连接 */
    conn = PQconnectdb(conninfo);


 /* 查看连接是否成功 */
 if (PQstatus(conn) == CONNECTION_BAD)
 {
```

```
 fprintf(stderr, "Connection to database failed.\n");
 fprintf(stderr, "%s", PQerrorMessage(conn));
 exit_nicely(conn);
}

/* 开启事务块 */
res = PQexec(conn, "BEGIN");
if (PQresultStatus(res) != PGRES_COMMAND_OK)
{
 fprintf(stderr, "BEGIN command failed\n");
 PQclear(res);
 exit_nicely(conn);
}

PQclear(res);

const char *stmt_name = "test_stmt";
/* 注意若存在参数，则以$1，$2 来表示 */
const char *stmt = "select * from pg_class where relname=$1";

Oid param_types[1];
param_types[0] = 0; //自动推测参数类型
```

```
/* 提交一个请求创建一个预备语句并且等待完成，当前 stmt 语句只有 1 个参数。 *
/
res = PQprepare(conn, stmt_name, stmt, 1, param_types);
if (PQresultStatus(res) != PGRES_COMMAND_OK)
{
 fprintf(stderr, "PQprepare failed\n");
 PQclear(res);
 exit_nicely(conn);
}

PQclear(res);

const char* theRelname= "pg_type";
const char* param_values[1];
param_values[0] = theRelname;

int param_lengths[1];
param_lengths[0] = 1;

int param_formats[1];
param_formats[0] = 0;

/* 使用 PQexecPrepared 发送一个请求来用给定参数执行一个预备语句，并且等待
```

```
结果。 */
 res = PQexecPrepared(conn, stmt_name, 1, param_values, param_length
s,
                      param_formats, 0);

 if (PQresultStatus(res) != PGRES_TUPLES_OK)
 {
  fprintf(stderr, "PQexecPrepared statement didn't return tuples properly\n
");
  PQclear(res);
  exit_nicely(conn);
 }

 /* 这里依然采用 PQnfields 获取字段数，采用 PQfname 获取各字段名 */
 nFields = PQnfields(res);
 for (i = 0; i < nFields; i++)
  printf("%-15s", PQfname(res, i));

 printf("\n\n");

 /* 采用 PQntuples 获取元组数量，采用 PQgetvalue 获取各行元组数据 */
 for (i = 0; i < PQntuples(res); i++)
 {
```

```
    for (j = 0; j < nFields; j++)
        printf("%-15s", PQgetvalue(res, i, j));
    printf("\n");
}

PQclear(res);

/* 结束事务 */
res = PQexec(conn, "END");
PQclear(res);

/* 关闭连接，清理 */
PQfinish(conn);
return 0;
}
```

5.6. 异步命令处理

PQexec 函数对于在普通的同步应用中提交命令是足以胜任的。不过，它的一些缺点可能对某些用户很重要：

1. PQexec 会等待命令完成。该应用可能有其他的工作要做（例如维护用户界面），这时它将不希望阻塞等待回应。

2. 因为客户端应用的执行在它等待结果时会被挂起，对于应用来说很难决定要不要尝试取消正在进行的命令（这可以在一个信号处理器中完成，但别无他法）。
3. PQexec 只能返回一个 PGresult 结构。如果提交的命令串包含多个 SQL 命令， 除了最后一个 PGresult 之外都会被 PQexec 丢弃。
4. PQexec 总是收集命令的整个结果,把它缓存在一个单一的 PGresult 中。虽然这简化了应用的错误处理逻辑,它对于包含很多行的结果并不现实。

不想受到这些限制的应用可以改用构建 PQexec 的底层函数：PQsendQuery 以及 PQgetResult。还有 PQsendQueryParams、PQsendPrepare、 PQsendQueryPrepared、 PQsendDescribePrepared 以及 PQsendDescribePortal， 它们可以与 PQgetResult 一起使用来分别复制 PQexecParams、 PQprepare、 PQexecPrepared、 PQdescribePrepared 和 PQdescribePortal 的功能。

5.6.1. 使用预备语句

❖ PQsendPrepare

发送一个请求用给定参数创建一个预备语句，而不等待完成。

```
int PQsendPrepare(PGconn *conn,
                  const char *stmtName,
                  const char *query,
                  int nParams,
```

```
                    const Oid *paramTypes);
```

这个函数是 PQprepare 的异步版本：如果它能发送这个请求，则返回 1；如果不能，则返回 0。在成功调用之后，调用 PQgetResult 判断服务器是否成功创建了预备语句。这个函数的参数的处理和 PQprepare 一样。和 PQprepare 类似，它不能在 2.0 协议的连接上工作。

- PQsendQueryPrepared

发送一个请求用给定参数执行一个预备语句，而不等待结果。

```
int PQsendQueryPrepared(PGconn *conn,
                        const char *stmtName,
                        int nParams,
                        const char * const *paramValues,
                        const int *paramLengths,
                        const int *paramFormats,
                        int resultFormat);
```

这个函数与 PQsendQueryParams 类似，但是要执行的命令是通过一个之前已经命名的预备语句指定， 而不是一个给出的查询字符串。该函数的参数处理和 PQexecPrepared 一样。和 PQexecPrepared 类似，它不能在 2.0 协议的连接上工作。

5.6.2. 提交命令

- PQsendQuery

向服务器提交一个命令而不等待结果。如果该命令被成功发送则返回 1，否则返回 0（此时，可以用 PQerrorMessage 获取关于失败的信息）。

```
int PQsendQuery(PGconn *conn, const char *command);
```

在成功调用 PQsendQuery 后，调用 PQgetResult 一次或者多次来获取结果。在 PQgetResult 返回一个空指针之前，都不能再次调用 PQsendQuery，返回的空指针指示该命令已经完成。

PQsendQueryParams

向服务器提交一个命令和单独的参数，而不等待结果。

```
int PQsendQueryParams(PGconn *conn,
                      const char *command,
                      int nParams,
                      const Oid *paramTypes,
                      const char * const *paramValues,
                      const int *paramLengths,
                      const int *paramFormats,
                      int resultFormat);
```

这个函数等效于 PQsendQuery，不过查询参数可以独立于查询字符串分开指定。该函数的参数处理和 PQexecParams 一样。和 PQexecParams 类似，它不能在 2.0 协议的连接上工作，并且它只允许在查询字符串中有一条命令。

5.6.3. 获取结果

使用 PQgetResult 函数等待来自于一个之前的 PQsendQuery、PQsendQueryParams、 PQsendPrepare、 PQsendQueryPrepared、

PQsendDescribePrepared 或 PQsendDescribePortal 调用的结果，并且返回它。当该命令完成并且没有更多结果时，将返回一个空指针。

```
PGresult *PQgetResult(PGconn *conn);
```

PQgetResult 必须被反复调用直到它返回一个空指针，空指针表示该命令完成（如果在没有命令活动时被调用，PQgetResult 将立即返回一个空指针）。

5.6.4. 示例

异步命令处理示例如下：

```
#include <stdio.h>
#include <stdlib.h>
#include "libpq-fe.h"

static void
exit_nicely(PGconn *conn)
{
    PQfinish(conn);
    exit(1);
}

int
main(int argc, char **argv)
{
```

```
    const char *conninfo;
    PGconn      *conn;
    PGresult    *res;
    int          nFields;
    int          i, j;
  const char *stmt;
  const char *sql;

    /*
     * 如果用户在命令行上提供了一个参数，将它用作连接信息串。
     */
    if (argc > 1)
        conninfo = argv[1];
    else
        conninfo = "user = pwadmin password = 1234Abcde dbname = p
anweidb";

    /* 建立到数据库的一个连接 */
    conn = PQconnectdb(conninfo);

    /* 检查看后端连接是否成功建立 */
    if (PQstatus(conn) != CONNECTION_OK)
    {
```

```
        fprintf(stderr, "Connection to database failed: %s",
                PQerrorMessage(conn));
        exit_nicely(conn);
    }

    /* 设置总是安全的搜索路径，这样恶意用户就无法取得控制。 */
    if (!PQsendQuery(conn, "SELECT pg_catalog.set_config('search_path', '',
 false)"))
    {
        fprintf(stderr, "SET failed: %s", PQerrorMessage(conn));
    exit_nicely(conn);
    }
    /* 获取 PQsendPrepare 调用结果，PQgetResult 必须被反复调用直到它返回一
个空指针，空指针表示该命令完成 */
  res = PQgetResult(conn);
  while(res != NULL)
  {   PQclear(res);
    res = PQgetResult(conn);
  }

    /*
     * 任何时候不再需要 PGresult 时，应该 PQclear 它来避免内存泄露
     */
```

```
    PQclear(res);

    /*
     * 我们的测试案例这里涉及使用一个游标，对它我们必须用在一个事务块内。
     * 我们可以在一个单一的 "select * from pg_database" 的 PQexec() 中做
整个事情，
     * 但是作为一个好的例子它太琐碎。
     */

    /* 开始一个事务块 */
    res = PQexec(conn, "BEGIN");
    if (PQresultStatus(res) != PGRES_COMMAND_OK)
    {
        fprintf(stderr, "BEGIN command failed: %s", PQerrorMessage(con
n));
        PQclear(res);
        exit_nicely(conn);
    }

    PQclear(res);

    /*
     * 从 pg_database 取得行，它是数据库的系统目录
```

```
    */
  stmt = "getdb";
  sql =  "select * from pg_database";

  /* 创建一个预备语句 */
    if (!PQsendPrepare(conn, stmt, sql, 0, NULL))
    {
        fprintf(stderr, "Send Prepare statement select * from pg_database
 failed: %s", PQerrorMessage(conn));
        exit_nicely(conn);
    }

  /* 获取 PQsendPrepare 调用结果，PQgetResult 必须被反复调用直到它返回一个
空指针，空指针表示该命令完成 */
  res = PQgetResult(conn);
  while(res != NULL)
  {  PQclear(res);
    res = PQgetResult(conn);
  }

  /* 执行预备语句 */
    if (!PQsendQueryPrepared(conn, stmt, 0, NULL, NULL,NULL, 0))
    {
```

```
        fprintf(stderr, "Send select * from pg_database failed: %s", PQerr
orMessage(conn));
        exit_nicely(conn);
    }

  /* 获取执行结果 */
  res = PQgetResult(conn);
    if (PQresultStatus(res) != PGRES_TUPLES_OK)
    {
        fprintf(stderr, "Get results failed: %s", PQerrorMessage(conn));
        PQclear(res);
        exit_nicely(conn);
    }

    /* 首先，打印出属性名 */
    nFields = PQnfields(res);
    for (i = 0; i < nFields; i++)
        printf("%-15s", PQfname(res, i));
    printf("\n\n");

    /* 接下来，打印出行 */
    for (i = 0; i < PQntuples(res); i++)
    {
```

```
        for (j = 0; j < nFields; j++)
            printf("%-15s", PQgetvalue(res, i, j));
        printf("\n");
    }

    PQclear(res);

    /* 关闭入口，我们不需要考虑检查错误 */
    res = PQexec(conn, "CLOSE myportal");
    PQclear(res);

    /* 结束事务 */
    res = PQexec(conn, "END");
    PQclear(res);

    /* 关闭到数据库的连接并且清理 */
    PQfinish(conn);

    return 0;
}
```

5.7. 逐行获取查询结果

5.7.1. libpq 设置逐行返回结果

在 5.5 同步命令处理中 libpq 会收集一个 SQL 命令的整个结果并且把它作为单个 PGresult 返回给应用。这对于返回大量行的命令是行不通的。对于这类情况，应用可以使用 PQsendQuery 和 PQgetResult 的单行模式。在这种模式中，结果行以一次一行的方式被返回给应用。

要进入到单行模式，在一次成功的 PQsendQuery（或者其他兄弟函数）调用后立即调用 PQsetSingleRowMode。这种模式选择只对当前正在执行的查询有效。然后反复调用 PQgetResult，直到它返回空。如果该查询返回一行数据，会将其作为单个的 PGresult 对象返回，其状态代码是 PGRES_SINGLE_TUPLE 而非 PGRES_TUPLES_OK。在最后一行之后或者紧接着该查询返回零行之后，一个状态为 PGRES_TUPLES_OK 的零行对象会被返回，这就是代表不会有更多行的信号（但是注意仍然有必要继续调用 PQgetResult 直到它返回空）。所有这些 PGresult 对象将包含相同的行描述数据（列名、类型等等），这些数据和通常一个查询的 PGresult 对象的相同，使用后每一个对象都应该用 PQclear 释放。

5.7.2. 示例

```
#include <stdio.h>

#include <stdlib.h>

#include "libpq-fe.h"
```

```
static void
exit_nicely(PGconn *conn)
{
    PQfinish(conn);
    exit(1);
}
int main(int argc, char **argv)
{
    const char *conninfo;
    PGconn      *conn;
    PGresult    *res;
    int i,j;
    /*
     * 如果用户在命令行上提供了一个参数，将它用作连接信息串。
     * 否则默认用设置 dbname=postgres 并且为所有其他链接参数使用环境变量
或默认值。
     */
    if (argc > 1)
        conninfo = argv[1];
    else
        conninfo = "dbname = postgres";
```

```
/* 建立到数据库的一个连接 */
conn = PQconnectdb(conninfo);


/* 检查看后端连接是否成功建立 */
if (PQstatus(conn) != CONNECTION_OK)
{
    fprintf(stderr, "Connection to database failed: %s",
            PQerrorMessage(conn));
    exit_nicely(conn);
}


const char *DML_statement="select * from pg_database;";
int status = PQsendQuery(conn,DML_statement);
int isSingleMode=PQsetSingleRowMode(conn);
res = PQgetResult(conn);
printf("print out 1 if it is single mode: %d\n", isSingleMode);
int isPrintName = 1;
int nFields;
while (res != 0)
{
    /* 最后的空行 */
    if (PQresultStatus(res)==PGRES_TUPLES_OK)
    {
```

```
        PQclear(res);
        break;
    }
    /* 逐行返回结果 */
    else if (PQresultStatus(res) == PGRES_SINGLE_TUPLE)
    {
        nFields = PQnfields(res);  /* 获取属性数量 */
        if(isPrintName) /* 打印属性名 */
        {
            for (i = 0; i < nFields; i++)
                printf("%-15s", PQfname(res, i)); /* 获取属性名并打印 */
            printf("\n\n");
            isPrintName = 0;
        }

        for (j = 0; j < nFields; j++) /* 循环获取元组每个属性的值 */
            printf("%-15s", PQgetvalue(res, 0, j));
        printf("\n");
        PQclear(res);                 /* 清理 */
        res = PQgetResult(conn); /* 继续获取结果 */
    }
    else  /* 未知错误 */
    {
```

```
                printf("Error Occured: %d\n", PQresultStatus(res));
                PQclear(res);
                break;
            }
}

/* 关闭到数据库的连接并且清理 */
PQfinish(conn);
return 0;
}
```

5.8. 事件系统

5.8.1. 概览

libpq 的事件系统，允许开发者按照接口要求自行编写时间处理器并进行注册，事件系统可将 libpq 事件通知已注册的事件处理器，例如 PGconn 以及 PGresult 对象的创建和析构。典型的使用情况是应用可以将自己的数据与一个 PGconn 或者 PGresult 对象关联在一起，并且提供恰当的时机释放这些数据。

每一个已注册的事件处理器与两部分数据相关，对于 libpq 它们只是透明的 void *指针。当事件处理器被注册到一个 PGconn 时，会有一个应用提供的转移指针。该转移指针在 PGconn 及其产生的所有 PGresult 的生命期内都不会改变。因此，如果使用它，它必须指向长期存在的数据。此外，还有一个 instance data

指针，在每一个 PGconn 和 PGresult 对象中默认都为 NULL。这个指针可以使用 PQinstanceData、PQsetInstanceData、PQresultInstanceData 和 PQresultSetInstanceData 函数操纵。注意和转移指针不同，一个 PGconn 的实例数据不会被从它创建的 PGresult 自动继承。libpq 不知道转移指针和实例数据指针实际指向什么，这些数据的释放也由开发者自行编写的事件处理器来负责。

5.8.2. 事件类型和触发函数

枚举 PGEventId 命名了事件系统处理的事件类型。它的所有值的名称都以 PGEVT 开始。对于每一种事件类型，都有一个相应的事件信息结构用来承载传递给事件处理器的参数。事件类型及其触发函数和相关说明如下表：

事件类型	触发函数和说明
PGEVT_REGISTER	当 PQregisterEventProc 被调用时，注册事件会发生
PGEVT_CONNRESET	连接重置事件在 PQreset 或 PQresetPoll 完成时被触发，只有重置成功才会触发该事件。
PGEVT_CONNDESTROY	连接销毁事件在调用 PQfinish 时会被触发。
PGEVT_RESULTCREATE	在任何生成一个结果的查询执行函数

	被执行时，结果创建事件会被触发。这些函数包括 PQgetResult。这个事件只有在结果被成功地创建之后才会被触发。
PGEVT_RESULTCOPY	结果复制事件在调用 PQcopyResult 时会被触发，这个事件只会在复制完成后才被触发。
PGEVT_RESULTDESTROY	结果销毁事件在调用 PQclear 时会被触发

5.8.3. 示例

```
/* libpq 事件的头文件（注意：libpq-events.h 已包括 libpq-fe.h） */
#include <libpq-events.h>
#include <stdlib.h>
#include <memory.h>
#define DATALENGTH 10
/* instanceData */
typedef struct
{
    int n;
    char *str;
```

```
} mydata;


/* 事件处理器，接口如下 */
static int myEventProc(PGEventId evtId, void *evtInfo, void *passThrough);


mydata *get_mydata(PGconn *conn)
{
    int i;
    mydata *data = (mydata *)malloc(sizeof(mydata));
  data->str = (char *)malloc(DATALENGTH);
  data->n = DATALENGTH;
  for (i = 0;i<DATALENGTH; ++i)
        data->str[i] = ('a' + i);
    return data;
}


mydata *dup_mydata(mydata *src_data)
{
    int i;
    mydata *data = (mydata *)malloc(sizeof(mydata));
  data->str = (char *)malloc(src_data->n);
  data->n = src_data->n;
  for (i = 0;i<data->n; ++i)
```

```
        data->str[i] = src_data->str[i];
    return data;
}

void free_mydata(mydata *data)
{
    free(data->str);
    free(data);
}

int
main(void)
{
    mydata *data;
    PGresult *res, *res_copy;
    PGconn *conn =
        PQconnectdb("dbname=panweidb");

    if (PQstatus(conn) != CONNECTION_OK)
    {
        fprintf(stderr, "Connection to database failed: %s",
                PQerrorMessage(conn));
        PQfinish(conn);
```

```
    return 1;
}

/* 在任何应该接收事件的连接上调用一次。
 * 发送一个 PGEVT_REGISTER 给 myEventProc。
 */
if (!PQregisterEventProc(conn, myEventProc, "mydata_proc", NULL))
{
    fprintf(stderr, "Cannot register PGEventProc\n");
    PQfinish(conn);
    return 1;
}

/* conn 的 instanceData 可用 */
data = PQinstanceData(conn, myEventProc);

/* 发送一个 PGEVT_RESULTCREATE 给 myEventProc */
res = PQexec(conn, "SELECT 1 + 1");

/* 结果的 instanceData 可用 */
data = PQresultInstanceData(res, myEventProc);

/* 如果使用了 PG_COPYRES_EVENTS，发送一个 PGEVT_RESULTCOPY 给 my
```

```
EventProc */
    res_copy = PQcopyResult(res, PG_COPYRES_TUPLES | PG_COPYRES_EVEN
TS);

    /* 如果在 PQcopyResult 调用时使用了 PG_COPYRES_EVENTS，结果的 insta
nceData 可用。*/
    data = PQresultInstanceData(res_copy, myEventProc);

    /* 两个清除都发送一个 PGEVT_RESULTDESTROY 给 myEventProc */
    PQclear(res);
    PQclear(res_copy);

    /* 发送一个 PGEVT_CONNDESTROY 给 myEventProc */
    PQfinish(conn);

    return 0;
}

static int
myEventProc(PGEventId evtId, void *evtInfo, void *passThrough)
{
    switch (evtId)
    {
```

```
        case PGEVT_REGISTER:    /* PQregisterEventProc 调用时触发 */
        {
            PGEventRegister *e = (PGEventRegister *)evtInfo;
            mydata *data = get_mydata(e->conn);

            /* 将应用相关的数据与连接关联起来 */
            PQsetInstanceData(e->conn, myEventProc, data);
            break;
        }

        case PGEVT_CONNRESET:   /* 调用 PQreset 或 PQresetPoll 且完成时被触
发，本例不触发 */
        {
            PGEventConnReset *e = (PGEventConnReset *)evtInfo;
            mydata *data = PQinstanceData(e->conn, myEventProc);

            if (data)
              memset(data, 0, sizeof(mydata));
            break;
        }

        case PGEVT_CONNDESTROY:     /* 调用 PQfinish 时触发 */
        {
```

```
            PGEventConnDestroy *e = (PGEventConnDestroy *)evtInfo;
            mydata *data = PQinstanceData(e->conn, myEventProc);

            /* 因为连接正在被销毁，释放示例数据 */
            if (data)
              free_mydata(data);
            break;
        }

        case PGEVT_RESULTCREATE: /* 调用 PQexec 时触发 */
        {
            PGEventResultCreate *e = (PGEventResultCreate *)evtInfo;
            mydata *conn_data = PQinstanceData(e->conn, myEventProc);
            mydata *res_data = dup_mydata(conn_data);

            /* 把应用相关的数据与结果（从 conn 复制过来）关联起来 */
            PQresultSetInstanceData(e->result, myEventProc, res_data);
            break;
        }

        case PGEVT_RESULTCOPY: /* 调用 PQcopyResult，并指定了 PG_COPYR
ES_EVENTS 时触发 */
        {
```

```
            PGEventResultCopy *e = (PGEventResultCopy *)evtInfo;
            mydata *src_data = PQresultInstanceData(e->src, myEventPro
c);
            mydata *dest_data = dup_mydata(src_data);

            /* 把应用相关的数据与结果（从一个结果复制过来）关联起来 */
            PQresultSetInstanceData(e->dest, myEventProc, dest_data);
            break;
        }

        case PGEVT_RESULTDESTROY:  /* 调用 PQclear 时触发 */
        {
            PGEventResultDestroy *e = (PGEventResultDestroy *)evtInfo;
            mydata *data = PQresultInstanceData(e->result, myEventProc);

            /* 因为结果正在被销毁，释放实例数据 */
            if (data)
              free_mydata(data);
            break;
        }

        /* 未知事件 ID，只返回 true。 */
        default:
```

```
            break;
        }

    return true; /* 事件处理成功 */
}
```

5.9. 处理大对象

PanWeiDB V2.0-S3.0.0_Beta libpq 客户端接口为访问大对象所提供的功能。大对象接口按照 Unix 文件系统的接口建模，也有相似的 open、read、write、lseek 等。

所有使用这些函数对大对象的操作都必须发生在一个 SQL 事务块中，因为大对象文件描述符只在事务期间有效。

在执行任何一个这种函数期间如果发生一个错误，该函数将会返回一个其他的不可能值，典型的是 0 或-1。一个关于该错误的消息亦会被保存在连接对象中，可以通过 PQerrorMessage 检索到。

使用这些函数的客户端应用应该包括头文件 libpq/libpq-fs.h 并链接 libpq 库。

5.9.1. 创建一个大对象

PanWeiDB V2.0-S3.0.0_Beta 提供 lo_creat 和 lo_create 函数创建大对象。

- lo_creat

```
Oid lo_creat(PGconn *conn, int mode);
```

lo_creat 函数创建一个新的大对象。其返回值是分配给这个新大对象的 OID 或者 InvalidOid（0）表示失败。 mode 不再使用且会被忽略，为了和以前的发行兼容，它最好被设置为 INV_READ、INV_WRITE 或 INV_READ | INV_WRITE（这些符号常量定义在头文件 libpq/libpq-fs.h 中）。

示例：

```
inv_oid = lo_creat(conn, INV_READ|INV_WRITE);
```

❖ lo_create

```
Oid lo_create(PGconn *conn, Oid lobjId);
```

lo_create 函数也创建一个新的大对象。分配给该大对象的 OID 可以通过 lobjId 指定，如果该 OID 已经被某个大对象使用时会产生错误。如果 lobjId 是 InvalidOid（0），则 lo_create 会分配一个未使用的 OID（这时和 lo_creat 的行为相同）。返回值是分配给新大对象的 OID 或 InvalidOid（0）表示发生错误。

示例：

```
inv_oid = lo_create(conn, desired_oid);
```

5.9.2. 导入一个大对象

要将一个操作系统文件导入成一个大对象，调用：

❖ lo_import

```
Oid lo_import(PGconn *conn, const char *filename);
```

filename 指定了要导入为大对象的操作系统文件名。返回值是分配给新大对象的 OID 或 InvalidOid（0）表示发生错误。注意该文件是被客户端接口库而不是服务器所读取，因此它必须存在于客户端文件系统中并且对于客户端应用是可读的。

❖ lo_import_with_oid

```
Oid lo_import_with_oid(PGconn *conn, const char *filename, Oid lobjId);
```

也可以导入一个新大对象。分配给新大对象的 OID 可以用 lobjId 指定，如果这样做，该 OID 已经被某个大对象使用时会产生错误。如果 lobjId 是 InvalidOid（0），则 lo_import_with_oid 会分配一个未使用的 OID（这和 lo_import 的行为相同）。返回值是分配给新大对象的 OID 或 InvalidOid（0）表示发生错误。

5.9.3. 导出一个大对象

要把一个大对象导出到一个操作系统文件，调用：

```
int lo_export(PGconn *conn, Oid lobjId, const char *filename);
```

lobjId 参数指定要导出的大对象的 OID，filename 参数指定操作系统文件名。注意该文件是被客户端接口库而不是服务器写入。成功返回 1，错误返回-1。

5.9.4. 打开一个现有的大对象

要打开一个现有的大对象进行读写，调用：

```
int lo_open(PGconn *conn, Oid lobjId, int mode);
```

lobjId 参数指定要打开的大对象的 OID。mode 位控制着打开对象是为了只读（INV_READ）、只写（INV_WRITE）或者读写（这些符号常量定义在头文件 libpq/libpq-fs.h 中）。lo_open 返回一个（非负）大对象描述符以便后面用于 lo_read、lo_write、lo_lseek、lo_lseek64、lo_tell、lo_tell64、lo_truncate、lo_truncate64 以及 lo_close。该描述符只在当前事务期间有效。如果打开错误将会返回-1。

服务器目前并不区分模式 INV_WRITE 和 INV_READ |INV_WRITE：在两种情况中都允许从描述符读取。但是在这些模式和单独的 INV_READ 之间有明显的区别：使用 INV_READ 我们不能向描述符写入，从中读取的数据则反映了该大对象在活动事务快照时刻的内容（该快照在 lo_open 被执行时创建），而不管之后被该事务或其他事务写入的内容。从一个以 INV_WRITE 模式打开的描述符读取的数据所有其他已提交事务以及当前事务所作的写入。这与普通 SQL 命令 SELECT 的 REPEATABLE READ 和 READ COMMITTED 事务模式之间的区别相似。

如果大对象的 SELECT 特权不可用，或者如果在指定了 INV_WRITE 时 UPDATE 特权不可用，则 lo_open 将会失败。这些特权检查可以用 lo_compat_privileges 运行时参数禁用。

示例：

```
inv_fd = lo_open(conn, inv_oid, INV_READ|INV_WRITE);
```

5.9.5. 向一个大对象写入数据

调用 lo_write 向一个大对象写入数据。

```
int lo_write(PGconn *conn, int fd, const char *buf, size_t len);
```

从 buf（大小必须是 len）中写出 len 字节到大对象描述符 fd。参数 fd 必须是已经由前面的 lo_open 返回的大对象描述符。函数将返回实际写入的字节数（在当前的实现中，除非出错，返回的字节数总是等于 len）。在出错时，返回值为-1。

参数 len 被声明为类型 size_t，该函数会拒绝超过 INT_MAX 的长度值。在实际中，被传送的数据最好是每块最多数兆字节。

5.9.6. 从一个大对象读取数据

要从一个大对象读取数据，调用：

```
int lo_read(PGconn *conn, int fd, char *buf, size_t len);
```

从大对象描述符 fd 中读取最多 len 字节到 buf （大小必须是 len）中。参数 fd 必须是已经由前面的 lo_open 返回的大对象描述符。实际读出的字节数将被返回，如果先到达了大对象的末尾返回值可能会小于 len。出错时返回值为-1。

参数 len 被声明为类型 size_t，该函数会拒绝超过 INT_MAX 的长度值。在实际中，被传送的数据最好是每块最多数兆字节。

5.9.7. 在一个大对象中查找

要改变一个大对象描述符的当前读或写位置，调用：

```
int lo_lseek(PGconn *conn, int fd, int offset, int whence);
```

该函数将大对象文件描述符 fd 的当前位置指针移动到由 offset 指定的新位置。whence 的可用值是 SEEK_SET（从对象开头定位）、SEEK_CUR （从当前位置定位）以及 SEEK_END（从对象末尾定位）。返回值是新位置的指针，或者是-1 表示出错。

在处理可能超过 2GB 大小的大对象时，换用

```
pg_int64 lo_lseek64(PGconn *conn, int fd, pg_int64 offset, int whence);
```

该函数的行为和 lo_lseek 相同，但是它能接受一个超过 2GB 的 offset 并/或传送一个超过 2GB 的结果。注意如果新位置的指针超过 2GB，lo_lseek 会失败。

5.9.8. 获取一个大对象的查找位置

要得到一个大对象描述符的当前读或写位置，调用：

```
int lo_tell(PGconn *conn, int fd);
```

如果出现错误，返回值是-1。

在处理可能超过 2GB 大小的大对象时，换用：

```
pg_int64 lo_tell64(PGconn *conn, int fd);
```

该函数和 lo_tell 的行为相同，但是它能传递超过 2GB 的结果。注意如果当前读/写位置超过 2GB，lo_tell 将会失败。

5.9.9. 截断一个大对象

要将一个大对象截断成一个给定长度，调用：

```
int lo_truncate(PGcon *conn, int fd, size_t len);
```

该函数将大对象描述符 fd 截断为长度 len。参数 fd 必须是已经由前面的 lo_open 返回的大对象描述符。如果 len 超过了大对象的当前长度，大对象将会被使用空字节（'\0'）扩展到指定长度。成功时 lo_truncate 返回 0，失败时返回值为-1。

描述 fd 的读/写位置不变。

尽管参数 len 被声明为类型 size_t，lo_truncate 会拒绝超过 INT_MAX 的长度值。

在处理可能超过 2GB 大小的大对象时，换用：

```
int lo_truncate64(PGcon *conn, int fd, pg_int64 len);
```

该函数和 lo_truncate 的行为相同，但它能够接受超过 2GB 的 len 值。

5.9.10. 关闭一个大对象描述符

要关闭一个大对象描述符，调用：

```
int lo_close(PGconn *conn, int fd);
```

其中 fd 是由 lo_open 返回的大对象描述符。成功时，lo_close 返回 0，失败时返回-1。

在事务末尾仍然保持打开的任何大对象描述符都会自动被关闭。

5.9.11. 移除一个大对象

要从数据库中移除一个大对象，调用：

```
int lo_unlink(PGconn *conn, Oid lobjId);
```

lobjId 参数指定要移除的大对象的 OID。成功时返回 1，失败时返回-1。

5.9.12. 示例

```
#include <stdio.h>
#include <stdlib.h>

#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

#include "libpq-fe.h"
#include "libpq/libpq-fs.h"

#define BUFSIZE           1024
```

```
/*
 * importFile
 *      把文件 "in_filename" 作为一个大对象 "lobjOid" 载入到数据库
 *
 */
static Oid
importFile(PGconn *conn, char *filename)
{
    Oid             lobjId;
    int             lobj_fd;
    char            buf[BUFSIZE];
    int             nbytes,
                    tmp;
    int             fd;

    /*
     * 打开要读入的文件
     */
    fd = open(filename, O_RDONLY, 0666);
    if (fd < 0)
    {                                   /* error */
        fprintf(stderr, "cannot open unix file\"%s\"\n", filename);
```

```
    }

    /*
     * 创建大对象
     */
    lobjId = lo_creat(conn, INV_READ | INV_WRITE);
    if (lobjId == 0)
        fprintf(stderr, "cannot create large object");

/*
     * 打开一个已有大对象
     */
    lobj_fd = lo_open(conn, lobjId, INV_WRITE);

    /*
     * 从该 Unix 文件读取并写入到大对象
     */
    while ((nbytes = read(fd, buf, BUFSIZE)) > 0)
    {
        tmp = lo_write(conn, lobj_fd, buf, nbytes);
        if (tmp < nbytes)
            fprintf(stderr, "error while reading \"%s\"", filename);
    }
```

```
    close(fd);
    lo_close(conn, lobj_fd);

    return lobjId;
}

/*
 * pickout
 *     从大对象指定位置读取长度为 len 的数据
 *
 */
static void
pickout(PGconn *conn, Oid lobjId, int start, int len)
{
    int         lobj_fd;
    char        *buf;
    int         nbytes;
    int         nread;

  /* 打开一个已有大对象 */
    lobj_fd = lo_open(conn, lobjId, INV_READ);
    if (lobj_fd < 0)
```

```
        fprintf(stderr, "cannot open large object %u", lobjId);

    /* 将大对象文件描述符 fd 的当前位置指针移动到 start 位置 */
    lo_lseek(conn, lobj_fd, start, SEEK_SET);
    buf = malloc(len + 1);

    nread = 0;
    while (len - nread > 0)
    {
    /* 从一个大对象读取数据 */
        nbytes = lo_read(conn, lobj_fd, buf, len - nread);
        buf[nbytes] = '\0';
        fprintf(stderr, ">>> %s", buf);
        nread += nbytes;
        if (nbytes <= 0)
            break;                /* no more data? */
    }
    free(buf);
    fprintf(stderr, "\n");
    /* 关闭一个大对象 */
    lo_close(conn, lobj_fd);
}
```

```
/*
 * overwrite
 *     向大对象指定位置写入长度为 len 的数据
 *
 */
static void
overwrite(PGconn *conn, Oid lobjId, int start, int len)
{
    int         lobj_fd;
    char        *buf;
    int         nbytes;
    int         nwritten;
    int         i;

    lobj_fd = lo_open(conn, lobjId, INV_WRITE);
    if (lobj_fd < 0)
        fprintf(stderr, "cannot open large object %u", lobjId);

  /* 将大对象文件描述符 fd 的当前位置指针移动到 start 位置 */
    lo_lseek(conn, lobj_fd, start, SEEK_SET);
    buf = malloc(len + 1);

    for (i = 0; i < len; i++)
```

```
        buf[i] = 'X';
    buf[i] = '\0';

    nwritten = 0;
    while (len - nwritten > 0)
    {
    /* 向大对象写入数据 */
        nbytes = lo_write(conn, lobj_fd, buf + nwritten, len - nwritten);
        nwritten += nbytes;
        if (nbytes <= 0)
        {
            fprintf(stderr, "\nWRITE FAILED!\n");
            break;
        }
    }
    free(buf);
    fprintf(stderr, "\n");
    lo_close(conn, lobj_fd);
}

/*
 * exportFile -
 *     把大对象 "lobjOid" 导出成文件 "out_filename"
```

```
 *
 */
static void
exportFile(PGconn *conn, Oid lobjId, char *filename)
{
    int           lobj_fd;
    char          buf[BUFSIZE];
    int           nbytes,
                  tmp;
    int           fd;

    /*
     * 打开大对象
     */
    lobj_fd = lo_open(conn, lobjId, INV_READ);
    if (lobj_fd < 0)
        fprintf(stderr, "cannot open large object %u", lobjId);

    /*
     * 打开要写入的文件
     */
    fd = open(filename, O_CREAT | O_WRONLY | O_TRUNC, 0666);
    if (fd < 0)
```

```
    {                               /* 错误 */
        fprintf(stderr, "cannot open unix file\"%s\"",
                filename);
    }

    /*
     * 从大对象读入并写出到 Unix 文件
     */
    while ((nbytes = lo_read(conn, lobj_fd, buf, BUFSIZE)) > 0)
    {
        tmp = write(fd, buf, nbytes);
        if (tmp < nbytes)
        {
            fprintf(stderr, "error while writing \"%s\"",
                    filename);
        }
    }

    lo_close(conn, lobj_fd);
    close(fd);

    return;
}
```

```
static void
exit_nicely(PGconn *conn)
{
    PQfinish(conn);
    exit(1);
}

int
main(int argc, char **argv)
{
    char        *in_filename,
                *out_filename;
    char        *database;
    Oid          lobjOid;
    PGconn      *conn;
    PGresult    *res;

    if (argc != 4)
    {
        fprintf(stderr, "Usage: %s database_name in_filename out_filenam
e\n",
                argv[0]);
```

```
    exit(1);
}

database = argv[1];
in_filename = argv[2];
out_filename = argv[3];

/*
 * 设置连接
 */
conn = PQsetdb(NULL, NULL, NULL, NULL, database);

/* 检查看看后端连接是否成功建立 */
if (PQstatus(conn) != CONNECTION_OK)
{
    fprintf(stderr, "Connection to database failed: %s",
            PQerrorMessage(conn));
    exit_nicely(conn);
}

/* 设置总是安全的搜索路径，这样恶意用户就无法取得控制权。 */
res = PQexec(conn,
             "SELECT pg_catalog.set_config('search_path', '', false)");
```

```
    if (PQresultStatus(res) != PGRES_TUPLES_OK)
    {
        fprintf(stderr, "SET failed: %s", PQerrorMessage(conn));
        PQclear(res);
        exit_nicely(conn);
    }
    PQclear(res);

    res = PQexec(conn, "begin");
    PQclear(res);
    printf("importing file \"%s\" ...\n", in_filename);

/* 调用创建大对象、打开大对象、写入大对象函数将输入文件中的数据导入到新的大对象中。
 * 除此之外，可以直接调用 lo_import 函数创建一个新的大对象并从文件导入数据。
 */
/*  lobjOid = importFile(conn, in_filename); */
    lobjOid = lo_import(conn, in_filename);
    if (lobjOid == 0)
        fprintf(stderr, "%s\n", PQerrorMessage(conn));
    else
    {
        printf("\tas large object %u.\n", lobjOid);
```

```
    /* 读取大对象中位于 1000-2000 位置的数据 */
        printf("picking out bytes 1000-2000 of the large object\n");
        pickout(conn, lobjOid, 1000, 1000);

        /* 向大对象中位于 1000-2000 的位置写入数据 */
    printf("overwriting bytes 1000-2000 of the large object with X's\n");
        overwrite(conn, lobjOid, 1000, 1000);

/* 调用打开大对象、打开要写入的文件、读取大对象、写入文件函数将大对象中的数据导出到输出文件。
 * 除此之外，可以直接调用 lo_export 函数直接将大对象中的数据导出到文件。
 */
        printf("exporting large object to file \"%s\" ...\n", out_filename);
/*      exportFile(conn, lobjOid, out_filename); */
        if (lo_export(conn, lobjOid, out_filename) < 0)
            fprintf(stderr, "%s\n", PQerrorMessage(conn));
    }

    res = PQexec(conn, "end");
    PQclear(res);
    PQfinish(conn);
    return 0;
```

5.10. 其他功能

5.10.1. 杂项函数

下表给出 libpq 支持的杂项函数。主要用于内存释放，密码加密，生成空对象等一些特殊场景。

函数	说明
PQfreemem	释放 libpq 分配的内存，尤其是 PQescapeByteaConn、PQescapeBytea、PQunescapeBytea 和 PQnotifies 分配的内存。
PQconninfoFree	释放 PQconndefaults 或 PQconninfoParse 分配的数据结构。
PQencryptPassword	这个函数旨在用于那些希望发送类似于 ALTER USER joe PASSWORD 'pwd' 命令的客户端应用。不在这样一个命令中发送原始的明文密码是一个好习惯，因为它可能被暴露在命令日志、活动显示等等中。相反，在发送之前使用这个函数可以将口令转换为加密的形式。

PQmakeEmptyPGresult	用给定的状态，构造一个空 PGresult 对象。
PQfireResultCreateEvents	为每一个在 PGresult 对象中注册的事件过程触发一个 PGEVT_RESULTCREATE 事件。
PQcopyResult	为一个 PGresult 对象创建一个拷贝。这个拷贝不会以任何方式链接到源结果，并且当该拷贝不再需要时，必须调用 PQclear 进行清理。
PQsetResultAttrs	设置 PGresult 对象的字段属性。
PQsetvalue	设置一个 PGresult 对象的一个元组域值。
PQresultAlloc	为一个 PGresult 对象分配附属存储。
PQlibVersion	返回所使用的 libpq 版本。

5.10.2. 控制函数

libpq 提供下列控制函数，控制执行过程中的各项细节。

函数	说明
PQclientEncoding	返回客户端编码 ID。

PQsetClientEncoding	设置客户端编码。
PQsetErrorVerbosity	设置 PQerrorMessage 和 PQresultErrorMessage 返回的消息的细节程度。
PQsetErrorContextVisibility	设置如何处理 PQerrorMessage 和 PQresultErrorMessage 返回的消息中的 CONTEXT 域。
PQtrace	启用对客户端/服务器通讯的跟踪，把跟踪信息输出到一个调试文件流中。
PQuntrace	禁用 PQtrace 打开的跟踪。

5.10.3. 批处理函数

libpq 提供 PQexecParamsBatch 接口实现批处理功能,比 PQexecParams 接口多了 nBatch 参数，参数说明如下:

函数	说明
PGconn *conn	连接句柄
const char *command	SQL 文本串
int nParams	参数个数
int nBatch	批量操作数

函数	说明
const Oid *paramTypes	绑定参数类型
const char * const *paramValues	绑定参数的值
const int *paramLengths	参数长度
const int *paramFormats	参数格式（文本或二进制）
int resultFormat	返回结果格式（文本或二进制）

示例：

```
#include <stdio.h>
#include <stdlib.h>
#include "libpq-fe.h"

static void
exit_nicely(PGconn *conn)
{
    PQfinish(conn);
    exit(1);
}

int main(int argc, char **argv)
{
    const char *conninfo;
```

```
  PGconn      *conn;
  PGresult    *res;
    int i;
    conninfo = "host = localhost user = libpq password = Aa@123 dbnam
e = panweidb port =25638";

  /* Make a connection to the database */
  conn = PQconnectdb(conninfo);

  /* Check to see that the backend connection was successfully made */
  if (PQstatus(conn) != CONNECTION_OK)
  {
    fprintf(stderr, "%s", PQerrorMessage(conn));
    exit_nicely(conn);
  }

    res = PQexec(conn, "drop table if exists test;create table test(id integ
er,name varchar(50));");
    if (PQresultStatus(res) != PGRES_COMMAND_OK)
  {
    fprintf(stderr, "sql failed: %s", PQerrorMessage(conn));
    PQclear(res);
    exit_nicely(conn);
```

```
  }
    else
    {
        printf("sql success\n");
    }
    PQclear(res);


    const char command[] = "INSERT INTO test (id, name)"
                           "VALUES($1::integer, $2::varchar(50))";
    int nParams = 2;
    int nBatchCount=4;
    const Oid paramTypes[] = {23,1043};
    int binaryVal1 = htonl(1);
    int binaryVal2 = htonl(2);
    int binaryVal3 = htonl(3);
    int binaryVal4 = htonl(4);
    const char *const paramValues[][2]={{(const char *)&binaryVal1,"123"},
                                        {(const char *)&binaryVal2,"234"},
                                        {(const char *)&binaryVal3,"345"},
                                        {(const char *)&binaryVal4,"456"}};
    const int paramLengths[][2] = {{sizeof(int), 3},{sizeof(int), 3},{sizeof(int),
3},{sizeof(int), 3}};
    const int paramFormats[] = {1, 1};
```

```
    int resultFormat = 0;


    printf("-----test batch insert -----\n");
    res = PQexecParamsBatch(conn,      /*连接句柄*/
                    command,      /* SQL 文本串 */
                    nParams ,      /* 参数个数*/
                    nBatchCount,   /* 批量操作数*/
                    paramTypes,    /*绑定参数类型*/
                    paramValues[0],   /*绑定参数的值*/
                    paramLengths[0],   /* 参数长度*/
                    paramFormats,    /* 参数格式（文本或二进制） */
                    resultFormat);     /* 返回结果格式（文本或二进制）*/
    if (PQresultStatus(res) != PGRES_COMMAND_OK)
{
    fprintf(stderr, "batch insert failed: %s", PQerrorMessage(conn));
    PQclear(res);
    exit_nicely(conn);
}
    else
    {
        printf("batch insert success\n");
    }
```

```
    PQclear(res);
    printf("-----print result -----\n");
    res = PQexec(conn, "select * from test;");
    for (i = 0; i < PQntuples(res); i++)
    {
        const char* id = NULL;
        const char* name = NULL;
        id = PQgetvalue(res, i, 0);
        name = PQgetvalue(res, i, 1);
        printf("id = %s, name = %s \n", id, name);
    }

    PQclear(res);
    PQfinish(conn);
    return 0;
}
```

6. Lua 接口开发指南

6.1. 介绍

Lua 语言通过 LuaSQL 访问 PanWeiDB V2.0-S3.0.0_Beta 数据库,LuaSQL 为使用 Lua 语言的应用程序定义了一组访问和操作 PanWeiDB V2.0-S3.0.0_Beta 数据库的标准接口。

6.2. 安装 LuaSQL

6.2.1. 安装 Lua

```
yum install readline-devel
curl -R -O http://www.lua.org/ftp/lua-5.3.0.tar.gz
tar zxf lua-5.3.0.tar.gz
cd lua-5.3.0
make linux test
make install
```

6.2.2. 安装 LuaRocks

yum install git

```
wget https://luarocks.github.io/luarocks/releases/luarocks-3.9.0.tar.gz
tar zxpf luarocks-3.9.0.tar.gz
cd luarocks-3.9.0
```

```
./configure

sudo make bootstrap

yum install postgresql-devel

luarocks install luasql-postgres
```

6.3. 安装 PanWeiDB V2.0-S3.0.0_Beta

按照安装说明安装 PanWeiDB V2.0-S3.0.0_Beta，并配置环境变量。如果是使用安装包自带安装程序进行安装，则环境变量会自动配置无需手动配置；如果是手动安装数据库，则需配置以下环境变量（只作为参数，应按实际的环境进行配置）：

```
export GAUSSHOME=/usr/local/panweidb

export PATH=$GAUSSHOME/bin:$PATH

export LD_LIBRARY_PATH=$GAUSSHOME/lib:$LD_LIBRARY_PATH
```

6.4. 使用 luasql.postgres 连接数据库

6.4.1. 导入

任何使用 luasql.postgres 的程序都需要导入 luasql.postgres 模块。

```
package.cpath = package.cpath .. ';/root/luarocks-3.9.0/lua_modules/lib/l

ua/5.3/?.so'; --luasql 所在目录

local driver = require "luasql.postgres"
```

6.4.2. 连接数据库

6.4.2.1. 获取数据库连接

```
env = driver.postgres()

-- connect 函数参数："数据库名" "用户名" "密码" "IP 地址" 端口口

conn = env:connect("postgres","panweidb","Aa@123456","127.0.0.1",5432)
```

6.4.2.2. 关闭数据库连接

关闭连接时调用 conn 的 close()方法即可：

```
conn = env:connect("postgres","panweidb","Aa@123456","127.0.0.1",5432)

conn:close()
```

6.5. 执行 SQL 语句并处理结果

向数据库发出执行 SQL 的请求时，服务端执行完成后将返回执行结果。

6.5.1. 执行查询

6.5.1.1. 直接执行

调用 conn 的 execute 函数执行插入语句：

```
conn:execute("insert into tb_test(id, name) values(1,'Anna')")
```

6.5.1.2. 预编译方式执行

调用 conn:execute()，使用占位符表示 SQL 中的变量，再变量值传入：

```
name = 'Jack'
conn:execute(string.format("insert into tb_test(id, name) values(2,'%s')", na
me))
```

6.5.2. 使用游标获取结果

```
cur = conn:execute("select * from tb_test")
row = cur:fetch ({}, "a")
while row do
    -- do something

    row = cur:fetch (row, "a")
end
```

6.5.3. 执行更新

要更改数据(执行一个 insert，update 或者 delete)，可以使用 conn:execute 函数执行完整的 SQL，也可以使用预编译的方式绑定参数后再执行。下面的示例将以预编译的方式执行。

6.5.3.1. 插入

```
conn:execute(string.format("insert into tb_test values ('%s', '%s')", 2, 'Jack
'))
```

6.5.3.2. 更新

```
conn:execute(string.format("update tb_test set name = '%s' where id = 1",
'Elsa'))
```

6.5.3.3. 删除

```
conn:execute("delete from tb_test where id = 1")
```

6.5.4. 创建或修改数据库对象

要创建、更改或者删除一个类似表或者视图这样的数据库对象， 可以调用 conn:execute() 方法。

```
    conn:execute("create table tb_test(id integer, name text)")
    conn:execute("create view v_tb_test as select * from tb_test")
```

6.5.5. 执行存储过程

下面的例子展示了如何调用存储过程。

```
sql_procedure = [[CREATE OR REPLACE PROCEDURE pro_test(a integer,inout
 b integer) AS
                  begin
                    b := a+b;
                  end
]]
conn:execute(sql_procedure)
conn:execute("call pro_test(1,5)")
```

下面的例子展示了如何调用函数。

```
sql_function = [[CREATE OR REPLACE FUNCTION func_test (a int,out b int)
RETURNS int AS
    $$
        begin
            b := a+2;
            return;
        end;
    $$ LANGUAGE 'plpgsql';
]]
conn:execute(sql_function)
conn:execute("select func_test(1)")
```

6.5.6. 处理大数据类型

6.5.6.1. 二进制类型

二进制数据可以使用大对象特性以一种特殊的格式将二进制数据存储在一个独立的表中， 然后通过在表中保存一个类型为 OID 的值来引用该表。以下为该方法的使用示例。

❖ 使用 OID 存储

1) 创建表

```
conn:execute("create table lo_test_tab (name text, img oid)")
```

2) 导入数据

该步骤会将操作系统文件“image”导入成一个大对象并生成一个 oid，查询 lo_test_tab 表时则显示 oid。

```
conn:execute("insert into lo_test_tab values('beautiful image', lo_import('/tmp/image'))")
```

3） 导出数据

该操作将数据库中存储的大对象导出到一个操作系统文件。

```
conn:execute("select lo_export(lo_test_tab.img, '/tmp/image-bak') from lo_test_tab where name = 'beautiful image'")
```

6.5.6.2. 字符类型

PanWeiDB V2.0-S3.0.0_Beta 中 TEXT 类型与 VARCHAR 类型都是可变长的字符类型，区别在于 VARCHAR 类型通过 VARCHAR(n)中的 n 来限制最大长度，而 TEXT 类型没有。TEXT 类型与 VARCHAR 类型几乎没有性能差别，TEXT 类型最多可存储 1G 数据 。

使用 TEXT 类型存储数据时，可用以下方式来进行读写。

1） 创建表

```
conn:execute("create table images (id int, msg text)")
```

2） 插入数据

```
conn:execute(string.format("insert into images values(1, '%s')","the text data"))
```

3） 读取数据

```
cur = conn:execute("select msg from images where id = 1")
```

```
row = cur:fetch({}, "a")

print(row.msg)
```

6.6. 附录 1：主要接口

LuaSQL 提供建立和管理与 PanWeiDB V2.0-S3.0.0_Beta 数据库连接、执行 SQL 语句和对结果集进行储存管理的若干接口。主要接口说明如下：

- env:connect(sourcename[,username[,password[,hostname[,port]]]])

  此方法用于创建新的数据库会话并返回新的 connection 对象。参数说明如下：

关键字	参数说明
sourcename	数据库名称
username	用户名
password	密码
hostname	数据库所在主机 IP 地址
port	连接端口号，默认为 5432

  返回值：connection 对象（连接 PanWeiDB V2.0-S3.0.0_Beta 数据库实例的对象）。
  调用示例见附录 2。

- conn:execute(statement[vars_list])

  此方法执行被参数化的 SQL 语句（即占位符，而不是 SQL 文字）。LuaSQL 支持用 %s 标志的占位符。参数说明如下：

关键字	参数说明
statement	SQL 语句

vars_list	变量列表，匹配 query 中%s 为占位符

返回值：cursor 对象。

调用示例见附录 2。

❖ conn:commit()

此方法将当前挂起的事务提交到数据库。

参数：无。

返回值：成功时返回 true，当操作无法执行或未执行时返回 false。

调用示例见附录 2。

❖ conn:rollback()

此方法回滚当前挂起事务。

参数：无。

返回值：成功时返回 true，当操作无法执行或未执行时返回 false。

调用示例见附录 2。

❖ cur:fetch([table[,modestring]])

此方法提取查询结果集的下一行，并返回一个元组。如果无参调用 fetch，则结果值将直接返回给调用者。

参数说明如下：

关键字	参数说明
table	如果 fetch 调用带了 table，结果将被复制到 table 中，并返回更改后的 table。
modestring	指示如何构造结果表： “n”：结果表将具有数字索引(默认) “a”：结果表将具有字母数字索引

返回值：data，或者如果没有更多行则返回 nil。

调用示例见附录 2。

- cur:close()

  此方法用于关闭该游标。

  参数：无。

  返回值：成功时返回 true，该游标已经关闭则返回 false。

  调用示例见附录 2。

- conn:close()

  此方法用于关闭连接 conn。只有当与之相关的所有游标都已关闭且连接仍处于打开状态时才会成功。

  参数：无。

  返回值：成功时返回 true，失败时返回 false。

  调用示例见附录 2。

- env:close()

  此方法用于关闭该环境 env。只有在所有与之相关的连接都先关闭的情况下才会成功。

  参数：无。

  返回值：成功时返回 true，当该环境已经关闭时返回 false。

  调用示例见附录 2。

6.7. 附录 2：调用示例

```
package.cpath = package.cpath .. ';/root/luarocks-3.9.0/lua_modules/lib/l
ua/5.3/?.so'; --luasql 所在目录
local driver = require "luasql.postgres"
env = assert (driver.postgres())
conn = assert (env:connect("postgres","panweidb","Aa@123456","127.0.0.1",
```

```
5432))
print("connect successful!")
--删除表
conn:execute("drop table tb_test")
--创建表
assert (conn:execute("create table tb_test(id integer not null primary key,
name text)"))
print("create table successful!")
--插入数据
assert (conn:execute("insert into tb_test values (1, 'Anna')"))
assert (conn:execute(string.format("insert into tb_test values ('%s', '%s')", 2,
 'Jack')))
print("insert successful!")
--查询数据
cur = assert (conn:execute("select * from tb_test"))
row = cur:fetch ({}, "a")
while row do
  print(string.format("fetch: id = %s, name: %s", row.id, row.name))

  row = cur:fetch (row, "a")
end
--rollback
conn:setautocommit(false)
assert (conn:execute("insert into tb_test values (3, 'Mike')"))
```

```
conn:rollback()

conn:commit()

conn:setautocommit(true)

cur = assert (conn:execute("select * from tb_test"))

row = cur:fetch ({}, "a")

while row do

  print(string.format("after rollback: id = %s, name: %s", row.id, row.nam

e))


  row = cur:fetch (row, "a")

end

--更新数据

conn:execute(string.format("update tb_test set name = '%s' where id = 1",

'Elsa'))

print("update successful!")

cur = assert (conn:execute("select * from tb_test"))

row = cur:fetch ({}, "a")

while row do

  print(string.format("after update: id = %s, name: %s", row.id, row.nam

e))


  row = cur:fetch (row, "a")

end

--删除数据
```

```
conn:execute("delete from tb_test where id = 1")

cur = assert (conn:execute("select * from tb_test"))

row = cur:fetch ({}, "a")

while row do

  print(string.format("after delete: id = %s, name: %s", row.id, row.nam

e))


  row = cur:fetch (row, "a")

end


cur:close()

conn:close()

env:close()
```

7. OCI 接口开发指南

7.1. 介绍

PanWeiDB V2.0-S3.0.0_Beta 数据库通过提供 OCI 驱动为使用 C/C++语言的应用程序提供访问 PanWeiDB V2.0-S3.0.0_Beta 的接口。

7.2. panweidb OCI 主要接口

PanWeiDB V2.0-S3.0.0_Beta OCI 驱动由具有建立和管理与 PanWeiDB V2.0-S3.0.0_Beta 数据库连接、执行 SQL 语句和对结果集进行储存管理的若干功能类组成。

- OCIEnvCreate

  创建并初始化 OCI 函数的工作环境句柄:

  示例:

```
OCIEnvCreate((OCIEnv **) &envhp, (ub4) OCI_DEFAULT,
                (dvoid *) 0, (dvoid * (*)(dvoid *,size_t)) 0,
                (dvoid * (*)(dvoid *, dvoid *, size_t)) 0,
            (void (*)(dvoid *, dvoid *)) 0, (size_t) 0, (dvoid **) 0);
```

- OCIAttrSet

  设置句柄或描述符属性的值。

  示例:

```
OCIAttrSet((dvoid *) svchp, (ub4) OCI_HTYPE_SVCCTX,
```

```
            (dvoid *) srvhp, (ub4) 0,
        (ub4) OCI_ATTR_SERVER, (OCIError *) errhp);
```

- OCIAttrGet

  获取句柄或描述符属性的值

  示例：

```
OCIAttrGet((dvoid *)parmp, (ub4)OCI_DTYPE_PARAM,
                        (dvoid *)&collection_typecode, (ub4 *)0,
                        (ub4)OCI_ATTR_COLLECTION_TYPECODE,
                    (OCIError *)ctx->errhp)
```

- OCIHandleAlloc

  用于分配和初始化用户指定类型的句柄。

  示例：

```
OCIHandleAlloc( (dvoid *) NULL, (dvoid **) &envhp, (ub4) OCI_HTYPE_ENV,
            52, (dvoid **) &tmp);
```

- OCIHandleFree

  释放用户指定的句柄指向的资源。

  示例：

```
OCIHandleFree((dvoid *) ctxptr->envhp, (ub4) OCI_HTYPE_ENV);
```

- OCIDescriptorAlloc

分配和初始化 OCI 描述符或 LOB 定位器。

示例：

```
OCIDescriptorAlloc((dvoid *)ctx.envhp, (dvoid **)&parm,
                   (ub4)OCI_DTYPE_PARAM, 0, (dvoid **)0);
```

- OCIDescriptorFree

  释放已经分配的描述符。

  示例：

```
OCIDescriptorFree((dvoid *)envhp, (ub4)OCI_HTYPE_ENV);
```

- OCILogon

  创建一个简单的登录会话。

  示例：

```
OCILogon(envhp, errhp, &svchp, username, strlen (username),
         password, strlen (password), "", 0)
```

- OCILogoff

  用来释放传递给 OCILogon 的服务上下文句柄，断开由 OCILogon 建立的数据库连接。

  示例：

```
OCILogoff((dvoid *) svchp, errhp)
```

- OCIStmtPrepare
- 准备要执行的 SQL 或 PL/SQL 语句。

  示例：

```
OCIStmtPrepare(stmthp, errhp, (text *) create_type_person,
                        (ub4) strlen((char *) create_type_person),
                        (ub4) OCI_NTV_SYNTAX, (ub4) OCI_DEFAULT)
```

- OCIStmtExecute

  将应用程序请求与服务器关联。

  示例：

```
OCIStmtExecute(svchp, stmthp, errhp, (ub4) 1, (ub4) 0,
                        (OCISnapshot *) NULL, (OCISnapshot *) NULL,
                        (ub4) OCI_DEFAULT)
```

- OCIStmtFetch

  从（可滚动的）结果集中获取一行。

  示例：

```
OCIStmtFetch(stmthp, errhp, (ub4) 1,  (ub4) OCI_FETCH_NEXT,
                        (ub4) OCI_DEFAULT)
```

- OCIStmtRelease

  释放通过调用 OCIStmtPrepare()获得的语句句柄。

  示例：

```
OCIStmtRelease ((OCIStmt *)stmthp, (OCIError *)errhp,tag ,
```

```
    (ub4)strlen((char  *)tag),OCI_DEFAULT)
```

7.3. 设置 OCI 驱动

安装 OCI 驱动前需要先装好 PanWeiDB V2.0-S3.0.0_Beta Libpq 和 unixODBC 以及 ODBC,并配置环境变量。

7.3.1. 安装 libpq

安装 Libpq 有两种方式。

- 通过提供的 Libpq 安装包安装

  解压 PanWeiDB V2.0-S3.0.0_Beta 提供的 Libpq，新建/usr/local/libpq 目录，将解压后的 include 和 lib 目录拷贝到/usr/local/libpq 目录下，配置环境变量如下：

```
export LD_LIBRARY_PATH=/usr/local/libpq/lib:$LD_LIBRARY_PATH
export C_INCLUDE_PATH=/usr/local/libpq/include:$C_INCLUDE_PATH
export CPLUS_INCLUDE_PATH=/usr/local/libpq/include:$CPLUS_INCLUDE_PATH
```

- 通过 PanWeiDB V2.0-S3.0.0_Beta 安装包安装

  安装过程可参考 PanWeiDB V2.0-S3.0.0_Beta 安装指南中的数据库安装步骤。新建/usr/local/libpq 目录，将数据库安装目录中的 include 和 lib 目录拷贝到/usr/local/libpq 目录下，配置上述环境变量。

7.3.2. 安装 unixODBC

要使用驱动， 必须先安装 unixODBC。

离线安装：



unixODBC-2.3.9.tar.gz

```
tar zxvf unixODBC-2.3.9.tar.gz

cd unixODBC-2.3.9


./configure --enable-gui=no #如果要在鲲鹏服务器上编译，请追加一个 confi
gure 参数： --build=aarch64-unknown-linux-gnu

make

#安装可能需要 root 权限

make install
```

7.3.3. 安装 ODBC

ODBC 安装可以选择安装 opengGauss 提供的安装包或者由 PanWeiDB V2.0-S3.0.0_Beta 提供的安装包。

7.3.3.1. 安装包安装 openGauss-2.0.0-ODBC

- 下载对应平台的 openGauss-2.0.0-ODBC（目前仅支持 centos_x86_64、openeuler_aarch64、openeuler_x84_64）

  地址：https://opengauss.org/zh/download.html

如图：

openGauss Connectors

	centos_x86_64	openeuler_aarch64	openeuler_x86_64
JDBC_2.0.0	下载 SHA256	下载 SHA256	下载 SHA256
ODBC_2.0.0	下载 SHA256	下载 SHA256	下载 SHA256
libpq_2.0.1	下载 SHA256	下载 SHA256	下载 SHA256

- 替换客户端 openGauss 驱动程序

  解压 openGauss-2.0.0-ODBC.tar.gz，将解压后 lib 目录下的库拷贝到“/usr/local/lib”目录下。

7.3.3.2. 安装包安装 PanWeiDB V2.0-S3.0.0_Beta ODBC

- 解压由 PanWeiDB V2.0-S3.0.0_Beta 提供的 ODBC 驱动安装包。
- 将解压后的 lib 目录以及 odbc/lib 目录下的库拷贝到“/usr/local/lib”目录下。

7.3.3.3. 手动编译安装

- 下载 ODBC 驱动源码

  地址如下：https://gitee.com/opengauss/openGauss-connector-odbc

- 编译

  首先修改 configure 文件如下（红色框为增加内容）：

```
4456 if test "$with_libpq" != yes; then
4457   if test -d "$with_libpq"; then
4458     PATH="$PATH:$with_libpq/bin"
4459     CPPFLAGS="$CPPFLAGS -I$with_libpq/include -I$with_libpq/include/postgresql/internal -I$with_libpq/include/postgresql/server"
4460     LDFLAGS="$LDFLAGS -L$with_libpq/lib"
4461   else
4462     if test -x "$with_libpq"; then
4463       PG_CONFIG=$with_libpq
4464     else
4465       as_fn_error $? "specified pg_config not found" "$LINENO" 5
4466     fi
```

  执行如下命令进行编译：

```
sh ./build.sh -bd $GAUSSHOME
```

  将 output 下的 GaussDB-Kernel-XXX-0dbc.tar.gz 解压，然后将 output

/odbc/lib 目录下的库拷贝到“/usr/local/lib” 目录下。

7.3.3.4. 配置数据源

1. 配置 ODBC 驱动文件。 在“/usr/local/etc/odbcinst.ini” 文件中添加以下内容。

```
vim /usr/local/etc/odbcinst.ini
```

配置	说明
[panweidbMPP]	# 驱动器名称， 对应数据源 DSN 中的驱动名
Driver64=/usr/local/lib/psqlodbcw.so	# 驱动动态库的路径
setup=/usr/local/lib/psqlodbcw.so	# 驱动安装路径， 与 Driver64 中动态库的路径一致

2. 配置数据源文件。 该步骤并不是必须的， 取决于获取连接的方式，请参见 7.4 使用 OCI 连接数据库。 在“/usr/local/etc/odbc.ini ” 文件中添加以下内容。

配置	说明
[Test] Description=Test Driver=panweidbMPP	# 数据源的名称。 # 数据源说明 # 驱动名， 对应 odbcinst.ini 中的 DriverName
Servername=172.16.12.57	# 服务器的 IP 地址

Database=postgres Username=yuntu	# 要连接的数据库的名称 # 数据库用户名称
Password= Ttest@123	# 数据库用户密码
Port=5432	# 服务器的端口号
Sslmode=allow	# 开启 SSL 模式： disable 否， allow 可能 ， prefer 可能， requrie 是，verify-ca 是， verify-full 是

7.3.3.5. 配置环境变量

```
vim ~/.bashrc
```

在配置文件中追加以下内容。

```
export LD_LIBRARY_PATH=/usr/local/lib/:$LD_LIBRARY_PATH

export ODBCSYSINI=/usr/local/etc

export ODBCINI=/usr/local/etc/odbc.ini
```

执行如下命令使配置生效

```
source ~/.bashrc
```

7.3.3.6. 测试数据源配置

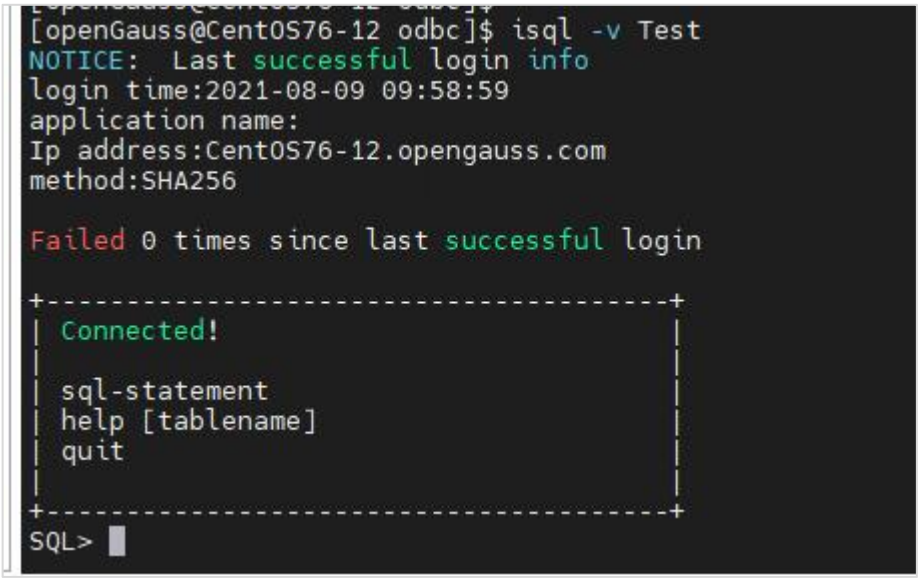
上述配置中该数据源的 DSN 为 Test,可以通过 isql 来验证配置是否正确。命令如下。

```
isql -v Test
```

如果数据源配置中未指定用户名和密码，则用 isql 测试连接时需要指定用户密码：

```
isql -v Test yuntu Ttest@123
```

如果显示以下信息，表示配置正确，连接成功。



如果显示查找不到 psqlodbcw.so 动态库，如图：



在确认动态库确实存在之后，该错误可能是由于当前机器上缺少 odbc 驱动库的依赖库造成的，可以使用 ldd 命令观察缺失的动态库后安装对应的依赖库。

7.3.4. 安装 OCI

7.3.4.1. 安装包安装 PanWeiDB V2.0-S3.0.0_Beta OCI

- 解压由 PanWeiDB V2.0-S3.0.0_Beta 提供的 OCI 驱动安装包,并将路径添加到 LD_LIBRARY_PATH 环境变量里，再将 LD_LIBRARY_PATH 添加到

LIBRARY_PATH 环境变量里。

- 将解压后的.so 和.a 文件拷贝到“/usr/local/lib” 目录下,解压出来的 include 文件夹放到/usr/include/路径下

7.4. 使用 OCI 连接数据库

7.4.1. 连接数据库

7.4.1.1. 获取 OCI 连接

在使用 OCIEnvCreate 接口（或者 OCIEnvInit，OCIInitialize）分配环境句柄，以及 OCIHandleAlloc 接口分配错误句柄后，获取 OCI 数据库连接用 OCILogon 接口。如果使用 OCIInitilize 接口创建环境,则需要搭配 OCITerminate 接口使用来结束程序。

```
OCIEnv        *envhp;   // 环境句柄
OCIServer     *srvhp;   // 服务句柄
OCIError      *errhp;    // 错误句柄
OCISvcCtx     *svchp;   //服务上下文句柄
/* 1.分配环境和错误句柄 */
if(OCIEnvCreate((OCIEnv **) &envhp, (ub4) OCI_DEFAULT,
                    (dvoid *) 0, (dvoid * (*)(dvoid *,size_t)) 0,
                    (dvoid * (*)(dvoid *, dvoid *, size_t)) 0,
                    (void (*)(dvoid *, dvoid *)) 0, (size_t) 0, (dvoid **) 0))
{
```

```
printf("FAILED: OCIEnvCreate()\n");
return OCI_ERROR;
}
If(OCIHandleAlloc( (dvoid *) envhp, (dvoid **) &errhp, (ub4) OCI_HTYPE_ER
ROR,
            (size_t)0, (dvoid **) 0))
{
printf("FAILED: OCIHandleAlloc()\n");
return OCI_ERROR;
}
 /* 2.建立连接并创建简单会话*/
if(OCILogon(envhp, errhp, &svchp, username, strlen (username),
                password, strlen (password), "", 0))
{
printf("FAILED: OCILogon()\n");
return OCI_ERROR;
}
```

7.4.1.2. 关闭 OCI 连接

关闭连接时调用 OCILogoff()函数关闭数据库连接，程序结束前调用 OCIHandleFree 释放分配的句柄:

```
if(OCILogoff(svchp, errhp))
```

```
{
printf("FAILED: OCILogoff()\n");
return OCI_ERROR;
}
if(svchp)
    OCIHandleFree((dvoid *)svchp, (ub4)OCI_HTYPE_SVCCTX);
if(errhp)
    OCIHandleFree((dvoid *)errhp, (ub4)OCI_HTYPE_ERROR);
if(envhp)
    OCIHandleFree((dvoid *)envhp, (ub4)OCI_HTYPE_ENV);
```

7.5. 执行 SQL 语句并处理结果

向数据库发出执行 SQL 的请求时，需要使用语句句柄，并调用 OCIStmtPrepare 和 OCIStmtExecute 执行查询，服务端执行完成后查询的结果将保存在语句句柄中，使用 OCIStmtFetch 获取结果。

7.5.1. 执行查询、获取结果

调用 OCIAllocHandle()函数获取语句句柄，使用 OCIStmtPrepare 预编译语句，使用 OCIDefineByPos 绑定缓冲区，然后调用 SQLExecute 执行查询，最后调用 OCIStmtFetch 获取结果集。

```
OCIStmt *stmthp; //语句句柄
text *sqlstmt=(text *)" select C1 from T2" ;
```

```
if(OCIHandleAlloc ((dvoid *) envhp, (dvoid **) &errhp, OCI_HTYPE_ERROR,
                (size_t) 0, (dvoid **) 0))
{
    printf("FAILED: OCIHandleAlloc()\n");
return OCI_ERROR
}
int *c1 = (int *) malloc (sizeof(int) * GET_ROWS);
OCIDefine *bndhp[0];
ub4 stmrow, stmrowEx;
OCIParam* colhd = NULL;
int i;
i = '+';
memset(c1, 0, sizeof(int) * GET_ROWS);
/*执行语句*/
OCIStmtPrepare(stmthp, errhp, sqlstmt, strlen((char *)sqlstmt), OCI_NTV_SY
NTAX, 0);

if(OCIStmtExecute(svchp, stmthp, errhp, 0, 0, 0, 0, 0))
{
    printf( "FAILED: OCIStmtExecute()\n" );
}

/*绑定缓冲区到相对应的列上*/
if(OCIDefineByPos(stmthp, &bndhp[0], errhp, 1
```

```
(dvoid *)c1, (sb4)sizeof(int), (ub2)SQLT_INT,
(dvoid *)0, (ub2 *)0, (ub2 *)0, (ub4)OCI_DEFAULT))
{
    printf( "FAILED: OCIDefineByPos()\n" );
    return OCI_ERROR;
}
if (OCIDefineArrayOfStruct(bndhp[0], errhp, sizeof(int), 0, 0, 0))
{
    printf( "FAILED: OCIDefineArrayOfStruct ()\n" );
    return OCI_ERROR;
}
/*获取结果*/
If(OCIStmtFetch(stmthp, errhp, GET_ROWS, OCI_FETCH_NEXT, 0))
{
    printf( "FAILED: OCIStmtFetch ()\n" );
    return OCI_ERROR;
}
```

7.5.2. 执行插入

要更改数据(执行一个 insert，update 或者 delete)，可以使用 bind 的方式绑定参数后再执行。

调用 OCIHandleAlloc 申请语句句柄，在语句通过调用 OCIStmtPrepare 准备完成后，可以通过 OCIBindByName（或者 OCIBindByPos）绑定参数，再调用 OCIStmtExecute 执行语句。

```
OCIStmt *stmthp; //语句句柄
text *sqlstmt=(text *)" insert into T2(C1) values(:a)" ;
if(OCIHandleAlloc ((dvoid *) envhp, (dvoid **) &errhp, OCI_HTYPE_ERROR,
                (size_t) 0, (dvoid **) 0))
{
    printf("FAILED: OCIHandleAlloc()\n");
return OCI_ERROR
}
int custno = 9;

OCIStmtPrepare(stmthp, errhp, sqlstmt, strlen((char *)sqlstmt), OCI_NTV_SY
NTAX, 0);

if(OCIBindByName(stmthp, &bnd1p, errhp, (text *) ":a" ,
(sb4) -1, (dvoid *) &custno,
(sb4) sizeof(int), SQLT_INT,
(dvoid *) 0, (ub2 *)0, (ub2 *)0, (ub4) 0, (ub4 *) 0,
(ub4) OCI_DEFAULT))
{
    printf( "FAILED: OCIBindByName()\n" );
```

```
    return OCI_ERROR;
}

if(OCIStmtExecute(svchp, stmthp, errhp, 0, 0, 0, 0, 0))
{
    printf( “FAILED: OCIStmtExecute()\n” );
}
```

7.5.3. 获取结果

调用 OCIStmtFetch 来获取执行的结果

```
//测试获取 SQL 生成的结果集中的行集
    if(OCIStmtFetch(stmthp, errhp, GET_ROWS, OCI_FETCH_NEXT, 0))
        printf("FAILED: OCIStmtFetch()\n");
```

7.6. 处理数据类型

OCI 支持 panweidb 中各种类型的读写操作，例如数字类型，字符串类型，日期类型，大对象类型等。

例如：

```
CREATE TABLE T1 (C1 int, C2 real, C3 CHAR(100), C4 VARCHAR(100), C5 FL
OAT, C6 NUMERIC(22,7), C7 Date, C8 bigint, C9 timestamp (6) with time z
one, C10 clob, c11 bfile)” ;
```

7.6.1. 大对象类型

7.6.1.1. 插入记录

```
OCIStmt *stmthp;

text *sqlstmt = (text *) “insert into T1(C10) value (‘panwei’)” ;

if(OCIHandleAlloc ((dvoid *) envhp, (dvoid **) &errhp, OCI_HTYPE_ERROR,
                (size_t) 0, (dvoid **) 0))
{
    printf("FAILED: OCIHandleAlloc()\n");
return OCI_ERROR
}

if(OCIStmtPrepare(stmthp, errhp, sqlstmt, strlen((char *)sqlstmt), OCI_NTV_S
YNTAX, 0))
{
printf( “FAILED: OCIStmtPrepare()\n” );
return OCI_ERROR;
}

if(OCIStmtExecute(svchp, stmthp, errhp, 0, 0, 0, 0, 0))
{
```

```
    printf( “FAILED: OCIStmtExecute()\n” );
}
```

7.6.1.2. 读取 LOB 大对象定位器

调用 OCIDescriptorAlloc 申请大对象描述符句柄，通过 OCIDefineByPos 定义绑定结果，将定位器绑定到大对象描述符上以便后续调用大对象接口。也可以再申请完大对象描述符句柄后调用 OCILobCreateTemporary 创建临时 LOB 对象。

```
OCIEnv *envhp;
    OCIServer *srvhp;
    OCIError *errhp;
    OCISvcCtx *svchp;
    OCIStmt *stmthp;

   /*可进行不同的 sql 测试，单个绑定和复合绑定可以选择下边的 sqlstmt*/
   text *sqlstmt=(text *)"select  c10 from T2 where c1 = 998999";
  int i;
  int flag;

  int      *c1 = (int*) malloc (sizeof(int) * GET_ROWS);

  OCIDefine *bndhp[2];
```

```
  ub4       stmrow, stmrowEx;
  OCIParam* colhd = NULL;
  i = '+';
  memset(c1, 0, sizeof(int) * GET_ROWS);


    if(OCIEnvCreate ((OCIEnv **)&envhp, (ub4)OCI_DEFAULT, (dvoid *)0,
                  (dvoid * (*)(dvoid *, size_t))0,
                  (dvoid * (*)(dvoid *, dvoid *, size_t))0,
                  (void (*)(dvoid *, dvoid *))0, (size_t)0, (dvoid **)0))
  {
    printf("FAILED: OCIEnvCreate()\n");
    return OCI_ERROR;
  }


    if(OCIHandleAlloc ((dvoid *) envhp, (dvoid **) &errhp, OCI_HTYPE_ERR
OR,
                     (size_t) 0, (dvoid **) 0))
  {
    printf("FAILED: OCIHandleAlloc()\n");
    return OCI_ERROR;
  }


  if(OCIHandleAlloc( (dvoid *) envhp, (dvoid **) &stmthp,
            OCI_HTYPE_STMT, (size_t) 0, (dvoid **) 0))
```

```
  {
    printf("FAILED: OCIHandleAlloc()\n");
    return OCI_ERROR;
  }

    int total =10;
    OCILobLocator **lobLocator = (OCILobLocator **)malloc(sizeof(OCILobLo
cator *)*total);

    OCILobLocator **lobLocator_dst = (OCILobLocator **)malloc(sizeof(OCIL
obLocator *)*total);

    for(i =0; i<total; ++i)
    {
        if (OCIDescriptorAlloc(envhp, (dvoid **)&lobLocator[i], OCI_DTYPE_L
OB, 0, 0))
        {
            printf("FAILED: lob_locatorOCIDescriptorAlloc()\n");
            report_error(errhp);
        }
        if (OCIDescriptorAlloc(envhp, (dvoid **)&lobLocator_dst[i], OCI_DTYP
E_LOB, 0, 0))
        {
            printf("FAILED: lob_locatorOCIDescriptorAlloc()\n");
```

```
            report_error(errhp);
        }
    }

    /*
     * Connect and create an user session
*/

if (OCILogon(envhp, errhp, &svchp, username, strlen (username),
        password, strlen (password), dsn, strlen(dsn)))
{
    printf("FAILED: log_on()\n");
    return OCI_ERROR;
}

//测试准备一条 SQL
if(OCIStmtPrepare(stmthp, errhp, sqlstmt, strlen((char *)sqlstmt),
 OCI_NTV_SYNTAX, 0))
        {
                printf("FAILED: OCIStmtPrepare()\n");
                report_error(errhp);
                return OCI_ERROR;
        }
```

```
if (OCIDefineByPos(stmthp, &bndhp[0], errhp, 1,
  (dvoid *)lobLocator, sizeof(OCILobLocator *), (ub2)SQLT_CLOB,
  (dvoid *)0, (ub2 *)0, (ub2 *)0, (ub4)OCI_DEFAULT))
{
  printf("FAILED: OCIDefineByPos2()\n");
  report_error(errhp);
  return OCI_ERROR;
}

if (OCIDefineArrayOfStruct(bndhp[0], errhp, sizeof(OCILobLocator *),
  0, 0, 0))
{
  printf("FAILED: OCIDefineArrayOfStruct2()\n");
  report_error(errhp);
  return OCI_ERROR;
}

//测试 SQL 执行
if(OCIStmtExecute(svchp, stmthp, errhp, 0, 0, 0, 0, 0))
      {
          report_error(errhp);
          printf("FAILED: OCIStmtExecute()\n");
      }
```

```
    //测试获取 SQL 生成的结果集中的行集
    if(OCIStmtFetch(stmthp, errhp, total, OCI_FETCH_NEXT, 0))
    {
        printf("FAILED: OCIStmtFetch()\n");
        report_error(errhp);

    }
```

7.6.1.3. 操作大对象类型

再获取了 LOB 对象定位器之后，可以调用 OCI 大对象接口操作大对象，包括 OCILobAssign，OCILobGetLength，OCILobIsEqual，OCILobRead，OCILobWrite，OCILobIsEqual。

```
    OCIAttrGet(stmthp, OCI_HTYPE_STMT, &stmrow, 0, OCI_ATTR_ROW_COUN
T, errhp);

    for(i=0; i<stmrow; ++i)
    {
        int len = 0;
        int len_dst = 0;
        boolean flag = 0;
        boolean flag1 = 0;
        boolean isEqual = 0;
```

```
    if(OCILobLocatorIsInit(envhp, errhp, lobLocator[i],&flag)){
        printf("FAILED: OCILobLocatorIsInit()\n");
        report_error(errhp);
        return OCI_ERROR;
    }
    printf("OCILobLocatorIsInit is %d\n", flag);

    if(OCILobIsOpen(svchp,errhp, lobLocator[i], &flag1)){
        printf("FAILED: OCILobIsOpen()\n");
        report_error(errhp);
        return OCI_ERROR;
    }
    printf("OCILobIsOpen -1 is %d\n", flag1);

    if(OCILobOpen(svchp, errhp, lobLocator[i],OCI_LOB_READWRITE)){
       printf("FAILED: OCILobOpen()\n");
report_error(errhp);
return OCI_ERROR;
    }

    if(OCILobIsOpen(svchp,errhp, lobLocator[i], &flag1)){
        printf("FAILED: OCILobIsOpen()\n");
        report_error(errhp);
```

```
            return OCI_ERROR;
        }
        printf("OCILobIsOpen - 2 is %d\n", flag1);

        if(OCILobAssign(envhp, errhp, lobLocator[i], &lobLocator_dst[i]))
        {
            printf("FAILED: OCILobAssign()\n");
            report_error(errhp);
            return OCI_ERROR;
        }

        if(OCILobIsEqual(envhp, lobLocator[i], lobLocator_dst[i], &isEqual))
        {
            printf("FAILED: OCILobAssign()\n");
            return OCI_ERROR;
        }
        printf("OCILobIsEqual = %d\n", isEqual);

        if(OCILobGetLength(svchp, errhp, lobLocator[i], &len)){
            printf("FAILED: OCILobGetLength()\n");
report_error(errhp);
return OCI_ERROR;
        }
        printf("len = %d\n", len);
```

```
        if(OCILobGetLength(svchp, errhp, lobLocator_dst[i], &len_dst)){
            printf("FAILED: OCILobGetLength()\n");
    report_error(errhp);
    return OCI_ERROR;
        }
        printf("len_dst = %d\n", len_dst);

        ub4 amount = 2;
        char readStr[10000]={0};
        if(OCILobRead(svchp, errhp, lobLocator[i], &amount, 1, (void *)rea
dStr, 2, NULL, NULL, 0, 0)){
            printf("FAILED: OCILobRead()\n");
            report_error(errhp);
            return OCI_ERROR;
        }
        printf("readStr=%s\n", readStr);

        ub4 amount1 = 3;
        char *writeStr = "panweidb";
        if(OCILobWrite(svchp, errhp, lobLocator[i], &amount1, 2, (void *)wri
teStr, 3, 0, NULL, NULL, 0, 0)){
            printf("FAILED: OCILobWrite()\n");
            report_error(errhp);
```

```
            return OCI_ERROR;
        }

        if(OCILobTrim(svchp, errhp, lobLocator[i], 6)){
            printf("FAILED: OCILobTrim()\n");
            report_error(errhp);
            return OCI_ERROR;
        }

        if(OCILobClose(svchp, errhp, lobLocator[i])){
            printf("FAILED: OCILobClose()\n");
    report_error(errhp);
    return OCI_ERROR;
        }
        if(OCILobClose(svchp, errhp, lobLocator_dst[i])){
            printf("FAILED: OCILobClose()\n");
    report_error(errhp);
    return OCI_ERROR;
       }
    }
```

7.7. 句柄和描述符属性管理

OCI 可以通过调用 OCIDescribeAny，OCIAttrGet，OCIAttrSet，OCIParamGet 接口对句柄和描述符属性资源进行管理。

7.7.1. 句柄属性管理

在分配完环境句柄后，可以调用 OCIAttrSet 和 OCIAttrGet 对句柄属性进行设置和获取

```
oratext phost[10] = {0};


    sword r2 =  OCIAttrGet(envhp, OCI_HTYPE_ENV, phost, &plen, OCI_ATT
R_LDAP_HOST, errhp);
    printf("OCI_ATTR_LDAP_HOST=%s,res=%s\n", phost,getErrStr(r2));


    sword r3 = OCIAttrGet(stmthp, OCI_HTYPE_STMT, &uvalue, &plen, OCI_A
TTR_BIND_COUNT, errhp);
    printf("OCI_ATTR_BIND_COUNT=%d,res=%s\n", uvalue,getErrStr(r3));


    OCIEnv *envhpnew = NULL;
    sword r4 = OCIAttrGet(stmthp, OCI_HTYPE_STMT, &envhpnew, &plen, O
CI_ATTR_ENV, errhp);
    printf("OCI_ATTR_ENV,res=%s\n", getErrStr(r4));
```

```
    boolean bl;
    sword r5 = OCIAttrGet(stmthp, OCI_HTYPE_STMT, &bl, &plen, OCI_ATTR_
FETCH_ROWID, errhp);
    printf("OCI_ATTR_FETCH_ROWID=%d,res=%s \n", bl,getErrStr(r5));


    uvalue = 0;
    sword r6 = OCIAttrGet(stmthp, OCI_HTYPE_STMT, &uvalue, &plen, OCI_A
TTR_STMT_STATE, errhp);
    printf("OCI_ATTR_STMT_STATE=%d,res=%s\n", uvalue,getErrStr(r6));


    uvalue = 0;
    sword r7 = OCIAttrGet(stmthp, OCI_HTYPE_STMT, &uvalue, &plen, OCI_A
TTR_PARAM_COUNT, errhp);
    printf("OCI_ATTR_PARAM_COUNT=%d,res=%s\n", uvalue,getErrStr(r7));




sword status = 0;
    static OCIServer *svrhp =  (OCIServer *)0;
    OCIHandleAlloc(envhp1, (dvoid **)&svrhp, OCI_HTYPE_SERVER, (size_t)0,
(dvoid **)0);
    status = OCIServerAttach(svrhp, errhp, (text *)dsn, (sb4)strlen((char *)d
sn),OCI_DEFAULT);
```

```
    OCIHandleAlloc(envhp1,(dvoid **)&svchp,OCI_HTYPE_SVCCTX,(size_t)0,(d
void **)0);
    sword rc = OCIAttrSet(svchp, OCI_HTYPE_SVCCTX, (dvoid *)svrhp, (ub4)
0,OCI_ATTR_SERVER, errhp);
    printf("设置 OCI_ATTR_SERVER 的结果为:%s\n",getErrStr(rc));

    static OCISession *sesnhp = (OCISession *)0;

    OCIHandleAlloc(envhp1, (dvoid **)&sesnhp, OCI_HTYPE_SESSION, (size_t)
0, (dvoid **)0);

    rc = OCIAttrSet(sesnhp, OCI_HTYPE_SESSION, (dvoid *)username, (ub4)s
trlen((char *)username), OCI_ATTR_USERNAME, errhp);
    printf("设置 OCI_ATTR_USERNAME 的结果为:%s\n", getErrStr(rc));

    rc = OCIAttrSet(sesnhp, OCI_HTYPE_SESSION, (dvoid*)password, (ub4)st
rlen((char *)password), OCI_ATTR_PASSWORD, errhp);
    printf("设置 OCI_ATTR_PASSWORD 的结果为:%s\n", getErrStr(rc));
    status = OCISessionBegin(svchp,errhp,sesnhp,OCI_CRED_RDBMS,OCI_DEF
AULT);
```

7.7.2. 描述符属性管理

描述符可以通过调用 OCIDescribeAny 和 OCIParamGet 接口来获取属性。

```
if(OCIDescribeAny(svchp, errhp, "t2", strlen("t2"), OCI_OTYPE_NAME, 1, OCI_
PTYPE_TABLE, (OCIDescribe *)dschp))
    {
        printf("FAILED: log_on()\n");
        report_error(errhp);
        return OCI_ERROR;
    }

    OCIParam *parmh = (OCIParam *) 0;
    if (OCIAttrGet((void *)dschp, OCI_HTYPE_DESCRIBE, (void *)&parmh, (ub
4 *)0,
                   OCI_ATTR_PARAM, errhp))
        return OCI_ERROR;

    ub1 param_type;
    if (OCIAttrGet((void *)parmh, OCI_DTYPE_PARAM, (void *)&param_type,
(ub4 *)0,
                   OCI_ATTR_PTYPE, errhp))
        return OCI_ERROR;
    printf("OCI_ATTR_PTYPE=%d %d\n", param_type, OCI_PTYPE_TABLE);

    ub2 numcols = 0;
    if (OCIAttrGet((void *)parmh, OCI_DTYPE_PARAM, (void *)&numcols, (ub
4 *)0,
```

```
     OCI_ATTR_NUM_COLS, errhp))
         return OCI_ERROR;


    OCIParam *collsthd = (OCIParam *) 0;
    if (OCIAttrGet((void *)parmh, OCI_DTYPE_PARAM, (void *)&collsthd, (ub
4 *)0,
     OCI_ATTR_LIST_COLUMNS, errhp)==OCI_NO_DATA)
        return OCI_ERROR;


    if (OCIAttrGet((void *)collsthd, OCI_DTYPE_PARAM, (void *)&param_type,
 (ub4 *)0,
                 OCI_ATTR_PTYPE, errhp))
         return OCI_ERROR;
    printf("OCI_ATTR_PTYPE=%d %d\n", param_type, OCI_PTYPE_LIST);


    for(int i=0;i<numcols;++i)
    {
         printf("\n 第 %d 列描述符信息如下:  \n", i+1);
         if (OCIParamGet((void *)collsthd, OCI_DTYPE_PARAM, errhp, (void *
*)&colhd, (ub4)i+1))
             return OCI_ERROR;


         OraText  *namep;
         ub4 sizep =0;
```

```
        OCIAttrGet((dvoid*) colhd, (ub4) OCI_DTYPE_PARAM, (dvoid**) &na
mep, (ub4 *) &sizep, (ub4)OCI_ATTR_NAME, (OCIError *)errhp);
        printf("OCI_ATTR_NAME=%s, %d\n", namep, sizep);

    if (OCIAttrGet((void *)colhd, OCI_DTYPE_PARAM, (void *)&param_type,
(ub4 *)0,
                OCI_ATTR_PTYPE, errhp))
        return OCI_ERROR;
    printf("OCI_ATTR_PTYPE=%d %d\n", param_type, OCI_PTYPE_COL);

        OCIAttrGet((dvoid*) colhd, (ub4) OCI_DTYPE_PARAM, (dvoid**) &na
mep, (ub4 *) &sizep, (ub4)OCI_ATTR_SCHEMA_NAME, (OCIError *)errhp);
        printf("OCI_ATTR_SCHEMA_NAME=%s, %d\n", namep, sizep);

        ub2 dtype;
        OCIAttrGet((dvoid*) colhd, (ub4) OCI_DTYPE_PARAM, (dvoid*) &dtyp
e, (ub4 *) &sizep, (ub4)OCI_ATTR_DATA_TYPE, (OCIError *)errhp);
        printf("OCI_ATTR_DATA_TYPE=%d, %d\n", dtype, sizep);

        OCIAttrGet((dvoid*) colhd, (ub4) OCI_DTYPE_PARAM, (dvoid*) &dtyp
e, (ub4 *) &sizep, (ub4)OCI_ATTR_DATA_SIZE, (OCIError *)errhp);
        printf("OCI_ATTR_DATA_SIZE=%d, %d\n", dtype, sizep);

        OCIAttrGet((dvoid*) colhd, (ub4) OCI_DTYPE_PARAM, (dvoid **) &na
```

```
mep, (ub4 *) &sizep, (ub4) OCI_ATTR_TYPE_NAME, (OCIError *)errhp);
        printf("OCI_ATTR_TYPE_NAME=%s,%d\n", namep, sizep);

        OCIAttrGet((dvoid*) colhd, (ub4) OCI_DTYPE_PARAM, (dvoid **) &dt
ype, (ub4 *) &sizep, (ub4)OCI_ATTR_PRECISION, (OCIError *)errhp);
        printf("OCI_ATTR_PRECISION=%d, %d\n", dtype, sizep);

        OCIAttrGet((dvoid*) colhd, (ub4) OCI_DTYPE_PARAM, (dvoid **) &dt
ype, (ub4 *) &sizep, (ub4)OCI_ATTR_SCALE, (OCIError *)errhp);
        printf("OCI_ATTR_SCALE=%d, %d\n", dtype, sizep);

        //if(colhd)
        //      OCIDescriptorFree(colhd, (ub4)OCI_DTYPE_PARAM);
    }
```

7.8. 服务与会话

OCI 提供 OCIServerAttach，OCIServerDetach，OCISessionBegin，OCISessionEnd 接口管理服务与会话。

```
/*
    * Connect and create an user session
    */
    sword status = 0;
    static OCIServer    *svrhp = (OCIServer *)0;
```

```
    OCIHandleAlloc(envhp, (dvoid **)&svrhp, OCI_HTYPE_SERVER, (size_t)0,
(dvoid **)0);
    status = OCIServerAttach(svrhp, errhp, (text *)dsn, (sb4)strlen((char *)d
sn), OCI_DEFAULT);

status = OCISessionBegin(svchp, errhp, sesnhp, OCI_CRED_RDBMS, OCI_DEF
AULT);
if (status != OCI_SUCCESS)
{
    printf("Connection  failed \n");
    report_error(errhp);
}else
    printf("Connection - Success \n");

OCISessionEnd (svchp, errhp, sesnhp, OCI_DEFAULT );
OCIHandleFree((dvoid *) sesnhp, OCI_HTYPE_SESSION);
sesnhp = NULL;

sword rc = OCIServerDetach(svrhp, errhp, OCI_DEFAULT );
printf("传入正确的参数,结果为:%s\n",getErrStr(rc));
```

7.9. 事务管理

OCI 提供 OCITransCommit，OCITransRollback，OCITransStart，OCITransPrepare，OCITransForget 管理事务。

```
if(OCITransStart(svchp,errhp, 1000, 0))
        report_error(errhp);
    printf("OCITransStart success\n");

    // //测试准备一条 SQL
    OCIStmtPrepare(stmthp, errhp, sqlstmt, strlen((char *)sqlstmt),
     OCI_NTV_SYNTAX, 0);

    //测试 SQL 执行
    if(OCIStmtExecute(svchp, stmthp, errhp, GET_ROWS, 0, 0, 0, OCI_DEFAU
LT))
    {
        report_error(errhp);
        printf("FAILED: OCIStmtExecute()\n");
    }
    if(OCITransCommit(svchp,errhp,0))
        report_error(errhp);
    printf("OCITransCommit success\n");

if(OCITransRollback(svchp,errhp,0))
```

```
        report_error(errhp);
```

7.10. 杂项接口

OCI 提供 OCIErrorGet 来获取 OCI 执行过程中的错误信息。

提供 OCIServerVersion 来获取数据库服务端的版本。

```
text   msgbuf[512];
    text buf2[512];
    sb4    errcode = 0;

    memset((void *) msgbuf, (int)'\0', (size_t)512);
    memset((void *) buf2, (int)'\0', (size_t)512);

    OCIErrorGet(errhp, (ub4) 1, buf2, &errcode,
                        msgbuf, (ub4) sizeof(msgbuf), (ub4) OCI_HTYPE_ER
ROR);

    printf("ERROR CODE = %d\n", errcode);
    printf("%.*s\n", 512, msgbuf);

char version[1024] = {0};
    if(OCIServerVersion(svchp, errhp, version, (ub4)1024, OCI_HTYPE_SVCCT
X))
    {
```

```
        report_error(errhp);
        printf("FAILED: OCIServerVersion()\n");
        return OCI_ERROR;
    }
```

7.11. 日志系统

OCI 支持日志系统，来帮助用户查看运行状态和问题定位等。

7.11.1. 日志的级别

OCI 的日志级别通过 LIBOCI_LOGPATH 环境变量来设置。

```
export LIBOCI_LOGLEVEL=$log_level
```

- $log_level 是日志级别，不设置的情况为 DEFAULT。

```
log_level
{
    DEFAULT,
    FATAL,
    ERROR,
    WARN,
    INFO,
    DEBUG
} log_level;
```

- DEFAULT：默认级别（如果不设置变为他），什么都不打印。
- FATAL：严重的错误级别，只打印 FATAL 级别日志。
- ERROR：错误级别，打印 ERROR、FATAL 的日志。
- WARN：告警级别，打印 WARN、ERROR、FATAL 的日志。
- INFO：信息级别，打印 INFO、WARN、ERROR、FATAL 的日志。
- DEBUG：调试级别，打印 DEBUG、INFO、WARN、ERROR、FATAL 的日志。

7.11.2. 日志的文件格式

1. 日志路径 LIBOCI_LOGPATH 环境变量来设置。

```
export LIBOCI_LOGPATH=$log_level
```

- $LIBOCI_LOGPATH 是日志路径，不设置的情况为/tmp/oci/。

2. 日志的文件名有以下组成

```
oci_$pid_$date.log
```

- $pid 是调用 OCI 驱动的进程号。
- $date 是运行的日期。

7.11.3. 日志内容的格式

在打印日志时需要打印的信息包括：日志的 C 文件、行号、时间、日志级别、日志内容等信息

例如：

```
[OCINumber.c:549][2022-12-05 18:35:23] [INFO]: ==== OCINumberToInt ===
=
```

```
[OCINumber.c:713][2022-12-05 18:35:23] [ERROR]: overflow error
```

7.12. 分段操作

OCI 在对非常大的数据类型操作时，可以执行数据的分段插入、更新和获取。通过应用程序在运行时分割均匀的或不均匀的片段大小。将 OCIBindByPos（）或 OCIBindByPos2（）或 OCIBindByName（）或 OCIBindByName2（）调用的 mode 参数设置为 OCI_DATA_AT_EXEC 时，表示 OCI 应用程序在运行时动态为插入或更新操作提供数据，OCIDefineByPos（）或 OCIDefineByPos2（）调用的 mode 参数设置为 OCI_DYNAMIC_FETCH 时，它表示应用程序在提取时动态地为接收数据提供分配空间。当 OCIStmtFetch2 或 OCIStmtExecute 函数返回码为 OCI_NEED_DAT。

OCI 提供 OCIStmtGetPieceInfo（获取分段操作的分段信息）和 OCIStmtSetPieceInfo（设置分段操作的分段信息）。

7.12.1. 执行插入或者更新流程

```
if(OCIStmtPrepare(stmthp0, errhp, sqlstmt0, strlen((char *)sqlstmt0), OCI_NTV_SYNTAX, 0))
    {
        report_error(errhp);
        printf("FAILED: log_on()\n");
        return OCI_ERROR;
```

```
    }

    if(OCIStmtExecute(svchp, stmthp0, errhp, 1, 0, 0, 0, OCI_DEFAULT))
    {
        report_error(errhp);
        printf("FAILED: OCIStmtExecute()\n");
        return OCI_ERROR;
    }

    //测试准备一条 SQL
    if(OCIStmtPrepare(stmthp, errhp, sqlstmt, strlen((char *)sqlstmt), OCI_N
TV_SYNTAX, 0))
    {
        report_error(errhp);
        printf("FAILED: log_on()\n");
        return OCI_ERROR;
    }

    if(OCIBindByName(stmthp, &bnd2p, errhp, (text *) ":c2",
                        (sb4) -1, (dvoid *) c2,
                        (sb4) DATA_SIZE, SQLT_CHR,
                        (dvoid *) 0, (ub2 *)0, (ub2 *)0, (ub4) 0, (ub4 *)
 0,
                        (ub4) OCI_DATA_AT_EXEC))
```

```
    {
        printf("FAILED: OCIBindByName()\n");
        return OCI_ERROR;
    }

    if(OCIBindByName(stmthp, &bnd3p, errhp, (text *) ":c3",
                        (sb4) -1, (dvoid *) c3,
                        (sb4) DATA_SIZE, SQLT_CHR,
                        (dvoid *) 0, (ub2 *)0, (ub2 *)0, (ub4) 0, (ub4 *)
0,
                        (ub4) OCI_DATA_AT_EXEC))
    {
        printf("FAILED: OCIBindByName()\n");
        return OCI_ERROR;
    }

    ub1     piece;
    ub4     alenp2 = PIECE_SIZE, alenp3 = PIECE_SIZE;

    dvoid *hdlptr = (dvoid *) 0;
    ub4 hdltype;
    ub1 in_outp;
    ub4 iterp;
    ub4 idxp;
```

```
    ub1 piecep;
    char *c ;
    //测试 SQL 执行
    while(1)
    {
        int status = OCIStmtExecute(svchp, stmthp, errhp, 1, 0, 0, 0, OCI_
DEFAULT);
        switch(status)
        {
            case OCI_NEED_DATA:
                if(OCIStmtGetPieceInfo(stmthp, errhp, &hdlptr, &hdltype, &i
n_outp, &iterp, &idxp, &piecep))
                    printf("ERROR: OCIStmtGetPieceInfo returned");
                printf("bnd%dp, piecep=%d\n", hdlptr==bnd2p?2:3, piece
p);
                set_piece(&piece);

                if(hdlptr == bnd2p)
                    c=c2;
                else
                    c=c3;

                if (OCIStmtSetPieceInfo(hdlptr, OCI_HTYPE_BIND, errhp, (dv
oid *)c, &alenp2, piece, (dvoid *) 0, NULL))
```

```
                {
                    printf("ERROR: OCIStmtSetPieceInfo returned %d \n", st
atus);
                    break;
                }
                status = OCI_NEED_DATA;
                break;

            case OCI_SUCCESS:
                break;

             default:
                printf( "oci exec returned %d \n", status);
                report_error(errhp);
                status = 0;
        }
        if (!status)
            break;
    }
```

7.12.2. 执行查询操作

```
OCIStmtPrepare(stmthp, errhp, sqlstmt, strlen((char *)sqlstmt),
```

```
 OCI_NTV_SYNTAX, 0);


//测试 SQL 执行
if(OCIStmtExecute(svchp, stmthp, errhp, 0, 0, 0, 0, 0))
        printf("FAILED: OCIStmtExecute()\n");


//绑定缓冲区到相对应的列上
if(OCIDefineByPos(stmthp, &bndhp[0], errhp, 1,
(dvoid *)c1, (sb4)sizeof(int), (ub2)SQLT_INT,
(dvoid *)0, (ub2 *)0, (ub2 *)0, (ub4)OCI_DEFAULT))
{
    printf("FAILED: OCIDefineByPos0()\n");
    report_error(errhp);
    return OCI_ERROR;
}


if (OCIDefineByPos(stmthp, &bndhp[1], errhp, 2,
        (dvoid *)c2, (sb4)CHAR_SIZE, (ub2)SQLT_CHR,
        (dvoid *)0, (ub2 *)0, (ub2 *)0, (ub4)OCI_DYNAMIC_FETCH))
{
    printf("FAILED: OCIDefineByPos2()\n");
    report_error(errhp);
    return OCI_ERROR;
}
```

```
if (OCIDefineByPos(stmthp, &bndhp[2], errhp, 3,
    (dvoid *)c3, (sb4)CHAR_SIZE, (ub2)SQLT_CHR,
    (dvoid *)0, (ub2 *)0, (ub2 *)0, (ub4)OCI_DYNAMIC_FETCH))
{
    printf("FAILED: OCIDefineByPos2()\n");
    report_error(errhp);
    return OCI_ERROR;
}

if (OCIDefineArrayOfStruct(bndhp[0], errhp, sizeof(int),
    0, 0, 0))
{
    printf("FAILED: OCIDefineArrayOfStruct0()\n");
    report_error(errhp);
    return OCI_ERROR;
}
if (OCIDefineArrayOfStruct(bndhp[1], errhp, CHAR_SIZE,
    0, 0, 0))
{
    printf("FAILED: OCIDefineArrayOfStruct2()\n");
    report_error(errhp);
    return OCI_ERROR;
}
```

```
    if (OCIDefineArrayOfStruct(bndhp[2], errhp, CHAR_SIZE,
        0, 0, 0))
    {
        printf("FAILED: OCIDefineArrayOfStruct2()\n");
        report_error(errhp);
        return OCI_ERROR;
    }

    #define PIECE_SIZE 10
    char buf1[100];
    ub4    alen   = PIECE_SIZE;
    ub1    piece = OCI_FIRST_PIECE;
    dvoid *hdlptr = (dvoid *) 0;
    ub4 hdltype = OCI_HTYPE_DEFINE, iter = 0, idx = 0;
    ub1    in_out = 0;
    sb2    indptr = 0;
    ub2    rcode = 0;
    //测试获取 SQL 生成的结果集中的行集
    int status = OCIStmtFetch(stmthp, errhp, 1, OCI_FETCH_NEXT, 0);
    while (status == OCI_NEED_DATA)
    {
        if(OCIStmtGetPieceInfo(stmthp, errhp, &hdlptr, &hdltype, &in_out, &
iter, &idx, &piece))
            printf("FAILED: OCIStmtGetPieceInfo()\n");
```

```
        //printf("bndhp[1]=%p, bndhp[2]=%p, hdlptr=%p\n", bndhp[1], bnd
hp[2], hdlptr);
        if(OCIStmtSetPieceInfo((dvoid *)hdlptr, (ub4)hdltype,
                                errhp, (dvoid *) &buf1, &alen, piece,
                                (dvoid *)&indptr, &rcode))
            printf("FAILED: OCIStmtGetPieceInfo()\n");

        memset(buf1, 0, alen);
        status = OCIStmtFetch(stmthp,errhp, (ub4) 1,
                        (ub2) OCI_FETCH_NEXT, (ub4) OCI_DEFAULT);

        printf("buf1:%s , %d\n", buf1, alen);
    }
    //总共返的记录的行数
    OCIAttrGet(stmthp, OCI_HTYPE_STMT, &stmrow, 0, OCI_ATTR_ROW_COUN
T, errhp);
    printf("stmrow = %d\n", stmrow);
```

7.13. 游标支持

OCIStmtFetch2 接口调用支持游标操作，可以分为可滚动和不可滚动两种。如果想要进行游标操作，在调用 OCIStmtExecute 接口时需要使用 OCI_STMT_SCROLLABLE_READONLY 模式。

不可滚动是指只能往后读取，并且只有下面参数有效：

- OCI_DEFAULT：具有与 OCI_FETCH_NEXT 相同的效果。
- OCI_FETCH_NEXT：从当前位置获取下一行。

可滚动是指前后都能读取参数，下列参数有效。

- OCI_FETCH_CURRENT ：获取当前行。
- OCI_FETCH_FIRST：获取结果集中的第一行。
- OCI_FETCH_LAST：获取结果集中的最后一行。
- OCI_FETCH_PRIOR：将结果集定位在结果集中当前行的前一行。您可以使用此模式从“前一行”中提取多行。
- OCI_FETCH_ABSOLUTE：使用绝对定位获取结果集中的行号。
- OCI_FETCH_RELATIVE：使用相对定位获取结果集中的行号。

```
//测试获取 SQL 生成的结果集中的行集
    printf("OCI_FETCH_ABSOLUTE 5\n");
    int ret = OCIStmtFetch2(stmthp, errhp, 1, OCI_FETCH_ABSOLUTE, 5, 0);
    test_print(stmthp, errhp, c1, c2, c3, ret);

    printf("\nOCI_FETCH_PRIOR 0\n");
```

```
ret = OCIStmtFetch2(stmthp, errhp, 1, OCI_FETCH_PRIOR, 0, 0);
test_print(stmthp, errhp, c1, c2, c3, ret);

printf("\nOCI_FETCH_RELATIVE 3\n");
ret = OCIStmtFetch2(stmthp, errhp, 1, OCI_FETCH_RELATIVE, 3, 0);
test_print(stmthp, errhp, c1, c2, c3, ret);

printf("\nOCI_FETCH_RELATIVE -1\n");
ret = OCIStmtFetch2(stmthp, errhp, 1, OCI_FETCH_RELATIVE, -1, 0);
test_print(stmthp, errhp, c1, c2, c3, ret);

printf("\nOCI_FETCH_NEXT  0\n");
ret = OCIStmtFetch2(stmthp, errhp, 1, OCI_FETCH_NEXT, 0, 0);
test_print(stmthp, errhp, c1, c2, c3, ret);

printf("\nOCI_DEFAULT  0\n");
ret = OCIStmtFetch2(stmthp, errhp, 1, OCI_DEFAULT, 0, 0);
test_print(stmthp, errhp, c1, c2, c3, ret);

printf("\nOCI_FETCH_CURRENT  0\n");
ret = OCIStmtFetch2(stmthp, errhp, 1, OCI_FETCH_CURRENT, 0, 0);
test_print(stmthp, errhp, c1, c2, c3, ret);

printf("\nOCI_FETCH_LAST 0\n");
```

```
    ret = OCIStmtFetch2(stmthp, errhp, 1, OCI_FETCH_LAST, 0, 0);
    test_print(stmthp, errhp, c1, c2, c3, ret);

    printf("\nOCI_FETCH_FIRST 0\n");
    ret = OCIStmtFetch2(stmthp, errhp, 1, OCI_FETCH_FIRST, 0, 0);
    test_print(stmthp, errhp, c1, c2, c3, ret);
```

7.14. 内存池

OCI 提供内存池开关，以便打开或者关闭 OCI 内存池的使用（这并不影响结果的输出）。

内存池开关，默认为关（在环境变量中 export_LIBOCI_MEMORYPOOL=ON 可开启）。

7.15. 连接池

连接池是多个会话使用一组（池）可重复使用的物理连接来平衡负载。

此功能的一个用途是在服务端从数据库服务器获得多个并发的数据请求时，应用程序可以在应用程序初始化期间在每个环境中创建一个池（或一组池）。

连接池本身通常配置有物理连接的共享池，转换为包含相同数量的专用服务器进程的后端服务器池。

在下面便是建立连接池及其使用。

```
//创建池子
    OCICPool *poolhp;
```

```
    if(OCIHandleAlloc((void *) envhp, (void **) &poolhp, OCI_HTYPE_CPOOL,
 (size_t) 0, (void **) 0))
    {
        report_error(errhp);
        printf("FAILED: OCIHandleAlloc()\n");
        return OCI_ERROR;
    }

    char    *poolName = NULL;
    int   poolNameLen = 0;

    if(OCIConnectionPoolCreate(envhp,errhp, poolhp,
                    (dvoid *)&poolName, &poolNameLen,
                    "Test8",strlen("Test8"),
                    (ub4) 2, (ub4) 4, (ub4) 1,
                    (text *)username,strlen(username),
                    (text *)password,strlen(password),
                    OCI_DEFAULT))
    {
        report_error(errhp);
        printf("FAILED: OCIConnectionPoolCreate()\n");
        return OCI_ERROR;
    }
//第一个用户
```

```
    static OCIServer *svrhp = (OCIServer *)0;
    OCIHandleAlloc(envhp, (dvoid **)&svrhp, OCI_HTYPE_SERVER, (size_t)0,
(dvoid **)0);
    OCIAttrSet(svchp, OCI_HTYPE_SVCCTX, (dvoid *)svrhp, (ub4)0, OCI_ATTR
_SERVER, errhp);

    if (OCIServerAttach(svrhp, errhp, (text *)poolName, (sb4)strlen((char *)
poolName), OCI_CPOOL))
    {
        report_error(errhp);
        printf("FAILED: OCIServerAttach()\n");
        return OCI_ERROR;
    }

    OCIHandleAlloc((void *)envhp, (void **)&usrhp, (ub4)OCI_HTYPE_SESSIO
N, (size_t)0, (void **)0);
    OCIAttrSet((void *)usrhp, (ub4)OCI_HTYPE_SESSION,(text *)username,strl
en(username), OCI_ATTR_USERNAME, errhp);
    OCIAttrSet((void *)usrhp, (ub4)OCI_HTYPE_SESSION, (text *)password,strl
en(password), OCI_ATTR_PASSWORD, errhp);

    if(OCISessionBegin(svchp, errhp, usrhp, OCI_CRED_RDBMS, OCI_DEFAUL
T))
    {
```

```
        printf("FAILED: OCISessionBegin()\n");
        report_error(errhp);
        return OCI_ERROR;
    }
    OCIAttrSet((void *)svchp, (ub4)OCI_HTYPE_SVCCTX, (void *)usrhp, (ub4)0,
OCI_ATTR_SESSION,errhp);
    //测试
    text *sqlstmt=(text *)"insert into t2(c1,c4) values(1,'111')";
    //测试准备一条 SQL
    if(OCIStmtPrepare(stmthp, errhp, sqlstmt, strlen((char *)sqlstmt),
    OCI_NTV_SYNTAX, 0))
    {
        printf("FAILED: OCIStmtPrepare()\n");
        report_error(errhp);
        return OCI_ERROR;
    }
    //测试 SQL 执行
    if(OCIStmtExecute(svchp, stmthp, errhp, 1, 0, 0, 0, 0))
    {
        report_error(errhp);
        printf("FAILED: OCIStmtExecute()\n");
        return OCI_ERROR;
    }
    if(OCITransCommit(svchp,errhp,0))
```

```
    {
        report_error(errhp);
        printf("FAILED: OCITransCommit()\n");
        return OCI_ERROR;
    }
    //测试退出
    if(OCISessionEnd(svchp,errhp,usrhp,OCI_DEFAULT)){
        printf("FAILED: OCISessionEnd()\n");
        report_error(errhp);
        return OCI_ERROR;
    }
    if (OCIServerDetach(svrhp, errhp, OCI_DEFAULT))
    {
        printf("FAILED: OCIServerDetach()\n");
        report_error(errhp);
        return OCI_ERROR;
    }
//销毁第一个池子
    if(OCIConnectionPoolDestroy(poolhp, errhp, OCI_DEFAULT))
    {
        printf("FAILED: OCIConnectionPoolDestroy()\n");
        report_error(errhp);
        return OCI_ERROR;
    }
```

```
if(OCIHandleFree((void *)poolhp, OCI_HTYPE_CPOOL))
  {
      printf("FAILED: OCIHandleFree()\n");
      report_error(errhp);
      return OCI_ERROR;
  }
```

8. ODBC 接口开发指南

8.1. 介绍

PanWeiDB V2.0-S3.0.0_Beta 数据库通过提供 ODBC 驱动为使用 C/C++语言的应用程序提供访问 PanWeiDB V2.0-S3.0.0_Beta 的接口。但同时也兼容 openGauss 的 ODBC 驱动，可以通过 openGauss 的 ODBC 驱动或者 PanWeiDB V2.0-S3.0.0_Beta 提供的 ODBC 驱动访问 PanWeiDB V2.0-S3.0.0_Beta。

8.2. openGauss ODBC 主要接口

PanWeiDB V2.0-S3.0.0_Beta ODBC 驱动由具有建立和管理与 PanWeiDB V2.0-S3.0.0_Beta 数据库连接、执行 SQL 语句和对结果集进行储存管理的若干功能类组成。

- SQLSetEnvAttr

设置控制环境各方面的属性，环境属性如下所示：

属性	说明
SQL_ATTR_ODBC_VERSION	指定 ODBC 版本。
SQL_CONNECTION_POOLING	连接池属性。
SQL_OUTPUT_NTS	指明驱动器返回字符串的形式。

示例：

```
SQLSetEnvAttr(V_OD_Env, SQL_ATTR_ODBC_VERSION, (void*)SQL_OV_ODBC3,
```

```
0);
```

❖ SQLAllocHandle

分配环境、连接、语句或描述符的句柄，它替代了 ODBC 2.x 函数 SQLAllocEnv、SQLAllocConnect 及 SQLAllocStmt。

示例：

```
V_OD_erg = SQLAllocHandle(SQL_HANDLE_DBC, V_OD_Env, &V_OD_hdbc);
```

❖ SQLSetConnectAttr

设置控制连接各方面的属性，下面列出一些连接属性和取值，详细信息请参考 https://msdn.microsoft.com/en-us/library/windows/desktop/ms714177(v=vs.85).aspx

连接属性	取值
SQL_ATTR_ACCESS_MODE	用于表明是否需要连接来支持会导致更新的 SQL 语句。 SQL_MODE_READ_ONLY SQL_MODE_READ_WRITE
SQL_ATTR_ASYNC_ENABLE	指定连接相关的语句中的函数调用是否异步执行。 SQL_ASYNC_ENABLE_OFF SQL_ASYNC_ENABLE_ON
SQL_ATTR_AUTOCOMMIT	自动提交，还是手动提交。 SQL_AUTOCOMMIT_OFF

	SQL_AUTOCOMMIT_ON
SQL_ATTR_CONNECTION_DEAD	连接的状态。 SQL_CD_TRUE（连接已丢失） SQL_CD_FALSE（连接仍活跃）
SQL_ATTR_LOGIN_TIMEOUT	保存秒数的 SQLUINTEGER 类型变量，传入空指针则关闭该功能。
SQL_ATTR_ODBC_CURSORS	用于驱动管理器使用 ODBC 游标。 SQL_CUR_USE_IF_NEEDED SQL_CUR_USE_ODBC SQL_CUR_USE_DRIVER

示例：

```
SQLSetConnectAttr(V_OD_hdbc, SQL_ATTR_AUTOCOMMIT, (SQLPOINTER)SQL_
AUTOCOMMIT_ON, 0);
```

- SQLConnect

在驱动程序和数据源之间建立连接。连接上数据源之后，可以通过连接句柄访问到所有有关连接数据源的信息，包括程序运行状态、事务处理状态和错误信息。

示例：

```
V_OD_erg = SQLConnect(V_OD_hdbc, (SQLCHAR*) "Test", SQL_NTS,
                          (SQLCHAR*) "yuntu", SQL_NTS,  (SQLCHAR*) "Tte
```

```
st@123", SQL_NTS);
      if ((V_OD_erg != SQL_SUCCESS) && (V_OD_erg != SQL_SUCCESS_WITH
_INFO))
      {
            printf("Error SQLConnect %d\n",V_OD_erg);
            SQLFreeHandle(SQL_HANDLE_ENV, V_OD_Env);
            exit(0);
  }
```

- SQLSetStmtAttr

  设置相关语句的属性。

  示例：

```
SQLSetStmtAttr(V_OD_hstmt,SQL_ATTR_QUERY_TIMEOUT,(SQLPOINTER *)3,0);
```

- SQLExecDirect

  使用参数的当前值，执行一条准备好的语句。对于一次只执行一条 SQL 语句，SQLExecDirect 是最快的执行方式。

  示例：

```
SQLExecDirect(V_OD_hstmt,"drop table IF EXISTS test_stu",SQL_NTS);
```

- SQLPrepare

  准备一个将要进行的 SQL 语句。

  示例：

```
SQLPrepare(V_OD_hstmt,"insert into test_stu values(?)",SQL_NTS);
```

- SQLBindParameter

将一条 SQL 语句中的一个参数标志和一个缓冲区绑定起来。

示例：

```
SQLBindParameter(V_OD_hstmt,1,SQL_PARAM_INPUT,SQL_C_SLONG,SQL_INTE

GER,0,0,

                              &value,0,NULL);
```

- SQLExecute

  如果语句中存在参数标记的话，SQLExecute 函数使用参数标记参数的当前值，执行一条准备好的 SQL 语句。

  示例：

```
SQLExecute(V_OD_hstmt);
```

- SQLColAttribute

  返回结果集中一列的描述符信息。

  示例：

```
SQLColAttribute(V_OD_hstmt,1,SQL_DESC_NAME,desc_name,100,NULL,NULL);



 printf("SQLColAtrribute  %s\n",desc_name);
```

- SQLFetch

  从结果集中取下一个行集的数据，并返回所有被绑定列的数据。

  示例：

```
ret = SQLFetch(V_OD_hstmt);
```

- SQLGetData

  SQLGetData 返回结果集中某一列的数据。可以多次调用它来部分地检索不定长度的数据。

  示例：

```
ret = SQLGetData(V_OD_hstmt, 1, SQL_C_CHAR, buf, sizeof(buf), &ind);
```

- SQLFreeHandle

  释放与指定环境、连接、语句或描述符相关联的资源，它替代了 ODBC 2.x 函数 SQLFreeEnv、SQLFreeConnect 及 SQLFreeStmt。

  示例：

```
SQLFreeHandle(SQL_HANDLE_STMT,V_OD_hstmt);
```

- SQLDisconnect

  关闭一个与特定连接句柄相关的连接。

  示例：

```
SQLDisconnect(V_OD_hdbc);
```

8.3. 设置 ODBC 驱动

安装 ODBC 驱动前需要先装好 PanWeiDB V2.0-S3.0.0_Beta Libpq 和 unixODBC,并配置环境变量。

8.3.1. 安装 PanWeiDB V2.0-S3.0.0_Beta ODBC

8.3.1.1. 通过 PanWeiDB V2.0-S3.0.0_Beta 安装包安装

安装过程可参考 PanWeiDB V2.0-S3.0.0_Beta 安装指南中的数据库安装步骤。新建/usr/local/libpq 目录，将数据库安装目录中的 include 和 lib 目录拷贝到/usr/local/libpq 目录下，配置上述环境变量。

8.3.2. 安装 unixODBC

要使用驱动， 必须先安装 unixODBC。

离线安装：



unixODBC-2.3.9.tar.gz

```
tar zxvf unixODBC-2.3.9.tar.gz

cd unixODBC-2.3.9


./configure --enable-gui=no #如果要在鲲鹏服务器上编译，请追加一个 confi
gure 参数： --build=aarch64-unknown-linux-gnu

make

#安装可能需要 root 权限

make install
```

8.3.3. 安装 ODBC

ODBC 安装可以选择安装 opengGauss 提供的安装包或者由 PanWeiDB V2.0-S3.0.0_Beta 提供的安装包。

8.3.3.1. 安装包安装 openGauss-2.0.0-ODBC

- 下载对应平台的 openGauss-2.0.0-ODBC（目前仅支持 centos_x86_64、openeuler_aarch64、openeuler_x84_64）

地址：https://opengauss.org/zh/download.html

如图：

openGauss Connectors

	centos_x86_64	openeuler_aarch64	openeuler_x86_64
JDBC_2.0.0	下载 SHA256	下载 SHA256	下载 SHA256
ODBC_2.0.0	下载 SHA256	下载 SHA256	下载 SHA256
libpq_2.0.1	下载 SHA256	下载 SHA256	下载 SHA256

- 替换客户端 openGauss 驱动程序

  解压 openGauss-2.0.0-ODBC.tar.gz，将解压后 lib 目录下的库拷贝到“/usr/local/lib”目录下。

8.3.3.2. 安装包安装 PanWeiDB V2.0-S3.0.0_Beta ODBC

- 解压由 PanWeiDB V2.0-S3.0.0_Beta 提供的 ODBC 驱动安装包。
- 将解压后的 lib 目录以及 odbc/lib 目录下的库拷贝到“/usr/local/lib”目录下。

8.3.3.3. 手动编译安装

- 下载 ODBC 驱动源码

  地址如下：https://gitee.com/opengauss/openGauss-connector-odbc

- 编译

  首先修改 configure 文件如下（红色框为增加内容）：

```
4456 if test "$with_libpq" != yes; then
4457     if test -d "$with_libpq"; then
4458         PATH="$PATH:$with_libpq/bin"
4459         CPPFLAGS="$CPPFLAGS -I$with_libpq/include -I$with_libpq/include/postgresql/internal -I$with_libpq/include/postgresql/server"
4460         LDFLAGS="$LDFLAGS -L$with_libpq/lib"
4461     else
4462         if test -x "$with_libpq"; then
4463             PG_CONFIG=$with_libpq
4464         else
4465             as_fn_error $? "specified pg_config not found" "$LINENO" 5
4466         fi
```

执行如下命令进行编译：

```
sh ./build.sh -bd $GAUSSHOME
```

将 output 下的 GaussDB-Kernel-XXX-0dbc.tar.gz 解压，然后将 output /odbc/lib 目录下的库拷贝到“/usr/local/lib” 目录下。

8.3.3.4. 配置数据源

1. 配置 ODBC 驱动文件。 在“/usr/local/etc/odbcinst.ini” 文件中添加以下内容。

```
vim /usr/local/etc/odbcinst.ini
```

配置	说明
[panweidbMPP]	# 驱动器名称， 对应数据源 DSN 中的驱动名
Driver64=/usr/local/lib/psqlodbcw.so	# 驱动动态库的路径
setup=/usr/local/lib/psqlodbcw.so	# 驱动安装路径， 与 Driver64 中动态库的路径一致

2. 数据源文件。 该步骤并不是必须的， 取决于获取连接的方式，请参见 8.4 使用 ODBC 连接数据库。 在“/usr/local/etc/odbc.ini ” 文件中添加以下内容。

配置	说明
[Test]	# 数据源的名称。

Description=Test Driver=panweidbMPP	# 数据源说明 # 驱动名， 对应 odbcinst.ini 中的 DriverName
Servername=172.16.12.57	# 服务器的 IP 地址
Database=postgres Username=yuntu	# 要连接的数据库的名称 # 数据库用户名称
Password= Ttest@123	# 数据库用户密码
Port=5432	# 服务器的端口号
Sslmode=allow	# 开启 SSL 模式： disable 否，allow 可能 ，prefer 可能，requrie 是，verify-ca 是， verify-full 是

8.3.3.5. 配置环境变量

```
vim ~/.bashrc
```

在配置文件中追加以下内容。

```
export LD_LIBRARY_PATH=/usr/local/lib/:$LD_LIBRARY_PATH

export ODBCSYSINI=/usr/local/etc

export ODBCINI=/usr/local/etc/odbc.ini
```

执行如下命令使配置生效。

```
source ~/.bashrc
```

8.3.3.6. 测试数据源配置

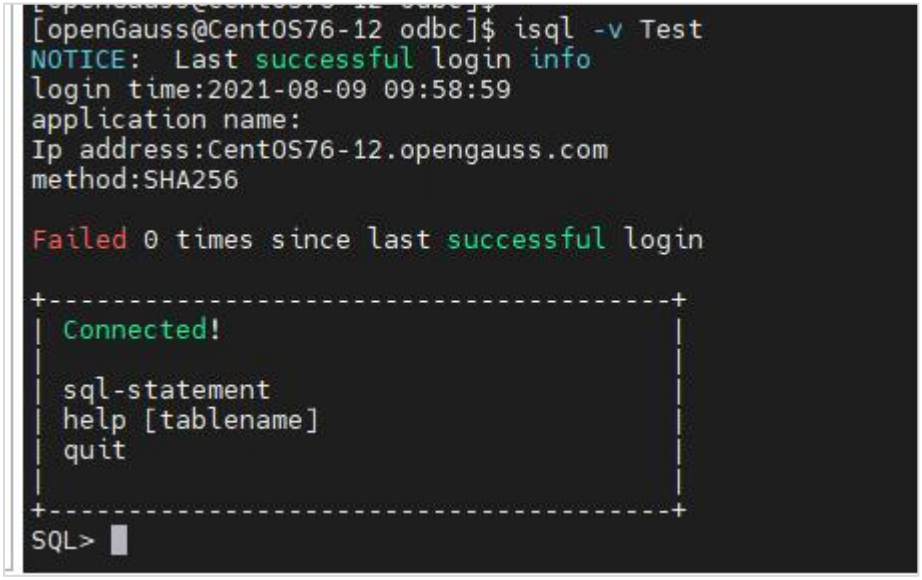
上述配置中该数据源的 DSN 为 Test,可以通过 isql 来验证配置是否正确。命令如下。

```
isql -v Test
```

如果数据源配置中未指定用户名和密码，则用 isql 测试连接时需要指定用户密码：

```
isql -v Test yuntu Ttest@123
```

如果显示以下信息，表示配置正确，连接成功。



如果显示查找不到 psqlodbcw.so 动态库，如图：



在确认动态库确实存在之后，该错误可能是由于当前机器上缺少 odbc 驱动

库的依赖库造成的，可以使用 ldd 命令观察缺失的动态库后安装对应的依赖库。

8.4. 使用 ODBC 连接数据库

8.4.1. 连接数据库

8.4.1.1. 获取 ODBC 连接

获取连接有两种方法。

❖ 通过调用 SQLConnect()函数连接到数据库：

```
SQLHENV         V_OD_Env;   // 环境句柄

SQLHDBC         V_OD_hdbc;   // 连接句柄

SQLRETURN  retcode;   // 返回值

SQLINTEGER     V_OD_erg; //调用 接口返回值

 /* 1.分配环境句柄 */

V_OD_erg = SQLAllocHandle(SQL_HANDLE_ENV,SQL_NULL_HANDLE,&V_OD_E

nv);

if ((V_OD_erg != SQL_SUCCESS) && (V_OD_erg != SQL_SUCCESS_WITH_INFO))



{

printf("Error AllocHandle\n");

exit(0);

}

 /* 2.设置 ODBC 环境属性（版本信息）*/
```

```
SQLSetEnvAttr(V_OD_Env, SQL_ATTR_ODBC_VERSION, (void*)SQL_OV_ODBC3,
 0);
/* 3.分配连接句柄 */
V_OD_erg = SQLAllocHandle(SQL_HANDLE_DBC, V_OD_Env, &V_OD_hdbc);

if ((V_OD_erg != SQL_SUCCESS) && (V_OD_erg != SQL_SUCCESS_WITH_INFO))

{
SQLFreeHandle(SQL_HANDLE_ENV, V_OD_Env);
exit(0);
}
 /* 4.设置连接属性 */
SQLSetConnectAttr(V_OD_hdbc, SQL_ATTR_AUTOCOMMIT,
                  (SQLPOINTER)SQL_AUTOCOMMIT_ON, 0);
/* 5. 连接数据源，这里的“userName”与“password”分别表示连接数据库的用户
名和用户密码，请根据实际情况修改。
如果 odbc.ini 文件中已经配置了用户名密码，那么这里可以留空（""）。 但是不建议
这么做，因为一旦 odbc.ini 权限管理不善，将导致数据库用户密码泄露. */
V_OD_erg = SQLConnect(V_OD_hdbc, (SQLCHAR*) "Test", SQL_NTS,  (SQLC
HAR*) "panweidb", SQL_NTS,  (SQLCHAR*) "Aa@123456", SQL_NTS);
 if ((V_OD_erg != SQL_SUCCESS) && (V_OD_erg != SQL_SUCCESS_WITH_INF
O))
{
```

```
printf("Error SQLConnect %d\n",V_OD_erg);
SQLFreeHandle(SQL_HANDLE_ENV, V_OD_Env);
exit(0);
}
 printf("Connected !\n");
```

❖ 通过调用 SQLDriverConnect()函数连接到数据库：

```
SQLHENV         V_OD_Env;   // 环境句柄
SQLHDBC         V_OD_hdbc;   // 连接句柄
SQLRETURN retcode;   // 返回值
SQLINTEGER     V_OD_erg; //调用 接口返回值
 /* 1.分配环境句柄 */
V_OD_erg = SQLAllocHandle(SQL_HANDLE_ENV,SQL_NULL_HANDLE,&V_OD_E
nv);
if ((V_OD_erg != SQL_SUCCESS) && (V_OD_erg != SQL_SUCCESS_WITH_INFO))


{
printf("Error AllocHandle\n");
exit(0);
}
 /* 2.设置 ODBC 环境属性（版本信息）*/
SQLSetEnvAttr(V_OD_Env, SQL_ATTR_ODBC_VERSION, (void*)SQL_OV_ODBC3,
 0);
/* 3.分配连接句柄 */
```

```
V_OD_erg = SQLAllocHandle(SQL_HANDLE_DBC, V_OD_Env, &V_OD_hdbc);


if ((V_OD_erg != SQL_SUCCESS) && (V_OD_erg != SQL_SUCCESS_WITH_INFO))


{
SQLFreeHandle(SQL_HANDLE_ENV, V_OD_Env);
exit(0);
}
 /* 4.设置连接属性 */
SQLSetConnectAttr(V_OD_hdbc, SQL_ATTR_AUTOCOMMIT,
                  (SQLPOINTER) SQL_AUTOCOMMIT_ON, 0);
/* 5. 连接数据源 */
V_OD_erg = SQLDriverConnect(V_OD_hdbc, NULL, (SQLCHAR*)"Driver=Gauss
MPP;Servername=127.0.0.1;Port=5432;Database=postgres;UserName=panwe
idb;Password=Aa@123456", SQL_NTS, str, sizeof(str), &strl,SQL_DRIVER_COM
PLETE);
 if ((V_OD_erg != SQL_SUCCESS) && (V_OD_erg != SQL_SUCCESS_WITH_INF
O))
{
printf("Error SQLConnect %d\n",V_OD_erg);
SQLFreeHandle(SQL_HANDLE_ENV, V_OD_Env);
exit(0);
}
```

```
printf("Connected !\n");
```

8.4.1.2. 关闭 ODBC 连接

关闭连接时调用 SQLDisconnect()函数关闭数据库连接：

```
rc = SQLDisconnect(V_OD_hdbc);
if (!SQL_SUCCEEDED(rc))
{
  printf("SQLDisconnect failed");
  fflush(stdout);
  exit(1);
}
rc = SQLFreeHandle(SQL_HANDLE_DBC,V_OD_hdbc);
if (!SQL_SUCCEEDED(rc))
{
  printf("SQLFreeHandle failed");
  fflush(stdout);
  exit(1);
}
V_OD_hdbc = NULL;
rc = SQLFreeHandle(SQL_HANDLE_ENV, V_OD_Env);
if (!SQL_SUCCEEDED(rc))
{
  printf("SQLFreeHandle failed");
```

```
    fflush(stdout);
    exit(1);
}
V_OD_Env= NULL;
```

8.4.2. 连接参数

名称	缩写	类型	定义
Description		string	数据源说明
Servicename		string	数据源服务主机名或 IP
Port		int	数据库端口
Username		string	数据库用户
Password		string	数据库用户密码
UseDeclareFetch	B6	bool	是否使用 Declare 和 Fetch
Fetch	A7	int	批量读取的最大行数
Socket	A8	int	Socket buffer size
ReadOnly	A0	bool	数据库是否只读
MaxVarcharSize	B0	int	Varchar 的最大长度
KeepaliveTime	D1	int	TCP KEEPALIVE 设置：空闲时间
KeepaliveInterval	D2	int	TCP KEEPALIVE 设置：间隔

连接参数可以在连接串中指定：

```
ret = SQLDriverConnect(conn, NULL, (SQLCHAR*)"Driver=PostgreSQL;Servern
ame=192.168.114.28;Port=5411;Database=atlasdb;UserName=atlasdb;Passw
ord=atlasdb;ReadOnly=Yes;Fetch=200", SQL_NTS, str, sizeof(str), &strl, SQL_
DRIVER_COMPLETE);
```

8.5. 执行 SQL 语句并处理结果

向数据库发出执行 SQL 的请求时，需要使用语句句柄，并调用 SQLExecDirect 或 SQLExecute 发出查询，服务端执行完成后查询的结果将保存在语句句柄中。

8.5.1. 执行查询

8.5.1.1. 直接执行

调用 SQLAllocHandle()函数获取句柄，调用 SQLExecDirect()函数执行查询。

```
HSTMT V_OD_hstmt = SQL_NULL_HSTMT; //语句句柄
ret = SQLAllocHandle(SQL_HANDLE_STMT, V_OD_hdbc, &V_OD_hstmt);
if (!SQL_SUCCEEDED(ret))
{
  printf("failed to allocate stmt handle");
  return;
}
```

```
/*执行语句*/
ret = SQLExecDirect(V_OD_hstmt, (SQLCHAR *) "select * from test_stu", SQL
_NTS);
/*获取结果*/
ret = SQLFetch(V_OD_hstmt);
if (ret == SQL_NO_DATA)
  return;
if (ret == SQL_SUCCESS)
{
char buf[40];
SQLLEN ind;
/*获取列数据*/
  ret = SQLGetData(V_OD_hstmt, 1, SQL_C_CHAR, buf, sizeof(buf), &ind);
}
```

8.5.1.2. 预编译方式执行

调用 SQLAllocHandle()函数获取句柄，使用 SQLPrepare 预编译语句，使用 SQLBindParameter 绑定参数，最后调用 SQLExecute 执行查询

```
/*初始化句柄*/
ret = SQLAllocHandle(SQL_HANDLE_STMT, V_OD_hdbc, &V_OD_hstmt);
if (!SQL_SUCCEEDED(ret))
{
  printf("failed to allocate stmt handle");
```

```
  exit(1);
}
/*预编译 SQL*/
ret = SQLPrepare(V_OD_hstmt, (SQLCHAR *) "SELECT * FROM test_stu WHER
E id = ?", SQL_NTS);
CHECK_STMT_RESULT(ret, "SQLPrepare failed", V_OD_hstmt);
SQLLEN cbParam1 = SQL_NTS;
/*绑定参数*/
ret = SQLBindParameter(V_OD_hstmt, 1, SQL_PARAM_INPUT,
                SQL_C_SLONG,   /* value type */
               SQL_INTEGER,     /* param type */
                20,        /* column size */
                0,        /* dec digits */
                "1",     /* param value ptr */
                0,        /* buffer len */
                &cbParam1      /* StrLen_or_IndPtr */);
CHECK_STMT_RESULT(ret, "SQLBindParameter failed", V_OD_hstmt);
/*执行*/
ret = SQLExecute(V_OD_hstmt);
CHECK_STMT_RESULT(ret, "SQLExecute failed", V_OD_hstmt);
/*获取结果*/
/*获取结果*/
ret = SQLFetch(V_OD_hstmt);
```

```
if (ret == SQL_NO_DATA)
  return;
if (ret == SQL_SUCCESS)
{
char buf[40];
SQLLEN ind;
/*获取列数据*/
  ret = SQLGetData(V_OD_hstmt, 1, SQL_C_CHAR, buf, sizeof(buf), &ind);
}
```

8.5.2. 使用游标获取结果

默认情况下，驱动程序将一次性获取查询的所有结果，当某个查询的结果集很庞大时，一次获取所有结果很不合理，因此 ODBC 驱动程序提供了一种基于数据库游标的结果集处理方法，只获取少量的行缓存在客户端。通过重新定位游标来获取下一批数据。

游标不能在所有情况下使用。有许多限制会使驱动程序一次性获取整个结果集：

- 连接不能使用自动提交模式 ，因为后端会在事务结束时关闭游标，因此在自动提交模式下，事务结束后，在从游标获取数据之前，游标已经被关闭。
- UseDeclareFetch 参数设置为 true，默认缓存 100 行数据，为 false 时将不会使用游标，直接获取整个结果集。
- 给定的查询语句必须是单条语句，不能是用分号连接在一起的多个语句。

❖ 设置 fetch size，且 fetch size 不能为 0.

示例：

```
int rc;
int i;
/*关闭自动提交*/
rc = SQLSetConnectAttr(V_OD_hdbc, SQL_AUTOCOMMIT, (SQLPOINTER) SQL_
AUTOCOMMIT_OFF, 0);
  CHECK_CONN_RESULT(rc, "SQL_AUTOCOMMIT off failed", V_OD_hdbc);
/*执行 SQL*/
rc = SQLExecDirect(V_OD_hstmt, (SQLCHAR *) "SELECT 'foo' || g FROM gene
rate_series(1, 3210) g", SQL_NTS);
CHECK_STMT_RESULT(rc, "SQLExecDirect failed", V_OD_hstmt);
/*获取结果*/
for (;; i++)
{
char buf[40];
  SQLLEN ind;
  rc = SQLFetch(V_OD_hstmt);
  if (rc == SQL_NO_DATA)
    break;
  if (rc != SQL_SUCCESS)
  {
    char sqlstate[32] = "";
```

```
    SQLINTEGER nativeerror;
    SQLSMALLINT textlen;
    SQLGetDiagRec(SQL_HANDLE_STMT, V_OD_hstmt, 1, sqlstate, &nativeerr
or,
                NULL, 0, &textlen);
                 /*在读取游标的过程中，如果调用用 SQLEndTran 提交或回滚了事务，
游标将被关闭*/
    if (strcmp(sqlstate, "HY010") == 0)
    {
      printf("SQLFetch failed with HY010 (which probably means that the
cursor was closed at commit/rollback)");
      break;
    }
    else
      CHECK_STMT_RESULT(rc, "SQLGetDiagRec failed", V_OD_hstmt);
  }

  rc = SQLGetData(V_OD_hstmt, 1, SQL_C_CHAR, buf, sizeof(buf), &ind);
  CHECK_STMT_RESULT(rc, "SQLGetData failed", V_OD_hstmt);
}
```

8.5.3. 执行更新

要更改数据(执行一个 insert，update 或者 delete)，可以使用 SQLExecDirect 函数执行完整的 SQL，也可以使用预编译的方式绑定参数后再执行。下面的示例将以预编译的方式执行。

8.5.3.1. 删除

```
rc = SQLAllocHandle(SQL_HANDLE_STMT, V_OD_hdbc, &V_OD_hstmt);
rc = SQLPrepare(V_OD_hstmt, (SQLCHAR *) "delete from test_stu where id=
 ? ", SQL_NTS);
cbParam1 = SQL_NTS;
rc = SQLBindParameter(V_OD_hstmt, 1, SQL_PARAM_INPUT,
SQL_C_SLONG,  /* value type */
SQL_INTEGER,  /* param type */
20,     /* column size */
0,     /* dec digits */
“1”,     /* param value ptr */
0,     /* buffer len */
&cbParam1  /* StrLen_or_IndPtr */);
CHECK_STMT_RESULT(rc, "SQLBindParameter failed", V_OD_hstmt);
rc = SQLExecute(V_OD_hstmt);
CHECK_STMT_RESULT(rc, "SQLExecute failed", V_OD_hstmt);
```

8.5.3.2. 批处理删除

```
SQLINTEGER *ids = NULL;

SQLLEN *bufflenIds = NULL;

SQLUSMALLINT

/* batchcount 为批处理数 */

for (i = 0; i < batchcount; i++) {

ids[i] = i;

bufflenIds[i]=sizeof(ids[i]);

}


rc = SQLAllocHandle(SQL_HANDLE_STMT, V_OD_hdbc, &hstmt);

rc = SQLPrepare(hstmt, (SQLCHAR *) "delete from test_stu where id= ? ",

SQL_NTS);

rc = SQLSetStmtAttr(hstmt, SQL_ATTR_PARAMSET_SIZE, (SQLPOINTER batchc

ount), sizeof(batchcount));

rc = SQLBindParameter(hstmt, 1, SQL_PARAM_INPUT,

SQL_C_SLONG,  /* value type */

SQL_INTEGER,  /* param type */

sizeof(ids[0]),     /* column size */

0,     /* dec digits */

&(ids[0]),     /* param value ptr */

0,     /* buffer len */
```

```
bufflenIds   /* StrLen_or_IndPtr */);
CHECK_STMT_RESULT(rc, "SQLBindParameter failed", hstmt);
rc = SQLExecute(hstmt);
CHECK_STMT_RESULT(rc, "SQLExecute failed", hstmt);
```

8.5.3.3. 更新

```
rc = SQLAllocHandle(SQL_HANDLE_STMT, V_OD_hdbc, &V_OD_hstmt);
rc = SQLPrepare(V_OD_hstmt, (SQLCHAR *) "update stu_id set name = 'Lis
a' where id = ? ", SQL_NTS);
cbParam1 = SQL_NTS;
rc = SQLBindParameter(V_OD_hstmt, 1, SQL_PARAM_INPUT,
          SQL_C_SLONG,   /* value type */
          SQL_INTEGER,   /* param type */
          20,     /* column size */
                0,     /* dec digits */
           "100",     /* param value ptr */
          0,     /* buffer len */
          &cbParam1   /* StrLen_or_IndPtr */);
CHECK_STMT_RESULT(rc, "SQLBindParameter failed", V_OD_hstmt);
rc = SQLExecute(V_OD_hstmt);
CHECK_STMT_RESULT(rc, "SQLExecute failed", V_OD_hstmt);
```

8.5.3.4. 批处理更新

```
SQLINTEGER *ids = NULL;

SQLLEN *bufflenIds = NULL;

/* batchcount 为批处理数 */

for (i = 0; i < batchcount; i++) {

ids[i] = i;

bufflenIds[i]=sizeof(ids[i]);

}


rc =  SQLAllocHandle(SQL_HANDLE_STMT, V_OD_hdbc, &hstmt);

rc = SQLPrepare(hstmt, (SQLCHAR *) "INSERT INTO test_stu VALUES (2, ?) ",

 SQL_NTS);

cbParam1 = SQL_NTS;

rc = SQLSetStmtAttr(hstmt, SQL_ATTR_PARAMSET_SIZE, (SQLPOINTER batchc

ount), sizeof(batchcount));

rc = SQLBindParameter(hstmt, 1, SQL_PARAM_INPUT,

          SQL_C_SLONG,   /* value type */

          SQL_INTEGER,   /* param type */

          sizeof(ids[0]),     /* column size */

          0,     /* dec digits */

          &(ids[0]),     /* param value ptr */

          0,     /* buffer len */
```

```
            bufflenIds   /* StrLen_or_IndPtr */);
CHECK_STMT_RESULT(rc, "SQLBindParameter failed", hstmt);
rc = SQLExecute(hstmt);
CHECK_STMT_RESULT(rc, "SQLExecute failed", hstmt);
```

8.5.3.5. 插入

```
rc = SQLAllocHandle(SQL_HANDLE_STMT, V_OD_hdbc, &V_OD_hstmt);
rc = SQLPrepare(V_OD_hstmt, (SQLCHAR *) "INSERT INTO test_stu VALUES
(2, ?) ", SQL_NTS);
cbParam1 = SQL_NTS;
rc = SQLBindParameter(V_OD_hstmt, 1, SQL_PARAM_INPUT,
            SQL_C_CHAR,   /* value type */
            SQL_CHAR,   /* param type */
            20,     /* column size */
            0,     /* dec digits */
            “Jennifer”,     /* param value ptr */
            0,     /* buffer len */
            &cbParam1   /* StrLen_or_IndPtr */);
CHECK_STMT_RESULT(rc, "SQLBindParameter failed", V_OD_hstmt);
rc = SQLExecute(V_OD_hstmt);
CHECK_STMT_RESULT(rc, "SQLExecute failed", V_OD_hstmt);
```

8.5.3.6. 批处理插入

```
SQLINTEGER *ids = NULL;
SQLLEN *bufflenIds = NULL;
/* batchcount 为批处理数 */
for (i = 0; i < batchcount; i++) {
ids[i] = i;
bufflenIds[i]=sizeof(ids[i]);
}

rc =  SQLAllocHandle(SQL_HANDLE_STMT, V_OD_hdbc, &V_OD_hstmt);
rc = SQLPrepare(V_OD_hstmt, (SQLCHAR *) "INSERT INTO test_stu VALUES
(2, ?) ", SQL_NTS);
cbParam1 = SQL_NTS;
rc = SQLBindParameter(V_OD_hstmt, 1, SQL_PARAM_INPUT,
          SQL_C_SLONG,   /* value type */
          SQL_INTEGER,   /* param type */
          sizeof(ids[0]),     /* column size */
          0,     /* dec digits */
          &(ids[0]),     /* param value ptr */
          0,     /* buffer len */
          bufflenIds   /* StrLen_or_IndPtr */);
CHECK_STMT_RESULT(rc, "SQLBindParameter failed", V_OD_hstmt);
rc = SQLExecute(V_OD_hstmt);
CHECK_STMT_RESULT(rc, "SQLExecute failed", V_OD_hstmt);
```

8.5.4. 创建或修改数据库对象

要创建、更改或者删除一个类似表或者视图这样的数据库对象， 可以调用 SQLExecDirect() 方法.

例如创建表：

```
HSTMT V_OD_hstmt = SQL_NULL_HSTMT; //语句句柄

ret = SQLAllocHandle(SQL_HANDLE_STMT, V_OD_hdbc, &V_OD_hstmt);

if (!SQL_SUCCEEDED(ret))

{

  printf("failed to allocate stmt handle");

  return;

}

ret = SQLExecDirect(V_OD_hstmt, (SQLCHAR *) "create table test(c1 int,c2 i

nt)", SQL_NTS);
```

8.6. 调用存储过程

采用预编译 SQL 的方式执行存储过程：

- 使用 SQLPrepare 准备待执行的 SQL，其中函数的调用形式为{call funcname(?,...)}
- 使用 SQLBindParameter 绑定待调用的函数的参数
- 使用 SQLExecute 执行 SQL

- 使用 SQLFetch 和 SQLGetData 获取执行结果

例子：

```
/*获取语句句柄*/
rc = SQLAllocHandle(SQL_HANDLE_STMT, V_OD_hdbc, &V_OD_hstmt);
/*预编译 SQL，调用函数时使用{call funcname(?,...)} */
SQLPrepare(V_OD_hstmt, (SQLCHAR *) "{ call length(?) }", SQL_NTS);
cbParams[paramno] = SQL_NTS;
/*绑定参数 */
SQLBindParameter(V_OD_hstmt, 1, SQL_PARAM_INPUT,
        SQL_C_CHAR, /* value type */
        SQL_CHAR,        /* param type */
          20,           /* column size */
          0,            /* dec digits */
          (SQLCHAR*)"testvalue",       /* param value ptr */
          0,            /* buffer len */
          NULL          /* StrLen_or_IndPtr */);
/*执行*/
SQLExecute(V_OD_hstmt);
/* 获取执行结果 */
rc =SQLFetch(V_OD_hstmt);
/* 通过 SQLGetData 获取并返回数据。*/
while(rc != SQL_NO_DATA)
```

```
{
        char  buf[40];
        SQLLEN  ind;
        SQLGetData(V_OD_hstmt,  1,  SQL_C_CHAR,  buf,  sizeof(buf),  &ind);
        printf("SQLGetData  ----ID  =  %s\n",buf);
        rc  =  SQLFetch(V_OD_hstmt);
 };
```

8.7. 处理数据类型

8.7.1. 二进制类型

panweidb 提供两种不同的方法存储二进制数据。 二进制数据可以使用二进制数据类型 BYTEA 存储在表中，或者使用大对象特性以一种特殊的格式将二进制数据存储在一个独立的表中， 然后通过在表中保存一个类型为 OID 的值来引用该表。

8.7.1.1. BYTEA

使用 BYTEA 类型存储二进制数据，例如：

```
CREATE  TABLE  lo_test_tab  (id  int,  img  bytea);
```

8.7.2. 插入记录

```
/*二进制数据*/
char param1[20] = { 1, 2, 3, 4, 5, 6, 7, 8 };
SQLLEN cbParam1;
/*分配语句句柄*/
rc = SQLAllocHandle(SQL_HANDLE_STMT, V_OD_hdbc, &V_OD_hstmt);
if (!SQL_SUCCEEDED(rc))
{
  printf("failed to allocate stmt handle");
  exit(1);
}
/*预编译 SQL*/
rc = SQLPrepare(V_OD_hstmt, (SQLCHAR *) "INSERT INTO lo_test_tab VALUE
S (1, ?)", SQL_NTS);
CHECK_STMT_RESULT(rc, "SQLPrepare failed", V_OD_hstmt);
/*绑定参数*/
cbParam1 = 8;
rc = SQLBindParameter(V_OD_hstmt, 1, SQL_PARAM_INPUT,
          SQL_C_BINARY,  /* value type */
          SQL_LONGVARBINARY,  /* param type */
          200,       /* column size */
          0,       /* dec digits */
```

```
            param1,     /* param value ptr */
            0,          /* buffer len */
            &cbParam1     /* StrLen_or_IndPtr */);
CHECK_STMT_RESULT(rc, "SQLBindParameter failed", V_OD_hstmt);


/* 执行*/
rc = SQLExecute(V_OD_hstmt);
CHECK_STMT_RESULT(rc, "SQLExecute failed", V_OD_hstmt);
```

在这里，SQLBindParameter 时指定参数类型为 SQL_LONGVARBINARY，再将参数值的指针(char *类型)传入函数，即可将二进制数据写入。

8.7.3. 读取记录

```
char buf[100];
SQLLEN ind;
/*分配句柄*/
rc = SQLAllocHandle(SQL_HANDLE_STMT, V_OD_hdbc, &V_OD_hstmt);
/*执行查询*/
SQLExecDirect(V_OD_hstmt, (SQLCHAR *) "SELECT id, img FROM lo_test_tab
 WHERE id = 1", SQL_NTS);
/*获取结果*/
rc = SQLFetch(V_OD_hstmt);
```

```
CHECK_STMT_RESULT(rc, "SQLFetch failed", V_OD_hstmt);
/*读取数据*/
rc = SQLGetData(V_OD_hstmt, 2, SQL_C_BINARY, buf, sizeof(buf), &ind);
CHECK_STMT_RESULT(rc, "SQLGetData failed", V_OD_hstmt);
```

8.7.3.1. 大对象

使用大对象存储二进制数据，例如：

```
create table image(name text, img oid);
```

8.7.4. 插入记录

```
/* 分配语句句柄 */
SQLAllocHandle(SQL_HANDLE_STMT, V_OD_hdbc, &V_OD_hstmt);
/* 使用服务端函数 lo_import 创建一个大对象，将其返回的大对象 OID 存入 image.
img 字段中； /tmp/bfile.from 是预先准备好的服务端文件；执行用户需要具有 sys
admin 权限。*/
SQLExecDirect(V_OD_hstmt,(SQLCHAR *)"insert into image values('test imag
e', lo_import('/tmp/bfile.from'))",SQL_NTS);
```

注意：这里使用的 lo_import 是服务端函数，不是客户端的 libpq 接口。

8.7.5. 读取记录

```
/* 将大对象导出到服务端文件/tmp/bfile.to 中。*/
```

```
SQLExecDirect(V_OD_hstmt,(SQLCHAR *)"select lo_export(image.img, '/tmp/bfile.to') from image where name = 'test image'",SQL_NTS);
```

【注意】
这里使用的 lo_export 是服务端函数，不是客户端的 libpq 接口。

8.7.6. 字符类型

panweidb 中 TEXT 类型与 VARCHAR 类型都是可变长的字符类型，区别在于 VARCHAR 类型通过 VARCHAR(n)中的 n 来限制最大长度，而 TEXT 类型没有。TEXT 类型与 VARCHAR 类型几乎没有性能差别，TEXT 类型最多可存储 1G 数据 。

使用 TEXT 类型存储数据时，可用 getString()，getCharacterStream(), setString(), setCharacterStream()来进行读写。

例如：

```
CREATE TABLE images (id int, msg text);
```

8.7.6.1. 插入记录

```
str=" text value for test" ;
SQLLEN cbParam1 = SQL_NTS;
/*分配语句句柄*/
rc = SQLAllocHandle(SQL_HANDLE_STMT, V_OD_hdbc, &V_OD_hstmt);
if (!SQL_SUCCEEDED(rc))
{
```

```
  printf("failed to allocate stmt handle");
  exit(1);
}
/*预编译 SQL*/
rc = SQLPrepare(V_OD_hstmt, (SQLCHAR *) "INSERT INTO images  VALUES
(1, ?)", SQL_NTS);
CHECK_STMT_RESULT(rc, "SQLPrepare failed", V_OD_hstmt);
/*绑定参数*/
cbParam1 = 8;
rc = SQLBindParameter(V_OD_hstmt, 1, SQL_PARAM_INPUT,
          SQL_C_CHAR,  /* value type */
          SQL_CHAR,  /* param type */
          200,       /* column size */
          0,       /* dec digits */
          str,     /* param value ptr */
          0,       /* buffer len */
          &cbParam1     /* StrLen_or_IndPtr */);
CHECK_STMT_RESULT(rc, "SQLBindParameter failed", V_OD_hstmt);

/* 执行*/
rc = SQLExecute(V_OD_hstmt);
CHECK_STMT_RESULT(rc, "SQLExecute failed", V_OD_hstmt);
```

8.7.6.2. 读取记录

```
char buf[100];
SQLLEN ind;
/*分配句柄*/
rc = SQLAllocHandle(SQL_HANDLE_STMT, V_OD_hdbc, &V_OD_hstmt);
/*执行查询*/
SQLExecDirect(V_OD_hstmt, (SQLCHAR *) "SELECT id, img FROM images  W
HERE id = 1", SQL_NTS);
/*获取结果*/
rc = SQLFetch(V_OD_hstmt);
CHECK_STMT_RESULT(rc, "SQLFetch failed", V_OD_hstmt);
/*读取数据*/
rc = SQLGetData(V_OD_hstmt, 2, SQL_C_CHAR, buf, sizeof(buf), &ind);
CHECK_STMT_RESULT(rc, "SQLGetData failed", V_OD_hstmt);
```

8.7.7. 数字类型

数字类型有整数类型、任意精度数字类型、浮点类型和序数类型。下面用整数类型 integer、任意精度数字类型 numeric、 浮点类型 real 和 double precision 举例说明。

8.7.7.1. 插入数据

- ❖ 整数类型 integer 的插入

```
    int     rc;
    char    buf[200];
    SQLINTEGER  intparam;
    SQLLEN  cbParam1;

    SQLExecDirect(hstmt,(SQLCHAR *)"drop table IF EXISTS test_integer;",SQ
L_NTS);
    SQLExecDirect(hstmt,(SQLCHAR *)"create table test_integer(intg integer);
",SQL_NTS);

    rc = SQLPrepare(hstmt, (SQLCHAR *) "insert into test_integer values (?);
", SQL_NTS);

    /* bind param  */
    intparam = 3;
    cbParam1 = sizeof(intparam);
    rc = SQLBindParameter(hstmt, 1, SQL_PARAM_INPUT,
                          SQL_C_SLONG,  /* value type */
                          SQL_INTEGER,  /* param type */
                          0,       /* column size (ignored for SQL_INTEG
ER) */
                          0,       /* dec digits */
                          &intparam,  /* param value ptr */
```

```
                                sizeof(intparam),
                                &cbParam1);

    /* Execute */
    rc = SQLExecute(hstmt);
    if (!SQL_SUCCEEDED(rc))
    {
        printf("[failed] test_insert_select: insert into test_integer values (?)
");
        exit(1);
    }
```

❖ 任意精度数字类型 numeric 的插入

```
SQL_NUMERIC_STRUCT numericparam, numericparamT;
    SQLLEN     cbParam_nmr_1;
    char     sql[100];

    SQLExecDirect(hstmt,(SQLCHAR *)"drop table IF EXISTS test_numeric;",S
QL_NTS);
    SQLExecDirect(hstmt,(SQLCHAR *)"create table test_numeric(nmr numeri
c);",SQL_NTS);

    rc = SQLPrepare(hstmt, (SQLCHAR *) "insert into test_numeric values
```

```
(?);", SQL_NTS);

    /* 25.212 */
    build_numeric_struct(&numericparam, 1, (char *)"7C62", 5, 3);

    cbParam_nmr_1 = sizeof(numericparam);
    rc = SQLBindParameter(hstmt, 1, SQL_PARAM_INPUT,
                          SQL_C_NUMERIC,  /* value type */ //指时 num
ericparam 的类型   -- 表示将 C 语言中的 SQL_C_NUMERIC 类型变量（在 C 语言实
际是 SQL_NUMERIC_STRUCT numericparam）绑定到 SQL_NUMERIC 类型的参数
中
                          SQL_NUMERIC,  /* param type */ //代绑定的参
数类型类型 select ? form ...; 表示代绑定的 ? 对应的参数类型
                          0,       /* column size (ignored for SQL_INTER
VAL_SECOND) */
                          0,       /* dec digits */
                          &numericparam, /* param value ptr */
                          sizeof(numericparam),
                          &cbParam_nmr_1);

    /* Execute */
    rc = SQLExecute(hstmt);
    if (!SQL_SUCCEEDED(rc))
```

```
    {
        /* 失败 */
    }
```

上述代码片段中的 build_numeric_struct 函数实现如下：

```
static void
build_numeric_struct(SQL_NUMERIC_STRUCT *numericparam,
            unsigned char sign, char *hexstr,
            unsigned char precision, unsigned char scale)
{
  int       len;

  /* parse the hex-encoded value */
  memset(numericparam, 0, sizeof(SQL_NUMERIC_STRUCT));

  numericparam->sign = sign;
  numericparam->precision = precision;
  numericparam->scale = scale;

  len = 0;
  while (*hexstr)
  {
    if (*hexstr == ' ')
    {
```

```
      hexstr++;
      continue;
    }
    if (len >= SQL_MAX_NUMERIC_LEN)
    {
      fprintf(stderr, "hex-encoded numeric value too long\n");
      exit(1);
    }
    numericparam->val[len] =
      hex_to_int(*hexstr) << 4 | hex_to_int(*(hexstr + 1));
    hexstr += 2;
    len++;
  }
}
```

❖ 浮点类型 real 和 double precision 的插入

```
    SQLDOUBLE  dbParam, dbParamT;
    SQLREAL  ftParam, ftParamT;
    SQLLEN  cbParam_db_1;
    SQLLEN  cbParam_ft_2;

    SQLExecDirect(hstmt,(SQLCHAR *)"drop table IF EXISTS test_float;",SQL_
NTS);
```

```
    SQLExecDirect(hstmt,(SQLCHAR *)"create table test_float(dbpre double
precision, rl real);",SQL_NTS);


    rc = SQLPrepare(hstmt, (SQLCHAR *) "insert into test_float values (?, ?);
", SQL_NTS);


    /* bind param  */
    dbParam = 3.1234;
    cbParam_db_1 = sizeof(dbParam);
    rc = SQLBindParameter(hstmt, 1, SQL_PARAM_INPUT,
                          SQL_C_DOUBLE,  /* value type */
                          SQL_DOUBLE,  /* param type */
                          0,       /* column size (ignored for SQL_INTEG
ER) */
                          0,       /* dec digits */
                          &dbParam,     /* param value ptr */
                          sizeof(dbParam),
                          &cbParam_db_1);


    ftParam = 4.1234;
    cbParam_ft_2 = sizeof(ftParam);
    rc = SQLBindParameter(hstmt, 2, SQL_PARAM_INPUT,
                          SQL_C_FLOAT,  /* value type */
```

```
                            SQL_FLOAT,   /* param type */
                            0,       /* column size (ignored for SQL_INTEG
ER) */
                            0,       /* dec digits */
                            &ftParam,     /* param value ptr */
                            sizeof(ftParam),
                            &cbParam_ft_2);


    /* Execute */
    rc = SQLExecute(hstmt);
    if (!SQL_SUCCEEDED(rc))
    {
        printf("[failed] test_insert_select: insert into test_float values (?, ?)
\n");
        exit(1);
    }
```

8.7.7.2. 读取数据

❖ 整数类型 integer 的读取

```
    rc = SQLPrepare(hstmt, (SQLCHAR *) "select intg from test_integer;", S
QL_NTS);
    /* Execute */
    rc = SQLExecute(hstmt);
```

```
    if (!SQL_SUCCEEDED(rc))
    {
        printf("[failed] test_insert_select: select intg from test_integer;");
        exit(1);
    }
    else
    {
        /* print result */
        rc = SQLFetch(hstmt);

        rc = SQLGetData(hstmt, 1, SQL_C_CHAR, buf, sizeof(buf), NULL);

        printf("test_integer.intg:\n     %s\n", buf);
    }
    printf ("test_insert_select: test_integer end \n\n ");

    rc = SQLFreeStmt(hstmt, SQL_CLOSE);
```

❖ 任意精度数字 numeric 的读取

```
    rc = SQLExecDirect(hstmt, (SQLCHAR *) "select * from test_numeric", S
QL_NTS);

    rc = SQLFetch(hstmt);
```

```
    // 下面是以十六进制显示。
    // rc = SQLGetData(hstmt, 1, SQL_C_NUMERIC, &numericparamT, sizeof
(numericparamT), NULL);
    // printf("[hexadecimal] test_numeric.nmr:\n      sign %u prec %u scal
e %d val %02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X%02X\n",
    //        numericparam.sign, numericparam.precision, numericparam.sc
ale,
    //        numericparam.val[0], numericparam.val[1],
    //        numericparam.val[2], numericparam.val[3],
    //        numericparam.val[4], numericparam.val[5],
    //        numericparam.val[6], numericparam.val[7],
    //        numericparam.val[8], numericparam.val[9],
    //        numericparam.val[10], numericparam.val[11],
    //        numericparam.val[12], numericparam.val[13],
    //        numericparam.val[14], numericparam.val[15]);

    rc = SQLGetData(hstmt, 1, SQL_C_CHAR, buf, sizeof(buf), NULL);
    printf("[character string] test_numeric.nmr:\n    %s\n", buf);

    rc = SQLFreeStmt(hstmt, SQL_CLOSE);
```

❖ 浮点类型 real 和 double precision 的读取

```
    rc = SQLPrepare(hstmt, (SQLCHAR *) "select * from test_float;", SQL_N
TS);
    /* Execute */
    rc = SQLExecute(hstmt);
    if (!SQL_SUCCEEDED(rc))
    {
        printf("[failed] test_insert_select: select * from test_float;\n");
        exit(1);
    }
    else
    {
        /* print result */
        rc = SQLFetch(hstmt);

        SQLGetData(hstmt, 1, SQL_C_DOUBLE, &dbParamT, sizeof(dbParam
T), NULL);
        printf("test_float.dbpre:\n     %lf\n", dbParamT);

        SQLGetData(hstmt, 2, SQL_C_FLOAT, &ftParamT, sizeof(ftParamT), N
ULL);
        printf("test_float.dbpre:\n     %f\n", ftParamT);
    }
```

```
    rc = SQLFreeStmt(hstmt, SQL_CLOSE);
```

8.7.8. 时间/日期类型

日期/时间类型有 timestamp 、date、time 、interval ； 下面以 timestamp 、date、time 举例说明。

8.7.9. 插入数据

```
    SQLHENV         hEnv     = SQL_NULL_HENV;
    SQLHDBC         hDbc     = SQL_NULL_HDBC;
    SQLHSTMT        hStmt    = SQL_NULL_HSTMT;
    SQLRETURN       rc       = SQL_SUCCESS;
    SQLINTEGER      RETCODE = 0;

    TIMESTAMP_STRUCT sTime, sTimeT ;
    DATE_STRUCT dTime, dTimeT ;
    TIME_STRUCT tTime, tTimeT ;
    SQLLEN cbID = 0;
    SQLCHAR result[200]={0};

    rc = SQLExecDirect(hStmt,(SQLCHAR*)"drop table IF EXISTS test_timesta
mp_date_time",SQL_NTS);
```

```
    if (!SQL_SUCCEEDED(rc))
        goto exit;

    rc = SQLExecDirect(hStmt,(SQLCHAR*)"create table test_timestamp_date
_time(tmsp timestamp, dt date, tm time)",SQL_NTS);
    if (!SQL_SUCCEEDED(rc))
        goto exit;

    rc = SQLExecDirect(hStmt,(SQLCHAR*)"insert into test_timestamp_date_ti
me values('2012-12-14 12:12','2013-11-15', '04:12:03')",SQL_NTS);
    if (!SQL_SUCCEEDED(rc))
        goto exit;

    SQLPrepare(hStmt,(SQLCHAR*)"insert into test_timestamp_date_time val
ues(?, ?, ?)",SQL_NTS);

    sTime.year = 1992;
    sTime.month = 12;
    sTime.day = 12;
    sTime.hour = 9;
    sTime.minute = 10;
    sTime.second = 31;
    SQLBindParameter(hStmt,1,SQL_PARAM_INPUT,SQL_C_TYPE_TIMESTAMP,S
```

```
QL_TYPE_TIMESTAMP,100,0,(SQLPOINTER)&sTime,sizeof(TIMESTAMP_STRUCT),
&cbID);


    dTime.year = 1982;
    dTime.month = 12;
    dTime.day = 11;
    SQLBindParameter(hStmt,2,SQL_PARAM_INPUT,SQL_C_TYPE_DATE ,SQL_T
YPE_DATE ,100,0,(SQLPOINTER)&dTime,sizeof(DATE_STRUCT),&cbID);


    tTime.hour = 10;
    tTime.minute = 10;
    tTime.second = 22;
    SQLBindParameter(hStmt,3,SQL_PARAM_INPUT,SQL_C_TYPE_TIME ,SQL_TY
PE_TIME ,100,0,(SQLPOINTER)&tTime,sizeof(TIME_STRUCT),&cbID);


    rc = SQLExecute(hStmt);
    if ((!SQL_SUCCEEDED(rc)))
            goto exit;
```

8.7.10. 读取数据

```
    rc = SQLExecDirect(hStmt,(SQLCHAR*)"select * from test_timestamp_dat
e_time;",SQL_NTS);
    if (!SQL_SUCCEEDED(rc))
```

```
        goto exit;


    rc = SQLFetch(hStmt);
    while(rc != SQL_NO_DATA)
    {
        /* test SQLGetData */
        SQLGetData(hStmt,1,SQL_C_TYPE_TIMESTAMP,(SQLPOINTER)&sTimeT,s
izeof(sTimeT),&cbID);
        sprintf((char*)result, "%.4d-%.2d-%.2d %.2d:%.2d:%.2d", sTimeT.yea
r, sTimeT.month, sTimeT.day,sTimeT.hour,sTimeT.minute,sTimeT.second);
        printf("TIMESTAMP = %s\n\n",result);

        SQLGetData(hStmt,2,SQL_C_TYPE_DATE ,(SQLPOINTER)&dTimeT,sizeof
(dTimeT),&cbID);
        sprintf((char*)result, "%.4d-%.2d-%.2d", dTimeT.year, dTimeT.month,
 dTimeT.day);
        printf("DATE = %s\n\n",result);

        SQLGetData(hStmt,3,SQL_C_TYPE_TIME ,(SQLPOINTER)&tTimeT,sizeof
(tTimeT),&cbID);
        sprintf((char*)result, "%.2d:%.2d:%.2d", tTimeT.hour,tTimeT.minute,t
TimeT.second);
        printf("TIME = %s\n\n",result);
```

```
        rc = SQLFetch(hStmt);
    };

    if (!SQL_SUCCEEDED(rc) && rc != SQL_NO_DATA)
        goto exit;
```

9. peewee 框架开发指南

9.1. 介绍

Peewee 是一种简单而小的 ORM。它有很少的（但富有表现力的）概念，使它易于学习和直观的使用。

9.2. 安装 peewee

安装 peewee 需要安装 peewee 包和数据库对应的 python 语言驱动 psycopg2。

9.2.1. 安装 psycopg2

安装 psycopg2 模块前需要先装好 PanWeiDB V2.0-S3.0.0_Beta Libpq，并配置环境变量。

9.2.1.1. 安装 PanWeiDB V2.0-S3.0.0_Beta Libpq

安装 Libpq 有两种方式。

- 通过提供的 Libpq 安装包安装

解压 PanWeiDB V2.0-S3.0.0_Beta 提供的 Libpq，新建/usr/local/libpq 目录，将解压后的 lib 目录拷贝到/usr/local/libpq 目录下，配置环境变量如下：

export LD_LIBRARY_PATH=/usr/local/libpq/lib:$LD_LIBRARY_PATH

❖ 通过安装 PanWeiDB V2.0-S3.0.0_Beta 安装包安装

按照安装说明安装 PanWeiDB V2.0-S3.0.0_Beta，并配置环境变量。如果是使用安装包自带安装程序进行安装，则环境变量会自动配置无需手动配置；如果是手动安装数据库，则需配置以下环境变量（只作为参数，应按实际的环境进行配置）：

```
export GAUSSHOME=/usr/local/panweidb

export PATH=$GAUSSHOME/bin:$PATH

export LD_LIBRARY_PATH=$GAUSSHOME/lib:$LD_LIBRARY_PATH
```

9.2.1.2. 安装 python3 和 gcc

推荐使用 python3，安装命令如下：

```
yum install -y python3 python3-devel gcc
```

9.2.1.3. 安装 psycopg2 模块

推荐使用源码包安装，获取 psycopg2 源码包：

```
# 1.卸载已安装的 psycopg2（如果没有安装则跳过，此步骤可能需要 root 用户执行）

pip3 uninstall psycopg2 -y


# 2.上传并解压源码包

unzip Psycopg-panwei.zip

# 3.安装
```

```
cd Psycopg-panwei

python3 setup.py build

# 此步骤可能需要 root 用户执行

python3 setup.py install


# 4.检查安装结果

进入 python3 命令行，执行 import psycopg2 可以正常导入即表示安装成功。
```

9.2.2. 安装 peewee

安装至本地可以通过命令：

```
pip install peewee
```

9.3. 使用 peewee 连接数据库

9.3.1. 导入

任何使用 peewee 的程序都需要导入 peewee 包。

```
from peewee import *
```

9.3.2. 连接数据库

在连接 panweidb 数据库时，使用连接 postgres 的模式。要连接到 PostgreSQL 数据库，我们将使用 PostgresqlDatabase。第一个参数始终是数据库的名称，之后可以指定任意 psycopg2 parameters。

```
psql_db = PostgresqlDatabase('my_database', user=pwadmin)


class BaseModel(Model):
    class Meta:
        database = psql_db


class User(BaseModel):
    username = CharField()
```

也可使用 URL 连接

```
from peewee import *
from playhouse.db_url import connect

db = connect('postgresql://panweidb:password@localhost:5432/my_datab
ase')

class BaseModel(Model):
    class Meta:
        database = db
```

9.3.2.1. 定义表结构体

peewee 支持将一个类映射为数据库中对应的一张表。

```
from peewee import *


db = PostgresqlDatabase('my_database', user=pwadmin)


class Person(Model):
    name = CharField()
    birthday = DateField()


    class Meta:
        database = db
```

9.4. 执行 SQL 语句并处理结果

9.4.1. 创建或修改数据库对象

为了开始使用我们的模型，必须先打开到数据库的连接并创建表。Peewee 将运行必要的 CREATE TABLE 查询，另外创建任何约束和索引。

```
class Person(Model):
    name = CharField(index=False,column_name='name1')
    birthday = DateField(null=True)
    is_relative = BooleanField()
```

```
    class Meta:
        database = db
# 创建表
Person.create_table()

# 创建多个可以在列表后面加
# database.create_tables([Person])
```

创建表后，如果选择修改数据库架构（通过添加、删除或更改列），则需要：

- 删除表并重新创建。
- 运行一个或多个 ALTER TABLE 查询。Peewee 附带了一个模式迁移工具，可以大大简化这一过程。

9.4.2. 删除数据库对象

删除数据库对象时使用 drop_tables()函数，可以删除给定模型列表的表、索引和相关元数据。

```
db.drop_tables([Person])
```

9.4.3. 导出数据库对象

如果模型已经存在数据库中,则直接通过 python -m pwiz 批量创建 Model。

```
# 指定 postgres，用户为 pwadmin，host 为 127.0.0.1，数据库为 test
python -m pwiz -e postgres -u pwadmin -H 127.0.0.1 --password test >
```

testModel.py

然后，输入密码，pwiz 脚本会自动创建 Model，内容如下：

```
from peewee import *


database = PostgresqlDatabase('test', **{'host': 127.0.0.1, 'user': pwadmin,
 'password': ''})


class UnknownField(object):
    def __init__(self, *_, **__): pass


class BaseModel(Model):
    class Meta:
        database = database


class Person(BaseModel):
    birthday = DateField()
    is_relative = IntegerField()
    name = CharField()


    class Meta:
        table_name = 'person'
```

9.4.4. 执行更新

9.4.4.1. 插入

插入数据很简单。我们可以指定数据 insert() 以四种不同的方式：

```
# 直接添加一条数据
p = Person(name='lcg', birthday=date(1990, 12, 20), is_relative=True)
p.save()

# 直接 create，create 之后还会返回创建的这条数据，方便进行其他操作
p= Person.create(name='yyy', birthday=date(2004, 2, 7), is_relative=True)

# 使用属性作为参数
p = Person.insert(name='zzz', birthday=date(2001,12,2),
is_relative=False).execute()

# 使用列属性和值映射
Note.insert({
    Person.name: 'meow',
    Person.birthday: datetime.datetime.now(),
    Person.relative: True}).execute()
```

9.4.4.2. 更新

```
query = Person.update(name="zzz").where(Person.is_relative=True)
query.execute()
```

9.4.4.3. 删除

删除不接受任何参数。

```
# 删除所有在 00 年前出生的人的信息，返回删除个数
n = Person.delete().where(Person.birthday< datetime.date(2000, 1, 1)).exec
ute()
```

因为删除（和更新）查询不支持联接，所以可以使用子查询根据相关表中的值删除行。例如，下面是如何删除 is_relative 为“False”的所有人的信息：

```
# 获取 is_relative=False 的人的 id
people = Person.select(Person.id).where(Person.is_relative == False)

# 删除前面获取到的 ID 的信息
Person.delete().where(Person.id.in_(people)).execute()
```

9.4.5. 执行查询

9.4.5.1. 查询单条数据

调用 SQL 语句查询：

```
# get 的括号中给定条件，可以一次性多个条件
people= Person.get(Person.name == 'yyy', is_relative == False)
```

9.4.5.2. 查询多条数据

```
# 保留 is_relative 的名字

people= Person.select(Person.name).where(Person.is_relative==True)
```

9.4.6. 获取结果

9.4.6.1. 遍历数组

select 方法获取到的结果可以顺序遍历以获取结果集中所有结果。

```
for i in people:

print(i.name)
```

9.4.7. 执行存储过程

下面的例子展示了如何调用存储过程。

```
sql_procedure = " CREATE OR REPLACE PROCEDURE pro_test(a integer,inou

t b integer) as \

                    begin \

                        b := a+b; \

                    end"

cursor = db.excute_sql(sql_procedure)

cursor = db.excute_sql("call pro_test(1,5)")

for value, in cursor:

print(value)
```

下面的例子展示了如何调用函数。

```
sql_function = "CREATE OR REPLACE FUNCTION func_test (a int,out b int)
RETURNS int AS \
                $$ \
                begin \
                    b := a+2; \
                return ;  \
                end; \
                $$ LANGUAGE 'plpgsql';"
cursor = db.excute_sql(sql_function)
cursor = db.excute_sql("select func_test(1)")
for value, in cursor:
print(value)
```

9.4.8. 处理数据类型

9.4.8.1. 二进制类型

PanWeiDB V2.0-S3.0.0_Beta 提供两种不同的方法存储二进制数据。 二进制数据可以使用二进制数据类型 BYTEA 存储在表中，或者使用大对象特性以一种特殊的格式将二进制数据存储在一个独立的表中， 然后通过在表中保存一个类型为 OID 的值来引用该表。以下为 BYTEA 类型使用示例。

1) 创建表

```
class ImageData(Model):
image = BlobField(default=b'emptydata')
class Meta:
        database = db


ImageData.create_table()
```

2) 插入数据

```
data = bytes(range(256))
i = ImageData(image=data)
save()
```

3) 读取数据

```
i2 = ImageData.get(i.id)
assert i2.image.tobytes() == data


ImageData.create(image = b'string')
assert ImageData.select(ImageData.image).where(ImageData.image ==  b's
tring').count()==1
```

9.4.8.2. 字符类型

PanWeiDB V2.0-S3.0.0_Beta 中 TEXT 类型与 VARCHAR 类型都是可变长的字符类型，区别在于 VARCHAR 类型通过 VARCHAR(n)中的 n 来限制最大长

度，而 TEXT 类型没有。TEXT 类型与 VARCHAR 类型几乎没有性能差别，TEXT 类型最多可存储 1G 数据。

1) 创建表

```
class String1(Model):
col1 = TextField()# text 类型
col2 = CharField()# varchar 类型
class Meta:
        database = db


String1.create_table()
```

2) 插入数据

```
a='test1'
b='test2'
s = String1(col1=a,col2=b)
s.save()
```

3) 读取数据

```
s1 = String1.get(1)
assert s1.col1==a
assert s1.col2==b
```

9.4.8.3. 数字类型

数字类型有整数类型、任意精度数字类型、浮点类型和序数类型。下面用整数类型 integer、任意精度数字类型 numeric 和 double precision 举例说明。

1)创建表

```
class Number1(Model):
col1 = IntegerField()# integer 类型
col2 = DecimalField()# numeric 类型
col3 = DoubleField()# double precision 类型
class Meta:
        database = db


Number1.create_table()
```

2)插入数据

```
a = 1
b= decimal.Decimal(2.141)
c = 3.1415926
n = Number1(col1=a,col2=b,col3=c)
n.save()
```

3)读取数据

```
n1=Number1.get(1)
print(n1.col1)
print(n1.col2)
print(n1.col3)
```

9.4.8.4. 时间/日期类型

日期/时间类型有 timestamp 、date、time 、interval ； 下面以 timestamp 、date、time 举例说明。

1)创建表

```
class DateTime1(Model):
col1 = DateTimeField() # timestamp 类型
col2 = DateField() # date 类型
col3 = TimeField() # time 类型
    class Meta:
        database = db

DateTime1.create_able()
```

2)插入数据

```
_now = datetime.now().replace(microsecond = 0)
d = DateTime1(col1=_now,col2=_now,col3=_now.time())
d.save()
```

3)读取数据

```
dt1=DateTime1.get(1)
assert isinstance(dt1.col1,datetime)
assert isinstance(dt1.col2,date)
assert isinstance(dt1.col3,time)
```

9.5. 附录 1：主要接口

主要接口说明如下：

- PostgresqlDatabase(database[,register_unicode=True[,encoding=None[,isolation_level=None]]])

  PostgreSQL 数据库实现。此方法用于创建新的数据库会话并返回新的 connection 对象。参数说明如下：

关键字	参数说明
dbname	数据库名称
parameters	任意 psycopg2 parameters
register_unicode	注册 Unicode 类型。
encoding	数据库编码。
isolation_level	隔离级别常量，在 psycopg2.extensions 模块。

  调用示例见附录 2。

- connect(url, **connect_params)

  创建一个 Database 来自给定连接 URL 的实例

  实例：postgresql://panweidb:my_password@localhost:5432/my_database 将创建一个 PostgresqlDatabase 实例。提供用户名和密码，以及要连接的主机和端口。

  参数说明如下：

关键字	参数说明
url	url 字符串，存储连接信息

调用示例见附录 2。

- Model 类:
- create_table([safe=True[, **options]])方法

创建模型表、索引、约束和序列。

参数说明如下:

关键字	参数说明
safe	如果设置为 True ，创建表查询将包括 IF NOT EXISTS 。

调用示例见附录 2。

- drop_table([safe=True[, **options]])方法

如果设置为 True ，创建表查询将包括 IF EXISTS。

参数:

关键字	参数说明
safe	插入的记录数

调用示例见附录 2。

- create(**query)方法

在表中插入新行并返回相应的模型实例。

参数:

关键字	参数说明
query	字段名到值的映射。

返回相应的模型实例

调用示例见附录 2。

- insert([__data=None[, **insert]])

  创建插入查询。

  参数:

关键字	参数说明
__data	dict 字段到要插入的值。
insert	字段名到值的映射。

  调用示例见附录 2。

- update([__data=None[, **update]])

  创建更新查询。

  参数:

关键字	参数说明
__data	dict 字段到值。
update	字段名到值的映射。

  调用示例见附录 2。

- get(*query, **filters)

  检索与给定筛选器匹配的单个模型实例。

参数：

关键字	参数说明
query	查询条件
filters	将字段名映射为 django 样式筛选器的值。

返回值：与指定筛选器匹配的模型实例。

调用示例见附录 2。

- select(*fields)

创建选择查询。如果没有显式提供字段，则查询将默认选择模型上定义的所有字段，除非将查询用作子查询，在这种情况下，默认情况下只选择主键。

参数：

关键字	参数说明
fields	模型类、字段实例、函数或表达式的列表。如果没有提供参数，则默认情况下将选择给定模型的所有列。

返回：ModelSelect 查询。

调用示例见附录 2。

- delete()

创建删除查询。

返回删除的个数。

调用示例见附录 2。

9.6. 附录 2：调用示例

```
from peewee import *

import datetime

from datetime impotr date

import logging


#输出 SQL 日志

#logger = logging.getLogger('peewee')

#logger.addHandler(logging.StreamHandler())

#logger.setLevel(logging.DEBUG)


#新建数据库 db

db = PostgresqlDatabase('test',host='127.0.0.1',port=5432,

user='pwadmin',password='password')

db.connect()


class Person(Model):

name = CharField(index=False,column_name='name1')

birthday = DateField()

is_relative = BooleanField()


class Meta:

database = db
```

```
def create_p():
#创建表
Person.create_table()
#创建多个，可以在[]中写多个表名
#database.create_tables([Person])

def insert_p():
#添加一条数据
p = Person(name='lcg', birthday=date(1990,12,20), is_relative=True)
p.save()

#直接 create，create 之后还会返回创建的这条数据，方便进行其他操作
p = Person.create(name='yyy', birthday=date(2004,2,7), is_relative=False)

#使用属性作为参数
p = Person.insert(name='zzz', birthday=date(2001,12,2), is_relative=False)
    .execute()

#使用列属性和值映射
Person.insert({
Person.name: 'meow',
Person.birthday: datetime.datetime.now(),
Person.is_relative: True}).execute()
```

```
def update_p():
query = Person.update(is_relative=True).where(Person.name =='zzz')
query.execute()

def select_p():
#get 的括号中给定条件，可以一次性多个条件
people = Person.get(Person.name == 'yyy', Person.is_relative == False)
print(people.name)

#保留 is_relative 的名字
people = Person.select(Person.name).where(Person.is_relative == True)
for i in people:
print(i.name)

def delete_p():
#删除所有在 18 年前出生的人的信息，返回删除个数
n = Person.delete().where(Person.birthday < datetime.date(2000,1,1))
.execute()
#获取 is_relative=False 的人的 id
people = Person.select(Person.id).where(Person.is_relative == False)
#删除前面获取到的 ID 的信息
Person.delete().where(Person.id.in_(people)).execute()
```

```
def  drop_p():

db.drop_tables([Person])



create_p()

insert_p()

update_p()

select_p()

delete_p()

drop_p()
```

10. Perl 接口开发指南

10.1. 介绍

PanWeiDB V2.0-S3.0.0_Beta 可以使用 Perl DBI 模块与 Perl 集成，Perl DBI 模块是 Perl 编程语言的数据库访问模块，它定义了一组提供标准数据库接口的方法，变量和约定。本文对 Perl 连接 PanWeiDB V2.0-S3.0.0_Beta 的步骤以及连接后的使用方法做了简单说明。

10.2. 相关依赖版本

Perl：建议 v5.8 及以上版本

DBI：Perl 数据库访问模块，建议 v1.627 及以上版本

DBD::Pg：PostgreSQL perl 驱动，建议 v3.14.2 及以上版本（安装依赖 postgresql-devel）

10.3. DBI 主要类方法

DBI 是 Perl 数据库访问模块，通过 DBD::Pg 驱动与 PanWeiDB V2.0-S3.0.0_Beta 建立连接。

- connect

  与特定数据库的连接（会话）。连接成功，返回一个数据库句柄对象。

  示例：

```
$dbh = DBI->connect($dsn, $user, $password,
                    { RaiseError => 1, AutoCommit => 0 });
```

- prepare

  准备 SQL 语句。

- execute

  执行 SQL 语句。

- fetchrow_array

  获取结果行。

- do

  执行 SQL 语句。
  do 不能用于 select 语句，因为 do 不返回语句句柄，所以获取不到数据。

- disconnect

  断开会话。

10.4. 使用 Perl 连接数据库

10.4.1. 导入

导入 DBI 模块。

```
use DBI;
use strict;
```

10.4.2. 连接数据库

10.4.2.1. URL 格式

连接 PanWeiDB V2.0-S3.0.0_Beta 有如下 url 格式：

```
dbi:Pg:database_name

dbi:Pg:database_name@hostname:port

dbi:Pg:database=database_name;host=hostname;port=port
```

URL 中各参数的含义如下：

- hostname

  服务端的主机名，默认为 localhost。

- port

  服务端的端口号，默认为 5432。

- database_name

  数据库名称，默认连接的数据库是与用户同名的数据库。比如连接用户为 pwuser，如果不指定 database 参数，则默认连接到 pwuser 这个数据库。

10.4.2.2. 获取 Perl 连接

使用 DBI->connect 获取连接：

```
$dbh = DBI->connect($dsn, $user, $password,
        { RaiseError => 1, AutoCommit => 1 }) or die $DBI::errstr;
```

10.4.2.3. 关闭 Perl 连接

```
$dbh->disconnect();
```

10.5. 执行 SQL 语句并处理结果

向数据库发出执行 SQL 的请求时，调用 do，prepare 方法执行相关语句，其中 do 只能用于非 select 语句，它不返回句柄。

10.5.1. 执行查询

10.5.1.1. 使用 prepare 查询

```
my $alexa = 3;
my $stmt = qq(SELECT id, name, address, salary  from COMPANY where i
d<?;);
my $sth = $dbh->prepare( $stmt );
my $rv = $sth->execute($alexa) or die $DBI::errstr;
if($rv < 0){
   print $DBI::errstr;
}
while(my @row = $sth->fetchrow_array()) {
      print "ID = ". $row[0] . "\n";
      print "NAME = ". $row[1] ."\n";
      print "ADDRESS = ". $row[2] ."\n";
```

```
        print "SALARY =  ". $row[3] ."\n\n";
}
$sth->finish;
```

10.5.2. 执行更新

10.5.2.1. 删除

```
# do 方法
my $stmt = qq(DELETE from COMPANY where ID=2;);
my $rv = $dbh->do($stmt) or die $DBI::errstr;
if( $rv < 0 ){
   print $DBI::errstr;
}else{
   print "Total number of rows deleted : $rv\n";
}

# do 绑定变量
$stmt = qq(DELETE from COMPANY where ID=?;);
$rv = $dbh->do($stmt,undef,2) or die $DBI::errstr;
if( $rv < 0 ){
   print $DBI::errstr;
}else{
```

```
   print "Total number of rows deleted : $rv\n";
}

# prepare 方法
my $sth = $dbh->prepare(q{
    DELETE from COMPANY where ID=?});
$sth->bind_param( 1, 3 );
$sth->execute() or die $DBI::errstr;
$sth->finish;
```

10.5.2.2. 插入

10.5.3. Insert 插入数据

```
# do 方法
my $stmt = qq(INSERT INTO COMPANY (ID,NAME,AGE,ADDRESS,SALARY)
      VALUES (1, 'Paul', 32, 'California', 20000.00 ));
my $rv = $dbh->do($stmt) or die $DBI::errstr;

# prepare 方法
my $sth = $dbh->prepare(q{
    INSERT INTO t1(id,name,age,address,salary)
```

```
        VALUES (?,?,?,?,?)
});
$sth->bind_param( 1, 2 );
$sth->bind_param( 2, 'Allen' );
$sth->bind_param( 3, 25 );
$sth->bind_param( 4, 'Texas' );
$sth->bind_param( 5, 15000.00 );
$sth->execute() or die $DBI::errstr;
$sth->finish;


# 事务
my @data = ([1,'Paul', 32, 'California', 20000.00], [2,'Allen', 25, 'Texas', 15
000.00]);
$sth = $dbh->prepare(qq{INSERT INTO t1(id,name,age,address,salary)
        VALUES (?,?,?,?,?)});
$dbh->begin_work();   # start transaction
foreach my $row(@data) {
    $sth->execute(@$row);
}
$dbh->commit();   # end transaction
```

10.5.4. copy 插入数据

```
my @data= (
    "1,Paul,32,California,20000\n",
    "2,Allen,25,Texas,15000\n",
    "3,Teddy,23,Norway,20000\n",
    "4,Mark,25,Rich-Mond,65000\n",
);
my $sql=qq{COPY COMPANY(ID,NAME,AGE,ADDRESS,SALARY) FROM STDIN
WITH DELIMITER ','};
$dbh->do($sql);
foreach my $line (@data) {
    $dbh->pg_putcopydata($line);
}
$dbh->pg_putcopyend();
```

10.5.4.1. 更新

```
# do 方法
my $stmt = qq(UPDATE COMPANY set SALARY = 25000.00 where ID=1;);
my $rv = $dbh->do($stmt) or die $DBI::errstr;
if( $rv < 0 ){
   print $DBI::errstr;
```

```
}else{
    print "Total number of rows updated : $rv\n";
}

# prepare 方法
my $sth = $dbh->prepare(q{
    UPDATE COMPANY set SALARY = ? where ID=?});
$sth->bind_param( 1, 25000.00 );
$sth->bind_param( 2, 2 );
$sth->execute() or die $DBI::errstr;
$sth->finish;
```

10.5.5. LOB 类型读写

10.5.5.1. BYTEA 类型

```
Create table generic_file_table (
column_1 varchar(20),
column_2 varchar(20),
bytea_column bytea);
```

插入数据：

```
my $datum1 = "foo";

my $datum2 = "123";

my $file = "/path/to/file.dat";

my $contents;


##open the file and store it

open my $FH, $file or die "Could not open file: $!";

{

local $/ = undef;

$contents = <$FH>;

};

close $FH;

print "$contents";


##preparte SQL

my $q = q{

INSERT INTO generic_file_table (column_1,column_2,bytea_column) VALUES

 (?, ?, ?)};

my $sth = $dbh->prepare($q);

##bind variables and specifically set #3 to bytea; then execute.

$sth->bind_param(1,$datum1);

$sth->bind_param(2,$datum2);

$sth->bind_param(3,$contents, { pg_type => DBD::Pg::PG_BYTEA });
```

```
$sth->execute();

$sth->finish();
```

读取数据：

```
my $sParam = "123";

$dbh->{LongReadLen}=2_000_000;

$dbh->{LongTruncOk}=1;

my $sth= $dbh->prepare( "SELECT bytea_column FROM generic_file_table

WHERE column_2 = ?" );

$sth->execute($sParam);

my $sRes = $sth->fetchrow_array();

$sth->finish();

$dbh->disconnect();
```

10.5.5.2. BLOB 类型

```
Create table test_blob (

column_1 varchar(20),

column_2 varchar(20),

blob_column blob);
```

插入数据：

```
my $datum1 = "foo";

my $datum2 = "123";
```

```
my $file = "/path/to/file.dat";
my $contents;

##open the file and store it
open my $FH, $file or die "Could not open file: $!";
{
local $/ = undef;
$contents = <$FH>;
};
close $FH;
$contents =~ s/(.)/sprintf '%02x', ord $1/seg; # 转换为 hexstring
print "$contents";

##preparte SQL
my $q = q{
INSERT INTO test_blob (column_1,column_2,blob_column) VALUES (?, ?, ?)};
my $sth = $dbh->prepare($q);
$sth->bind_param(1,$datum1);
$sth->bind_param(2,$datum2);
$sth->bind_param(3,$contents);
$sth->execute();
$sth->finish();
```

读取数据：

```
my $sParam = "123";

$dbh->{LongReadLen}=2_000_000;

$dbh->{LongTruncOk}=1;

my $sth= $dbh->prepare( "SELECT blob_column FROM test_blob WHERE co

lumn_2 = ?" );

$sth->execute($sParam);

my $sRes = $sth->fetchrow_array();

$sth->finish();

$dbh->disconnect();
```

10.5.5.3. CLOB 类型、TEXT 类型

```
CREATE TABLE COMPANY (

ID INT PRIMARY KEY      NOT NULL,

NAME            CLOB     NOT NULL,

AGE             INT      NOT NULL,

ADDRESS         TEXT

);
```

插入数据：

```
my $id= 1;

my $name = "Twitter";
```

```
my $age = 22;
my $address = "1 Queen's Road Central , Hongkong";
my $sth = $dbh->prepare("INSERT INTO company
                         (id, name, age, address)
                          values
                         (?,?,?,?)");
$sth->execute($id,$name,$age, $address)
           or die $DBI::errstr;
$sth->finish();
$dbh->commit or die $DBI::errstr;
```

读取数据：

```
my $age= 20;
my $sth = $dbh->prepare("SELECT name, age,address
                         FROM company
                         WHERE age> ?");
$sth->execute( $age) or die $DBI::errstr;

# 循环输出所有数据
while ( my @row = $sth->fetchrow_array() )
{
        print join('\t', @row)."\n";
}
```

```
$sth->finish();


$dbh->disconnect();
```

10.5.6. 创建或修改数据库对象

```
# do 方法
my $stmt = qq(CREATE TABLE T1(ID INT, COL VARCHAR(20)););
my $rv = $dbh->do($stmt) or die $DBI::errstr;
if( $rv < 0 ){
   print $DBI::errstr;
}else{
    print "Create Success\n";
}
```

10.6. 存储过程或函数调用

10.6.1. 无参数

```
# CREATE FUNCTION
CREATE OR REPLACE FUNCTION "public"."function1" () RETURNS integer AS
$body$
```

```
DECLARE
  myvar INTEGER;
BEGIN
  SELECT INTO myvar fld FROM table1 LIMIT 1;
  RETURN myvar;
END;
$body$
LANGUAGE 'plpgsql';


# Perl
my $sth = $dbh->prepare("SELECT function1()");
$sth->execute( ) or die $DBI::errstr;


# 循环输出所有数据
while ( my @row = $sth->fetchrow_array() )
{
        print join('\t', @row)."\n";
}


$sth->finish();
$dbh->disconnect();
```

10.6.2. 带参数

```
# CREATE FUNCTION

CREATE OR REPLACE FUNCTION "public"."function2" (uname_val varchar) R

ETURNS integer AS

$body$

DECLARE

  myvar INTEGER;

BEGIN

  SELECT INTO myvar fld FROM table1 where uname = uname_val ;

  RETURN myvar;

END;

$body$

LANGUAGE 'plpgsql';


# Perl

my $uname_val = 'test';

my $sth = $dbh->prepare("SELECT function2(?)");

$sth->execute( $uname_val) or die $DBI::errstr;


# 循环输出所有数据

while ( my @row = $sth->fetchrow_array() )

{
```

```
        print join('\t', @row)."\n";
}


$sth->finish();
```

11. PHP 接口开发指南

11.1. 介绍

PanWeiDB V2.0-S3.0.0_Beta 数据库通过 PHP 官方提供的 pdo_pgsql 和 pgsql 动态链接库集成，为使用 PHP 语言的应用程序提供访问 PanWeiDB V2.0-S3.0.0_Beta 的接口。

11.1.1. PDO 类主要方法

- beginTransaction

  启动一个事务。

- commit

  提交一个事务。

- rollBack

  回滚一个事务。

- exec

  执行一条 SQL 语句并返回受影响行数。

- query

  执行 SQL 语句，返回结果集。

- prepare

  备要执行的 SQL 语句并返回 PDOStatement 对象。

- setAttribute

  设置属性。

11.2. 启用动态链接库

PHP 连接 PanWeiDB V2.0-S3.0.0_Beta 数据库需启用相应的数据库驱动。

11.2.1. Windows 平台

以 Windows10 操作系统和 PHP 版本 7.4.18 为例子，安装流程如下：

官网下载 ZIP 包 https://windows.php.net/download#php-7.4，解压后进入解压目录复制一份配置文件 php.ini-development 改名为 php.ini，修改 php.ini 文件支持动态链接库。

搜索 pdo_pgsql 快速定位到如下

```
;extension=pdo_pgsql

;extension=pgsql
```

去掉分号修改为

```
extension=pdo_pgsql

extension=pgsql
```

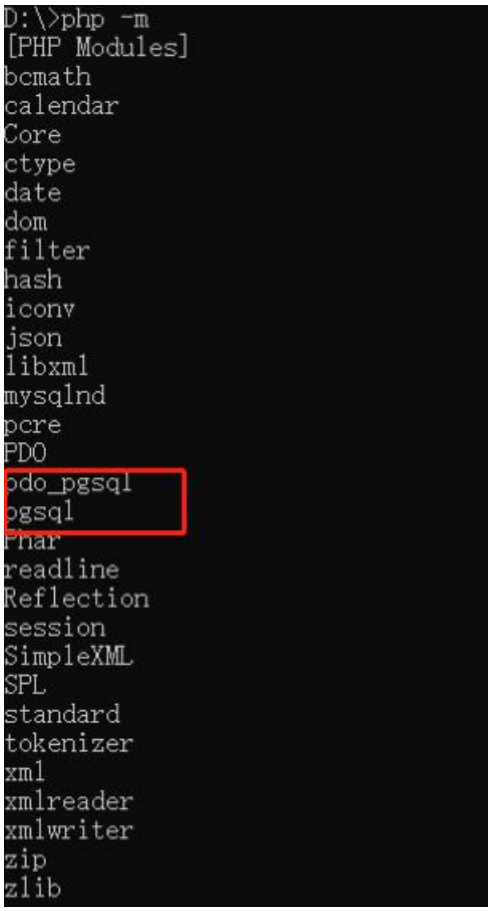
搜索 extension_dir 快速定位到如下

```
;extension = “ext”
```

去掉分号修改拓展地址为

```
extension = “ext”
```

命令行输入 php -m 检查是否启用成功，若有 pdo_pgsql 和 pgsql 表示以成功启用。



温馨提示：查询 PHP 加载使用的配置文件地址

```
php -i|findstr php.ini
```

11.2.2. Linux 平台

启用 pdo_pgsql 和 pgsql 前,需先本地安装 PanWeiDB V2.0-S3.0.0_Beta 数据库；若数据库安装在远程，本地需单独安装依赖 postgresql-devel。

PHP 是支持跨平台的，绝大部分 Linux 操作系统都支持，下面以 Centos7.4 和 PHP7.4.16 为例子。

11.2.2.1. 安装 PanWeiDB V2.0-S3.0.0_Beta

请参照《PanWeiDB V2.0-S3.0.0_Beta 安装升级指南》，安装 PanWeiDB V2.0-S3.0.0_Beta。

11.2.2.2. 安装 postgresql-devel

```
yum install -y postgresql-devel
```

11.2.2.3. 安装驱动拓展

Linux 已安装 PHP 要连接 panweidb 需安装驱动拓展，下面假设用户已经安装好了 PHP 为例子安装拓展:

执行命令 `php -v` 查看 PHP 版本，从官网下载对应版本安装包上传到服务器解压 。

安装 pgsql 拓展

```
cd ${PHP 源码目录}/ext

cd pgsql

#执行 `whereis phpize` 查找 phpize 文件路径后执行该文件

/usr/bin/phpize

#phpize 执行成功后执行 `whereis php-config` 找到的 php-config 文件作为 wit

h-php-config 的选项值

./configure  --with-php-config=/usr/bin/php-config

make

make install
```

安装 pdo_pgsql 拓展

```
cd ../pdo_pgsql

/usr/bin/phpize

./configure --with-php-config=/usr/bin/php-config

make

make install
```

- make install 编译后动态链接库存放路径命令行会有提示
- 请记住该路径，后面修改 extension_dir 会使用到

```
[root@172.16.103.105 php-7.4.16]$ make install
Installing shared extensions:     /usr/lib/php/extensions/no-debug-non-zts-20190902/
Installing PHP CLI binary:        /usr/bin/
Installing PHP CLI man page:      /usr/php/man/man1/
Installing phpdbg binary:         /usr/bin/
Installing phpdbg man page:       /usr/php/man/man1/
Installing PHP CGI binary:        /usr/bin/
Installing PHP CGI man page:      /usr/php/man/man1/
Installing build environment:     /usr/lib/php/build/
Installing header files:          /usr/include/php/
Installing helper programs:       /usr/bin/
  program: phpize
  program: php-config
Installing man pages:             /usr/php/man/man1/
  page: phpize.1
  page: php-config.1
/usr/local/php-7.4.16/build/shtool install -c ext/phar/phar.phar /usr/bin/phar.phar
ln -s -f phar.phar /usr/bin/phar
Installing PDO headers:           /usr/include/php/ext/pdo/
[root@172.16.103.105 php-7.4.16]$
```

执行命令 `php --ini` 找 php.ini 配置文件的位置，Loaded Configuration File 表示该文件位置，如果为 none；则根据 Configuration File (php.ini) Path 指向的目录，将解压出来的安装包中的 php.ini-development 文件复制到该目录下，并修改文件名称为 php.ini 。

```
[root@172.16.103.105 incremental]$ php --ini
Configuration File (php.ini) Path: /usr/lib
Loaded Configuration File:         /usr/lib/php.ini
Scan for additional .ini files in: (none)
Additional .ini files parsed:      (none)
```

若 php.ini 中尚未添加拓展地址，请添加如下拓展地址，该地址将存放拓展的动态链接库文件

```
extension_dir = "/usr/lib/php/extensions/no-debug-non-zts-20190902"
```

修改 php.ini 添加依赖库

```
extension=pgsql

extension=pdo_pgsql
```

执行命令 `php -m` 查看是否存在 pgsql 和 pdo_pgsql 两个模块，存在表示安装拓展成功

```
[root@172.16.103.105 pdo_pgsql]$ php -m
[PHP Modules]
Core
ctype
date
dom
fileinfo
filter
hash
iconv
json
libxml
pcre
PDO
pdo_pgsql
pdo_sqlite
pgsql
Phar
posix
Reflection
session
SimpleXML
SPL
sqlite3
standard
tokenizer
xml
xmlreader
xmlwriter
```

11.3. 使用 PDO 连接数据库

11.3.1. 连接数据库

11.3.1.1. 获取数据库连接

使用 new PDO()获取连接：

```
$conn = new PDO( “pgsql:host=$host;port=$port;dbname=$dbname” , $use

rname, $password);
```

连接参数设置方式：

```
$conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

其它参数请参考 PHP 文档：PHP：连接与连接管理 - Manual

11.3.1.2. 关闭数据库连接

关闭连接时将变量设置为 null 即可：

```
$conn = new PDO( “pgsql:host=$host;port=$port;dbname=$dbname” , $use
rname, $password);
$conn = null;
```

11.3.1.3. 获取持久化连接

必须在实例化时指定该参数，通过实例对象设置参数无效。

```
new PDO(“pgsql:host=$host;port=$port;dbname=$dbname”, $username, $p
assword, array(
PDO::ATTR_PERSISTENT => true
));
```

11.4. 执行 SQL 语句并处理结果

向数据库发出执行 SQL 的请求时，需要使用 cursor 发出查询，服务端执行完成后将返回执行结果。

11.4.1. 直接执行

```
<?php
try {
$conn = new PDO( “pgsql:host=172.16.19.117;port=5432;dbname=panweid
```

```
b" ," pwadmin" ," pwase@admin" );
$rs = $conn->query( "select * from hzy.t" );
if ($rs) {
foreach($rs as $row) {
print_r($row);
}
}
$conn = null;
} catch(Exception $e) {
  print $e->getMessage();
  die();
}
?>
```

执行上面代码结果如下：

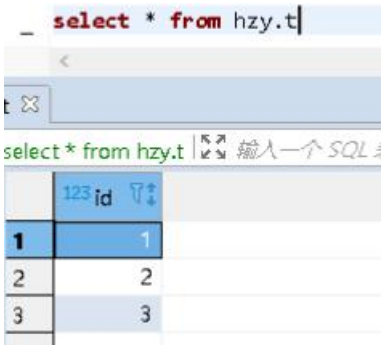
```
Array
(
    [id] => 1
    [0] => 1
)
Array
(
    [id] => 2
    [0] => 2
)
```

11.4.2. 预编译方式执行

调用$conn->prepare($sql)，使用占位符表示 SQL 中的变量，再变量值传入。

```
<?php
try {
$conn = new PDO( “pgsql:host=172.16.19.117;port=5432;dbname=panweid
b” ,” pwadmin” ,” pwase@admin” );
$stmt = $conn->prepare( “insert into hzy.t(id) values(?)” );
$stmt->bindParam( “:id” , $id);
$id = 3;
$stmt->execute();
$conn = null;
} catch(Exception $e) {
  print $e->getMessage();
  die();
}
?>
```

数据库中查询到 3 被成功写入：



11.5. 存储过程

11.5.1. 创建存储过程

创建存储过程和普通 SQL 的执行一致，调用数据库连接实例的 exec()方法即可。

下面的例子展示了如何创建存储过程：

```
<?php
try {
$conn = new PDO( “pgsql:host=172.16.19.117;port=5432;dbname=panweid
b” ,” pwadmin” ,” pwase@admin” );
$sql = " CREATE OR REPLACE PROCEDURE hzy.pro_test(a integer,inout b in
teger) AS begin b := a+b; end;";
$conn->exec($sql);
$conn = null;
} catch(Exception $e) {
  print $e->getMessage();
  die();
```

```
}
?>
```

11.5.2. 调用存储过程

非预编译调用存储过程和执行普通 SQL 步骤一致，不再赘述；预编译调用存储过程和普通 SQL 的过程一致，区别在于设置占位符参数。需通过 bindParam()方法设置占位符参数。第一个参数是占位符的索引位置，第二个参数是变量，若 JDBC 支持可获取到存储过程的输出值(经测试不支持)，第三个参数是存储过程参数对应的类型。

下面的例子展示了如何调用存储过程：

```
<?php
try {
$conn = new PDO( “pgsql:host=172.16.19.117;port=5432;dbname=panweid
b” ,” pwadmin” ,” pwase@admin” );
$stmt = $conn->prepare( “call hzy.pro_test(?,?));
$stmt->bindParam(1, $a, PDO::PARAM_INT);
$stmt->bindParam(2, $b, PDO::PARAM_INT);
$stmt->execute();
$conn = null;
} catch(Exception $e) {
  print $e->getMessage();
  die();
```

```
}
?>
```

11.6. 函数

11.6.1. 创建函数

创建函数和普通 SQL 的执行一致，调用数据库连接实例的 exec()方法即可。

下面的例子展示了如何创建函数：

```
<?php
try {
$conn = new PDO( “pgsql:host=172.16.19.117;port=5432;dbname=panweid
b” ,” pwadmin” ,” pwase@admin” );
$sql = " CREATE OR REPLACE FUNCTION hzy.fun_test(a integer,b integer) r
eturns integer language ‘plpgsql’ as $body$begin return a+b; end$body$;
";
$conn->exec($sql);
$conn = null;
} catch(Exception $e) {
  print $e->getMessage();
  die();
}
?>
```

11.6.2. 调用函数

调用函数和执行普通查询 SQL 步骤一致。

下面的例子展示了如何调用函数：

```
<?php
try {
$conn = new PDO( “pgsql:host=172.16.19.117;port=5432;dbname=panweid
b” ,” pwadmin” ,” pwase@admin” );
$sql = "select hzy.fun_test(1,6)";
$rs = $conn->query($sql);
foreach($rs as $row) {
print_r($row);
}
$conn = null;
} catch(Exception $e) {
  print $e->getMessage();
  die();
}
?>
```

11.7. 处理大数据类型

11.7.1. 二进制类型

PanWeiDB V2.0-S3.0.0_Beta 提供两种不同的方法存储二进制数据。 二进制数据可以使用二进制数据类型 BYTEA 存储在表中，或者使用大对象特性以一种特殊的格式将二进制数据存储在一个独立的表中， 然后通过在表中保存一个类型为 OID 的值来引用该表。

11.7.1.1. BYTEA 插入记录

插入二进制数据，需将二进制数据转为二进制流，该流作为第二个参数传给 bindParam()方法，并且指定数据类型为 PDO::PARAM_LOB。

使用 BYTEA 类型存储二进制数据，例如：

```
<?php
try {
$conn = new PDO( “pgsql:host=172.16.19.117;port=5432;dbname=panweid
b” ,” pwadmin” ,” pwase@admin” );
$sql = "insert into hzy.t2 values(?)";
$fp = fopen( “/tmp/a.png” , “rb” );
$stmt->bindParam(1, $fp, PDO::PARAM_LOB);
$stmt->execute();
$conn = null;
} catch(Exception $e) {
```

```
  print $e->getMessage();
  die();
}
?>

// 最终可以从数据库中查询到图片成功插入
```

11.7.1.2. BYTEA 读取记录

查询二进制数据类型字段数据时，先预编译 SQL 后执行，再通过 bindColumn()方法绑定列，调用 fetch(PDO::FETCH_BOUND)将数据转成二进制流到 bindColumn 方法的第二个参数中。

```
<?php
try {
$conn = new PDO( “pgsql:host=172.16.19.117;port=5432;dbname=panweid
b” ,” pwadmin” ,” pwase@admin” );
$sql = "select bytea_field from hzy.t2";
$stmt->prepare($sql);
$stmt->execute();
$stmt->bindColumn(1, $bytea, PDO::PARAM_LOB);
$stmt->fetch(PDO::FETCH_BOUND);
file_put_contents( “/tmp/b.png” , $bytea, true);
```

```
$conn = null;
} catch(Exception $e) {
  print $e->getMessage();
  die();
}
?>

// 最终可以在/tmp 目录下查看到读取出来的图片 b.png
```

11.7.2. 字符类型

PanWeiDB V2.0-S3.0.0_Beta 中 TEXT 类型与 VARCHAR 类型都是可变长的字符类型，区别在于 VARCHAR 类型通过 VARCHAR(n)中的 n 来限制最大长度，而 TEXT 类型没有。TEXT 类型与 VARCHAR 类型几乎没有性能差别，TEXT 类型最多可存储 1G 数据 。

使用 TEXT 类型存储数据时，可用以下方式来进行读写。

例如：

```
CREATE TABLE images (id int, msg text);
```

12. Python 接口开发指南

12.1. 介绍

Python 语言通过 psycopg2 模块访问 PanWeiDB V2.0-S3.0.0_Beta 数据库，psycopg2 模块为使用 Python 语言的应用程序定义了一组访问和操作 PanWeiDB V2.0-S3.0.0_Beta 数据库的标准接口。

12.2. 安装 psycopg2 模块

安装 psycopg2 模块前需要先装好 PanWeiDB V2.0-S3.0.0_Beta Libpq，并配置环境变量。

12.2.1. 安装 PanWeiDB V2.0-S3.0.0_Beta Libpq

12.2.1.1. 通过安装 PanWeiDB V2.0-S3.0.0_Beta 安装包安装

按照安装说明安装 PanWeiDB V2.0-S3.0.0_Beta，并配置环境变量。如果是使用安装包自带安装程序进行安装，则环境变量会自动配置无需手动配置；如果是手动安装数据库，则需配置以下环境变量（只作为参数，应按实际的环境进行配置）：

```
export GAUSSHOME=/usr/local/panweidb

export PATH=$GAUSSHOME/bin:$PATH

export LD_LIBRARY_PATH=$GAUSSHOME/lib:$LD_LIBRARY_PATH
```

12.2.2. 安装 python3 和 gcc

推荐使用 python3，安装命令如下：

```
yum install -y python3 python3-devel gcc
```

12.2.3. 安装 psycopg2 模块

推荐使用源码包安装，获取 psycopg2 源码包：

```
# 1.卸载已安装的 psycopg2（如果没有安装则跳过，此步骤可能需要 root 用户执行）
pip3 uninstall psycopg2 -y

# 2.上传并解压源码包
unzip Psycopg-panwei.zip
# 3.安装
cd Psycopg-panwei
python3 setup.py build
# 此步骤可能需要 root 用户执行
python3 setup.py install

# 4.检查安装结果
进入 python3 命令行，执行 import psycopg2 可以正常导入即表示安装成功。
```

12.2.4. psycopg2 对 python 的版本要求

psycopg2 1.0 支持 python 3

12.3. 使用 psycopg2 连接数据库

12.3.1. 导入

任何使用 psycopg2 的程序都需要导入 psycopg2 模块。

```
import psycopg2
```

12.3.2. 连接数据库

12.3.2.1. 获取数据库连接

使用 psycopg2.connect()获取连接：

```
conn = psycopg2.connect(database="postgres", user="panweidb", passwor
d="Aa@123456", host="127.0.0.1", port="5432")
```

12.3.2.2. 关闭数据库连接

关闭连接时调用 connect 的 close()方法即可：

```
conn = psycopg2.connect(database="postgres", user="panweidb", passwor
d="Aa@123456", host="127.0.0.1", port="5432")
conn.close()
```

12.4. 执行 SQL 语句并处理结果

向数据库发出执行 SQL 的请求时，需要使用 cursor 发出查询，服务端执行完成后将返回执行结果。

12.4.1. 执行插入

12.4.1.1. 直接执行

调用 conn.cursor()函数获取句柄，调用 cursor.execute 函数执行插入语句：

```
cursor = conn.cursor()

cursor.execute("insert into tb_test(id, name) values(1, 'Anna')")
```

12.4.1.2. 预编译方式执行

通过在 SQL 语句中使用%s 占位符，并将一个值序列作为函数的第二个参数，向 SQL 语句传递参数，例如：

```
cursor = conn.cursor()

name = 'Jack'

cursor.execute("insert into tb_test(id, name) values(2, %s)",(name,))
```

12.4.2. 使用游标获取结果

可以通过以下几种方式获取结果。

12.4.2.1. 遍历游标

该方法将顺序遍历获取结果集中所有结果。

```
cursor.execute("select * from tb_test")

for record in cursor:

# do something
```

12.4.2.2. cur.fetchone()

该方法将获取结果集中下 1 行结果，如果结果集中没有后续数据，则返回空。

```
cursor.execute("select * from tb_test")

record = cursor.fetchone()
```

12.4.2.3. cur.fetchall()

该方法将一次获取结果集中所有结果。

```
cursor.execute("select * from tb_test")

records = cursor.fetchall()

for record in records:

# do something
```

12.4.2.4. cur.fetchmany([size=fetchsize])

该方法将获取结果集中后续 fetchsize 行结果，如果不足则返回剩下的所有行，如果结果集中没有后续数据，则返回空。当结果集数量庞大时，建议指定 fetchsize 分批获取。

```
cursor.execute("select * from tb_test")
records = cursor.fetchmany(10)
for record in records:
# do something
```

12.4.3. 执行更新

要更改数据(执行一个 insert, update 或者 delete)，可以使用 cur.execute 函数执行完整的 SQL，也可以使用预编译的方式绑定参数后再执行。下面的示例将以预编译的方式执行。

12.4.3.1. 插入

```
cursor.execute("insert into tb_test(id, name) values(%s,%s)", (2, 'Taxol',))
```

12.4.3.2. 更新

```
cursor.execute("update tb_test set name = %s where id = %s" , ('Elsa',
1,))
```

12.4.3.3. 删除

```
cursor.execute("delete from tb_test where id = 2")
```

12.4.4. 创建或修改数据库对象

要创建、更改或者删除一个类似表或者视图这样的数据库对象， 可以调用 cursor.execute() 方法。

```
cursor.execute("create table tb_test(id integer, name text)")

cursor.execute("create view v_tb_test as select * from tb_test")
```

12.4.5. 执行存储过程

下面的例子展示了如何调用存储过程。

```
sql_procedure = " CREATE OR REPLACE PROCEDURE pro_test(a integer,inou
t b integer) as \
                  begin \
                      b := a+b; \
                  end"
cursor.execute(sql_procedure)
cursor.execute("call pro_test(1,5)")
result = cursor.fetchone()
```

下面的例子展示了如何调用函数。

```
sql_function = "CREATE OR REPLACE FUNCTION func_test (a int,out b int)
RETURNS int AS \
               $$ \
```

```
                begin \
                    b := a+2; \
                return ;  \
                end; \
                $$ LANGUAGE 'plpgsql';"
cursor.execute(sql_function)
cursor.execute("select func_test(1)")
result = cursor.fetchone()
```

12.4.6. 处理数据类型

12.4.6.1. 二进制类型

PanWeiDB V2.0-S3.0.0_Beta 提供两种不同的方法存储二进制数据。 二进制数据可以使用二进制数据类型 BYTEA 存储在表中，或者使用大对象特性以一种特殊的格式将二进制数据存储在一个独立的表中， 然后通过在表中保存一个类型为 OID 的值来引用该表。以下为这两种方法的使用示例。

- 使用 BYTEA 类型存储

1) 创建表

```
cursor.execute("create table lo_test_tab (id int, img bytea)")
```

2) 插入数据

```
with open("/data/test.data", 'rb') as file:
data = file.read()
```

```
cursor.execute("insert into lo_test_tab values(1, %s)", (psycopg2.Binary
(data),)
```

3) 读取数据

```
cursor.execute("select img from lo_test_tab where id = 1")
record = cursor.fetchone()
byteadata = record[0]
print(byteadata.tobytes())
```

❖ 使用 OID 存储

1) 创建表

```
cursor.execute("create table lo_test_tab (name text, img oid)")
```

2) 导入数据

该步骤会将操作系统文件“image”导入成一个大对象并生成一个 oid，查询 lo_test_tab 表时则显示 oid。

```
cursor.execute("insert into lo_test_tab values('beautiful image', lo_impo
rt('/tmp/image'))")
```

3) 导出数据

该操作将数据库中存储的大对象导出到一个操作系统文件。

```
cursor.execute("select lo_export(lo_test_tab.img, '/tmp/image-bak') fro
m lo_test_tab where name = 'beautiful image'")
```

12.4.6.2. 字符类型

PanWeiDB V2.0-S3.0.0_Beta 中 TEXT 类型与 VARCHAR 类型都是可变长的字符类型，区别在于 VARCHAR 类型通过 VARCHAR(n)中的 n 来限制最大长度，而 TEXT 类型没有。TEXT 类型与 VARCHAR 类型几乎没有性能差别，TEXT 类型最多可存储 1G 数据 。

使用 TEXT 类型存储数据时，可用以下方式来进行读写。

1) 创建表

```
cursor.execute("create table images (id int, msg text)")
```

2) 插入数据

```
cursor.execute("insert into  images  values(1, %s)", ("the text data",))
```

3) 读取数据

```
cursor.execute("select msg from images where id = 1")

record = cursor.fetchone()

byteadata = record[0]
```

12.4.6.3. 数字类型

数字类型有整数类型、任意精度数字类型、浮点类型和序数类型。下面用整数类型 integer、任意精度数字类型 numeric、 浮点类型 real 和 double precision 举例说明。

❖ 创建表

➢ 整数类型 integer 的建表

```
cursor.execute("create table test_int(Num int)")
```

- 任意精度数字类型 numeric 的建表

```
cursor.execute("create table test_numeric(Num numeric)")
```

- 浮点类型 real 和 double precision 的建表

```
cursor.execute("create table test_real(Num real)")

cursor.execute("create table test_doubleprecision(Num double precision)

")
```

❖ 插入数据

- 整数类型 integer 的插入

```
cursor.execute("insert into  test_int values(%s)" ,(1,))
```

- 任意精度数字类型 numeric 的插入

```
cursor.execute("insert into  test_numeric values(%s)" ,(123.456,))
```

- 浮点类型 real 和 double precision 的插入

```
# real 类型

cursor.execute("insert into  test_real values(%s)" ,(3.1234,))

# double precision 类型

cursor.execute("insert into  test_doubleprecison values(%s)" ,(14.1234,))
```

❖ 读取数据

- 整数类型 integer 的读取

```
cursor.execute("select Num from test_int")

record = cursor.fetchone()

Num = record[0]
```

- 任意精度数字类型 numeric 的读取

```
cursor.execute("select Num from test_numeric")
record = cursor.fetchone()
Num = record[0]
```

- 浮点类型 real 和 double precision 的读取

```
# real 类型
cursor.execute("select Num from test_real")
record = cursor.fetchone()
Num = record[0]
# double precision 类型
cursor.execute("select Num from test_doubleprecision")
record = cursor.fetchone()
Num = record[0]
```

12.4.6.4. 时间/日期类型

日期/时间类型有 timestamp 、date、time 、interval ； 下面以 timestamp 、time 举例说明。

- 创建表

```
# timestamp 类型
cursor.execute("create table test_timestamp(dt timestamp)")
# time 类型
cursor.execute("create table test_time(dt time)")
```

❖ 插入数据

```
# timestamp 类型
cursor.execute("insert into test_timestamp values(%s)" ,(datetime.datetime
(2018,10,1,10,30,30),))
# time 类型
cursor.execute("insert into test_time values(%s)" ,(datetime.time(10,30,30),))
```

❖ 读取数据

```
# timestamp 类型
cursor.execute("select dt from test_timestamp")
record = cursor.fetchone()
dt= record[0]
# time 类型
cursor.execute("select dt from test_time")
record = cursor.fetchone()
dt= record[0]
```

12.5. 附录 1：主要接口

psycopg2 模块提供建立和管理与 PanWeiDB V2.0-S3.0.0_Beta 数据库连接、执行 SQL 语句和对结果集进行储存管理的若干接口。主要接口说明如下：

❖ psycopg2.connect()

此方法用于创建新的数据库会话并返回新的 connection 对象。参数说明如下：

关键字	参数说明
dbname	数据库名称
user	用户名
password	密码
host	数据库所在主机 IP 地址
port	连接端口号，默认为 5432
sslmode	ssl 模式，ssl 连接时用

sslmode 为非必要参数。

返回值：connection 对象（连接 PanWeiDB V2.0-S3.0.0_Beta 数据库实例的对象）。

调用示例见附录 2。

❖ connection.cursor()

此方法用于返回新的 cursor 对象。参数说明如下：

关键字	参数说明
name	cursor 名称，默认为 None
cursor_factory	用于创造非标准 cursor，默认为 None
withhold	设置 HOLD 选项，默认为 False

返回值为 cursor 对象（用于整个数据库使用 Python 编程的 cursor）。

调用示例见附录 2。

❖ cursor.execute()

此方法执行被参数化的 SQL 语句（即占位符，而不是 SQL 文字）。psycopg2 模块支持用%s 标志的占位符。参数说明如下：

关键字	参数说明
query	待执行的 sql 语句
vars_list	变量列表，匹配 query 中%s 为占位符

返回值：无。

调用示例见附录 2。

❖ connection.commit()

此方法将当前挂起的事务提交到数据库。默认情况下，psycopg2 在执行第一个命令之前打开一个事务：如果不调用 commit()，任何数据操作的效果都将丢失。

参数：无。

返回值：无。

调用示例见附录 2。

❖ connection.rollback()

此方法回滚当前挂起事务。

参数：无。

返回值：无

调用示例见附录 2。

❖ cursor.fetchone()

此方法提取查询结果集的下一行，并返回一个元组。

参数：无。

返回值：单个元组，为结果集的第一条结果，当没有更多数据可用时，返回为“None”。

调用示例见附录 2。

❖ cursor.fetchall()

此方法获取查询结果的所有（剩余）行，并将它们作为元组列表返回。

参数：无。

返回值：元组列表，为结果集的所有结果。空行时则返回空列表。

调用示例见附录 2。

❖ cursor.close()

此方法用于关闭当前连接的游标。

参数：无。

返回值：无。

调用示例见附录 2。

❖ connection.close()

此方法用于关闭数据库连接。

参数：无。

返回值：无。

调用示例见附录 2。

12.6. 附录 2：调用示例

```
import os

import sys

import psycopg2
```

```
#数据库操作
def connectpanweidb():
    #连接数据库
    conn = psycopg2.connect(database="postgres", user="panweidb2", pas
sword="Aa@123456", host="127.0.0.1", port="25432")
    print('connect successful!')
    cursor = conn.cursor()
    #创建表
    cursor.execute("create table tb_test(id integer not null primary key, n
ame text)")
    print('create table successful!')
    #插入数据
    cursor.execute("insert into tb_test(id, name) values(1, 'Anna')")
    cursor.execute("insert into tb_test(id, name) values(%s, %s)" ,(2, 'Jack
',))
    conn.commit()
    print('insert successful!')
    #查询数据
    cursor.execute("select * from tb_test")
    #fetchall
    records = cursor.fetchall()
    for record in records:
```

```
        print('fetchall: id = ' + str(record[0]) + ', name = ' + str(record[1]))
    #fetchone
    cursor.execute("select * from tb_test")
    record = cursor.fetchone()
    print('fetchone: id = ' + str(record[0]) + ', name = ' + str(record[1]))
    #fetchmany
    cursor.execute("select * from tb_test")
    records = cursor.fetchmany(10)
    for record in records:
        print('fetchmany: id = ' + str(record[0]) + ', name = ' + str(record
[1]))
    #rollback
    cursor.execute("insert into tb_test(id, name) values(3, 'Mike')")
    conn.rollback()
    cursor.execute("select * from tb_test")
    records = cursor.fetchall()
    for record in records:
        print('atfer rollback: id = ' + str(record[0]) + ', name = ' + str(reco
rd[1]))
    #更新数据
    cursor.execute("update tb_test set name = %s where id = 1" ,('Elsa',))
    print('update successful!')
    cursor.execute("select * from tb_test")
```

```
    records = cursor.fetchall()
    for record in records:
        print('after update: id = ' + str(record[0]) + ', name = ' + str(recor
d[1]))
    #删除数据
    cursor.execute("delete from tb_test where id = 2")
    conn.commit()
    cursor.execute("select * from tb_test")
    records = cursor.fetchall()
    for record in records:
        print('after delete: id = ' + str(record[0]) + ', name = ' + str(record
[1]))
    cursor.close()
    conn.close()

if __name__=='__main__':
    connectpanweidb()
```

13. soci 接口开发指南

13.1. 介绍

soci 提供 C++接口访问 PanWeiDB V2.0-S3.0.0_Beta 数据库，soci 为使用 C++语言的应用程序定义了一组访问和操作 PanWeiDB V2.0-S3.0.0_Beta 数据库的标准接口。

13.2. Windows 搭建 soci 开发环境

13.2.1. 安装 Visual Studio

测试环境安装的是 Visual Studio Enterprise 2015（其他版本的 Visual Studio 也可以）。

【注意】
需要自定义安装，勾选 Windows8.1 的 SDK。

13.2.2. 安装 cmake

在 https://cmake.org/download/上下载。

建议选择 Latest Release 版本，下载后按照安装程序的提示安装即可。

13.2.3. 安装 PostgreSQL

在 https://www.enterprisedb.com/downloads/postgres-postgresql-downloads 上下载。

PostgreSQL 版本需要>=7.3，下载后按照安装程序的提示安装即可。

13.2.4. 下载 soci 源码

在 https://github.com/SOCI/soci 上下载。

建议选择 release 最新版本（当前为 release/4.0），下载 zip 包并解压到自选目录即可。

13.2.5. 编译 soci

13.2.5.1. cmake 生成 visual studio 工程

打开 VS2015 的 MSBuild 命令提示符，切到 soci 源码所在目录。

然后输出下面命令：

```
 cmake ./  -DWITH_POSTGRESQL=ON -DPOSTGRESQL_INCLUDE_DIR=...（libpq-fe.h 文件所在目录）-DPOSTGRESQL_LIBRARY=...(libpq.lib 文件所在的绝对路径）
```

测试环境为：

```
 cmake ./  -DWITH_POSTGRESQL=ON -DPOSTGRESQL_INCLUDE_DIR="C:\Program Files (x86)\PostgreSQL\10\include"  -DPOSTGRESQL_LIBRARY=" C:\Program Files (x86)\PostgreSQL\10\lib\libpq.lib"
```

13.2.5.2. msbuild 编译

输入下面命令（如果需要 Release 版本则将 Debug 改成 Release）：

```
msbuild SOCI.sln /t:build /p:Configuration=Debug
```

编译生成的的 dll 文件在 bin/Debug 目录下，lib 文件在 lib/Debug 目录下。

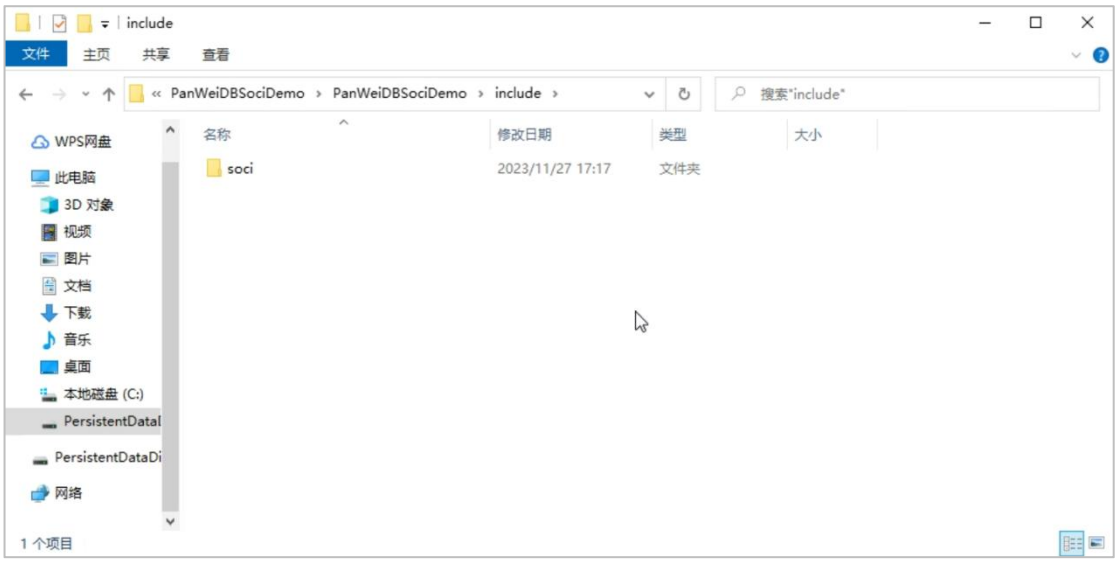
13.2.6. Demo 工程引入 soci

13.2.6.1. 创建 Demo 工程

打开 Visual Studio 新建项目，选择 Visual C++/Win32 控制台应用程序，修改名称为 panweidbSociDemo，点击确定，完成 Demo 工程的创建。

13.2.6.2. 引入头文件

创建 include 目录，将 soci 的头文件放入。



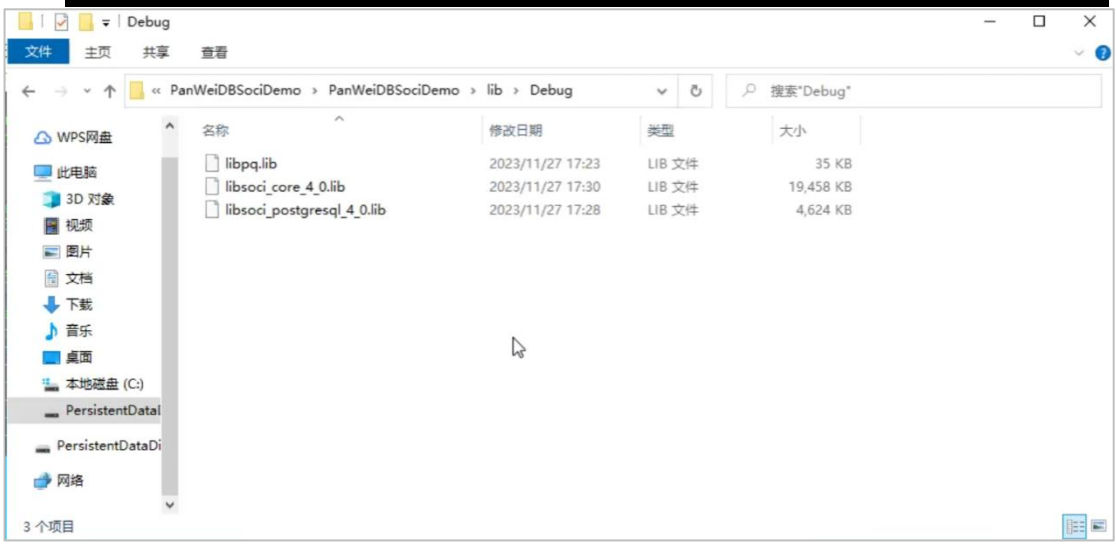
13.2.6.3. 引入 lib 文件

创建 lib 目录，并在 lib 目录中创建 Debug 和 Release 两个目录，分别将对应版本的 soci 和 PostgreSQL 的 lib 文件放入，有

```
libsoci_core_4_0.lib

libsoci_postgresql_4_0.lib

libpq.lib（该文件在 PostgreSQL 的安装路径下的 lib 目录中）
```



13.2.6.4. 修改包含目录

工程属性页中包含目录中添加：

```
$(SolutionDir)$(MSBuildProjectName)\include

C:\Program Files (x86)\PostgreSQL\10\include（PostgreSQL 的安装路径下的 include 目录）
```

13.2.6.5. 修改附加库目录

工程属性页中附加库目录中添加：

```
$(SolutionDir)$(MSBuildProjectName)\lib\$(Configuration)
```

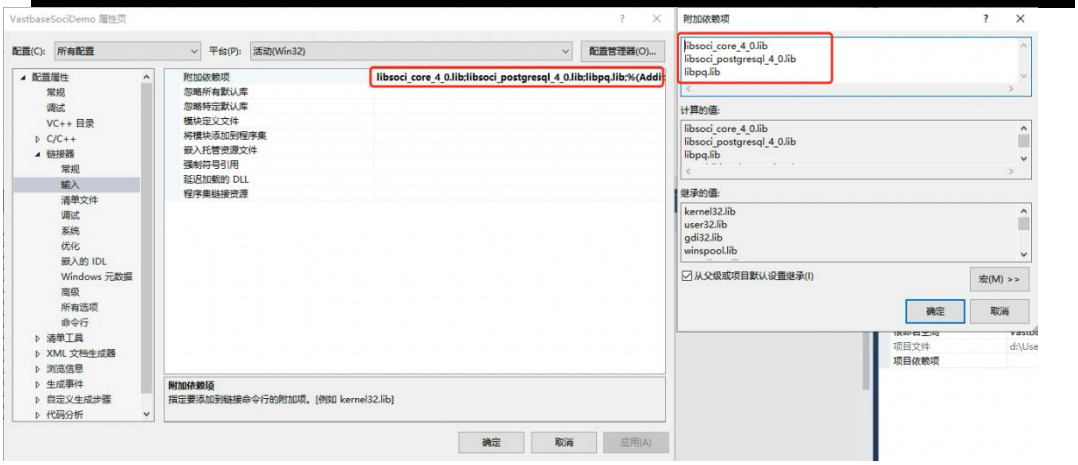
13.2.6.6. 修改附加依赖项

工程属性页中附加依赖项中添加：

```
libsoci_core_4_0.lib

libsoci_postgresql_4_0.lib

libpq.lib
```



13.2.6.7. 添加 dll 文件

将所需 dll 文件（可在 PostgreSQL 安装目录下的 bin 目录下找到）放入 exe 文件生成目录下：

```
libcrypto-1_1.dll

libiconv-2.dll

libintl-8.dll

libpq.dll

libssl-1_1.dll
```

13.3. 安装 PanWeiDB V2.0-S3.0.0_Beta

按照安装说明安装 PanWeiDB V2.0-S3.0.0_Beta，并配置环境变量。如果是使用安装包自带安装程序进行安装，则环境变量会自动配置无需手动配置；如果是手动安装数据库，则需配置以下环境变量（只作为参数，应按实际的环境进行配置）：

```
export GAUSSHOME=/usr/local/panweidb

export PATH=$GAUSSHOME/bin:$PATH

export LD_LIBRARY_PATH=$GAUSSHOME/lib:$LD_LIBRARY_PATH
```

注意
禁止与初始用户建立远程连接

13.4. 使用 soci::session 连接数据库

13.4.1. 导入

任何使用 soci::session 的程序都需要引入相应头文件。可使用 soci 命名空间

```
#include <soci/soci.h>

#include <soci/postgresql/soci-postgresql.h>


using namespace soci;
```

13.4.2. 连接数据库

13.4.2.1. 获取数据库连接

```
soci::session sql;
/*
  dbname:数据库名称
  user:数据库用户名称
  password:数据库用户密码
  host:远端访问 ip
*/
sql.open(soci::postgresql, "dbname=postgres user=panweidb_remote pass
word=Aa@123456 host=172.16.12.234");
```

13.4.2.2. 关闭数据库连接

关闭连接时调用 sql 的 close()方法即可:

```
sql.close();
```

13.5. 执行 SQL 语句并处理结果

向数据库发出执行 SQL 的请求时，服务端执行完成后将返回执行结果。

13.5.1. 执行查询

13.5.1.1. 直接执行

调用 sql<<执行插入语句：

```
sql<<"insert into tb_test(id, name) values(1,'Anna')";
```

13.5.1.2. 预编译方式执行

调用 sql<<执行插入语句，使用冒号占位符表示 SQL 中的变量，再调用 soci::use 将变量值传入：

```
std::string name = "Jack";

sql<<"insert into tb_test(id, name) values(2,:name)",use(name);
```

13.5.2. 使用 sql.prepare 获取多行结果

```
rowset<row> rs = (sql.prepare << "select * from tb_test");


for(rowset<row>::iterator it = rs.begin();it != rs.end();it++){
  const row& r = *it;
  cout << "id : " << r.get<int>(0)
       << " name : " << r.get<string>(1)
       << endl;
}
```

13.5.3. 执行更新

要更改数据(执行一个 insert, update 或者 delete), 可以使用 cur.execute 函数执行完整的 SQL, 也可以使用预编译的方式绑定参数后再执行。下面的示例将以预编译的方式执行。

13.5.3.1. 插入

```
sql<<"insert into tb_test values(:id,:name)",use(2),use(string("Jack"));
```

13.5.3.2. 更新

```
sql<<"update tb_test set name = :name where id = 1",use(string("Elsa"));
```

13.5.3.3. 删除

```
sql<<"delete from tb_test where id = 1";
```

13.5.4. 创建或修改数据库对象

要创建、更改或者删除一个类似表或者视图这样的数据库对象， 可以通过 sql<<来做。

```
    sql<<"create table tb_test(id integer, name text)";

    sql<<"create view v_tb_test as select * from tb_test";
```

13.5.5. 执行存储过程

下面的例子展示了如何调用存储过程。

```
string sql_procedure = "CREATE OR REPLACE PROCEDURE pro_test(a intege
r,inout b integer) AS "
      "begin b := a+b;end;";
sql<<sql_procedure;
sql<<"call pro_test(1,5)";
```

下面的例子展示了如何调用函数。

```
string sql_function = "CREATE OR REPLACE FUNCTION func_test (a int,out
b int) RETURNS int AS "
  "$$  begin b := a+2;return;  end;"
  "$$ LANGUAGE 'plpgsql';";
sql<<sql_function;
sql<<"select func_test(1)";
```

13.5.6. 处理大数据类型

13.5.6.1. 二进制类型

PanWeiDB V2.0-S3.0.0_Beta 提供两种不同的方法存储二进制数据。 二进制数据可以使用二进制数据类型 BYTEA 存储在表中，或者使用大对象特性以一种特殊的格式将二进制数据存储在一个独立的表中， 然后通过在表中保存一个类型为 OID 的值来引用该表。以下为这两种方法的使用示例。

❖ 使用 BYTEA 类型存储

1） 创建表

```
sql<<"create table lo_test_tab(id int, img bytea)";
```

2） 插入数据

```
int v = 0x0A0B0C0D;
unsigned char* b = reinterpret_cast<unsigned char*>(&v);
std::string data;
std::copy(b, b + sizeof(v), std::back_inserter(data));
sql << "insert into lo_test_tab values(1, :data)", use(data);
```

3） 读取数据

```
std::string byteadata;
sql << "select img from lo_test_tab where id = 1",into(byteadata);
cout << byteadata << endl;
```

❖ 使用 OID 存储

1） 创建表

```
sql<<"create table lo_test_tab (name text, img oid)";
```

2） 导入数据

该步骤会将文件“image”导入成一个大对象并生成一个 oid，查询 lo_test_tab 表时则显示 oid。

```
sql<< "insert into lo_test_tab values('beautiful image',:image)",use(lo_import(((postgresql_session_backend *)(sql.get_backend()))->conn_,"D://image
```

```
"));
```

3) 导出数据

该操作将数据库中存储的大对象导出为一个新文件。

```
Oid oid;
sql << "select img from lo_test_tab where name = 'beautiful image'",into
(oid);
lo_export(((postgresql_session_backend *)(sql.get_backend()))->conn_, oid,
"D://image-bak");
```

13.5.6.2. 字符类型

PanWeiDB V2.0-S3.0.0_Beta 中 TEXT 类型与 VARCHAR 类型都是可变长的字符类型，区别在于 VARCHAR 类型通过 VARCHAR(n)中的 n 来限制最大长度，而 TEXT 类型没有。TEXT 类型与 VARCHAR 类型几乎没有性能差别，TEXT 类型最多可存储 1G 数据 。

使用 TEXT 类型存储数据时，可用以下方式来进行读写。

1) 创建表

```
sql<<"create table images (id int, msg text)";
```

2) 插入数据

```
string text = "the text data";
sql<<"insert into images values(1, :text)",use(text);
```

3) 读取数据

```
string msg;
sql<<"select msg from images where id = 1",into(msg);
cout << msg << endl;
```

13.6. 附录 1：主要接口

soci 提供建立和管理与 PanWeiDB V2.0-S3.0.0_Beta 数据库连接、执行 SQL 语句和对结果集进行储存管理的若干接口。主要接口说明如下：

- session::open(backend_factory const &factory,std::string const &connectString)

此方法用于创建新的数据库会话。参数说明如下：

关键字	参数说明
factory	后端工厂，PostgreSQL 使用 soci::postgresql
connectString	连接参数字符串，举例为"dbname=postgres user=panweidb_remote password=Aa@123456 host=172.16.12.234 port=5432" dbname：数据库名称 user：用户名 password：密码 host：数据库所在主机 IP 地址 port：连接端口号，默认为 5432

返回值：无。

调用示例见附录 2。

❖ sql<<statement,use_list;

此方法执行被参数化的 SQL 语句（即占位符，而不是 SQL 文字）。soci 支持用冒号编写的占位符。参数说明如下：

关键字	参数说明
statement	SQL 语句
use_list	变量列表，用 use 表达式将 SQL 占位符（用冒号编写）与本地数据关联起来

返回值：无。
调用示例见附录 2。

❖ sql<<statement,into_list;

此方法执行 SQL 语句，获取结果值。参数说明如下：

关键字	参数说明
statement	SQL 语句
into_list	变量列表，用 into 表达式将结果与本地数据关联起来

返回值：无。

调用示例见附录 2。

❖ transaction::rollback()

此方法回滚当前挂起事务。

参数：无。

返回值：无。

调用示例见附录 2。

❖ (session)(sql).prepare<<statement

此方法获取查询结果集。

参数说明如下：

关键字	参数说明
statement	SQL 语句

返回值：rowset<row>类型值。

调用示例见附录 2。

❖ session::close()

此方法用于关闭该连接。

参数：无。

返回值：无。

调用示例见附录 2。

13.7. 附录 2：调用示例

```
#include "stdafx.h"

#include <string>

#include <iostream>

#include <soci/soci.h>

#include <soci/postgresql/soci-postgresql.h>


using namespace soci;
```

```
using namespace std;


int main(){
  session sql;
  try{
    sql.open(postgresql, "dbname=postgres user=panweidb_remote passw
ord=Aa@123456 host=172.16.12.234");
    cout << "connect successful!" << endl;
    //删除表
    sql << "drop table if exists tb_test";
    //创建表
    sql << "create table tb_test(id integer not null primary key, name text)
";
    cout << "create table successful!" << endl;
    //插入数据
    sql << "insert into tb_test values (1, 'Anna')";
    sql << "insert into tb_test values (:id, :name)", use(2), use((string)"Jac
k");
    cout << "insert successful!" << endl;
    //查询数据
    rowset<row> rs = (sql.prepare << "select * from tb_test");


    for(rowset<row>::iterator it = rs.begin();it != rs.end();it++){
      const row& r = *it;
```

```
        cout << "id : " << r.get<int>(0)
          << " name : " << r.get<string>(1)
          << endl;
    }

    //rollback
    transaction tr(sql);
    sql << "insert into tb_test values (3, 'Mike')";
    tr.rollback();

    rs = (sql.prepare << "select * from tb_test");
    for(rowset<row>::iterator it = rs.begin();it != rs.end();it++){
        const row& r = *it;
        cout << "after rollback: id : " << r.get<int>(0)
          << " name : " << r.get<string>(1)
          << endl;
    }
    //更新数据
    sql << "update tb_test set name = :name where id = 1",use((string)"El
sa");
    cout << "update successful!" << endl;
    rs = (sql.prepare << "select * from tb_test");
    for(rowset<row>::iterator it = rs.begin();it != rs.end();it++){
        const row& r = *it;
```

```
        cout << "after update: id : " << r.get<int>(0)
          << " name : " << r.get<string>(1)
          << endl;
      }
      //删除数据
      sql << "delete from tb_test where id = 1";
      rs = (sql.prepare << "select * from tb_test");
      for(rowset<row>::iterator it = rs.begin();it != rs.end();it++){
        const row& r = *it;
        cout << "after delete: id : " << r.get<int>(0)
          << " name : " << r.get<string>(1)
          << endl;
      }
    }
    catch (postgresql_soci_error const & e)
    {
      cerr << "PostgreSQL error: " << e.sqlstate() << " " << e.what() << end
l;
    }
    catch (exception const & e)
    {
      cerr << "Some other error: " << e.what() << endl;
    }
```

```
sql.close();


return 0;

  }
```

14. SQLAlchemy 框架开发指南

14.1. 介绍

SQLAlchemy 是 Python SQL 工具包和对象关系映射器，它为应用程序开发人员提供了 SQL 的全部功能和灵活性。

它提供了一整套著名的企业级持久性模式，专为高效和高性能的数据库访问而设计，并改编为简单的 python 域语言。

14.2. 安装 SQLAlchemy

安装 SQLAlchemy 需要安装 SQLAlchemy 包和数据库对应的 python 语言驱动 psycopg2。

14.2.1. 安装 psycopg2

安装 psycopg2 模块前需要先装好 PanWeiDB V2.0-S3.0.0_Beta Libpq，并配置环境变量。

14.2.1.1. 安装 PanWeiDB V2.0-S3.0.0_Beta Libpq

安装 Libpq 有两种方式。

- 通过提供的 Libpq 安装包安装

解压 PanWeiDB V2.0-S3.0.0_Beta 提供的 Libpq，新建/usr/local/libpq 目录，将解压后的 lib 目录拷贝到/usr/local/libpq 目录下，配置环境变量如下：

```
export LD_LIBRARY_PATH=/usr/local/libpq/lib:$LD_LIBRARY_PATH
```

❖ 通过安装 PanWeiDB V2.0-S3.0.0_Beta 安装包安装

按照安装说明安装 PanWeiDB V2.0-S3.0.0_Beta，并配置环境变量。如果是使用安装包自带安装程序进行安装，则环境变量会自动配置无需手动配置；如果是手动安装数据库，则需配置以下环境变量（只作为参数，应按实际的环境进行配置）：

```
export GAUSSHOME=/usr/local/panweidb

export PATH=$GAUSSHOME/bin:$PATH

export LD_LIBRARY_PATH=$GAUSSHOME/lib:$LD_LIBRARY_PATH
```

14.2.1.2. 安装 python3 和 gcc

推荐使用 python3，安装命令如下：

```
yum install -y python3 python3-devel gcc
```

14.2.1.3. 安装 psycopg2 模块

推荐使用源码包安装，获取 psycopg2 源码包：

```
# 1.卸载已安装的 psycopg2（如果没有安装则跳过，此步骤可能需要 root 用户执行）

pip3 uninstall psycopg2 -y


# 2.上传并解压源码包

unzip Psycopg-panwei.zip
```

```
# 3.安装

cd Psycopg-panwei

python3 setup.py build

# 此步骤可能需要 root 用户执行

python3 setup.py install


# 4.检查安装结果

进入 python3 命令行，执行 import psycopg2 可以正常导入即表示安装成功。
```

14.2.2. 安装 SQLAlchemy

安装至本地可以通过命令：

```
pip install SQLAlchemy
```

14.3. 使用 SQLAlchemy 连接数据库

14.3.1. 导入

任何使用 SQLAlchemy 的程序都需要导入 SQLAlchemy 包。

```
from sqlalchemy import *
```

14.3.2. 连接数据库

连接数据库使用 create_engine() 函数，会生成一个 Engine 基于 URL 的。这些 URL 如下 RFC-1738 形式，通常可以包括用户名、密码、主机名、数据库名以及用于其他配置的可选关键字参数。在某些情况下，接受文件路径，而

在其他情况下，“数据源名称”替换“主机”和“数据库”部分。数据库 URL 的典型形式为：

```
dialect+driver://username:password@host:port/database
```

dialect 名称包括 SQLAlchemy 支持的数据库的标识名，如 sqlite，mysql，postgresql，oracle 或 mssql。drivername 是要使用所有小写字母连接到数据库的 DBAPI 的名称。如果未指定，则将导入“默认”DBAPI（如果可用）-此默认值通常是该后端可用的最广泛的驱动程序。

建立连接如下：

```
# default
engine = create_engine('postgresql://user:pass@localhost/mydatabase')


# psycopg2
engine=create_engine('postgresql+psycopg2://user:pass@localhost/mydata
base')
```

SQLAlchemy 使用会话与数据库交谈，ORM 对数据库的“处理”是 Session。当我们第一次设置应用程序时，在与 create_engine()语句相同的级别上，我们定义一个 Session 类，它将作为新 Session 对象的工厂

```
from sqlalchemy import sessionmaker
DBSession = sessionmaker(bind=engine)
db_session = DBSession()
```

14.4. 执行 SQL 语句并处理结果

14.4.1. 创建或修改数据库对象

在使用 SQLAlchemy 执行数据库操作前需要先建立表的类对象，因为在 SQLAlchemy 中，一张表对应一个类。我们可以自己创建表，让 SQLAlchemy 访问这些表。

```
from sqlalchemy.ext.declarative import declarative_base

Base = declarative_base()


from sqlalchemy import Column,Integer,String
# 创建表模型
class User(Base):
    __tablename__ = 'user'
    id = Column(Integer,primary_key=True,autoincrement=True)
    name = Column(String(32))
    address = Column(String(64))


    def __repr__(self):
        return "<User(id='%s', name='%s', address='%s')>" % (
self.id,self.name,self.address)


Base.metadata.create_all(engine)
```

createTable()可以指定参数 ifNotExists=True，仅仅在表格不存在时才创建这个表格。

14.4.2. 删除数据库对象

删除数据库对象时使用 drop_all()函数，此方法以及 create_all()方法接受可选参数 tables，它使用的迭代器是 sqlalchemy.sql.schema.Table 实例。可以通过这种方式控制要创建或删除的表。

```
Base.metadata.drop_all(bind=engine,tables=[User.__table__])
```

14.4.3. 执行更新

14.4.3.1. 插入

```
# 插入单条数据
u = User(name='用户 1',address='地址 1')
session.add(u)

# 插入多条数据
session.add_all([
    User(name='用户 2',address='地址 2'),
    User(name='用户 3',address='地址 3'),
])

# 提交数据，保存到数据库中
```

```
session.commit()
```

14.4.3.2. 更新

```
# 更新单条数据
res = session.query(User).filter(User.id==3).update({"name":"新用户"})
session.commit()


# 更新多条数据，返回更新行数
res2 = session.query(User).filter(User.id<=2).update({"name":"修改用户"})
print(res2)


session.commit()
```

14.4.3.3. 删除

```
res = session.query(User).filter(User.id==2).delete()


session.commit()
```

14.4.4. 执行查询

14.4.4.1. 简单查询

查询数据使用相关 query 函数查询：

```
# 查询所有用户
```

```
user_all = session.query(User).all()


# where 条件查询
user = session.query(User).filter(User.id>=2).all()


# 只取第一条数据
user = session.query(User).filter(User.id>=2).first()
```

14.4.4.2. 复杂条件查询

```
# and_, or_ 条件查询
from sqlalchemy.sql import and_,or_
r1 = session.query(User).filter(and_(User.id>3,User.name=='用户 4')).all()
r1 = session.query(User).filter(or_(User.id<2,User.name=='新用户')).all()


# 查询数据，指定查询数据加入别名
r2 = session.query(User.name.label('username'),User.id).firset()
print(r2.id,r2.username)


# 原生 sql 查询
r3 = session.query(User).from_statement(text("select * from User where na
me=:name")).params(name='修改用户').all()
  print(r3)
```

14.4.5. 获取结果

query 返回的是 list 类型，可以调用每一个值的属性输出结果。

```
for i in r3:

print(i.id,i.name,i.address)
```

14.4.6. 执行存储过程

下面的例子展示了如何调用存储过程。

```
connection = engine.raw_connection()

cursor = connection.cursor()

sql_procedure = " CREATE OR REPLACE PROCEDURE pro_test(a integer,inou

t b integer) as \

                    begin \

                        b := a+b; \

                    end"

cursor.execute(sql_procedure)

cursor.execute("call pro_test(1,5)")

result = cursor.fetchone()
```

下面的例子展示了如何调用函数。

```
connection = engine.raw_connection()

cursor = connection.cursor()

sql_function = "CREATE OR REPLACE FUNCTION func_test (a int,out b int)

RETURNS int AS \
```

```
                $$ \
                begin \
                    b := a+2; \
                return ;  \
                end; \
                $$ LANGUAGE 'plpgsql';"
cursor.execute(sql_function)
cursor.execute("select func_test(1)")
result = cursor.fetchone()
```

14.4.7. 处理数据类型

14.4.7.1. 二进制类型

PanWeiDB V2.0-S3.0.0_Beta 提供两种不同的方法存储二进制数据。二进制数据可以使用二进制数据类型 BYTEA 存储在表中，或者使用大对象特性以一种特殊的格式将二进制数据存储在一个独立的表中， 然后通过在表中保存一个类型为 OID 的值来引用该表。以下为 BYTEA 类型使用示例。

1)创建表:

```
from sqlalchemy import Column,Integer,LargeBinary
from sqlalchemy.ext.declarative import declarative_base
Base = declarative_base()


class ByteaTest(Base):
```

```
__tablename__ = 'bytetest'

id = Column(Integer,primary_key=True,autoincrement=True)

col1 = Column(LargeBinary())# bytea 类型


def __repr__(self):

return "<bytetest(col1='%s')>" %(self.col1)


Base.metadata.create_all(engine)
```

2)插入数据

```
data = bytes(range(256))

u = ByteaTest(col1=data)

session.add(u)

session.commit()
```

3)读取数据

```
b1 = session.query(ByteaTest).filter(ByteaTest.id==1).first()

print(b1.id,b1.col1)

assert b1.col1==data
```

14.4.7.2. 字符类型

PanWeiDB V2.0-S3.0.0_Beta 中 TEXT 类型与 VARCHAR 类型都是可变长的字符类型，区别在于 VARCHAR 类型通过 VARCHAR(n)中的 n 来限制最大长

度，而 TEXT 类型没有。TEXT 类型与 VARCHAR 类型几乎没有性能差别，TEXT 类型最多可存储 1G 数据 。

1)创建表

```
from sqlalchemy import Column,String,Text,Integer

from sqlalchemy.ext.declarative import declarative_base

Base = declarative_base()


class StringTest(Base):

__tablename__ = 'stringtest'

id = Column(Integer,primary_key=True,autoincrement=True)

col1 = Column(Text())# text 类型

col2 = Column(String(256))# varchar 类型


def __repr__(self):

return "<stringtest(col1='%s', col2='%s')>" %(self.col1,self.col2)


Base.metadata.create_all(engine)
```

2)插入数据

```
a='test1'

b='test2'

u = StringTest(col1=a,col2=b)

session.add(u)
```

```
session.commit()
```

3)读取数据

```
s1 = session.query(StringTest).filter(StringTest.id==1).first()
assert s1.col1 == a
assert s1.col2 == b
print(s1.id,s1.col1,s1.col2)
```

14.4.7.3. 数字类型

数字类型有整数类型、任意精度数字类型、浮点类型和序数类型。下面用整数类型 integer、任意精度数字类型 numeric 和 double precision 举例说明。

1)创建表

```
from sqlalchemy import Column,Integer,Numeric
from sqlalchemy.dialects.postgresql import DOUBLE_PRECISION
from sqlalchemy.ext.declarative import declarative_base
Base = declarative_base()

class NumberTest(Base):
__tablename__ = 'numbertest'
id = Column(Integer,primary_key=True,autoincrement=true)
col1 = Column(Integer)# integer 类型
col2 = Column(Numeric(precision=6,scale=4)# numeric 类型
col3 = Column(DOUBLE_PRECISION())# double precision 类型
```

```
def __repr__(self):
return "<NumberTest(col1='%s', col2='%s', col3='%s')>"
%(self.col1,self.col2,self.col3)

Base.metadata.create_all(engine)
```

2)插入数据

```
a = 1
b= decimal.Decimal(2.141)
c = 3.1415926
u = NumberTest(col1=a,col2=b,col3=c)
session.add(u)
session.commit()
```

3)读取数据

```
n1=session.query(NumberTest).filter(NumberTest.id==1).first()
print(n1.col1)
print(n1.col2)
print(n1.col3)
```

14.4.7.4. 时间/日期类型

日期/时间类型有 timestamp 、date、time 、interval ； 下面以 timestamp 、date、time 举例说明。

1)创建表

```
from sqlalchemy import Column,Integer,Date,DateTime,Time

from sqlalchemy.ext.declarative import declarative_base

Base = declarative_base()


class DatetimeTest(Base):

__tablename__ = 'datetimetest'

id = Column(Integer,primary_key=True,autoincrement=true)

col1 = Column(DateTime())# timestamp 类型

col2 = Column(Date())# date 类型

col3 = Column(Time())# time 类型


def __repr__(self):

return "<DatetimeTest(col1='%s', col2='%s', col3='%s')>"

%(self.col1,self.col2,self.col3)


Base.metadata.create_all(engine)
```

2)插入数据

```
_now = datetime.now().replace(microsecond = 0)

u = DateTime1(col1=_now,col2=_now,col3=_now.time())
```

```
session.add(u)

session.commit()
```

3)读取数据

```
d1=session.query(DatetimeTest).filter(DatetimeTest.id==1).first()

print(d1.id,d1.col1,d1.col2,d1.col3)

assert  isinstance(d1.col1,datetime)

assert  isinstance(d1.col2,date)

assert  isinstance(d1.col3,time)
```

14.5. 连接 panweidb 时需注意的问题

14.5.1. 连接

使用 SQLAlchemy 连接 panweidb 时使用 Postgresql 的数据库模式。连接串的 dialect 为连接 postgresql 时的 'postgresql' 。

14.5.2. 冲突时插入

14.5.2.1. 问题描述

在使用 insert 语句的 Insert.on_conflict_do_nothing() 函数时，会报如下错误

```
Traceback (most recent call last):
  File "/usr/local/python39/lib/python3.9/site-packages/SQLAlchemy-1.4.39-py3.9-linux-x86_64.egg/sqlalchemy/engine/base.py", line 1819, in _execute_context
    self.dialect.do_execute(
  File "/usr/local/python39/lib/python3.9/site-packages/SQLAlchemy-1.4.39-py3.9-linux-x86_64.egg/sqlalchemy/engine/default.py", line 732, in do_execute
    cursor.execute(statement, parameters)
psycopg2.errors.FeatureNotSupported: Conflict without target is not yet supported within INSERT ON CONFLICT KEY DO UPDATE statement.
```

14.5.2.2. 原因

从 9.5 版开始，PostgreSQL 允许通过 ON CONFLICT 条款 INSERT 语句。只有当候选行不违反任何唯一约束时，才会插入该行。在违反唯一约束的情况下，可能会发生第二个操作，该操作可以是“do update”（执行更新），表示应更新目标行中的数据，也可以是“do nothing”（不执行任何操作），表示静默跳过此行。

使用现有的唯一约束和索引确定冲突。可以使用 DDL 中所述的名称来标识这些约束，也可以通过声明组成索引的列和条件来推断这些约束。

SQLAlchemy 提供 ON CONFLICT 通过特定于 PostgreSQL 的支持 insert() 函数，提供生成方法 Insert.on_conflict_do_update() 和 Insert.on_conflict_do_nothing() 。

panweidb 连接时采用 postgresql 的模式，但尚未支持 ON CONFLICT DO NOTHING 的语法，在使用 Insert.on_conflict_do_nothing()函数时会出错。

14.5.2.3. 解决办法

将 ON CONFLICT DO NOTHING 改为 panweidb 相应的语法 ON DUPLICATE KEY UPDATE NOTHING。

打开 sqlalchemy.dialects.postgresql.base.py 文件，找到 vistit_on_conflict_do_nothing(self,on_conflict,**kw)函数，改动如下：

```
2494    def visit_on_conflict_do_nothing(self, on_conflict, **kw):
2495
2496        target_text = self._on_conflict_target(on_conflict, **kw)
2497
2498        if target_text:
2499            return "ON CONFLICT %s DO NOTHING" % target_text
2500        else:
2501            return "ON DUPLICATE KEY UPDATE NOTHING"
2502
```

14.6. 附录 1：主要接口

主要接口说明如下：

- create_engine(url, **kwargs)

此方法用于创建新的 Engine 实例。参数说明如下：

关键字	参数说明
url	指示数据库和连接参数的字符串
case_sensitive	如果为 false，则结果列名称将以不区分大小写的方式匹配
connect_args	将直接传递给 DBAPI 的选项字典
echo=False	如果为真，引擎将记录所有语句以及 repr() 其参数列表的默认日志处理程序
echo_pool=False	如果为 true，则连接池将记录信息输出

调用示例见附录 2。

- ccreate_all(bind=None, tables=None, checkfirst=True)

此方法创建存储在此元数据中的所有表。参数说明如下：

关键字	参数说明
bind	用于访问数据库；如果没有，则对此使用现有绑定 MetaData

tables	可选列表 Table 对象
checkfirst	默认为 true，不为目标数据库中已存在的表发出 creates。

调用示例见附录 2。

- drop_all(bind=None, tables=None, checkfirst=True)

此方法根据类创建表。参数说明如下：

关键字	参数说明
bind	用于访问数据库；如果没有，则对此使用现有绑定 MetaData
tables	可选列表 Table 对象
checkfirst	默认值为 true，仅对确认存在于目标数据库中的表发出 drops

调用示例见附录 2。

- session.add(instance, _warn=True)

此方法将对象放置在 Session 。它的状态将在下次刷新操作时保留到数据库中。参数说明如下：

关键字	参数说明
instance	需要添加的实例对象

调用示例见附录 2。

❖ Session.commit()

刷新挂起的更改并提交当前事务。如果没有正在进行的事务，该方法将首先“自动注册”一个新事务并提交。
调用示例见附录 2。

❖ Query.filter(*criterion)

将给定的筛选条件应用于 Query ，使用 SQL 表达式。

参数说明如下：

关键字	参数说明
criterion	筛选条件

调用示例见附录 2。

❖ Query.first()

此方法返回此操作的第一个结果 Query 如果结果不包含任何行，则为“无”。
调用示例见附录 2。

❖ Query.all()

此方法返回由此表示的结果 Query 作为一个列表。这将导致执行底层 SQL 语句。
调用示例见附录 2。

14.7. 附录 2：调用示例

```
from sqlalchemy import *

from sqlalchemy.orm import sessionmaker

from sqlalchemy.ext.declarative import declarative_base

from sqlalchemy import Column,Integer,String

from sqlalchemy.sql import and_,or_


connstr = 'postgresql+psycopg2://pwadmin:password@127.0.0.1/panweidb
'

engine = create_engine(connstr,echo=False)

DBSession = sessionmaker(bind=engine)

session = DBSession()


Base = declarative_base()

# 创建表模型

class User(Base):

__tablename__='users'

id = Column(Integer,primary_key=True,autoincrement=True)

name = Column(String(32))

address = Column(String(64))


def __repr__(self):

return "<User(id='%s', name='%s', address='%s')>"
```

```
%(self.id,self.name,self.address)


def drop_t():
Base.metadata.drop_all(bind=engine,tables=[User.__table__])


def add_t():
# 插入单条数据
u=User(name='用户 1',address='地址 1')
session.add(u)


# 插入多条数据
session.add_all([
User(name='用户 2',address='地址 2'),
User(name='用户 3',address='地址 3'),
])


# 提交数据，保存到数据库中
session.commit()


def update_t():
res = session.query(User).filter(User.id==3).update({"name":"新用户"})
session.commit()


res2 = session.query(User).filter(User.id<=2).update({"name":"修改用户"})
```

```
print(res2)


session.commit()


def select_t():
# 查询所有用户
 user_all = session.query(User).all()
print(user_all)
# where 条件查询
user = session.query(User).filter(User.id>=2).all()


# 只取第一条数据
user = session.query(User).filter(User.id>=2).first()
print(user)


# and_,or_条件查询
r1 = session.query(User).filter(and_(User.id>3,User.name=='用户 4')).all()
print(r1)
r2 = session.query(User).filter(or_(User.id<2,User.name=='新用户')).all()
print(r2)


# 查询数据，指定查询数据加入别名
r3 = session.query(User.name.label('username'),User.id).first()
print(r3.id,r3.username)
```

```
# 原生 sql 查询
r4 = session.query(User).from_statement(text("select * from users where
 name=:name")).params(name='修改用户').all()
for i in r4:
print(i.id,i.name,i.address)


def delete_t():
res = session.query(User).filter(User.id==2).delete()
session.commit()


Base.metadata.create_all(engine)
add_t()
update_t()
select_t()
delete_t()
drop_t()
```

15. SQLAPI++开发指南

15.1. 介绍

SQLAPI++ 是一个 C++库，它提供了统一的 API 来访问不同的 SQL 数据库,通过它来连接 PanWeiDB V2.0-S3.0.0_Beta 可以简单而方便的管理数据库。

15.2. sqlapi++适配

要使用 sqlapi++需要安装 gcc 和 sqlapi++库。

15.2.1. gcc 安装

打开命令行终端，输入 gcc -v 如看到如下提示，可跳过此步

```
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/usr/libexec/gcc/x86_64-linux-gnu/10.3.1/lto-wrapper
Target: x86_64-linux-gnu
Configured with: ../configure --prefix=/usr --mandir=/usr/share/man --infodir=/usr/share/info --enable-shared --enable-t
hreads=posix --enable-checking=release --with-system-zlib --enable-__cxa_atexit --disable-libunwind-exceptions --enable-
gnu-unique-object --enable-linker-build-id --with-linker-hash-style=gnu --enable-languages=c,c++,objc,obj-c++,fortran,lt
o --enable-plugin --enable-initfini-array --disable-libgcj --without-isl --without-cloog --enable-gnu-indirect-function
--build=x86_64-linux-gnu --with-stage1-ldflags=' -Wl,-z,relro,-z,now' --with-boot-ldflags=' -Wl,-z,relro,-z,now' --disab
le-bootstrap --with-tune=generic --with-arch_32=x86-64 --disable-multilib
Thread model: posix
Supported LTO compression algorithms: zlib
gcc version 10.3.1 (GCC)
```

若是没有使用 apt-get install gcc gcc-c++安装即可

15.2.2. sqlapi++包下载

前往 sqlapi++官网下载包，放在某个目录下，例如我这里放在 /home/panweidb 下

```
[panwei@localhost ~]$ ls
SQLAPI sqlapi_trail.tar.gz panweienv
[panwei@localhost ~]$ pwd
/home/panwei
```

输入 tar -zxvf sqlapi_trial.tar.gz 解压包，得到 SQLAPI 文件夹即可

```
./SQLAPI/
./SQLAPI/binaries-g++-7.2-amd64.tar.gz
./SQLAPI/binaries-g++-7.2-i386.tar.gz
./SQLAPI/binaries-g++-5.3-amd64.tar.gz
./SQLAPI/binaries-g++-4.8-amd64.tar.gz
./SQLAPI/bin
./SQLAPI/bin/sqltests
./SQLAPI/bin/sqltestus
./SQLAPI/bin/index.txt
./SQLAPI/bin/sqltest
./SQLAPI/binaries-g++-4.7-amd64.tar.gz
./SQLAPI/binaries-g++-4.6-amd64.tar.gz
./SQLAPI/binaries-g++-4.2-amd64.tar.gz
./SQLAPI/binaries-g++-6-amd64.tar.gz
./SQLAPI/binaries-g++-4.4-i386.tar.gz
./SQLAPI/binaries-g++-4.2-i386.tar.gz
./SQLAPI/samples/
```

进入 SQLAPI 文件夹，根据 gcc 版本和 cpu 型号解压提供的二进制包

```
bin                             binaries-g++-4.4-i386.tar.gz   binaries-g++-4.9-i386
binaries-g++-10-amd64.tar.gz    binaries-g++-4.6-amd64.tar.gz  binaries-g++-5.3-amd6
binaries-g++-10-i386.tar.gz     binaries-g++-4.6-i386.tar.gz   binaries-g++-5.3-i386
binaries-g++-4.1-amd64.tar.gz   binaries-g++-4.7-amd64.tar.gz  binaries-g++-6-amd64.
binaries-g++-4.1-i386.tar.gz    binaries-g++-4.7-i386.tar.gz   binaries-g++-6-i386.t
binaries-g++-4.2-amd64.tar.gz   binaries-g++-4.8-amd64.tar.gz  binaries-g++-7.2-amd6
binaries-g++-4.2-i386.tar.gz    binaries-g++-4.8-i386.tar.gz   binaries-g++-7.2-i386
binaries-g++-4.4-amd64.tar.gz   binaries-g++-4.9-amd64.tar.gz  documentation.html
```

```
./bin/
./bin/sqltests
./bin/sqltestus
./bin/index.txt
./bin/sqltestu
./bin/sqltest
./lib/
./lib/libsqlapiu.so
./lib/libsqlapi.so
./lib/index.txt
./lib/libsqlapiu.a
./lib/libsqlapi.a
```

配置环境变量，将 SQLAPI 文件夹下的 include 目录加到 CPLUS_INCLUDE_PATH 里

```
[panwei@localhost SQLAPI]$ export CPLUS_INCLUDE_PATH=/home/panwei/SQLAPI/includes:$CPLUS_INCLUDE_PATH
[panwei@localhost SQLAPI]$ echo $CPLUS_INCLUDE_PATH
/home/panwei/SQLAPI/include:
```

将 SQLAPI 文件夹下的 lib 目录添加到 LD_INCLUDE_PATH 里即可，方法与上面类似

15.3. 使用 sqlapi++连接数据库

15.3.1. 包含头文件

任何使用 sqlapi++的程序都只需包含一个 include<SQLAPI.h>即可

#include<SQLAPI.h>

15.3.2. 连接与断开

15.3.2.1. 连接数据库

一般来说，一个程序有一个连接对象即可，通过调用连接对象的 Connect 成员函数来配置连接属性。

```
SAConnection con;  //创建连接对象
con.Connect(
            _TSA("127.0.0.1,5432@test"),      // IP 地址，断开@数据库名
            _TSA("tester"),    // 用户名
            _TSA("testpwd"),    // 密码
            SA_PostgreSQL_Client);  //连接的数据库
```

15.3.2.2. 断开连接

直接调用 con 对象的 Disconnect 成员函数即可断开连接

```
con.Disconnect();  //断开连接
```

15.4. 执行 SQL 语句并处理结果

15.4.1. 执行查询

15.4.1.1. 绑定参数

Sqlapi++在执行 sql 语句前需要进行参数绑定

```
SACommand cmd;
cmd.setCommandText(
            _TSA("select * from testtable(id,firname, scdname,age) where
age > :1"));
```

上述语句声明了一个 sql 文本，将文本设置为一个模板，where 语句里带有一个参数

15.4.1.2. 输入参数

使用<<进行参数的输入：

```
cmd << 15L;
```

15.4.1.3. 执行查询

通过调用 Execute 执行查询语句：

```
cmd.Execute(); //执行 sql 语句
```

还可以通过调用 RowsAffected 函数查看执行语句后被影响的行数：

```
int  rows=cmd.RowsAffected();

cout<<rows;
```

15.4.2. 获取结果

通过调用 FetchNext()即可获取结果集。

```
While(cmd.FetchNext()){

      cout  <<" id:" <<cmd.Field(_TSA( "id" )).asLong()<<"     "<<

       "firname:" <<cmd.Field(_TSA( "firname" )).asString().GetMultiByteChars()

<<"     "<< "scdname:" <<cmd.Field(_TSA( "scdname" )).asString().GetMultiByt

eChars()<<"     "<<" age:" <<cmd.Field(_TSA( "age" )).asLong()<<endl;

}
```

Int 型需要调用 asLong,char 型需要调用 asString.GetMultiByteChars()

15.4.3. 执行更新

要更改数据(执行一个 insert, update 或者 delete), 可以直接使用 Execute 执行 sql 语句，也可以在 sql 语句中绑定参数，在自己输入，下面分别演示两种用法。

15.4.3.1. 插入

```
//绑定参数
SACommand insert(&con,_TSA("insert into testtable(id,firname,scdname,age)
 values(:1,:2:,3,:4);" ));
insert <<3L<<_TSA( "testfir" )<<_TSA( "testscd" )<<20L;
insert.Execute();


//直接执行
SACommand insert(&con,_TSA("insert into testtable(id,firname,scdname,age)
 values(2,'xxx','xxx',18);" ));
insert.Execute();
```

15.4.3.2. 更新

```
//绑定参数
SACcommand update(
    &con,
    _TSA( "update testtable set firname =: 1 where firname=: 2;" )
);
update<<_TSA( "joe" )<<_TSA( "mike" );
update.Execute();


//直接执行
SACcommand update(
```

```
    &con,
    _TSA( “update testtable set firname =’joe’ where firname=’mike’;” )
);
update.Execute();
```

15.4.3.3. 删除

```
//绑定参数
SACcommand del(
    &con,
    _TSA( “delete from testtable where firname=: 1;” )
);
del<<_TSA( “joe” );
del.Execute();

//直接执行
SACcommand del(
    &con,
    _TSA( “delete from testtable where firname=’joe’;” )
);
del.Execute();
```

15.4.4. 创建或修改数据库对象

要创建、更改或者删除一个类似表或者视图这样的数据库对象， 直接调用 Execute 方法执行创建数据库对象的 sql 语句即可。

```
SACommand creatdb(
    &con,
    _TSA( “create table testtb(id int, firname char(10),scdname char(10),ag
e int);” ));
Creatdb.Execute()
```

15.4.5. 执行存储过程

存储过程是一种在数据库中存储复杂程序，以便外部程序调用的一种数据库对象。 存储过程是为了完成特定功能的 SQL 语句集，经编译创建并保存在数据库中，用户可通过指定存储过程的名字并给定参数 (需要时)来调用执行。

使用 sqlapi++调用储存过程主要有 3 步：

初始化存储过程命令

```
SACommand cmd(&con, "TestProc");
```

绑定输入参数

```
//普通赋值
cmd.Param(_TSA("n1")).setAsLong() = 5L;
cmd.Param(_TSA("n2")).setAsLong() = 10L;
```

```
//使用流赋值

cmd << SAPos(_TSA("n1")) << 5L << SAPos(_TSA("n2")) << 10L;
```

执行储存过程

```
cmd.Execute();
```

15.4.6. 处理数据类型

15.4.6.1. 二进制类型

二进制类型有 blob，row,bytea 等，下面代码演示这三种类型的基本用法

- 创建表

```
SACommand cmd(&con,_TSA( "CREATE TABLE blob_type_t1

(

    BT_COL1 INTEGER,

    BT_COL2 BLOB,

    BT_COL3 RAW,

    BT_COL4 BYTEA

) ;" ));

cmd.Execute();
```

- 插入数据

```
SACommand cmd(&con,_TSA( "INSERT INTO blob_type_t1 VALUES(10,empty_

blob(),

HEXTORAW('DEADBEEF'),E'\\xDEADBEEF');" ));
```

```
cmd.Execute();
```

❖ 读取数据

```
SACommand cmd(&con,_TSA( “SELECT * FROM blob_type_t1;
” ));
cmd.Execute();
while(cmd.FetchNext()){
…
}
```

15.4.6.2. 字符类型

PanWeiDB V2.0-S3.0.0_Beta 中 TEXT 类型与 VARCHAR 类型都是可变长的字符类型，区别在于 VARCHAR 类型通过 VARCHAR(n)中的 n 来限制最大长度，而 TEXT 类型没有。TEXT 类型与 VARCHAR 类型几乎没有性能差别，TEXT 类型最多可存储 1G 数据 。

使用 TEXT 类型存储数据时，可用以下方式来进行读写。

❖ 创建表

```
cmd.setCommandText("create table images (id int, msg text)");
cmd.Execute();
```

❖ 插入数据

```
cmd.setCommandText("insert into images values(1, ?)" ("the text data"));
```

```
cmd.Execute();
```

❖ 读取数据

```
cmd.setCommandText("select msg from images where id = 1");
cmd.Execute();
while(cmd.FetchNext()){
        ......;
}
```

使用 varchar 类型存储数据时。可以使用以下方法来读写

❖ 创建表

```
cmd.setCommandText("create table images (id int, msg varchar(20))");
cmd.Execute();
```

❖ 插入数据

```
cmd.setCommandText("insert into images values(1, ?)" ("the text data"));
cmd.Execute();
```

❖ 读取数据

```
cmd.setCommandText("select msg from images where id = 1");
cmd.Execute();
while(cmd.FetchNext()){
        ......;
}
```

15.4.6.3. 时间/日期类型

要使用日期类型，主要有一下两步：

1.将日期/时间值绑定到输入变量（SQL 语句）或输入参数（存储过程/函数）中

2.读取日期/时间值（从结果集中的字段或从存储过程的输出参数）

首先，创建一个需要日期时间绑定的 sql 命令：

```
SACommand cmd(
&con,
_TSA("Update TEST set FDATETIME = :1"));
```

需要一个 SADate 时间类的实例来执行绑定。有几种方法可以创建它。首先是显式构造它：

```
SADateTime dtValue(
2022 /*year*/, 10 /*October*/, 6 /*day of month*/,
19 /*hours*/, 0 /*minutes*/, 0 /*seconds*/);
```

其次，可以从现有的 C 结构 tm 转换为 SADate 时间：

```
extern struct tm some_tm_Value;
SADateTime dtValue = some_tm_Value;
```

最后，绑定日期/时间值并执行

```
cmd << dtValue;
cmd.Execute();
```

假设要从名为 FDATETIMETEST 的表中检索命名的日期/时间字段

创建并执行一个返回日期时间类型数据的示例 SQL 命令：

```
SACommand cmd(
```

```
    &con,
    _TSA("SELECT FDATETIME FROM TEST"));
cmd.Execute();
```

使用 SADate 时间、标准 C 结构 tm 或 Windows DATE 数据类型来获取数据并访问日期/时间列值:

```
while(cmd.FetchNext())
{
    SADateTime dtValue = cmd.Field(_TSA("FDATETIME"));
    struct tm tmValue = dtValue;
    struct tm tmValue2 = (struct tm)cmd.Field(_TSA("FDATETIME")).asDateTi
me();
    ...
    DATE dateValue = dtValue;
    DATE dateValue2 = (DATE)cmd.Field(_TSA("FDATETIME")).asDateTime();
    ...
}
```

16. SQLObject 框架开发指南

16.1. 介绍

SQLObject 是一种流行的对象关系管理器，用于为数据库提供对象接口，其中表作为类，行作为实例，列作为属性。

SQLObject 包括一种基于 python 对象的查询语言，使 SQL 更加抽象，并为应用程序提供了实质性的数据库独立性。

16.2. 安装 SQLObject

安装 SQLObject 需要安装 SQLObject 包和数据库对应的 python 语言驱动 psycopg2。

16.2.1. 安装 psycopg2

安装 psycopg2 模块前需要先装好 PanWeiDB V2.0-S3.0.0_Beta Libpq，并配置环境变量。

16.2.1.1. 安装 PanWeiDB V2.0-S3.0.0_Beta Libpq

安装 Libpq 有两种方式。

- 通过提供的 Libpq 安装包安装

解压 PanWeiDB V2.0-S3.0.0_Beta 提供的 Libpq，新建/usr/local/libpq 目录，将解压后的 lib 目录拷贝到/usr/local/libpq 目录下，配置环境变量

如下：

```
export LD_LIBRARY_PATH=/usr/local/libpq/lib:$LD_LIBRARY_PATH
```

❖ 通过安装 PanWeiDB V2.0-S3.0.0_Beta 安装包安装

按照安装说明安装 PanWeiDB V2.0-S3.0.0_Beta，并配置环境变量。如果是使用安装包自带安装程序进行安装，则环境变量会自动配置无需手动配置；如果是手动安装数据库，则需配置以下环境变量（只作为参数，应按实际的环境进行配置）：

```
export GAUSSHOME=/usr/local/panweidb

export PATH=$GAUSSHOME/bin:$PATH

export LD_LIBRARY_PATH=$GAUSSHOME/lib:$LD_LIBRARY_PATH
```

16.2.1.2. 安装 python3 和 gcc

推荐使用 python3，安装命令如下：

```
yum install -y python3 python3-devel gcc
```

16.2.1.3. 安装 psycopg2 模块

推荐使用源码包安装，获取 psycopg2 源码包：

```
# 1.卸载已安装的 psycopg2（如果没有安装则跳过，此步骤可能需要 root 用户执行）

pip3 uninstall psycopg2 -y


# 2.上传并解压源码包

unzip Psycopg-panwei.zip
```

```
# 3.安装

cd Psycopg-panwei

python3 setup.py build

# 此步骤可能需要 root 用户执行

python3 setup.py install


# 4.检查安装结果

进入 python3 命令行，执行 import psycopg2 可以正常导入即表示安装成功。
```

16.2.2. 安装 SQLObject

安装至本地可以通过命令：

```
pip install sqlobject
```

16.3. 使用 SQLObject 连接数据库

16.3.1. 导入

任何使用 sqlobject 的程序都需要导入 sqlobject 包。

```
from sqlobject import *
```

16.3.2. 连接数据库

连接 URI 必须遵循标准 URI 语法：

```
scheme://[user[:password]@]host[:port]/database[?parameters]
```

SQLObject 支持 sqlite、mysql、postgres、firebird、interbase、maxdb、sapdb、mssql、sybase。在连接 panweidb 数据库时，使用连接 postgres 的模式。

连接 URL 如：

```
postgres://user@host/database?debug=&cache=

postgres:///full/path/to/socket/database

postgres://host:5432/database
```

参数包括:debug(默认值:False)、debugOutput(默认值:False)、cache(默认值:True)、autoCommit(默认值:True)、debugThreading(默认值:False)、logger(默认值:None)、loglevel(默认值:None)、schema(默认值:None)。

建立连接如下：

```
from sqlobject import *


connection_string = 'postgres://pwadmin:panwei_123@127.0.0.1:5432/pan
weidb'
connection = connectionForURI(connection_string)
sqlhub.processConnection = connection
```

sqlhub.processConnection 意味着在默认情况下，所有类都将使用我们刚刚建立的这个连接。如果没有定义 sqlhub，则需要使用 Person._connection=conn 来指定连接。

16.4. 执行 SQL 语句并处理结果

16.4.1. 创建或修改数据库对象

在使用 SQLObject 执行数据库操作前需要先建立表的类对象，因为在 SQLObject 中，一张表对应一个类。我们可以自己创建表，让 SQLObject 访问这些表。

```
class Person(SQLObject):

    firstName = StringCol()

    middleInitial = StringCol(length=1, default=None)

    lastName = StringCol()



Person.createTable()
```

createTable()可以指定参数 ifNotExists=True，仅仅在表格不存在时才创建这个表格。

16.4.2. 删除数据库对象

删除数据库对象时使用 Person.dropTable()函数,可以指定 ifExists=True，仅仅在表格存在时才删除这个表格。

```
Person.dropTable(ifExists = True)

assert not conn.tableExists(Person.sqlmeta.table)
```

16.4.3. 执行更新

16.4.3.1. 插入

```
Person(firstName = 'John', lastName = 'Doe')

Person(firstName = 'Bob', lastName = 'Hope')

Person(firstName = 'Doe', lastName = 'Mark')
```

16.4.3.2. 更新

```
p = Person.get(1)

p.firstName = 'David'

p.middleInitial = 'Q'

p2 = Person.get(2)

p2.set(firstName = 'Robert', lastName = 'Hope Jr.')
```

16.4.3.3. 删除

```
Person.deleteMany(OR(Person.q.firstName == 'Bob',

Person.q.firstName =='Fred'))

Person.deleteBy(firstName = 'Robert')
```

16.4.4. 执行查询

16.4.4.1. 根据 ID 号查询

调用 get()函数根据 ID 号查询，返回的对象类型是 sqlobject.sresults.SelectResults：

```
peeps = Person.get(1)

print(list(peeps))
```

16.4.4.2. 根据条件查询

查询数据使用相关 select 函数查询：

```
p1 = Person.select(OR(Person.q.firstName == 'John',

LIKE(Person.q.lastName, '%Hope%')))

print(list(p1))

p2 = Person.selectBy(firstName = 'John')[0]

print(p2)

p3 = Person.select(Person.q.firstName == 'Doe').count()

print(p3)
```

16.4.5. 获取结果

select 方法获取到的结果需要先转为列表类型再输出。

```
peeps = list(p1)

for i in peeps:

print(i)
```

17. xorm 框架开发指南

17.1. 介绍

GoLang 语言通过 xorm 框架访问 PanWeiDB V2.0-S3.0.0_Beta 数据库，xorm 是一个简单而强大的 Go 语言 ORM 库。通过它可以使数据库操作非常简便。

17.2. 安装 xorm

安装 xorm 需要安装 GoLang 包和数据库对应的 GoLang 语言驱动。

17.2.1. 安装 GoLang

若机器已经安装 GoLang 编译环境并可以正确编译 GoLang 程序，此处可以跳过。

1、获取 go1.16.6.linux-amd64.tar.gz 安装包，若可以联网的话，可以使用 wget 命令直接获取：

```
wget https://dl.google.com/go/go1.16.6.linux-amd64.tar.gz
```

2、进行解压，目录自行确定，建议解压到 /usr/local 目录下：

```
tar -C /usr/local -zxvf go1.16.6.linux-amd64.tar.gz
```

3、添加 /usr/local/go/bin 到 PATH 变量中,建议添加至 /etc/profile 中;以及添加 GOROOT,GOPATH 等变量:

```
export GOROOT=/usr/local/go

export GOPATH=$HOME/go

export GO111MODULE=on

export GOPROXY=https://goproxy.cn,direct
```

添加完成后 source /etc/profile

17.2.2. 安装 xorm

安装至本地可以通过命令:

```
go get xorm.io/xorm
```

17.2.3. 安装数据库对应的 GoLang 语言驱动

获取 Go 语言驱动的 pg 的代码，将其放在前面设置的 GOPATH/pkg/mod 中。在运行的项目代码中,如果没有 go.mod 文件,则运行如下命令生成 go.mod 文件。

```
go mod init golangproject # golangproject 根据实际项目名修改
```

在 go.mod 文件中，添加如下代码:

```
require pq v0.0.0

replace pq => /root/go/pkg/pq # pq 目录根据实际情况修改
```

导入 pq 通过命令:

```
go get pq
```

如果报错请检查是否存在重复的 pq 驱动，数据库的 Go 驱动无法与 PostgreSQL 的 Go 驱动并存。

17.3. 使用 xorm 连接数据库

17.3.1. 导入

任何使用 xorm 的程序都需要导入 xorm 和数据库对应的驱动。

```
import (
  _ "pq"
"xorm.io/xorm"
)
```

17.3.2. 连接数据库

17.3.2.1. 创建 Engine 引擎

一个 Engine 一般只对应一个数据库。Engine 通过调用 xorm.NewEngine 生成。

```
var engine *xorm.Engine
var err error
engine, err = xorm.NewEngine("postgres", "host=%s port=%d user=%s pass
word=%s dbname=%s sslmode=disable", host, port, user, password, dbna
```

```
me)
```

17.3.2.2. 定义表结构体

xorm 支持将一个 struct 映射为数据库中对应的一张表。

```
type User struct {
    Id    int64
    Name string  `xorm:"varchar(25) notnull unique 'usr_name' comment('
姓名')"`
}
```

17.4. 执行 SQL 语句并处理结果

17.4.1. 执行查询

17.4.1.1. 直接执行

调用 SQL 语句查询：

```
sql := "select * from userinfo"
results, err := engine.Query(sql)
```

17.4.1.2. 使用 xorm 中的方法查询

查询单条数据使用 Get 方法，在调用 Get 方法时需要传入一个对应结构体的指针：

```
user := new(User)
has, err := engine.Id(id).Get(user)
```

查询多条数据使用 Find 方法,Find 方法的第一个参数为 slice 的指针或 Map 指针，即为查询后返回的结果，第二个参数可选，为查询的条件 struct 的指针。

```
everyone := make([]User, 0)
err := engine.Find(&everyone)
```

17.4.2. 获取结果

17.4.2.1. 遍历数组

Find 方法获取到的结果可以顺序遍历以获取结果集中所有结果。

```
for i := range (&everyone){
        Fmt.Println(everyone[i])
}
```

17.4.3. 执行更新

要更改数据(执行一个 insert,update 或者 delete),可以使用 engine.Exec 函数执行完整的 SQL，也可以使用 xorm 的方法执行。下面的示例将以 xorm 的方法执行。

17.4.3.1. 插入

```
user := new(User)
user.Name = "myname"
affected, err := engine.Insert(user)
```

17.4.3.2. 更新

```
user := new(User)
user.Name = "myname"
affected, err := engine.Id(id).Update(user)
```

17.4.3.3. 删除

```
user := new(User)
affected, err := engine.Id(id).Delete(user)
```

17.4.4. 创建或修改数据库对象

要创建、更改或者删除一个类似表或者视图这样的数据库对象， 可以调用 engine.Exec() 方法执行 SQL 语句。对于 panweidb 特有的一些数据类型，更推荐使用 SQL 语句新建表。

```
engine.Exec("create table tb_test(id integer, name text)")
engine.Exec("create view v_tb_test as select * from tb_test")
```

也可以使用 xorm 中的 Sync2() 方法，该方法将数据库中的表与对应 Go 代码中的结构体做同步。若数据库中没有表 user，调用 Sync2()方法会根据 User 的结构自动创建一个 user 表。如果表 user 已经存在，Sync()方法会对比 User 结构与表结构的不同，对表做相应的修改。

```
err := engine.Sync2(new(User))
```

17.4.5. 执行存储过程

下面的例子展示了如何调用存储过程。

```
sql_procedure := ` CREATE OR REPLACE PROCEDURE pro_test(a integer,ino
ut b integer) as
                    begin
                        b := a+b;
                    end;`
engine.Exec(sql_procedure)
result,err := engine.Exec("call pro_test(1,5)")
fmt.Println(result,err)
```

下面的例子展示了如何调用函数。

```
sql_function := `CREATE OR REPLACE FUNCTION func_test (a int,out b int)
RETURNS int AS
                $$
                begin
```

```
                    b := a+2;
                return ;
                end;
                $$ LANGUAGE 'plpgsql';`
engine.Exec(sql_function)
result,err := engine.Query("select func_test(1)")
fmt.Println(result,err)
```

17.4.6. 处理数据类型

17.4.6.1. 二进制类型

PanWeiDB V2.0-S3.0.0_Beta 提供两种不同的方法存储二进制数据。二进制数据可以使用二进制数据类型 BYTEA 存储在表中，或者使用大对象特性以一种特殊的格式将二进制数据存储在一个独立的表中， 然后通过在表中保存一个类型为 OID 的值来引用该表。以下为这两种方法的使用示例。

- ❖ 使用 BYTEA 类型存储

1) 创建表

```
engine.Exec("create table lo_test_tab (id int, img bytea)")
```

2) 插入数据

```
file,_ := os.Open("image.png")
defer file.Close()
bytes,_ := ioutil.ReadAll(file)
engine.Exec("insert into lo_test_tab values(1,?)",bytes)
```

3) 读取数据

```
bytedata,_ := engine.Query("select img from lo_test_tab where id=1")

_ = IfNoFileToCreate("newimage.png")

if err := ioutil.WriteFile("newimage.png",bytedata[0]["img"],0666);err !=ni

l{

    fmt.Println("写入文件失败")

}
```

❖ 使用 OID 存储

1) 创建表

```
engine.Exec("create table lo_test_tab (name text, img oid)")
```

2) 导入数据

该步骤会将操作系统文件“image”导入成一个大对象并生成一个 oid，查询 lo_test_tab 表时则显示 oid。

```
engine.Exec("insert into  lo_test_tab values('beautiful image', lo_import

('/tmp/image'))")
```

3) 导出数据

该操作将数据库中存储的大对象导出到一个操作系统文件。

```
engine.Exec("select lo_export(lo_test_tab.img, '/tmp/image-bak') from l

o_test_tab where name = 'beautiful image'")
```

17.4.6.2. 字符类型

PanWeiDB V2.0-S3.0.0_Beta 中 TEXT 类型与 VARCHAR 类型都是可变长的字符类型，区别在于 VARCHAR 类型通过 VARCHAR(n)中的 n 来限制最大长

度，而 TEXT 类型没有。TEXT 类型与 VARCHAR 类型几乎没有性能差别，TEXT 类型最多可存储 1G 数据 。

使用 TEXT 类型存储数据时，可用以下方式来进行读写。

1) 创建表

```
engine.Exec("create table images (id int, msg text)")
```

2) 插入数据

```
engine.Exec("insert into images values(1, ?)" ("the text data"))
```

3) 读取数据

```
result,err := engine.QueryString("select msg from images where id = 1
")
fmt.Println(result[0]["img"])
```

17.4.6.3. 数字类型

数字类型有整数类型、任意精度数字类型、浮点类型和序数类型。下面用整数类型 integer、任意精度数字类型 numeric、浮点类型 real 和 double precision 举例说明。

1)创建表

- 整数类型 integer 的建表

```
type test_int struct{
    Num int
}
```

```
engine.Sync2(new(test_int))
```

- 任意精度数字类型 numeric 的建表

```
type test_numeric struct{
    Num string
}
engine.Exec("create table test_numeric(num numeric)")
```

- 浮点类型 real 和 double precision 的建表

```
# real 类型
type test_real struct{
    Num string
}
engine.Exec("create table test_real(num real)")
# double precision 类型
type test_doubleprecision struct{
    Num string
}
engine.Exec("create table test_doubleprecision(num double precision)")
```

2)插入数据

- 整数类型 integer 的插入

```
temp := new(test_int)
Temp.Num=3
affected,err := engine.Insert(temp)
Fmt.Println("affected,err")
```

- 任意精度数字类型 numeric 的插入

```
temp := new(test_numeric)
Temp.Num="123.456"
affected,err := engine.Insert(temp)
Fmt.Println("affected,err")
```

- 浮点类型 real 和 double precision 的插入

```
# real 类型
temp := new(test_real)
Temp.Num=3.1234
affected,err := engine.Insert(temp)
Fmt.Println("affected,err")
# double precision 类型
temp := new(test_doubleprecision )
Temp.Num=4.1234
affected,err := engine.Insert(temp)
Fmt.Println("affected,err")
```

3)读取数据

- 整数类型 integer 的读取

```
tmp := new(test_int)
rc,_ := engine.Get(tmp)
fmt.Println(tmp.Num)
fmt.Println(rc)
```

- 任意精度数字类型 numeric 的读取

```
tmp := new(test_numeric)
rc,_ := engine.Get(tmp)
fmt.Println(tmp.Num)
fmt.Println(rc)
```

- 浮点类型 real 和 double precision 的读取

```
# real 类型
tmp := new(test_real)
rc,_ := engine.Get(tmp)
fmt.Println(tmp.Num)
fmt.Println(rc)
# double precision 类型
tmp := new(test_doubleprecision)
rc,_ := engine.Get(tmp)
fmt.Println(tmp.Num)
fmt.Println(rc)
```

17.4.6.4. 时间/日期类型

日期/时间类型有 timestamp 、date、time 、interval ； 下面以 timestamp 、date、time 举例说明。

1)创建表

```
# timestamp 类型
type test_timestamp struct{
    Dt time.Time
```

```
}
engine.Exec("create table test_timestamp(dt timestamp)")
# date 类型
type test_date struct{
    Dt time.Time
}
engine.Exec("create table test_date(dt date)")
# time 类型
type test_time struct{
    Dt time.Time
}
engine.Exec("create table test_time(dt time)")
```

2)插入数据

```
# timestamp 类型
temp := new(test_timestamp)
temp.Dt= time.Date(2018,10,1,10,30,30,0,time.Local)
engine.Insert(temp)
# date 类型
temp := new(test_date)
temp.Dt= time.Date(2018,10,1,0,0,0,0,time.Local)
engine.Insert(temp)
# time 类型
temp := new(test_time)
```

```
temp.Dt= time.Date(1,1,1,10,10,10,time.Local)

engine.Insert(temp)
```

3)读取数据

```
# timestamp 类型

tmp := new(test_timestamp)

rc,_ := engine.Get(tmp)

fmt.Println(tmp.Dt)

fmt.Println(rc)

# date 类型

tmp := new(test_date)

rc,_ := engine.Get(tmp)

fmt.Println(tmp.Dt.Year(),tmp.Dt.Month(),tmp.Dt.Day())

fmt.Println(rc)

# time 类型

tmp := new(test_time)

rc,_ := engine.Get(tmp)

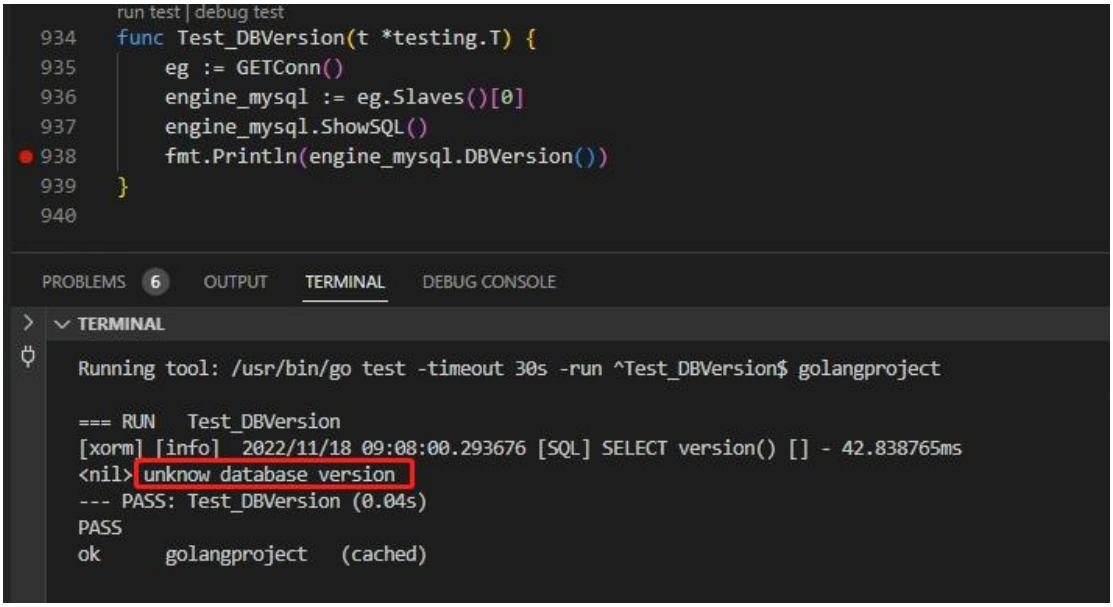
fmt.Println(tmp.Dt.Hour(),tmp.Dt.Minute(),tmp.Dt.Second())

fmt.Println(rc)
```

17.5. 修改 xorm 框架代码以正常获取 DBVersion

17.5.1. 问题

连接 PanWeiDB V2.0-S3.0.0_Beta 版本时，使用 xorm 框架调用 engine.DBVersion()无法获取数据库版本信息。



17.5.2. 影响范围

engine.DBVersion()方法

17.5.3. 原因

xorm 框架目前只适配了 Mysql、Postgres、Tidb、Oracle 等数据库，没有适配 panweidb。连接 panweidb 数据库时，调用的是连接 Postgres 时的函

数。xorm 框架查询版本号用了 select version()命令，期待返回的版本为 pg 的版本，PanWeiDB V2.0-S3.0.0_Beta 版本 select version()返回的是 panweidb 的版本，与其无法匹配，故无法识别到数据库版本信息。

17.5.4. 解决办法

在 xorm/dialects/postgres.go 文件的 func (db *postgres) Version(ctx context.Context, queryer core.Queryer) (*schemas.Version, error)函数中修改判断版本号的代码。

需要在获取到 version 后面的判断版本的代码最后加上：

```
else if strings.HasPrefix(version,"(panweidb ") {
versions := strings.Split(version, "on ")
return &schemas.Version{
Number: versions[0],
Level:   versions[1],
        }, nil
    }
```

具体修改代码位置如下图所示：

```
822
823     // Postgres: 9.5.22 on x86_64-pc-linux-gnu (Debian 9.5.22-1.pgdg90+1), compiled by gcc (Debia
824     // CockroachDB CCL v19.2.4 (x86_64-unknown-linux-gnu, built
825     if strings.HasPrefix(version, "CockroachDB") {
826         versions := strings.Split(strings.TrimPrefix(version, "CockroachDB CCL "), " ")
827         return &schemas.Version{
828             Number:  strings.TrimPrefix(versions[0], "v"),
829             Edition: "CockroachDB",
830         }, nil
831     } else if strings.HasPrefix(version, "PostgreSQL") {
832         versions := strings.Split(strings.TrimPrefix(version, "PostgreSQL "), " on ")
833         return &schemas.Version{
834             Number:  versions[0],
835             Level:   versions[1],
836             Edition: "PostgreSQL",
837         }, nil
838     } else if strings.HasPrefix(version, "(Vastbase ") {
839         versions := strings.Split(version, "on ")
840         return &schemas.Version{
841             Number: versions[0],
842             Level:  versions[1],
843         }, nil
844     }
```

修改后效果如下所示：

```
run test | debug test
934 func Test_DBVersion(t *testing.T) {
935     eg := GETConn()
936     engine_mysql := eg.Slaves()[0]
937     engine_mysql.ShowSQL()
938     fmt.Println(engine_mysql.DBVersion())
939 }
940

PROBLEMS 6  OUTPUT  TERMINAL  DEBUG CONSOLE

TERMINAL

=== RUN   Test_DBVersion
[xorm] [info]  2022/11/18 09:52:56.779967 [SQL] SELECT version() [] - 30.424923ms
&{(Vastbase G100 V2.2 (Build 10) Alpha) compiled at 2022-08-22 23:59:02 commit 8814 last mr   x86_64-unknown-linux-gnu, compiled by g++ (G
CC) 7.3.0, 64-bit } <nil>
--- PASS: Test_DBVersion (0.03s)
PASS
ok      golangproject   (cached)
```

17.6. 附录 1：主要接口

主要接口说明如下：

❖ Xorm.NewEngine()

此方法用于创建新的数据库会话并返回新的 connection 对象。参数说明如下：

关键字	参数说明

dbname	数据库名称
user	用户名
password	密码
host	数据库所在主机 IP 地址
port	连接端口号，默认为 5432
sslmode	ssl 模式，ssl 连接时用
sslcert	客户端证书路径，ssl 连接时用
sslkey	客户端秘钥路径，ssl 连接时用
sslrootcert	根证书路径，ssl 连接时用

返回值说明如下：

关键字	参数说明
engine	ORM 引擎
err	报错信息

调用示例见附录 2。

❖ engine.Exec()

此方法执行一个 SQL 命令。此时不管数据库是何种类型,都可以使用 ` 和 ？符号。参数说明如下：

关键字	参数说明
query	待执行的 sql 语句。
vars_list	变量列表，匹配 query 中?为占位符。

返回值说明如下：

关键字	参数说明
affected	结果影响行数。
err	报错信息。

调用示例见附录 2。

❖ engine.Query()

此方法执行一个 SQL 查询，即 Select 命令。在 panweidb 中支持原始 SQL 语句中使用 ` 和 ？ 符号。参数说明如下：

关键字	参数说明
query	待执行的 sql 语句
vars_list	变量列表，匹配 query 中?为占位符

返回值说明如下：

关键字	参数说明
result	查询结果，[]map[string][]byte 的形式
err	报错信息

调用示例见附录 2。

❖ engine.Insert()

插入数据使用 Insert 方法，可以插入单条数据，也可以批量插入多条数据。

参数：一个或多个 Struct 的指针，一个或多个 Struct 的 Slice 的指针。

返回值说明如下：

关键字	参数说明
affected	插入的记录数
err	报错信息

调用示例见附录 2。

- engine.Update()

更新数据。

参数：结构体指针或 Map[string]interface{}类型。

返回值说明如下：

关键字	参数说明
affected	更新的记录数
err	报错信息

调用示例见附录 2。

- engine.Delete()

删除数据。

参数：struct 的指针。

返回值说明如下：

关键字	参数说明

affected	删除的记录数
err	报错信息

调用示例见附录 2。

❖ engine.Get()

此方法用于查询单条数据，并使用返回的字段为传入的对象赋值。

参数：结构体。

返回值说明如下：

关键字	参数说明
has	该条记录是否存在
err	报错信息

调用示例见附录 2。

❖ engine.Find()

此方法用于返回所有符合条件的记录。

参数：对象切片的指针或 map 的指针。

返回值：如果有错误则返回报错信息

调用示例见附录 2。

17.7. 附录 2：调用示例

```
import (
  "fmt"
  "os"
```

```
  "reflect"
  "unsafe"
  _ "pq"
  "github.com/go-xorm/xorm"
)

const(
  host     =    "127.0.0.1"
  port     =    "5432"
  user     =    "pwadmin"
  password =    "pwase@admin"
  dbname   =    "panweidb"
)

type Student struct{
  Id int64
  Name string
}

var engine *xorm.Engine

func main(){
  var err error
  Info := fmt.Sprintf("host=%s port=%d user=%s password=%s dbname=%s
```

```
 sslmode=disable",host,port,user,password,dbname)
  engine,err = xorm.NewEngine("postgres",Info)
  if err != nil{
     fmt.Println(err)
  }
 #创建表
  engine.Sync2(new(Student))
  temp := new(Student)
  temp.Id=1
  temp.Name="xiaohong"
#插入数据
  engine.Insert(temp)
  engine.Exec("insert into Student values(2,?),","xiaoli")
#更新数据
  temp.Name="xiaoming"
  engine.ID(1).Update(temp)
#查询数据 Get 方法
  student := new(Student)
  engine.ID(1).Get(student)
  fmt.Println(student.Name)
#查询数据 Query 方法
  engine.Query("select * from Student")
temp := make([]Student,0)
rc := engine.Find(&temp)
```

```
    engine.ID(1).Delete(student)

}
```

17.7.1. 处理数据类型

17.7.1.1. 二进制类型

PanWeiDB V2.0-S3.0.0_Beta 提供两种不同的方法存储二进制数据。 二进制数据可以使用二进制数据类型 BYTEA 存储在表中，或者使用大对象特性以一种特殊的格式将二进制数据存储在一个独立的表中， 然后通过在表中保存一个类型为 OID 的值来引用该表。以下为 BYTEA 类型使用示例。

1） 创建表

```
class ImageData(SQLObject):

image = BLOBCol(default=b'emptydata',length=256)



ImageData.createTable(ifNotExists=True)
```

2） 插入数据

```
data = bytes(range(256))

i1 = ImageData(image=data)

i1id = i1.id

ImageData._connection.cache.clear()
```

3） 读取数据

```
i2 = ImageData.get(i1id)

assert i2.image == data



ImageData(image = b'string')

assert ImageData.selectBy(image=b'string').count()==1
```

17.7.1.2. 字符类型

PanWeiDB V2.0-S3.0.0_Beta 中 TEXT 类型与 VARCHAR 类型都是可变长的字符类型，区别在于 VARCHAR 类型通过 VARCHAR(n)中的 n 来限制最大长度，而 TEXT 类型没有。TEXT 类型与 VARCHAR 类型几乎没有性能差别，TEXT 类型最多可存储 1G 数据 。

1） 创建表

```
class String1(SQLObject):

col1 = StringCol()# text 类型

col2 = StringCol(length=256)# varchar 类型



String1.createTable(ifNotExists=True)
```

2） 插入数据

```
a='test1'

b='test2'
```

```
String1(col1=a,col2=b)
```

3) 读取数据

```
s1 = String1.get(1)

assert s1.col1==a

assert s1.col2==b
```

17.7.1.3. 数字类型

数字类型有整数类型、任意精度数字类型、浮点类型和序数类型。下面用整数类型 integer、任意精度数字类型 numeric 和 double precision 举例说明。

1)创建表

```
class Number1(SQLObject):

col1 = IntCol()# integer 类型

col2 = DecimalCol(size=6,precision=4)# numeric 类型

col3 = FloatCol()# double precision 类型



Number1.createTable(ifNotExists=True)
```

2)插入数据

```
a = 1

b= decimal.Decimal(2.141)

c = 3.1415926
```

```
Number1(col1=a,col2=b,col3=c)
```

3)读取数据

```
n1=Number1.get(1)

print(n1.col1)

print(n1.col2)

print(n1.col3)
```

17.7.1.4. 时间/日期类型

日期/时间类型有 timestamp 、date、time 、interval ； 下面以 timestamp 、date、time 举例说明。

1)创建表

```
class DateTime1(SQLObject):

col1 = DateTimeCol() # timestamp 类型

col2 = DateCol() # date 类型

col3 = TimeCol() # time 类型



DateTime1.createTable(ifNotExixts = True)
```

2)插入数据

```
_now = datetime.now().replace(microsecond = 0)

DateTime1(col1=_now,col2=_now,col3=_now.time())
```

3)读取数据

```
dt1=DateTime1.get(1)

assert  isinstance(dt1.col1,datetime)

assert  isinstance(dt1.col2,date)

assert  isinstance(dt1.col3,time)
```

17.8. 附录 1：主要接口

主要接口说明如下：

- connectionForURI(uri,oldUri=False)

此方法用于创建新的数据库会话并返回新的 connection 对象。参数说明如下：

关键字	参数说明
uri	当前需连接的 Uri
oldUri	是否存在旧的 Uri

返回值说明如下：

关键字	参数说明
conn	连接对象

调用示例见附录 2。

❖ createTable()

此方法根据类创建表。参数说明如下：

关键字	参数说明
ifNotExists	表是否不存在
createJoinTables	是否创建连接表
createIndexes	是否创建索引
applyConstraints	是否应用约束
connection	连接对象

返回值说明如下：

关键字	参数说明
extra_sql	需执行的 sql 语句

调用示例见附录 2。

❖ dropTable()

此方法根据类创建表。参数说明如下：

关键字	参数说明
ifExists	表是否存在
dropJoinTables	是否删除连接表
cascade	是否级联删除
connection	连接对象

调用示例见附录 2。

❖ selectBy()

此方法执行一个 SQL 查询，即 Select 命令。参数说明如下：

关键字	参数说明
connection	连接对象
**kw	查询条件

返回值说明如下：

关键字	参数说明
cls.SelectResultsClass(cls, conn._SO_columnClause(cls, kw),connection=conn)	调用其他函数，查询结果

调用示例见附录 2。

❖ delete()

根据 id 号删除数据。

参数说明如下：

关键字	参数说明
id	数据 id 号
connection	连接对象

调用示例见附录 2。

❖ deleteBy()

删除数据。

参数说明如下：

关键字	参数说明
connection	连接对象
**kw	删除条件

调用示例见附录 2。

❖ get()

此方法用于查询单条数据，并使用返回的字段为传入的对象赋值。

参数：

关键字	参数说明
id	查询数据的 id
connection	连接对象
selectResults	查询结果

调用示例见附录 2。

17.9. 附录 2：调用示例

```
from sqlobject import *
```

```
connection_string = 'postgres://pwadmin:panwei_123@127.0.0.1:5432/pan
weidb'
connection = connectionForURI(connection_string)
sqlhub.processConnection = connection

class Person(SQLObject):
firstName = StringCol()
middleInitial = StringCol(length=1,default=None)
lastName = StringCol()

Person._connection.debug = False

def create_p():
Person.createTable()

def insert_p():
Person(firstName='John1',lastName='Doe')
Person(firstName='John',lastName='Doe')
p=Person(firstName='Bob',lastName='Hope')
p.middleInitial = 'Q'
Person(firstName='John',lastName='David')

def update_p():
p2 = Person.get(1)
```

```
p2.middleInitial='F'
p2.set(firstName='Robert',lastName='Hope Jr.')


def delete_p():
  Person.delete(1)
Person.deleteMany(OR(Person.q.firstName == 'bob',
Person.q.firstName == 'fred'))
Person.deleteBy(firstName='Robert')


def select_p():
peeps = Person.select(Person.q.firstName=='John')
print(list(peeps))
peeps = Person.select(
OR(Person.q.firstName =='John',
LIKE(Person.q.lastName,'%Hope%')))
  print(list(peeps))
peeps = Person.select("""person.first_name = 'John' AND
person.last_name LIKE 'D%'""")
print(list(peeps))
peeps = Person.selectBy(firstName = 'John')
print(list(peeps))


def drop_p():
Person.dropTable(ifExists=True)
```

```
create_p()
insert_p()
update_p()
select_p()
delete_p()
select_p()
drop_p()
```